ICSE · CLASS 10 · COMPUTER APPLICATIONS

ICSE Java Programs

Complete theory, harder self-test questions and 350 fully tested programs — every chapter of the syllabus

☕ Java | JDK 21 | 350 programs | 15 chapters | 38 extras

01 ☕ Introduction to OOP and Java
02 🔢 Values and Data Types
03 ➕ Operators in Java
04 ⌨️ Input in Java
05 📐 Mathematical Library Methods
06 🔀 Conditional Constructs in Java
07 🔁 Iterative Constructs in Java
08 🔲 Nested for Loops
09 🏛️ Class as the Basis of all Computation
10 🧩 User-Defined Methods
11 🏗️ Constructors
12 📚 Library Classes
13 🔒 Encapsulation
14 🗃️ Arrays
15 🔤 String Handling

github.com/CodeWithAyanava/icse-java-programs

Standard Java · every program compiled and run on JDK 21 · November 2026

Contents

01 ☕ Introduction to OOP and Java 3
Theory 4
Test yourself — MCQs and short questions 11
Programs Q1 – Q5 (5) 14

02 🔢 Values and Data Types 18
Theory 19
Test yourself — MCQs and short questions 26
Programs Q6 – Q13 (8) 29

03 ➗ Operators in Java 35
Theory 36
Test yourself — MCQs and short questions 43
Programs Q14 – Q23 (10) 45

04 ⌨️ Input in Java 53
Theory 54
Test yourself — MCQs and short questions 58
Programs Q24 – Q31 (8) 61

05 📐 Mathematical Library Methods 68
Theory 69
Test yourself — MCQs and short questions 73
Programs Q32 – Q45 (14) 76

06 🔀 Conditional Constructs in Java 86
Theory 87
Test yourself — MCQs and short questions 98
Programs Q46 – Q77 (32) 101

07 🔁 Iterative Constructs in Java 128
Theory 129
Test yourself — MCQs and short questions 141
Programs Q78 – Q131 (54) 145

08 🔲 Nested for Loops 187
Theory 188
Test yourself — MCQs and short questions 194
Programs Q132 – Q175 (44) 198

09 🏛️ Class as the Basis of all Computation 225
Theory 226
Test yourself — MCQs and short questions 232
Programs Q176 – Q181 (6) 234

10 🧩 User-Defined Methods 242
Theory 243
Test yourself — MCQs and short questions 250
Programs Q182 – Q211 (30) 253

11 🏗️ Constructors 288
Theory 289
Test yourself — MCQs and short questions 293
Programs Q212 – Q237 (26) 296

12 📚 Library Classes 334
Theory 335
Test yourself — MCQs and short questions 339
Programs Q238 – Q249 (12) 342

13 🔒 Encapsulation 353
Theory 354
Test yourself — MCQs and short questions 358
Programs Q250 – Q254 (5) 360

14 🗃️ Arrays 366
Theory 367
Test yourself — MCQs and short questions 377
Programs Q255 – Q301 (47) 381

15 🔤 String Handling 426
Theory 427
Test yourself — MCQs and short questions 434
Programs Q302 – Q350 (49) 438

CHAPTER

01



Introduction to OOP and Java

Principles of OOP · Introduction to Java · Elementary concept of objects and classes

5 programs

Q1 – Q5

Theory

📑 Contents

1. Programming paradigms: POP and OOP
2. The four principles of OOP
3. Introduction to Java
4. Applets and applications
5. The Java compilation process
6. Features of Java
7. Elementary concept of objects and classes
8. Structure of a simple Java program
9. Differentiate between — quick table
10. One-line facts
11. Test yourself
12. Programs in this chapter

1. Programming paradigms: POP and OOP

A **programming paradigm** is a style or approach of writing programs.

Procedure Oriented Programming (POP) is an approach in which a program is divided into a set of **functions (procedures)**. The emphasis is on *doing things* (algorithms). Data is mostly global and moves freely from function to function. *Examples:* C, FORTRAN, Pascal, BASIC.

Object Oriented Programming (OOP) is an approach in which a program is organised around **objects** — units that combine **data** and the **functions (methods)** that operate on that data. The emphasis is on *data* rather than procedure. *Examples:* Java, C++, Python, C#.

Basis	Procedure Oriented Programming (POP)	Object Oriented Programming (OOP)
Emphasis	On functions / procedures (doing things)	On data (objects)
Division of program	Into functions	Into objects
Approach	Top-down	Bottom-up
Data security	Data moves freely between functions — less secure	Data is hidden inside objects — more secure
Access control	No access specifiers	Access specifiers (`private` , `protected` , `public`)
Adding new data/functions	Difficult	Easy
Reusability	Limited	High, through inheritance
Real-world modelling	Poor	Close to real-world entities
Examples	C, Pascal, FORTRAN	Java, C++, Python

> **NOTE**
>
> Java is called a **pure** object oriented language in ICSE books in the sense that every function must be written inside a class. (Strictly speaking it is not 100% pure because it has primitive data types such as `int` and `char` , which are not objects.)

2. The four principles of OOP

2.1 Data Abstraction

Abstraction is the act of representing the **essential features** of an entity **without including the background details** or explanations.

Real-life example: While driving a car we use the steering wheel, accelerator and brake. We do not need to know how the engine, fuel injection or gearbox work internally. *Java example:* We call `Math.sqrt(25)` without knowing the algorithm used inside it.

2.2 Encapsulation

Encapsulation is the **wrapping up of data and the functions** that operate on the data **into a single unit** (called a class). The data is kept hidden from the outside world and can be accessed only through the functions of the class — this is called **data hiding**.

Real-life example: A medicine capsule wraps different medicines in one shell. A bank keeps your balance hidden; you can change it only through deposit and withdrawal counters. *Java example:* A class with `private` data members and `public` methods.

2.3 Inheritance

Inheritance is the process by which **objects of one class acquire the properties and behaviour of objects of another class**. The class that is inherited from is the **base / super / parent class**; the class that inherits is the **derived / sub / child class**. It supports **reusability**.

Real-life example: A child inherits features from parents. A *Car* is a *Vehicle* — it inherits wheels, engine and speed from Vehicle and adds its own features such as air-conditioning. *Java keyword:* `extends`.

2.4 Polymorphism

Polymorphism (Greek: *many forms*) is the ability of a function or an object to **take more than one form**, i.e. the same message produces different results in different situations.

Real-life example: A person behaves as a student in school, a son at home and a customer in a shop. The word "open" means different actions for a door, a file and a bank account. *Java example:* **Method overloading** — several methods named `area()` that work on a circle, a square and a rectangle.

> **TIP**
>
> **Exam pattern:** "Name the OOP principle shown below" is very common. Look for the key idea: *hiding details* → **Abstraction** · *binding data + methods / data hiding* → **Encapsulation** · *reusing an existing class* → **Inheritance** · *same name, many forms* → **Polymorphism**.

Situation	Principle
A TV remote has buttons; the circuit inside is not shown	Abstraction
`private` data is reachable only through methods of the class	Encapsulation
`class Student extends Person`	Inheritance
`area(double r)` and `area(double l, double b)` in one class	Polymorphism
A class binds the properties and methods together	Encapsulation
A switch board shows only switches, not the wiring	Abstraction

3. Introduction to Java

Java is a **high-level, object oriented, platform-independent** programming language developed by **James Gosling** and his team at **Sun Microsystems** in **1991**. It was first named **Oak** and renamed **Java** in **1995**. Sun Microsystems was later acquired by **Oracle Corporation** (2010).

Some terms

Term	Meaning
JDK (Java Development Kit)	The package of tools needed to *develop* Java programs — compiler (`javac`), JVM, libraries, etc.
JRE (Java Runtime Environment)	The JVM + library classes needed only to *run* Java programs
API (Application Programming Interface)	The ready-made library of classes and packages supplied with Java (e.g. `java.lang` , `java.util`)
BlueJ	An IDE (Integrated Development Environment) designed for beginners, used in many ICSE schools

4. Java applets and Java applications

There are two types of Java programs.

Java application (stand-alone program): A Java program that runs **independently** on a computer under the control of the operating system with the help of the JVM. It must contain a **`main()` method**, from where execution begins. *Examples:* a program to calculate simple interest, a billing program, a desktop game.

Java applet: A **small Java program** that is designed to be **embedded in a web page (HTML document)** and run inside a **Java-enabled web browser** or an applet viewer. It does **not** need a `main()` method. *Examples:* an animated clock or a small game displayed on a web page.

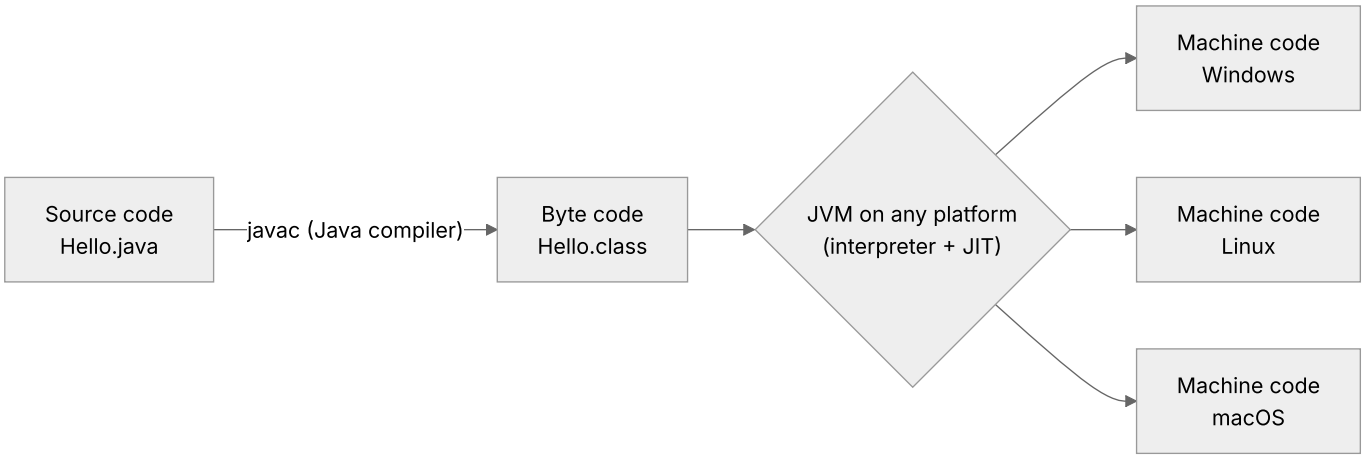
Basis	Java Application	Java Applet
Execution	Runs independently on the OS through the JVM	Runs inside a web browser / applet viewer
`main()` method	Required	Not required
Size	Can be of any size	Small
Access to local files	Allowed	Restricted (for security)
Example	A banking program	An animation in a web page

> **WARNING**
>
> Applets are **obsolete** in real-world Java (browsers stopped supporting them and the Applet API is deprecated in modern JDKs). However, the **definition and differences are still in the ICSE syllabus**, so learn them for the exam.

5. The Java compilation process

Term	Definition
Source code	The program written by the programmer in a high-level language (Java), saved with the extension `.java`

Term	Definition
Compiler	A translator that converts the **whole** source program into another form in one go and lists all errors together. The Java compiler is `javac`
Byte code	The **intermediate code** produced by the Java compiler. It is **not** specific to any machine and is stored in a `.class` file
Java Virtual Machine (JVM)	A **virtual (software) machine** that **interprets / translates byte code into machine code** of the host computer and executes it. Every platform has its own JVM
Object code / machine code	Code in binary (0s and 1s) that the processor of a particular computer can execute directly
Interpreter	A translator that translates and executes a program **line by line**
JIT compiler (Just-In-Time)	A part of the JVM that converts frequently used byte code into machine code at run time to speed up execution



Steps involved

1. The programmer writes the **source code** and saves it as `Hello.java`.
2. The **compiler** `javac Hello.java` checks the syntax and converts the source code into **byte code** (`Hello.class`).
3. The **JVM** (`java Hello`) loads the byte code, verifies it, and the interpreter/JIT compiler converts it into **machine code** for that computer.
4. The machine code is **executed** by the processor and the output is produced.

> **IMPORTANT**
>
> Java is **both compiled and interpreted**: compiled from source code to byte code, then interpreted by the JVM from byte code to machine code. This two-step process is what makes Java **platform independent** — the *same* `.class` file runs on any computer that has a JVM. ("**Write Once, Run Anywhere**" — WORA.)

Basis	Compiler	Interpreter
Translation	Whole program at once	Line by line
Errors	Listed together after compilation	Shown one at a time; stops at the first error
Speed of execution	Faster (translation done once)	Slower
Object file	Generated	Not generated

Basis	Source code	Byte code	Machine (object) code
Written/produced by	Programmer	`javac` compiler	JVM

Basis	Source code	Byte code	Machine (object) code
Extension	`.java`	`.class`	— (in memory)
Machine dependent?	No	No	Yes
Understood by	Humans	JVM	Processor

6. Features of Java

Feature	Meaning
Simple	Syntax is similar to C/C++ but removes complex features such as pointers and operator overloading
Object oriented	Everything is written inside classes; supports abstraction, encapsulation, inheritance and polymorphism
Platform independent	Byte code runs on any machine that has a JVM (WORA)
Portable	Programs can be moved from one system to another without change; sizes of data types are fixed on all machines
Robust	Strong type checking, automatic memory management (garbage collection) and exception handling make programs reliable
Secure	No explicit pointers; byte code is verified; programs run inside the JVM (sandbox)
Multithreaded	Can perform several tasks at the same time
Compiled and interpreted	Two-stage translation through byte code
Distributed	Supports networking; programs can use resources across the internet
Case sensitive	`Total`, `total` and `TOTAL` are three different identifiers
Automatic garbage collection	Memory occupied by unused objects is freed automatically

7. Elementary concept of objects and classes

7.1 Object

An **object** is an **identifiable entity** that has some **characteristics (state)** and **behaviour**.

- **State** = the values of its attributes (stored in **data members / instance variables**).
- **Behaviour** = what it can do (written as **member methods / functions**).

Real-world object	State (attributes)	Behaviour (methods)
Car	colour, brand, speed	accelerate(), brake(), turn()
Student	name, roll number, marks	study(), writeExam()
Bank account	account number, balance	deposit(), withdraw()
Mobile phone	brand, battery level	call(), sendMessage()

7.2 Class

A **class** is a **blueprint / template** that defines the **common attributes and behaviour** of a set of similar objects. Objects are **instances** of a class.

- **Class as a blueprint:** just as many houses are built from one architectural plan, many objects are created from one class.
- **Class as an object factory:** a class can produce any number of objects, each having its own copy of the data members. Hence a class is called an **object factory**.
- **Class as a user-defined data type:** once a class `Student` is defined, it can be used like a type to declare variables: `Student s1;`. Such a type is called a **composite / user-defined / reference** data type.

7.3 Message passing

Objects communicate with each other by **sending messages**, i.e. by **calling each other's methods**. A message = name of the object + name of the method + information (arguments) needed.

```
car1.accelerate(60);   // the message "accelerate by 60" is sent to the object car1
```

7.4 Creating an object

```
Box b1 = new Box();    // declares a reference b1 and creates an object using 'new'
```

- `new` allocates memory for the object and returns its reference (address).
- `b1` is a **reference variable** that refers to the object.

Basis	Class	Object
Nature	Blueprint / template / logical entity	Instance of a class / physical (real) entity
Memory	No memory for data is allocated when a class is defined	Memory is allocated when an object is created
Number	Defined once	Many objects can be created from one class
Keyword	Declared using `class`	Created using `new`
Example	`Car`	`car1`, `car2`

> **TIP**
>
> In questions like "*A class is called an object factory. Why?*" write: **Because a class is a blueprint from which any number of objects having similar attributes and behaviour can be created.**

---

8. Structure of a simple Java program

```
// Single-line comment: my first program
public class Hello                // class header – file must be saved as Hello.java
{
    public static void main(String[] args)   // execution begins here
    {
        System.out.println("Hello, ICSE!");  // statement ends with a semicolon
    }
}
```

Part	Meaning
`public`	Access specifier — the method/class is accessible from anywhere
`static`	`main()` can be called without creating an object
`void`	`main()` returns no value
`main`	Name of the method from which execution starts
`String[] args`	Array of strings that can receive command-line arguments
`System.out.println()`	Prints the output and moves the cursor to the next line
`System.out.print()`	Prints the output and keeps the cursor on the same line

9. Differentiate between — quick table

Pair	Key difference
POP vs OOP	Functions first vs data (objects) first
Class vs Object	Blueprint vs instance
Abstraction vs Encapsulation	Hiding *complexity* (showing essentials) vs *binding* data and methods together and hiding data
Applet vs Application	Runs in a browser, no `main()` vs stand-alone, has `main()`
Compiler vs Interpreter	Whole program at once vs line by line
Source code vs Byte code	`.java`, human-readable vs `.class`, read by the JVM
Byte code vs Machine code	Platform independent vs platform dependent
JDK vs JVM	Development kit (includes compiler) vs virtual machine that runs byte code
`print()` vs `println()`	Cursor stays on the same line vs moves to the next line

10. One-line facts

- Java was developed by **James Gosling** at **Sun Microsystems**; its original name was **Oak**.
- The Java compiler is **`javac`**; the command to run a program is **`java`**.
- The extension of a source file is **`.java`**; of a byte code file, **`.class`**.
- **JVM** makes Java platform independent; the JVM itself is **platform dependent**.
- A class is a **user-defined data type**; an object is an **instance** of a class.
- The keyword used to create an object is **`new`**.
- The principle that supports **reusability** is **inheritance**.
- **Method overloading** is an example of **polymorphism**.
- **Data hiding** is achieved through **encapsulation**.
- Execution of a Java application begins from the **`main()`** method.

🧠 Test Yourself

Part A — Multiple choice (choose the most appropriate option)

1. A `.class` file produced on Windows is copied to a Linux computer that has a JVM. What happens?
   (a) It must be recompiled with `javac` on Linux (b) It runs without recompilation (c) It runs only if the source file is also copied (d) It cannot run because machine code differs
2. Which statement is **false**?
   (a) Byte code is machine independent (b) The JVM is machine independent (c) `javac` produces byte code (d) The JIT compiler is part of the JVM
3. *Assertion (A):* Java is called both a compiled and an interpreted language.
   Reason (R): The JVM converts source code directly into machine code.
   (a) Both A and R are true and R explains A (b) Both true but R does not explain A (c) A is true, R is false (d) A is false, R is true
4. A class `Account` keeps `balance` as `private` and allows it to change only through `deposit()` and `withdraw()`. The best-matching principle is:
   (a) Inheritance (b) Polymorphism (c) Encapsulation (d) Abstraction
5. A user operates an ATM by pressing buttons without knowing how the machine contacts the bank server. This best illustrates:
   (a) Encapsulation (b) Abstraction (c) Inheritance (d) Polymorphism
6. Which of these is **not** needed to *run* a compiled Java application?
   (a) JVM (b) Byte code (c) `javac` (d) Library classes
7. Why is a class called an *object factory*?
   (a) It stores objects permanently (b) Many objects with similar attributes and behaviour can be produced from it (c) It manufactures methods (d) It allocates memory for itself
8. `Car c;` is written but no `new` is used. Which is correct?
   (a) An object of Car is created (b) Only a reference variable is declared; no object exists yet (c) A compile-time error occurs (d) Memory for all data members is allocated
9. Which pair is correctly matched?
   (a) Inheritance – `new` (b) Polymorphism – method overloading (c) Encapsulation – `extends` (d) Abstraction – `main()`
10. *Assertion (A):* An applet does not need a `main()` method.
    Reason (R): An applet is executed by a browser or applet viewer which controls its life cycle.
    (a) Both true and R explains A (b) Both true, R does not explain A (c) A true, R false (d) A false, R true
11. Which of the following is the **correct order**?
    (a) Byte code → source code → machine code (b) Source code → machine code → byte code (c) Source code → byte code → machine code (d) Machine code → byte code → source code
12. What happens if you save `public class Marks` in a file named `marks.java` on a case-sensitive system and compile it?
    (a) Compiles normally (b) Compile-time error — the public class name and file name must match exactly (c) Runtime error (d) The class is renamed automatically

Part B — Short answer questions

1. State two differences between POP and OOP.
2. Define abstraction and give one real-life example.
3. Why is Java platform independent though the JVM is platform dependent?
4. What is byte code? Why is it called intermediate code?
5. Define encapsulation. How is data hiding related to it?

6. Write two differences between a Java applet and a Java application.
7. Why is a class called a user-defined data type?
8. Identify the object, its state and its behaviour: *"Riya's red bicycle with 21 gears can speed up and stop."*
9. What is meant by message passing? Give an example statement.
10. Name the principle: (i) A `SavingsAccount` class reuses `Account`; (ii) `print(int)` and `print(String)` in the same class.

✅ Answers — Part A

Q	Ans	Reason
1	(b)	Byte code is platform independent; any JVM can run it
2	(b)	The JVM is **platform dependent** — each OS has its own JVM
3	(c)	R is false: the JVM converts **byte code** (not source code) into machine code
4	(c)	Binding data with methods and restricting access = encapsulation (data hiding)
5	(b)	Essential features shown, background details hidden
6	(c)	`javac` is needed only to compile, not to run
7	(b)	—
8	(b)	Only a reference is declared; it does not refer to any object yet
9	(b)	—
10	(a)	—
11	(c)	—
12	(b)	A `public` class must be saved in a file with exactly the same name

✅ Answers — Part B

1. POP emphasises functions and follows a top-down approach; OOP emphasises data (objects) and follows a bottom-up approach. In POP data moves freely between functions; in OOP data is hidden inside objects.
2. Abstraction is representing the essential features without including the background details. Example: we drive a car using the steering and pedals without knowing how the engine works.
3. Because the compiler produces machine-independent **byte code**, and a JVM (written separately for each platform) is available for every platform to run that same byte code.
4. Byte code is the code produced by the Java compiler and stored in a `.class` file. It is called intermediate code because it lies between source code and machine code and is not tied to any machine.
5. Encapsulation is the wrapping up of data and the methods that operate on it into a single unit (class). Because the data is kept inside the class and accessed only through its methods, it is hidden from the outside — this is data hiding.
6. An application runs independently and needs `main()`; an applet runs inside a browser and needs no `main()`. Applets have restricted access to local files; applications do not.
7. Because the programmer creates it, and it can be used, like `int` or `char`, to declare variables (`Student s;`).
8. Object: Riya's bicycle; state: colour red, 21 gears; behaviour: speed up, stop.
9. Objects communicate by calling each other's methods. Example: `car1.accelerate(60);`
10. (i) Inheritance (ii) Polymorphism (method overloading).

💻 Programs in this chapter

No.	Program
Q1	Display Student Details using print() and println()
Q2	Marks Table using Tab and New-Line Escape Sequences
Q3	Quotation and File Path using Escape Sequences
Q4	Class Box: Creating an Object and Finding its Volume
Q5	Class Car: Two Objects, Same Behaviour, Different States

Programs Q1 – Q5

Q1 Display Student Details using print() and println()

> Write a program to store a student's name, class and school in three String variables. Display the three details on three separate lines using System.out.println(). Then display all three details on a single line, separated by commas, using System.out.print().

Q_1.java

```
public class Q_1
{
    public static void main(String[] args)
    {
        String name = "Aarav Sen";
        String std = "X";
        String school = "Green Valley School";

        System.out.println("Name   : " + name);
        System.out.println("Class  : " + std);
        System.out.println("School : " + school);

        System.out.print(name + ", ");
        System.out.print(std + ", ");
        System.out.print(school);
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
name	String	to store the name of the student
std	String	to store the class of the student
school	String	to store the name of the school

Q2 Marks Table using Tab and New-Line Escape Sequences

> Write a program to store the marks obtained by a student in English, Mathematics and Science (85, 92 and 88 respectively) in integer variables. Calculate the total marks and display a neat table of subjects and marks using the escape sequences \t (horizontal tab) and \n (new line). Use only one System.out.print() statement to print the rows of the table.

Q_2.java

```
public class Q_2
{
    public static void main(String[] args)
    {
        int eng = 85, mat = 92, sci = 88;
        int total = eng + mat + sci;

        System.out.print("Subject\t\tMarks\n"
                       + "-------\t\t-----\n"
                       + "English\t\t" + eng + "\n"
                       + "Maths\t\t" + mat + "\n"
                       + "Science\t\t" + sci + "\n"
                       + "-------\t\t-----\n"
                       + "Total\t\t" + total + "\n");
```

```
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
eng	int	to store the marks in english
mat	int	to store the marks in mathematics
sci	int	to store the marks in science
total	int	to store the total marks

Q3 Quotation and File Path using Escape Sequences

Write a program to display the following three lines exactly as shown, using the escape sequences \", \' and \\ wherever required:

```
Swami Vivekananda said, "Arise, awake and stop not till the goal is reached."
It's Java's 'Write Once, Run Anywhere' promise.
File saved at: C:\ICSE\Java\Q_3.java
```

Q_3.java

```java
public class Q_3
{
    public static void main(String[] args)
    {
        String person = "Swami Vivekananda";
        char apos = '\'';

        System.out.println(person + " said, \"Arise, awake and stop not till the goal is reached.\"");
        System.out.println("It" + apos + "s Java" + apos + "s 'Write Once, Run Anywhere' promise.");
        System.out.println("File saved at: C:\\ICSE\\Java\\Q_3.java");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
person	String	to store the name of the speaker
apos	char	to store the apostrophe character

Q4 Class Box: Creating an Object and Finding its Volume

Define a class Box (written here as class Q_4 to match the file name) with the following specifications:

```
Data members  : double length, breadth, height
void assign(double l, double b, double h) : to assign the
                arguments to the data members
double volume() : to calculate and return the volume of the box
void display()  : to display the dimensions and the volume
```

Write a main method to create an object of the class and call the above methods to assign the dimensions 12.5, 8.0 and 4.0 and display the details.

Q_4.java

```java
public class Q_4
{
    double length, breadth, height;

    void assign(double l, double b, double h)
    {
```

```
        length = l;
        breadth = b;
        height = h;
    }

    double volume()
    {
        return length * breadth * height;
    }

    void display()
    {
        System.out.println("Length  = " + length);
        System.out.println("Breadth = " + breadth);
        System.out.println("Height  = " + height);
        System.out.println("Volume  = " + volume());
    }

    public static void main(String[] args)
    {
        Q_4 box = new Q_4();
        box.assign(12.5, 8.0, 4.0);
        box.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
length	double	to store the length of the box
breadth	double	to store the breadth of the box
height	double	to store the height of the box
l	double	to receive the length
b	double	to receive the breadth
h	double	to receive the height
box	Q_4	to refer to the box object

Q5 Class Car: Two Objects, Same Behaviour, Different States

Define a class Car (written here as class Q_5) to show that objects of the same class share behaviour but have their own state.

```
Data members : String brand, int speed
void setBrand(String b) : to assign the brand and set speed to 0
void accelerate(int inc) : to increase the speed by inc
void brake(int dec)      : to decrease the speed by dec; the
                           speed must never become negative
void show()              : to display the brand and the speed
```

In the main method, create two objects. Send the messages accelerate(60) and brake(20) to the first car, and accelerate(40) and brake(70) to the second car, displaying the state of each car after every message.

Q_5.java

```
public class Q_5
{
    String brand;
    int speed;

    void setBrand(String b)
    {
```

```
        brand = b;
        speed = 0;
    }

    void accelerate(int inc)
    {
        speed = speed + inc;
    }

    void brake(int dec)
    {
        speed = speed - dec;
        if (speed < 0)
            speed = 0;
    }

    void show()
    {
        System.out.println(brand + " is moving at " + speed + " km/h");
    }

    public static void main(String[] args)
    {
        Q_5 car1 = new Q_5();
        Q_5 car2 = new Q_5();
        car1.setBrand("Tata Nexon");
        car2.setBrand("Maruti Swift");

        car1.accelerate(60);
        car1.show();
        car1.brake(20);
        car1.show();

        car2.accelerate(40);
        car2.show();
        car2.brake(70);
        car2.show();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
brand	String	to store the brand of the car
speed	int	to store the current speed
b	String	to receive the brand name
inc	int	to receive the increase in speed
dec	int	to receive the decrease in speed
car1	Q_5	to refer to the first car object
car2	Q_5	to refer to the second car object

CHAPTER

02



Values and Data Types

Character set · ASCII and Unicode · Escape sequences · Tokens · Constants and variables · Data types · Type conversion

8 programs

Q6 – Q13

Theory

📑 Contents

1. Character set
2. ASCII and Unicode
3. Escape sequences
4. Tokens
5. Constants and variables
6. Data types
7. Type conversion
8. Evaluate the expression — solved examples
9. Predict the output
10. Find the errors
11. One-line facts
12. Test yourself
13. Programs in this chapter

1. Character set

The **character set** is the set of valid characters that a language can recognise. Java uses the **Unicode** character set. The characters are grouped as:

Group	Characters
Letters	`A – Z`, `a – z`
Digits	`0 – 9`
Special characters / delimiters	`+ - * / % = < > ! & | ^ ~ ? : ; , . ' " ( ) [ ] { } @ # _ $ \`
White spaces	blank space, tab (`\t`), new line (`\n`), carriage return, form feed

2. ASCII and Unicode

ASCII (American Standard Code for Information Interchange) is a **7-bit** code that represents **128** characters (codes **0 to 127**) — English letters, digits, punctuation and control characters.

Unicode is a **16-bit** (2-byte) code in Java's `char` type, able to represent **65,536** characters (codes 0 to 65535) covering the scripts of almost all the world's languages (Hindi, Bengali, Chinese, etc.). The first 128 Unicode characters are exactly the ASCII characters.

Basis	ASCII	Unicode
Bits used	7 bits (often stored in 8)	16 bits in Java `char`
Number of characters	128	65,536 (in a `char`)
Languages	English only	Almost all languages
Used by Java?	Subset of Unicode	Yes — Java's character set

ASCII codes you must remember

Character(s)	Codes
`'A'` – `'Z'`	**65 – 90**
`'a'` – `'z'`	**97 – 122**

Character(s)	Codes
`'0'` – `'9'`	**48 – 57**
Blank space `' '`	**32**
Difference between `'a'` and `'A'`	**32**

> **TIP**
>
> To convert a digit character to its value: `'7' - '0'` = 55 − 48 = **7**. To convert uppercase to lowercase without a library method: `(char)(ch + 32)`.

---

3. Escape sequences

An **escape sequence** is a **non-graphic character** represented by a **backslash (`\`) followed by a character**. It is used inside character or string literals to print characters that cannot be typed directly or that have a special meaning.

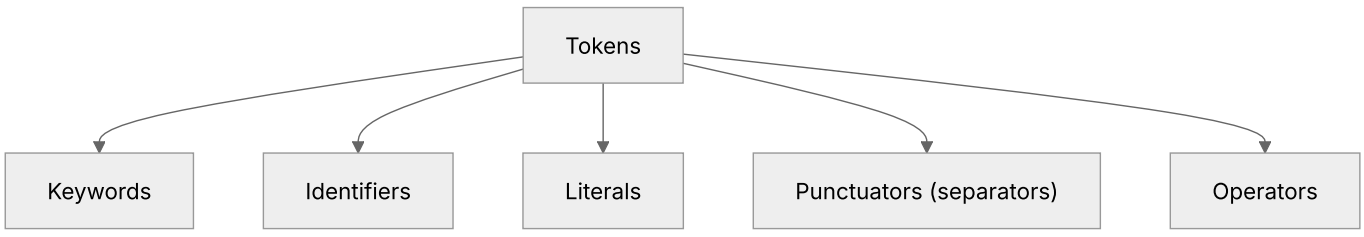
Escape sequence	Meaning	Example	Output
`\n`	New line	`"Hi\nThere"`	Hi *(new line)* There
`\t`	Horizontal tab	`"A\tB"`	A B
`\\`	Backslash	`"C:\\Java"`	C:\Java
`\"`	Double quote	`"\"OK\""`	"OK"
`\'`	Single quote	`'\''`	'

> **WARNING**
>
> `"C:\new"` does **not** print `C:\new` — `\n` is a new line! Write `"C:\\new"`. An escape sequence counts as **one** character: `"A\tB".length()` is **3**.

---

4. Tokens

A **token** is the **smallest individual unit** of a program that is meaningful to the compiler. Java has **five** types of tokens:



4.1 Keywords

Keywords are **reserved words** that have a **special meaning** for the compiler. They cannot be used as identifiers. All keywords are in **lowercase**.

Examples: `class`, `public`, `static`, `void`, `int`, `double`, `char`, `boolean`, `if`, `else`, `switch`, `case`, `default`, `break`, `continue`, `for`, `while`, `do`, `return`, `new`, `this`, `final`, `private`, `protected`, `import`, `package`, `extends`.

> **NOTE**
>
> `true`, `false` and `null` are **literals**, not keywords, but they also cannot be used as identifiers. `main`, `String` and `System` are **not** keywords — they are identifiers.

4.2 Identifiers

Identifiers are the **names given** to the different parts of a program such as variables, methods, classes and arrays.

Rules for naming identifiers

1. It may contain letters, digits, underscore (`_`) and dollar sign (`$`).
2. It **must not begin with a digit**.
3. It **must not be a keyword** (or `true`, `false`, `null`).
4. **No blank spaces** or other special characters are allowed.
5. It is **case sensitive** (`sum` and `Sum` differ). There is no limit on the length.

Identifier	Valid?	Reason
`total_marks`	✅	—
`$amount`	✅	`$` is allowed
`_count1`	✅	—
`2ndName`	❌	Starts with a digit
`net pay`	❌	Contains a space
`class`	❌	Keyword
`rate%`	❌	`%` not allowed
`Main`	✅	Not a keyword
`true`	❌	Reserved literal

Naming conventions (not compulsory but followed): class names begin with a capital letter (`Student`); variable and method names begin with a small letter and use camelCase (`netSalary`, `findSum()`); constants are in capitals (`PI`, `MAX_MARKS`).

4.3 Literals (constants)

Literals are **fixed values** that do not change during the execution of a program.

Type	Examples	Notes
Integer literal	`25`, `-108`, `0`, `50000L`	Whole numbers; `L` suffix = `long`
Real (floating-point) literal	`3.14`, `-0.5`, `2.5f`, `1.2E3`	`double` by default; `f` suffix = `float`
Character literal	`'A'`, `'7'`, `'$'`, `'\n'`	**One** character in single quotes
String literal	`"ICSE"`, `"7"`, `""`	Zero or more characters in double quotes
Boolean literal	`true`, `false`	Only two values
Null literal	`null`	Absence of a reference

> **IMPORTANT**
>
> `'A'` is a **character** literal; `"A"` is a **String** literal. `'AB'` is an **error** (more than one character in single quotes). `""` (empty string) is valid, but `''` (empty char) is **invalid**.

4.4 Punctuators (separators)

Punctuators are special symbols used to **separate** or **group** parts of a program: `;` `,` `.` `( )` `{ }` `[ ]`.

4.5 Operators

Operators are symbols that **perform operations** on operands: `+ - * / % = == < > && || ! ++ --` etc. (Chapter 03).

> **TIP**
>
> **Counting tokens:** in `int sum = a + 10;` the tokens are `int`, `sum`, `=`, `a`, `+`, `10`, `;` — **7 tokens** (1 keyword, 2 identifiers, 2 operators, 1 literal, 1 punctuator).

---

5. Constants and variables

A **variable** is a **named memory location** that stores a value which **can change** during program execution.

```
int marks;          // declaration
marks = 95;         // assignment (initialisation)
int age = 15;       // declaration with initialisation
int a = 5, b = 10;  // multiple declaration
```

Term	Meaning	Example
Declaration	Telling the compiler the name and type of a variable	`double rate;`
Static initialisation	Initial value given directly as a literal	`int n = 10;`
Dynamic initialisation	Initial value computed at run time from an expression	`double area = l * b;`

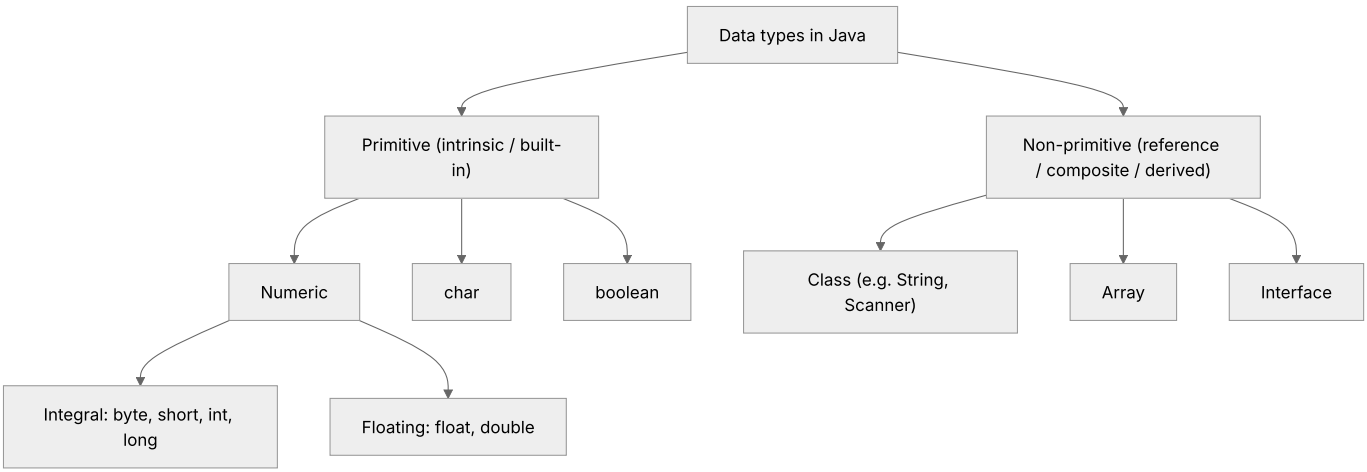
A **constant** (symbolic constant) is a variable whose value **cannot be changed** once assigned. It is declared using the keyword `final`.

```
final double PI = 3.14159;
PI = 3.14;          // ❌ compile-time error: cannot assign a value to final variable PI
```

---

6. Data types

A **data type** specifies the **type of value** a variable can hold, the **memory** it occupies and the **operations** that can be performed on it.



6.1 Primitive data types

Type	Size (bits)	Size (bytes)	Range	Default value
`byte`	8	1	−128 to 127	`0`
`short`	16	2	−32,768 to 32,767	`0`
`int`	32	4	-2^{31} to $2^{31} - 1$ (about $\pm 2.1 \times 10^9$)	`0`
`long`	64	8	-2^{63} to $2^{63} - 1$ (about $\pm 9.2 \times 10^{18}$)	`0L`
`float`	32	4	about $\pm 3.4 \times 10^{38}$ (6–7 significant digits)	`0.0f`
`double`	64	8	about $\pm 1.7 \times 10^{308}$ (15–16 significant digits)	`0.0d`
`char`	16	2	0 to 65,535 (`'\u0000'` to `'\uffff'`)	`'\u0000'`
`boolean`	1 bit of information (JVM-dependent storage)	—	`true` / `false`	`false`

> **NOTE**
>
> Default values apply only to **instance/class variables** (data members) and array elements. **Local variables** have no default value — using one before assigning it gives a compile-time error.

6.2 Non-primitive (reference) data types

Non-primitive data types are built from primitive types or other classes. They store a **reference (address)** of the memory where the actual data is kept. Examples: **class** (e.g. `String`, `Scanner`, a user-defined `Student`), **array**, **interface**.

Basis	Primitive data type	Non-primitive data type
Definition	Predefined (built-in) in Java	Created by the programmer or provided as classes
Stores	The actual value	A reference (address) to an object
Size	Fixed	Depends on the object
Default value	`0`, `0.0`, `false`, `'\u0000'`	`null`
Example	`int`, `char`, `double`	`String`, arrays, class `Student`

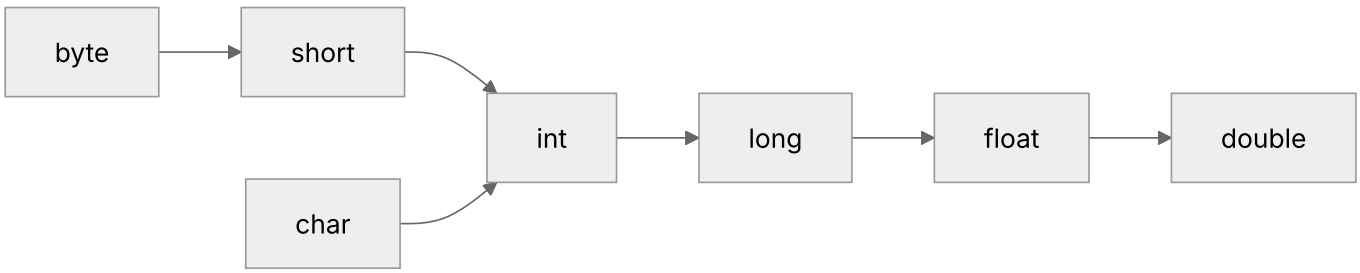
7. Type conversion

Type conversion is the process of converting a value of one data type into another.

7.1 Implicit type conversion (coercion / widening / type promotion)

Done **automatically by the compiler** when a value of a **smaller** type is assigned to or used with a **larger** type. There is **no loss of data**.

Hierarchy (lower → higher):



Type promotion in an expression: in a mixed expression, all operands are promoted to the **highest type present** — and `byte`, `short` and `char` are always promoted to at least `int`.

Expression (int i, long l, float f, double d, char c)	Result type
`i + d`	`double`
`c + i`	`int`
`c + c`	`int` (not `char`!)
`i * l`	`long`
`l + f`	`float`
`f * d`	`double`
`i / 2`	`int`

7.2 Explicit type conversion (type casting / narrowing)

Done **by the programmer** using the **cast operator** `(type)`. It is needed when converting a **larger** type to a **smaller** type; data may be **lost**.

```
double x = 9.87;
int n = (int) x;          // 9   — fractional part is truncated, not rounded
char ch = (char) 66;      // 'B'
double q = (double) 7 / 2; // 3.5 — 7 is cast first, then 7.0 / 2
double r = (double) (7 / 2); // 3.0 — 7 / 2 = 3 is computed first, then cast
```

Basis	Implicit conversion	Explicit conversion
Done by	Compiler (automatically)	Programmer, using `(type)`
Direction	Smaller → larger type (widening)	Larger → smaller type (narrowing)
Data loss	No	Possible
Also called	Coercion, type promotion	Type casting
Example	`double d = 10;`	`int n = (int) 9.8;`

WARNING

`int n = 9.8;` is a **compile-time error** ("possible lossy conversion from double to int"). Also `float f = 2.5;` is an error, because `2.5` is a `double` literal — write `2.5f`. And `long big = 3000000000;` is an error — the literal is too large for `int`; write `3000000000L`.

8. Evaluate the expression — solved examples

Q1. `char c = 'B'; int x = c + 2;` → `'B'` is 66 → `x` = **68**.

Q2. `System.out.println('A' + 'B');` → 65 + 66 = **131** (char + char → int).

Q3. `System.out.println("" + 'A' + 'B');` → `"" + 'A'` = `"A"`, then `"A" + 'B'` = **AB**.

Q4. `System.out.println('A' + 1 + "B");` → `'A' + 1` = 66, then `66 + "B"` = **66B**.

Q5. `char ch = 'a'; ch += 2; System.out.println(ch);` → shorthand assignment includes an automatic cast → **c**. (But `ch = ch + 2;` is a **compile-time error** because `ch + 2` is `int`.)

Q6. `double d = 17 / 4;` → integer division 17 / 4 = 4, then widened → **4.0**.

Q7. `int k = (int) 'z' - (int) 'a';` → 122 − 97 = **25**.

Q8. `System.out.println((char) ('a' + 25));` → 97 + 25 = 122 → **z**.

9. Predict the output

```
int a = 7, b = 2;
double d = a / b;
double e = (double) a / b;
char c = (char) (a + 'A');
System.out.println(d + " " + e + " " + c);
System.out.println(1 + 2 + "3" + 4 + 5);
```

Show answer and dry run

Step	Expression	Value
1	`a / b` (int ÷ int)	3 → `d` = 3.0
2	`(double) a / b`	7.0 / 2 = 3.5
3	`a + 'A'`	7 + 65 = 72 → `(char)` 72 = `'H'`
4	`1 + 2`	3 (numeric addition, left to right)
5	`3 + "3"`	`"33"` (string concatenation from here on)
6	`"33" + 4 + 5`	`"3345"`

Output

```
3.0 3.5 H
3345
```

10. Find the errors

```
int 1num = 10;          // (1)
char grade = "A";       // (2)
float rate = 7.5;       // (3)
long pop = 9000000000;  // (4)
boolean flag = 1;       // (5)
final int MAX = 50; MAX = 60;   // (6)
```

Show corrected code

```
int num1 = 10;              // (1) identifier cannot start with a digit
char grade = 'A';           // (2) char needs single quotes
float rate = 7.5f;          // (3) 7.5 is double; use the suffix f
long pop = 9000000000L;     // (4) literal exceeds int range; use the suffix L
boolean flag = true;        // (5) boolean takes only true/false, not 1
final int MAX = 50;         // (6) a final variable cannot be reassigned
```

11. One-line facts

- Java uses the **Unicode** character set; a `char` occupies **2 bytes**.
- ASCII uses **7 bits** and has **128** characters.
- The smallest unit of a program is a **token**; there are **5** types of tokens.
- All keywords are in **lowercase**.
- An identifier cannot **begin with a digit**.
- The keyword used to declare a constant is **`final`**.
- The default type of a real literal is **`double`**; of an integer literal, **`int`**.
- `boolean` is the only primitive type that **cannot be converted** to or from any other type.
- **Casting a real number to `int` truncates** (it does not round).
- `char + char` gives an **`int`**.
- `String` is a **class**, hence a **non-primitive** (reference) type.

🧠 Test Yourself

Part A — Multiple choice

1. What is the output of `System.out.println('a' + 'b' + "c");`?
   (a) abc (b) 195c (c) 195 (d) Error
2. Which declaration compiles **without error**?
   (a) `float f = 1.5;` (b) `char c = 65 + 1;` (c) `int i = 3.0;` (d) `byte b = 200;`
3. *Assertion (A):* `char ch = 'x'; ch = ch + 1;` gives a compile-time error.
   Reason (R): The expression `ch + 1` is of type `int`, which cannot be stored in a `char` without a cast.
   (a) Both true; R explains A (b) Both true; R does not explain A (c) A true, R false (d) A false, R true
4. Which of the following is **not** a valid identifier? (a) `_$x` (b) `Final` (c) `double` (d) `MAX_VALUE`
5. `int x = (int) -7.9;` stores:
   (a) −8 (b) −7 (c) −7.9 (d) Error

6. How many tokens are there in `if (a >= 10) b++;`?
   (a) 7 (b) 8 (c) 9 (d) 10
7. Which literal occupies the **most memory** when stored in its default type? (a) `'A'` (b) `100` (c) `100L` (d) `true`
8. `"A".length() + 'A'` evaluates to: (a) 66 (b) "1A" (c) 2 (d) 65
9. A local variable `int n;` is declared and immediately printed. The result is:
   (a) 0 (b) garbage value (c) compile-time error (d) runtime error
10. What is printed by `System.out.println((double) (5 / 2) + 5 / 2.0);`?
    (a) 5.0 (b) 4.5 (c) 5.5 (d) 4.0
11. Which statement is **true**?
    (a) Every Unicode character is an ASCII character (b) Every ASCII character is a Unicode character (c) ASCII is 16-bit (d) Unicode has 128 characters
12. `System.out.println("Tab:\t|" + "\\n|");` prints:
    (a) `Tab:<tab>|<newline>|` (b) `Tab:<tab>|\n|` (c) `Tab:\t|\n|` (d) Error

Part B — Short answer questions

1. What are tokens? Name all five types with one example each.
2. Distinguish between `'7'`, `"7"` and `7`.
3. Why is `float f = 3.14;` an error? Give two ways to correct it.
4. Give the output: `char c = 'C'; int n = c; System.out.println(c + " " + n + " " + (char)(n + 3));`
5. Explain dynamic initialisation with an example.
6. State the default values of `int`, `double`, `char`, `boolean` and `String` data members.
7. What is type promotion? What will be the type of `b + s` where `b` is `byte` and `s` is `short`?
8. Name the escape sequences for: new line, tab, backslash, double quote.
9. Differentiate between a keyword and an identifier.
10. Evaluate: `int x = 'z' - 'a' + 1;` and `double y = 1 / 2 + 1.0 / 2;`

✅ Answers — Part A

Q	Ans	Reason
1	(b)	`'a' + 'b'` = 97 + 98 = 195 (int addition), then `195 + "c"` = `"195c"`
2	(b)	`65 + 1` is a constant expression (66) within `char` range, so it is allowed
3	(a)	—
4	(c)	`double` is a keyword; `Final` (capital F) is valid
5	(b)	Casting truncates toward zero
6	(c)	`if ( a >= 10 ) b ++ ;` = 9
7	(c)	`long` = 8 bytes
8	(a)	1 + 65 = 66
9	(c)	Local variables are not given default values
10	(b)	`(double)(2)` = 2.0, plus 2.5 = 4.5
11	(b)	—
12	(b)	`\t` becomes a tab; `\\n` is a backslash followed by `n`

✅ Answers — Part B

1. A token is the smallest meaningful unit of a program. Keywords (`int`), identifiers (`sum`), literals (`25`), punctuators (`;`), operators (`+`).
2. `'7'` is a character literal (code 55, `char`); `"7"` is a String literal; `7` is an integer literal.
3. `3.14` is a `double` literal and cannot be stored in a `float` without loss. Correct: `float f = 3.14f;` or `float f = (float) 3.14;`
4. `C 67 F`
5. A variable is given its initial value at run time from an expression, e.g. `double area = length * breadth;`
6. `0` , `0.0` , `'\u0000'` , `false` , `null` .
7. Automatic conversion of operands to a higher type in an expression. `b + s` is of type `int` .
8. `\n` , `\t` , `\\` , `\"` .
9. A keyword is a reserved word with a fixed meaning (e.g. `while`) and cannot be used as a name; an identifier is a programmer-chosen name (e.g. `total`).
10. `x` = 122 − 97 + 1 = **26**; `y` = 0 + 0.5 = **0.5**.

💻 Programs in this chapter

No.	Program
Q6	ASCII Code of an Entered Character
Q7	Character for an Entered ASCII Code
Q8	Implicit Type Conversion
Q9	Explicit Type Casting
Q10	Swap Two Numbers using a Third Variable
Q11	Swap Two Numbers without a Third Variable
Q12	Next and Previous Character
Q13	Uppercase to Lowercase using ASCII Arithmetic

Programs Q6 – Q13

Q6 ASCII Code of an Entered Character

Write a program to accept a character from the user and display its ASCII (Unicode) code.

Example: Input: A Output: ASCII code of A = 65

Q_6.java

```java
import java.util.Scanner;

public class Q_6
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a character: ");
        char ch = sc.next().charAt(0);
        int code = (int) ch;
        System.out.println("ASCII code of " + ch + " = " + code);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
ch	char	to store the entered character
code	int	to store the ascii code

Q7 Character for an Entered ASCII Code

Write a program to accept an ASCII code (0 to 127) and display the character it represents. If the code is outside this range, display "Invalid ASCII code".

Example: Input: 97 Output: Character = a

Q_7.java

```java
import java.util.Scanner;

public class Q_7
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter an ASCII code (0-127): ");
        int code = sc.nextInt();
        if (code >= 0 && code <= 127)
        {
            char ch = (char) code;
            System.out.println("Character = " + ch);
        }
        else
        {
            System.out.println("Invalid ASCII code");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
code	int	to store the entered ascii code
ch	char	to store the corresponding character

Q8 Implicit Type Conversion

Write a program to demonstrate implicit (automatic) type conversion. Declare int i = 25, long l = 100000L, float f = 2.5f, double d = 4.75 and char c = 'A', and display the result and the resulting data type of each of the following:

```
(a) i + d    (b) c + i    (c) i * l    (d) l + f
(e) a double variable assigned the value of i
```

Q_8.java

```java
public class Q_8
{
    public static void main(String[] args)
    {
        int i = 25;
        long l = 100000L;
        float f = 2.5f;
        double d = 4.75;
        char c = 'A';

        double r1 = i + d;
        int r2 = c + i;
        long r3 = i * l;
        float r4 = l + f;
        double r5 = i;

        System.out.println("(a) i + d = " + r1 + "   (int + double gives double)");
        System.out.println("(b) c + i = " + r2 + "   (char + int gives int)");
        System.out.println("(c) i * l = " + r3 + "   (int * long gives long)");
        System.out.println("(d) l + f = " + r4 + "   (long + float gives float)");
        System.out.println("(e) double r5 = i stores " + r5 + "   (int widened to double)");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to store an integer value
l	long	to store a long value
f	float	to store a float value
d	double	to store a double value
c	char	to store a character
r1	double	to store the result of i + d
r2	int	to store the result of c + i
r3	long	to store the result of i * l
r4	float	to store the result of l + f
r5	double	to store i after widening

Q9 Explicit Type Casting

Write a program to demonstrate explicit type conversion (type casting). Accept two integers a and b, and a real number x from the user, and display:

```
(a) the quotient a / b without casting and with (double) casting
(b) the integer part of x using (int) casting
(c) the character whose code is the integer part of x, using
    (char) casting (only if that code lies from 0 to 127,
    otherwise display "Not a valid ASCII code")
(d) the value of x rounded off to the nearest whole number using
    only casting, i.e. (int)(x + 0.5) for x >= 0 and
    (int)(x - 0.5) for x < 0
```

If b is zero, display "Division by zero is not possible" for part (a).

Q_9.java

```java
import java.util.Scanner;

public class Q_9
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter two integers a and b: ");
        int a = sc.nextInt();
        int b = sc.nextInt();
        System.out.print("Enter a real number x: ");
        double x = sc.nextDouble();

        if (b != 0)
        {
            System.out.println("(a) a / b          = " + (a / b));
            System.out.println("    (double) a / b = " + ((double) a / b));
        }
        else
        {
            System.out.println("(a) Division by zero is not possible");
        }

        int whole = (int) x;
        System.out.println("(b) (int) x = " + whole);
        if (whole >= 0 && whole <= 127)
            System.out.println("(c) (char) " + whole + " = " + (char) whole);
        else
            System.out.println("(c) Not a valid ASCII code");

        int rounded;
        if (x >= 0)
            rounded = (int) (x + 0.5);
        else
            rounded = (int) (x - 0.5);
        System.out.println("(d) x rounded = " + rounded);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	int	to store the first integer
b	int	to store the second integer
x	double	to store the real number
whole	int	to store the integer part of x

Name of the Variable	Data Type	Purpose / Description
rounded	int	to store x rounded off

Q10 Swap Two Numbers using a Third Variable

> Write a program to accept two integers and swap (interchange) their values using a third variable. Display the values before and after swapping.

Q_10.java

```
import java.util.Scanner;

public class Q_10
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the first number: ");
        int a = sc.nextInt();
        System.out.print("Enter the second number: ");
        int b = sc.nextInt();

        System.out.println("Before swapping: a = " + a + ", b = " + b);
        int t = a;
        a = b;
        b = t;
        System.out.println("After swapping : a = " + a + ", b = " + b);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	int	to store the first number
b	int	to store the second number
t	int	to hold a value temporarily

Q11 Swap Two Numbers without a Third Variable

> Write a program to accept two integers and swap their values without using a third variable. Display the values before and after swapping.

Q_11.java

```
import java.util.Scanner;

public class Q_11
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the first number: ");
        int a = sc.nextInt();
        System.out.print("Enter the second number: ");
        int b = sc.nextInt();

        System.out.println("Before swapping: a = " + a + ", b = " + b);
        a = a + b;
        b = a - b;
        a = a - b;
        System.out.println("After swapping : a = " + a + ", b = " + b);
```

```
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	int	to store the first number
b	int	to store the second number

Q12 Next and Previous Character

Write a program to accept a character and display the character that comes just before it and just after it in the Unicode (ASCII) sequence.

Example: Input: M Output: Previous: L Next: N

Q_12.java

```java
import java.util.Scanner;

public class Q_12
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a character: ");
        char ch = sc.next().charAt(0);
        char prev = (char) (ch - 1);
        char next = (char) (ch + 1);
        System.out.println("Previous: " + prev + "   Next: " + next);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
ch	char	to store the entered character
prev	char	to store the previous character
next	char	to store the next character

Q13 Uppercase to Lowercase using ASCII Arithmetic

Write a program to accept a character. If it is an uppercase letter, convert it to lowercase by adding 32 to its ASCII code (without using any library method) and display it. Otherwise display "Not an uppercase letter".

Q_13.java

```java
import java.util.Scanner;

public class Q_13
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a character: ");
        char ch = sc.next().charAt(0);
        if (ch >= 'A' && ch <= 'Z')
        {
            char lower = (char) (ch + 32);
            System.out.println("Lowercase: " + lower);
        }
```

```
        else
        {
            System.out.println("Not an uppercase letter");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
ch	char	to store the entered character
lower	char	to store the lowercase letter

CHAPTER

03



Operators in Java

Forms and types of operators · Precedence and associativity · Prefix and postfix · Counters and accumulators · `new` and dot operators · Output statements

10 programs

Q14 – Q23

Theory

📑 Contents

1. Operators, operands and expressions
2. Forms of operators
3. Arithmetic operators
4. Increment and decrement operators
5. Relational operators
6. Logical operators
7. Assignment and shorthand operators
8. Ternary (conditional) operator
9. The `new` operator and the dot operator
10. Precedence and associativity
11. Counters and accumulators
12. Output: `print()` and `println()`, and the `+` operator with strings
13. Writing Java expressions
14. Step-by-step evaluation — solved examples
15. Differentiate between
16. One-line facts
17. Test yourself
18. Programs in this chapter

1. Operators, operands and expressions

- An **operator** is a symbol or token that performs an **operation** (arithmetic, comparison, logical, etc.) on one or more values.
- An **operand** is the value or variable on which an operator acts.
- An **expression** is a valid combination of operators, operands and constants that produces a value. E.g. in `a + b * 2`, `+` and `*` are operators; `a`, `b` and `2` are operands.

Kind of expression	Example	Result type
Arithmetic expression	`a * b + 5`	number
Relational / Boolean expression	`marks >= 40`	`boolean`
Logical expression	`(a > b) && (a > c)`	`boolean`
Pure expression	all operands of the same type, e.g. `int + int`	—
Mixed (impure) expression	operands of different types, e.g. `int + double`	highest type

> **NOTE**
>
> **Bitwise and shift operators** (`&`, `|`, `^`, `~`, `<<`, `>>`, `>>>`) are **not included** in the ICSE syllabus.

2. Forms of operators (unary, binary, ternary)

Form	Number of operands	Examples
Unary	1	unary `+`, unary `-`, `++`, `--`, `!`
Binary	2	`+ - * / %`, `< > <= >= == !=`, `&& ||`, `=`, `+=`
Ternary	3	`? :` (conditional operator)

> **TIP**
>
> The **only ternary operator** in Java is `?:`. The `-` symbol is **unary** in `-a` and **binary** in `a - b`.

Types of operators: arithmetic, relational, logical, assignment, increment/decrement, shorthand (compound assignment), conditional (ternary), and special operators (`new`, dot `.`, `instanceof`).

3. Arithmetic operators

Operator	Meaning	Example (`a = 17`, `b = 5`)	Result
`+`	Addition	`a + b`	22
`-`	Subtraction	`a - b`	12
`*`	Multiplication	`a * b`	85
`/`	Division (quotient)	`a / b`	**3** (integer division)
`%`	Modulus (remainder)	`a % b`	2

Important behaviour

Expression	Value	Rule
`17 / 5`	3	int ÷ int → int (fraction discarded)
`17 / 5.0`	3.4	at least one real operand → real division
`-17 / 5`	−3	integer division truncates **towards zero**
`-17 % 5`	−2	sign of `%` result = sign of the **dividend** (left operand)
`17 % -5`	2	—
`5 % 17`	5	if dividend < divisor, the remainder is the dividend
`7.5 % 2`	1.5	`%` also works on real numbers in Java
`10 / 0`	—	**ArithmeticException** at run time (`/ by zero`)
`10.0 / 0`	`Infinity`	no exception for floating-point division

> **WARNING**
>
> `double avg = (a + b) / 2;` with `int a, b` gives a **wrong (truncated) average**. Write `(a + b) / 2.0`.

4. Increment and decrement operators

`++` increases the value of a variable by 1; `--` decreases it by 1. They can be used only with **variables** (not constants: `5++` is an error).

Form	Name	Working ("change-then-use" or "use-then-change")
`++a`, `--a`	**Prefix**	First **change** the value, then **use** the new value in the expression
`a++`, `a--`	**Postfix**	First **use** the current value in the expression, then **change** it

```
int a = 5, b;
b = a++;   // b = 5, a = 6   (use, then increment)
b = ++a;   // a = 7, b = 7   (increment, then use)
```

> **IMPORTANT**
>
> **Evaluating an expression with several `++` / `--` :** go strictly **left to right**. Write the value each operand contributes, and update the variable immediately after each operand.

Worked example: `int a = 5; int b = a++ + ++a * a--;`

Operand	Value used	`a` after it
`a++`	5	6
`++a`	7	7
`a--`	7	6

`b = 5 + 7 * 7 = 5 + 49 = 54`, and finally `a = 6`.

5. Relational operators

Relational operators **compare** two values and return a `boolean` result (`true` / `false`).

Operator	Meaning	`5 ? 8`
`<`	less than	`true`
`>`	greater than	`false`
`<=`	less than or equal to	`true`
`>=`	greater than or equal to	`false`
`==`	equal to	`false`
`!=`	not equal to	`true`

> **WARNING**
>
> `=` is **assignment**; `==` is **comparison**. `if (a = 5)` is a compile-time error in Java (an `int` is not a `boolean`). Characters can be compared because they have codes: `'a' > 'B'` is `true` (97 > 66).

6. Logical operators

Logical operators combine two or more **boolean** expressions.

Operator	Name	Result is `true` when…
`&&`	Logical AND	**both** operands are `true`
`||`	Logical OR	**at least one** operand is `true`
`!`	Logical NOT (unary)	the operand is `false`

A	B	A && B	A || B	!A
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true
false	false	false	false	true

Short-circuit evaluation: `&&` does **not evaluate** the right operand if the left one is `false`; `||` does not evaluate the right operand if the left one is `true`.

```
int x = 5;
boolean r = (x > 10) && (++x > 5);   // left is false → ++x never runs
System.out.println(r + " " + x);     // false 5
```

Precedence among logical operators: `!` (highest) → `&&` → `||` (lowest).

7. Assignment and shorthand operators

The **assignment operator** `=` stores the value of the expression on its right in the variable on its left. It is evaluated **right to left**: `a = b = c = 10;` assigns 10 to all three.

Shorthand (compound assignment) operators combine an arithmetic operation with assignment.

Shorthand	Equivalent to	`a = 17` and `b = 5`
`a += b`	`a = a + b`	22
`a -= b`	`a = a - b`	12
`a *= b`	`a = a * b`	85
`a /= b`	`a = a / b`	3
`a %= b`	`a = a % b`	2

> **NOTE**
>
> A shorthand operator contains a **hidden cast** to the type of the left operand: `char ch = 'A'; ch += 2;` is valid (`'C'`), but `ch = ch + 2;` is an error. Also, `a *= b + 2` means `a = a * (b + 2)`, **not** `a * b + 2`.

8. Ternary (conditional) operator

Syntax: `variable = (condition) ? value_if_true : value_if_false;`

```
int max = (a > b) ? a : b;
String res = (n % 2 == 0) ? "Even" : "Odd";
char grade = (m >= 90) ? 'A' : (m >= 75) ? 'B' : 'C';    // nested ternary
```

Converting between ternary and if-else

```
// ternary
tax = (salary > 50000) ? salary * 0.1 : 0;

// equivalent if-else
if (salary > 50000)
```

```
    tax = salary * 0.1;
else
    tax = 0;
```

9. The `new` operator and the dot operator

- **`new` operator:** allocates **memory dynamically** (at run time) for an object or an array and returns a **reference** to it. `java Scanner sc = new Scanner(System.in); int arr[] = new int[10]; Student s1 = new Student();`
- **Dot (`.`) operator:** used to **access the members** (data members and methods) of a class or an object, and to refer to a class inside a package. `java s1.display(); // invoking a method of object s1 System.out.println(); // out is a member of class System Math.sqrt(25); // static method of class Math java.util.Scanner // class Scanner inside package java.util`

10. Precedence and associativity

Precedence decides **which operator is evaluated first** in an expression with more than one operator. **Associativity** decides the **order of evaluation of operators having the same precedence** (left to right or right to left).

Rank	Operators	Description	Associativity
1	`()` `[]` `.`	parentheses, array subscript, member access	Left → Right
2	`a++` `a--`	postfix	Left → Right
3	`++a` `--a` `+a` `-a` `!` `(type)`	prefix, unary, NOT, cast	**Right → Left**
4	`new`	object creation	Right → Left
5	`*` `/` `%`	multiplicative	Left → Right
6	`+` `-`	additive (and string concatenation)	Left → Right
7	`<` `<=` `>` `>=` `instanceof`	relational	Left → Right
8	`==` `!=`	equality	Left → Right
9	`&&`	logical AND	Left → Right
10	`||`	logical OR	Left → Right
11	`?:`	ternary	**Right → Left**
12	`=` `+=` `-=` `*=` `/=` `%=`	assignment	**Right → Left**

> **TIP**
>
> Memory aid: **U**nary → **A**rithmetic (`* / %` before `+ -`) → **R**elational → **E**quality → **L**ogical (`&&` before `||`) → **T**ernary → **A**ssignment.

11. Counters and accumulators

- A **counter** is a variable that **counts** the number of times something happens. It is usually incremented by **1** (or a fixed value). It is initialised to **0** (or 1). `count = count + 1;` or `count++;`
- An **accumulator** is a variable that **collects (accumulates) a running total** of values. It is initialised to **0** for a sum, or **1** for a product. `sum = sum + n;` or `prod = prod * n;`

Basis	Counter	Accumulator
Purpose	Counts occurrences	Adds/multiplies values
Increases by	A fixed amount (usually 1)	A varying amount
Example	`c++;`	`s += n;`

12. Output: `print()` and `println()`

`System.out.print()`	`System.out.println()`
Prints and keeps the cursor **on the same line**	Prints and moves the cursor to the **next line**
Must have an argument	Can be used without an argument (prints a blank line)

The `+` operator with strings: if **either** operand of `+` is a `String`, `+` performs **concatenation**. Evaluation is left to right:

Statement	Output
`System.out.println(2 + 3 + "A");`	`5A`
`System.out.println("A" + 2 + 3);`	`A23`
`System.out.println("A" + (2 + 3));`	`A5`
`System.out.println("Sum=" + 2 * 3);`	`Sum=6` (`*` has higher precedence)
`System.out.println('A' + 'B' + "C");`	`131C`

13. Writing Java expressions

Mathematical expression	Java expression
$ut + \frac{1}{2}at^2$	`u * t + 0.5 * a * t * t` (or `1.0 / 2 * a * t * t`)
$\frac{a+b}{c-d}$	`(a + b) / (c - d)`
$\frac{9}{5}C + 32$	`9.0 / 5 * c + 32`
$\frac{P \times R \times T}{100}$	`p * r * t / 100`
$a^2 + 2ab + b^2$	`a * a + 2 * a * b + b * b`
$\frac{x}{y} + \frac{y}{x}$	`x / y + y / x`

> **WARNING**
>
> `1 / 2 * a * t * t` is **always 0** for `int` arithmetic, because `1 / 2` is 0. Use `0.5` or `1.0 / 2`.

(Expressions with powers and roots using the `Math` class are covered in Chapter 05.)

14. Step-by-step evaluation — solved examples

Q1. `int a = 10, b = 4; a += a++ - --b * 2;`

The shorthand stores `a = a + (a++ - --b * 2)` — the **old value of `a` (10)** is taken first.

- `a++` → 10 (a becomes 11)
- `--b` → 3
- `a++ - --b * 2` = 10 − 3 × 2 = 4
- `a` = 10 + 4 = **14** (the 11 is overwritten), `b` = **3**

Q2. `int x = 7; x = x++ + x++;` → 7 + 8 = 15, so **x = 15**.

Q3. `int p = 3, q = 6; boolean r = p > q || q % p == 0 && !(p == q);`

- `p > q` → false
- `q % p == 0` → true; `!(p == q)` → true; `true && true` → true
- `false || true` → **true**

Q4. `int m = 5; int n = m-- * 2 + --m;` → 5 × 2 + 3 = **13**, m = **3**.

Q5. `double d = 25 / 2 + 25 % 2 + 2.5;` → 12 + 1 + 2.5 = **15.5**.

Q6. `int k = 4; System.out.println(k++ + " " + k + " " + ++k);` → **4 5 6**.

Q7. `int a = 5; boolean r = (a > 2) || (++a > 3); System.out.println(r + " " + a);` → short-circuit: **true 5**.

Q8. `char c = 'D'; c -= 2; int x = c + 1;` → `c` = `'B'`, `x` = **67**.

15. Differentiate between

Pair	Difference
`=` and `==`	Assignment vs comparison for equality
`/` and `%`	Quotient vs remainder
Prefix and postfix	Change-then-use vs use-then-change
Unary and binary	One operand vs two operands
`&&` and `||`	True only if both are true vs true if either is true
Counter and accumulator	Counts by a fixed step vs adds varying values
`print()` and `println()`	Same line vs next line
Precedence and associativity	Which operator first vs order among equal-precedence operators

16. One-line facts

- `%` gives the remainder; its sign follows the **dividend**.
- Integer division by zero throws an **`ArithmeticException`**.
- The **ternary** operator is the only operator with three operands.
- `!` has higher precedence than `&&`, which is higher than `||`.
- Assignment, ternary and unary operators are evaluated **right to left**.
- `new` allocates memory **dynamically** at run time.
- Increment/decrement operators cannot be applied to constants or expressions (`(a + b)++` is invalid).
- `a = a + 1`, `a += 1`, `a++` and `++a` (as standalone statements) all have the same effect.

🧠 Test Yourself

Part A — Multiple choice

1. `int a = 3; a = a++ + a++ * --a;` What is `a`?
   (a) 19 (b) 28 (c) 16 (d) 15
2. Which expression gives `true` for **every** odd integer `n`, including negative odd numbers?
   (a) `n % 2 == 1` (b) `n % 2 != 0` (c) `n / 2 == 1` (d) `n % 2 > 0`
3. `System.out.println(10 + 20 + "30" + 10 * 2);` prints
   (a) 303020 (b) 1020302 (c) 3030 20 (d) 603020
4. *Assertion (A):* `int x = 0; if (x != 0 && 10 / x > 1)` does not throw an exception.
   Reason (R): `&&` evaluates both operands before deciding the result.
   (a) Both true; R explains A (b) Both true; R does not explain A (c) A true, R false (d) A false, R true
5. What is the value of `-15 % 4 + 15 % -4`?
   (a) 0 (b) −6 (c) 6 (d) −2
6. `int p = 2; p *= p++ + 3;` gives `p` =
   (a) 10 (b) 12 (c) 7 (d) 15
7. Which statement is a **compile-time error**? (a) `int a = 5; a++;` (b) `int b = (int) 7.9;` (c) `int c = 5++;` (d) `double d = 5 / 2;`
8. `int a = 6, b = 4; System.out.println(a > b ? a % b == 2 ? "X" : "Y" : "Z");` prints
   (a) X (b) Y (c) Z (d) Error
9. The associativity of `=` is right to left. Hence `int a, b, c; a = b = c = 4;`
   (a) assigns 4 only to `c` (b) assigns 4 to all three (c) is an error (d) assigns 4 to `a` only
10. `int i = 1; i = i++ + i++ + i++;` gives `i` = (a) 3 (b) 6 (c) 4 (d) 9
11. What is printed? `int n = 7; System.out.println(n++ > 7 || n-- == 8); System.out.println(n);`
    (a) true 7 (b) true 8 (c) false 7 (d) true / 7 on separate lines
12. Which operator has the **lowest** precedence? (a) `||` (b) `?:` (c) `+=` (d) `!`

Part B — Short answer questions

1. Evaluate, given `int a = 8, b = 3;`: (i) `a / b * b + a % b` (ii) `a++ * 2 - --b`
2. Rewrite using the ternary operator: `if (x % 2 == 0) s = "Even"; else s = "Odd";`
3. Rewrite using if-else: `int d = (a > b) ? a - b : b - a;`
4. What is short-circuit evaluation? Give an example.
5. Write the Java expression (without using `Math` methods) for $\frac{a^3+b^3}{a^2-ab+b^2}$.
6. Give the output: `int x = 5, y = 2; System.out.println(x / y + " " + x % y + " " + (double) x / y);`
7. What are counters and accumulators? Write one statement of each.
8. State the use of the `new` operator and the dot operator with one example each.
9. Find the value of `k` after: `int k = 10; k -= k-- - --k;`
10. Why does `double avg = (m1 + m2 + m3) / 3;` sometimes give a wrong answer? Correct it.

✅ Answers — Part A

Q	Ans	Working
1	(a)	`a++` → 3 (a=4), `a++` → 4 (a=5), `--a` → 4; `*` before `+`: 3 + (4 × 4) = **19**. Option (b) is the trap of ignoring precedence: (3 + 4) × 4 = 28
2	(b)	For negative odd `n`, `n % 2` is −1

Q	Ans	Working
3	(a)	10 + 20 = 30 → `"3030"` → `10 * 2` = 20 is evaluated first (higher precedence) → `"303020"`
4	(c)	`&&` short-circuits, so R is false
5	(a)	−15 % 4 = −3; 15 % −4 = 3; total 0
6	(a)	`p = p * (p++ + 3)` = 2 × (2 + 3) = 10
7	(c)	Increment cannot be applied to a literal
8	(a)	6 > 4 → `6 % 4 == 2` → true → "X"
9	(b)	—
10	(b)	1 + 2 + 3 = 6
11	(d)	`n++ > 7` → 7 > 7 false, n = 8; `n-- == 8` → true, n = 7. Prints `true` then `7`
12	(c)	Assignment operators have the lowest precedence

✅ Answers — Part B

1. (i) 8 / 3 = 2, 2 × 3 = 6, 8 % 3 = 2 → **8** (ii) 8 × 2 − 2 = **14** (a becomes 9, b becomes 2)
2. `s = (x % 2 == 0) ? "Even" : "Odd";`
3. `if (a > b) d = a - b; else d = b - a;`
4. The right operand of `&&` / `||` is not evaluated when the result is already decided by the left operand. E.g. `(n != 0) && (100 / n > 2)` avoids division by zero.
5. `(a * a * a + b * b * b) / (a * a - a * b + b * b)`
6. `2 1 2.5`
7. A counter counts occurrences (`count++;`); an accumulator collects a running total (`sum += n;`).
8. `new` creates an object: `Scanner sc = new Scanner(System.in);` . Dot accesses a member: `sc.nextInt()` .
9. `k = k - (k-- - --k)` = 10 − (10 − 8) = **8**.
10. `int` ÷ `int` truncates the fraction. Correct: `double avg = (m1 + m2 + m3) / 3.0;`

💻 Programs in this chapter

No.	Program
Q14	Area and Perimeter of a Rectangle
Q15	Area and Circumference of a Circle
Q16	Temperature Conversion: Celsius and Fahrenheit
Q17	Simple Interest and Amount
Q18	Convert Seconds into Hours, Minutes and Seconds
Q19	Convert Days into Years, Months and Days
Q20	Prefix and Postfix Increment and Decrement
Q21	Larger of Two Numbers using the Ternary Operator
Q22	Shorthand (Compound Assignment) Operators
Q23	Total, Average and Percentage of Five Subjects

Programs Q14 – Q23

Q14 Area and Perimeter of a Rectangle

Write a program to accept the length and breadth of a rectangle and calculate and display its area and perimeter.

`Area = length x breadth` `Perimeter = 2 x (length + breadth)`

If either dimension is zero or negative, display "Invalid dimensions".

Q_14.java

```java
import java.util.Scanner;

public class Q_14
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the length: ");
        double l = sc.nextDouble();
        System.out.print("Enter the breadth: ");
        double b = sc.nextDouble();

        if (l <= 0 || b <= 0)
        {
            System.out.println("Invalid dimensions");
        }
        else
        {
            double area = l * b;
            double peri = 2 * (l + b);
            System.out.println("Area      = " + area);
            System.out.println("Perimeter = " + peri);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
l	double	to store the length
b	double	to store the breadth
area	double	to store the area
peri	double	to store the perimeter

Q15 Area and Circumference of a Circle

Write a program to accept the radius of a circle and calculate its area and circumference, taking the value of pi as 3.14159 stored in a constant.

`Area = pi x r x r` `Circumference = 2 x pi x r`

If the radius is zero or negative, display "Invalid radius".

Q_15.java

```java
import java.util.Scanner;

public class Q_15
```

```
{
    public static void main(String[] args)
    {
        final double PI = 3.14159;
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the radius: ");
        double r = sc.nextDouble();

        if (r <= 0)
        {
            System.out.println("Invalid radius");
        }
        else
        {
            double area = PI * r * r;
            double circum = 2 * PI * r;
            System.out.println("Area          = " + area);
            System.out.println("Circumference = " + circum);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
PI	double	to store the constant value of pi
r	double	to store the radius
area	double	to store the area
circum	double	to store the circumference

Q16 Temperature Conversion: Celsius and Fahrenheit

Write a program to accept a temperature in degree Celsius and convert it into degree Fahrenheit. Also accept a temperature in degree Fahrenheit and convert it into degree Celsius.

`F = (9 / 5) x C + 32` `C = (5 / 9) x (F - 32)`

Make sure the formula is written so that integer division does not spoil the result.

Q_16.java

```
import java.util.Scanner;

public class Q_16
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter temperature in Celsius: ");
        double c = sc.nextDouble();
        double f = 9.0 / 5 * c + 32;
        System.out.println(c + " C = " + f + " F");

        System.out.print("Enter temperature in Fahrenheit: ");
        double fah = sc.nextDouble();
        double cel = 5.0 / 9 * (fah - 32);
        System.out.println(fah + " F = " + cel + " C");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
c	double	to store the temperature in celsius
f	double	to store the converted fahrenheit value
fah	double	to store the temperature in fahrenheit
cel	double	to store the converted celsius value

Q17 Simple Interest and Amount

Write a program to accept the principal, the rate of interest (per annum) and the time (in years), and calculate the simple interest and the amount.

`Simple Interest = (P x R x T) / 100` `Amount = P + SI`

Display "Invalid input" if any value is negative.

Q_17.java

```java
import java.util.Scanner;

public class Q_17
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the principal: ");
        double p = sc.nextDouble();
        System.out.print("Enter the rate of interest: ");
        double r = sc.nextDouble();
        System.out.print("Enter the time in years: ");
        double t = sc.nextDouble();

        if (p < 0 || r < 0 || t < 0)
        {
            System.out.println("Invalid input");
        }
        else
        {
            double si = p * r * t / 100;
            double amt = p + si;
            System.out.println("Simple Interest = " + si);
            System.out.println("Amount          = " + amt);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
p	double	to store the principal
r	double	to store the rate of interest
t	double	to store the time in years
si	double	to store the simple interest
amt	double	to store the amount

Q18 Convert Seconds into Hours, Minutes and Seconds

Write a program to accept a time in seconds and convert it into hours, minutes and seconds using the / and % operators.

Example: Input: 7384 Output: 2 hours 3 minutes 4 seconds

Display "Invalid input" for a negative value.

Q_18.java

```java
import java.util.Scanner;

public class Q_18
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter time in seconds: ");
        long total = sc.nextLong();

        if (total < 0)
        {
            System.out.println("Invalid input");
        }
        else
        {
            long h = total / 3600;
            long m = (total % 3600) / 60;
            long s = total % 60;
            System.out.println(h + " hours " + m + " minutes " + s + " seconds");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
total	long	to store the time in seconds
h	long	to store the hours
m	long	to store the remaining minutes
s	long	to store the remaining seconds

Q19 Convert Days into Years, Months and Days

Write a program to accept a number of days and convert it into years, months and days, assuming 1 year = 365 days and 1 month = 30 days.

Example: Input: 1005 Output: 2 years 9 months 5 days

Display "Invalid input" for a negative value.

Q_19.java

```java
import java.util.Scanner;

public class Q_19
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of days: ");
        int days = sc.nextInt();
```

```
        if (days < 0)
        {
            System.out.println("Invalid input");
        }
        else
        {
            int y = days / 365;
            int rem = days % 365;
            int m = rem / 30;
            int d = rem % 30;
            System.out.println(y + " years " + m + " months " + d + " days");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
days	int	to store the number of days
y	int	to store the number of years
rem	int	to store the days left after years
m	int	to store the number of months
d	int	to store the remaining days

Q20 Prefix and Postfix Increment and Decrement

Write a program to show the difference between the prefix and postfix forms of the increment (++) and decrement (--) operators. Starting with a = 5 each time, find and display the values of a and b after each of these statements:

```
(a) b = a++;     (b) b = ++a;     (c) b = a--;     (d) b = --a;
(e) b = a++ + ++a;               (f) b = a-- - --a + a++;
```

Q_20.java

```
public class Q_20
{
    public static void main(String[] args)
    {
        int a, b;

        a = 5;
        b = a++;
        System.out.println("(a) b = a++          : a = " + a + ", b = " + b);

        a = 5;
        b = ++a;
        System.out.println("(b) b = ++a          : a = " + a + ", b = " + b);

        a = 5;
        b = a--;
        System.out.println("(c) b = a--          : a = " + a + ", b = " + b);

        a = 5;
        b = --a;
        System.out.println("(d) b = --a          : a = " + a + ", b = " + b);

        a = 5;
        b = a++ + ++a;
        System.out.println("(e) b = a++ + ++a    : a = " + a + ", b = " + b);

        a = 5;
```

```
        b = a-- - --a + a++;
        System.out.println("(f) b = a-- - --a + a++ : a = " + a + ", b = " + b);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	int	to store the operand value
b	int	to store the result of each statement

Q21 Larger of Two Numbers using the Ternary Operator

Write a program to accept two integers and display the larger of the two using the ternary (conditional) operator. If both are equal, display "Both are equal" (use a nested ternary operator).

Q_21.java

```
import java.util.Scanner;

public class Q_21
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter two integers: ");
        int a = sc.nextInt();
        int b = sc.nextInt();

        String result = (a == b) ? "Both are equal"
                      : (a > b) ? "Larger number = " + a
                      : "Larger number = " + b;
        System.out.println(result);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	int	to store the first number
b	int	to store the second number
result	String	to store the message to display

Q22 Shorthand (Compound Assignment) Operators

Write a program to accept two integers a and b. Using a fresh copy of a each time, apply the shorthand operators +=, -=, *=, /= and %= with b and display the results. If b is zero, display "Not possible" for /= and %=.

Q_22.java

```
import java.util.Scanner;

public class Q_22
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a and b: ");
        int a = sc.nextInt();
        int b = sc.nextInt();
        int x;
```

```
        x = a;
        x += b;
        System.out.println("a += b gives " + x);

        x = a;
        x -= b;
        System.out.println("a -= b gives " + x);

        x = a;
        x *= b;
        System.out.println("a *= b gives " + x);

        if (b != 0)
        {
            x = a;
            x /= b;
            System.out.println("a /= b gives " + x);
            x = a;
            x %= b;
            System.out.println("a %= b gives " + x);
        }
        else
        {
            System.out.println("a /= b : Not possible");
            System.out.println("a %= b : Not possible");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	int	to store the first number
b	int	to store the second number
x	int	to store a fresh copy of a

Q23 Total, Average and Percentage of Five Subjects

> Write a program to accept the marks obtained by a student in five subjects (each out of 100) and calculate the total marks, the average marks and the percentage. Display "Invalid marks" if any mark is outside the range 0 to 100.

Q_23.java

```
import java.util.Scanner;

public class Q_23
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter marks in five subjects:");
        int m1 = sc.nextInt();
        int m2 = sc.nextInt();
        int m3 = sc.nextInt();
        int m4 = sc.nextInt();
        int m5 = sc.nextInt();

        boolean valid = m1 >= 0 && m1 <= 100 && m2 >= 0 && m2 <= 100
                    && m3 >= 0 && m3 <= 100 && m4 >= 0 && m4 <= 100
                    && m5 >= 0 && m5 <= 100;
        if (!valid)
```

```
        {
            System.out.println("Invalid marks");
        }
        else
        {
            int total = m1 + m2 + m3 + m4 + m5;
            double avg = total / 5.0;
            double per = total * 100.0 / 500;
            System.out.println("Total      = " + total);
            System.out.println("Average    = " + avg);
            System.out.println("Percentage = " + per + "%");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
m1	int	to store marks in subject 1
m2	int	to store marks in subject 2
m3	int	to store marks in subject 3
m4	int	to store marks in subject 4
m5	int	to store marks in subject 5
valid	boolean	to check that all marks are valid
total	int	to store the total marks
avg	double	to store the average marks
per	double	to store the percentage

CHAPTER

04



Input in Java

Initialisation · Parameters · Packages · Scanner class · Types of errors · Types of comments

8 programs

Q24 – Q31

Theory

📑 Contents

1. Three ways of giving input
2. Packages and the `import` statement
3. The Scanner class
4. The `nextLine()` trap
5. Types of errors
6. Types of comments
7. Find the error
8. Differentiate between
9. One-line facts
10. Test yourself
11. Programs in this chapter

1. Three ways of giving input

Method	When the data is given	Example
Initialisation (assignment)	**Before execution** — the values are written in the program itself	`int a = 5, b = 10;`
Parameters (arguments)	**At the time of execution** — values are passed to a method when it is called	`velocity(10, 2.5, 4);` → `void velocity(double u, double a, double t)`
Input stream (Scanner)	**During execution** — the user types the data while the program is running	`int n = sc.nextInt();`

- **Initialisation** is simple, but the program always works on the same data; to change the data, the program must be edited and recompiled.
- **Parameters** allow the same method to work on different values each time it is called. (In BlueJ the values are typed into a dialog box when the method is invoked.)
- An **input stream** is a flow (sequence) of data from an input device (keyboard) to the program. `System.in` represents the **standard input stream** (keyboard).

2. Packages and the `import` statement

A **package** is a **collection (group) of related classes and interfaces**. It is like a folder that organises the Java library.

Package	Contains	Import needed?
`java.lang`	`Math`, `String`, `System`, wrapper classes (`Integer`, `Character`, ...)	**No** — imported **by default**
`java.util`	`Scanner` and other utility classes	Yes
`java.io`	Input/output classes (`BufferedReader`, `InputStreamReader`, ...)	Yes
`java.awt`, `java.applet`	Graphics / applet classes	Yes

The **`import` statement** makes the classes of a package available to a program. It is written at the **beginning** of the program, before the class.

```
import java.util.Scanner;   // imports only the Scanner class
import java.util.*;         // imports all classes of java.util (* = wildcard)
```

> **NOTE**
>
> `import` does **not** copy code into the program — it only tells the compiler where to find the class. `import java.util.*;` does **not** import sub-packages.

3. The Scanner class

The **Scanner** class of the `java.util` package is used to accept input from the keyboard (and other sources). It breaks the input into **tokens** using **whitespace** (space, tab, new line) as the **delimiter**.

Creating a Scanner object

```
Scanner sc = new Scanner(System.in);
```

Method	Return type	Reads
`nextShort()`	`short`	a short integer
`nextInt()`	`int`	an integer
`nextLong()`	`long`	a long integer
`nextFloat()`	`float`	a real number (float)
`nextDouble()`	`double`	a real number (double)
`nextBoolean()`	`boolean`	`true` / `false`
`next()`	`String`	**one word** (up to the next whitespace)
`nextLine()`	`String`	**the rest of the current line**, including spaces
`next().charAt(0)`	`char`	the first character of the next word

> **IMPORTANT**
>
> Scanner has **no `nextChar()`** method. A single character is read as `sc.next().charAt(0)`.

Example

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter marks: ");
int m = sc.nextInt();
```

If the user types `abc` instead of a number, the program stops with an **`InputMismatchException`** (a runtime error).

4. The `nextLine()` trap

`nextInt()`, `nextDouble()`, `next()` etc. read a token but **leave the new-line character (`\n`)** of the Enter key in the input buffer. A following `nextLine()` reads that leftover `\n` and returns an **empty string** at once.

```
int age = sc.nextInt();          // user types 15⏎  → "15" read, "\n" left behind
String name = sc.nextLine();     // ❌ reads the leftover "\n" → name = ""
```

Fix: add an extra `sc.nextLine();` to consume the leftover new line.

```java
int age = sc.nextInt();
sc.nextLine();                  // consumes the leftover "\n"
String name = sc.nextLine();    // ✅ now reads the full name
```

Basis	`next()`	`nextLine()`
Reads	One word	The whole line
Stops at	Whitespace	End of line (new line)
Spaces in input	Not included	Included
Leading whitespace	Skipped	Kept
Input `" Sachin Tendulkar"` gives	`"Sachin"`	`" Sachin Tendulkar"`

5. Types of errors

An **error** is a mistake in a program that either stops it from compiling/running or makes it give wrong results. A mistake is also called a **bug**, and removing it is **debugging**.

5.1 Syntax error (compile-time error)

Occurs when the **rules (grammar) of the language are violated**. It is detected by the **compiler**, and the program does not compile until it is corrected.

```java
int a = 5                   // missing semicolon
System.out.printn("Hi");    // misspelt method name
int x = 3.5;                // incompatible types (lossy conversion)
Int n = 10;                 // Int instead of int (case sensitive)
```

5.2 Runtime error

Occurs **during execution** when the program asks the computer to do something **impossible**. The program compiles successfully but stops abruptly (**abnormal termination**) with an **exception**.

```java
int a = 10 / 0;             // ArithmeticException: / by zero
int arr[] = new int[5];
arr[5] = 1;                 // ArrayIndexOutOfBoundsException
int n = sc.nextInt();       // user enters "abc" → InputMismatchException
"Java".charAt(10);          // StringIndexOutOfBoundsException
Integer.parseInt("12a");    // NumberFormatException
```

5.3 Logical error

Occurs when the program compiles and runs **without stopping** but gives **wrong output** because of a **mistake in the logic**. It is the **hardest** to detect, as neither the compiler nor the JVM reports it.

```java
double avg = (a + b + c) / 3;       // integer division — wrong average
double area = 2 * 3.14 * r;         // circumference formula used for area
if (n % 2 == 1) ... "Odd"           // wrong for negative odd numbers
```

Basis	Syntax error	Runtime error	Logical error
Cause	Grammar rules broken	Impossible operation during execution	Wrong logic
Detected by	Compiler	JVM, during execution	Programmer, by testing

Basis	Syntax error	Runtime error	Logical error
Program compiles?	No	Yes	Yes
Program runs?	No	Stops abruptly	Runs, wrong output
Example	Missing `;`	Division by zero	Wrong formula

6. Types of comments

Comments are non-executable remarks written in a program to **explain** it. They are **ignored by the compiler**.

Type	Syntax	Use
Single-line comment	`// text`	A short remark up to the end of the line
Multi-line comment	`/* text over several lines */`	A longer explanation
Documentation comment	`/** text */`	Used by the `javadoc` tool to produce documentation

```
/* Program to find the area of a circle
   Written for ICSE practice */
double area = 3.14 * r * r;   // area = pi r squared
```

> **WARNING**
>
> Multi-line comments **cannot be nested**: `/* outer /* inner */ still outer? */` — the first `*/` ends the comment and the rest causes a syntax error.

7. Find the error

```
import java.util.scanner;
class Input
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner();
        System.out.print("Enter radius")
        double r = sc.nextdouble();
        char c = sc.nextChar();
        System.out.println("Area = " + 3.14 * r * r);
    }
}
```

Show corrected code

```
import java.util.Scanner;                   // class name Scanner (capital S)
class Input
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in); // System.in must be passed
        System.out.print("Enter radius");    // missing semicolon
        double r = sc.nextDouble();          // nextDouble (capital D)
        char c = sc.next().charAt(0);        // there is no nextChar()
        System.out.println("Area = " + 3.14 * r * r);
```

```
    }
}
```

8. Differentiate between

Pair	Difference
Initialisation vs Scanner input	Data fixed before execution vs typed by the user during execution
Parameter vs Scanner input	Values passed when the method is called vs values typed during execution
`next()` vs `nextLine()`	One word vs whole line
Syntax vs logical error	Detected by compiler vs not detected; gives wrong output
Runtime vs logical error	Program stops abruptly vs program runs with wrong output
`//` vs `/* */`	Single line vs multiple lines
`java.lang` vs `java.util`	Imported by default vs must be imported

9. One-line facts

- `Scanner` belongs to the package **`java.util`**.
- The package imported by default is **`java.lang`**.
- `System.in` refers to the **standard input device (keyboard)**.
- Scanner uses **whitespace** as its default delimiter.
- A character is read using **`sc.next().charAt(0)`**.
- Entering a word when `nextInt()` expects a number causes **`InputMismatchException`**.
- **Logical errors** are the most difficult to find.
- `*` in `import java.util.*;` is called the **wildcard** character.
- Documentation comments begin with `/**`.

🧠 Test Yourself

Part A — Multiple choice

1. Input typed: `25⏎Ravi Kumar⏎`. Code: `int a = sc.nextInt(); String s = sc.nextLine();` What is stored in `s`?
   (a) `"Ravi Kumar"` (b) `"Ravi"` (c) `""` (empty string) (d) InputMismatchException
2. Input typed: `hello world⏎`. Which call returns `"hello"`?
   (a) `nextLine()` (b) `next()` (c) `nextLine().trim()` (d) `next().charAt(0)`
3. *Assertion (A):* `Math.sqrt()` can be used without any `import` statement.
   Reason (R): The `Math` class belongs to the `java.lang` package, which is imported automatically.
   (a) Both true; R explains A (b) Both true; R doesn't explain A (c) A true, R false (d) A false, R true
4. Which error is produced by `int n = sc.nextInt();` when the user types `4.5`?
   (a) Syntax error (b) Logical error (c) InputMismatchException (runtime) (d) It stores 4
5. A program prints the circumference when asked for area. The error is
   (a) Syntax (b) Runtime (c) Logical (d) Compile-time

6. Which of these compiles?
   (a) `char c = sc.nextChar();` (b) `String s = sc.next().charAt(0);` (c) `char c = sc.next().charAt(0);` (d) `int c = sc.nextLine();`
7. Input typed on one line: `10 20 30⏎`. Code: `int a = sc.nextInt(); int b = sc.nextInt(); int c = sc.nextInt();`
   (a) Error — each value must be on a new line (b) a=10, b=20, c=30 (c) a=10, b and c wait for more input (d) a=102030
8. Which is **not** a runtime error? (a) `10 / 0` (b) missing semicolon (c) `arr[10]` for a 10-element array (d) `"abc".charAt(5)`
9. What does `import java.util.*;` import?
   (a) All classes of `java.util` and its sub-packages (b) All classes of `java.util` only (c) Only `Scanner` (d) Nothing — it is invalid
10. *Assertion (A):* Logical errors are the hardest to detect.
    Reason (R): Neither the compiler nor the JVM reports them.
    (a) Both true; R explains A (b) Both true; R doesn't explain A (c) A true, R false (d) A false, R true

Part B — Short answer questions

1. Differentiate between initialisation and input through parameters.
2. Name the package that contains the Scanner class. Write the statement to import it.
3. Write statements to accept: (i) a character (ii) a long integer (iii) a full sentence.
4. Why does `nextLine()` sometimes return an empty string? How is this fixed?
5. Classify the errors: (i) `int x = "5";` (ii) `int y = 5 / 0;` (iii) `area = l + b;` for a rectangle.
6. State the three types of comments in Java with their syntax.
7. What is a package? Name two packages other than `java.util`.
8. What is debugging?

✅ Answers — Part A

Q	Ans	Reason
1	(c)	`nextInt()` leaves the `\n`; `nextLine()` reads it immediately
2	(b)	`next()` skips leading spaces and reads one word
3	(a)	—
4	(c)	`4.5` is not an int token
5	(c)	—
6	(c)	—
7	(b)	Whitespace (including spaces) separates tokens
8	(b)	A missing semicolon is a syntax error
9	(b)	Sub-packages are not imported by `*`
10	(a)	—

✅ Answers — Part B

1. In initialisation, values are written in the program before execution and remain the same for every run; in parameter input, values are supplied at the time of execution when the method is called, so they can change on every call.

2. `java.util` ; `import java.util.Scanner;`
3. (i) `char ch = sc.next().charAt(0);` (ii) `long n = sc.nextLong();` (iii) `String s = sc.nextLine();`
4. Because a previous `nextInt()` / `next()` etc. leaves the new-line character in the buffer, and `nextLine()` reads it. Fix: write an extra `sc.nextLine();` before reading the line.
5. (i) Syntax (incompatible types) (ii) Runtime (ArithmeticException) (iii) Logical
6. Single-line `// …` , multi-line `/* … */` , documentation `/** … */` .
7. A package is a group of related classes and interfaces. Examples: `java.lang` , `java.io` .
8. The process of finding and removing errors (bugs) from a program.

💻 Programs in this chapter

No.	Program
Q24	Input using Every Scanner Method
Q25	Volume and Surface Area of a Cuboid
Q26	Final Velocity: Input through Parameters
Q27	Distance Travelled: Input by Initialisation
Q28	Reading a Full Name with nextLine() after nextInt()
Q29	Bill Amount after a 10% Discount
Q30	Distance Conversion: Kilometres to Metres, Centimetres and Miles
Q31	Gross and Net Salary of an Employee

Programs Q24 – Q31

Q24 Input using Every Scanner Method

Write a program to accept the following data from the user using the appropriate methods of the Scanner class and display them:

```
(a) a short integer     - nextShort()
(b) an integer          - nextInt()
(c) a long integer      - nextLong()
(d) a float value       - nextFloat()
(e) a double value      - nextDouble()
(f) a single word       - next()
(g) a single character  - next().charAt(0)
(h) a complete sentence - nextLine()
```

Take care that the sentence is read correctly after the other inputs.

Q_24.java

```java
import java.util.Scanner;

public class Q_24
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a short integer: ");
        short s = sc.nextShort();
        System.out.print("Enter an integer: ");
        int i = sc.nextInt();
        System.out.print("Enter a long integer: ");
        long l = sc.nextLong();
        System.out.print("Enter a float value: ");
        float f = sc.nextFloat();
        System.out.print("Enter a double value: ");
        double d = sc.nextDouble();
        System.out.print("Enter a word: ");
        String word = sc.next();
        System.out.print("Enter a character: ");
        char ch = sc.next().charAt(0);
        sc.nextLine();
        System.out.print("Enter a sentence: ");
        String line = sc.nextLine();

        System.out.println();
        System.out.println("short     : " + s);
        System.out.println("int       : " + i);
        System.out.println("long      : " + l);
        System.out.println("float     : " + f);
        System.out.println("double    : " + d);
        System.out.println("word      : " + word);
        System.out.println("character : " + ch);
        System.out.println("sentence  : " + line);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	short	to store the short integer
i	int	to store the integer

Name of the Variable	Data Type	Purpose / Description
l	long	to store the long integer
f	float	to store the float value
d	double	to store the double value
word	String	to store a single word
ch	char	to store a single character
line	String	to store a complete sentence

Q25 Volume and Surface Area of a Cuboid

Write a program to accept the length, breadth and height of a cuboid and calculate its volume and total surface area.

`Volume = l x b x h` `Total Surface Area = 2 (lb + bh + hl)`

If any dimension is zero or negative, display "Invalid dimensions".

Q_25.java

```java
import java.util.Scanner;

public class Q_25
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter length, breadth and height: ");
        double l = sc.nextDouble();
        double b = sc.nextDouble();
        double h = sc.nextDouble();

        if (l <= 0 || b <= 0 || h <= 0)
        {
            System.out.println("Invalid dimensions");
        }
        else
        {
            double vol = l * b * h;
            double tsa = 2 * (l * b + b * h + h * l);
            System.out.println("Volume              = " + vol);
            System.out.println("Total surface area = " + tsa);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
l	double	to store the length
b	double	to store the breadth
h	double	to store the height
vol	double	to store the volume
tsa	double	to store the total surface area

Q26 Final Velocity: Input through Parameters

Write a program to calculate the final velocity of a body using the formula v = u + at, taking the input through parameters. Define a method void velocity(double u, double a, double t) that calculates and displays v. Call the method from main() for the following two sets of values:

```
u = 10 m/s, a = 2.5 m/s^2, t = 4 s
u = 0 m/s,  a = 9.8 m/s^2, t = 5 s
```

Q_26.java

```java
public class Q_26
{
    static void velocity(double u, double a, double t)
    {
        double v = u + a * t;
        System.out.println("u = " + u + ", a = " + a + ", t = " + t + ", v = " + v + " m/s");
    }

    public static void main(String[] args)
    {
        velocity(10, 2.5, 4);
        velocity(0, 9.8, 5);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
u	double	to receive the initial velocity
a	double	to receive the acceleration
t	double	to receive the time
v	double	to store the final velocity

Q27 Distance Travelled: Input by Initialisation

Write a program to calculate the distance travelled by a body using the formula s = ut + (1/2)at^2, where the values u = 20 m/s, a = 4 m/s^2 and t = 5 s are given to the variables by initialisation (assignment) inside the program.

Q_27.java

```java
public class Q_27
{
    public static void main(String[] args)
    {
        double u = 20;
        double a = 4;
        double t = 5;
        double s = u * t + 0.5 * a * t * t;
        System.out.println("Distance travelled = " + s + " m");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
u	double	to store the initial velocity
a	double	to store the acceleration
t	double	to store the time

Name of the Variable	Data Type	Purpose / Description
s	double	to store the distance travelled

Q28 Reading a Full Name with nextLine() after nextInt()

Write a program to accept the age of a person (an integer) followed by the person's full name (which may contain spaces) and display them. Show how the leftover new-line character after nextInt() is removed so that nextLine() reads the name correctly. Remove any extra spaces at the beginning and end of the name. If the age is negative, display "Invalid age".

Q_28.java

```java
import java.util.Scanner;

public class Q_28
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your age: ");
        int age = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter your full name: ");
        String name = sc.nextLine().trim();

        if (age < 0)
        {
            System.out.println("Invalid age");
        }
        else
        {
            System.out.println("Name : " + name);
            System.out.println("Age  : " + age + " years");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
age	int	to store the age
name	String	to store the full name

Q29 Bill Amount after a 10% Discount

A shop gives a discount of 10% on the total cost of an item. Write a program to accept the price of one item and the quantity purchased, and calculate and display the total cost, the discount and the amount to be paid. Display "Invalid input" if the price or the quantity is negative.

Q_29.java

```java
import java.util.Scanner;

public class Q_29
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the price of one item: ");
        double price = sc.nextDouble();
        System.out.print("Enter the quantity: ");
```

```
        int qty = sc.nextInt();

        if (price < 0 || qty < 0)
        {
            System.out.println("Invalid input");
        }
        else
        {
            double cost = price * qty;
            double disc = cost * 10 / 100;
            double pay = cost - disc;
            System.out.println("Total cost     = " + cost);
            System.out.println("Discount (10%) = " + disc);
            System.out.println("Amount to pay  = " + pay);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
price	double	to store the price of one item
qty	int	to store the quantity
cost	double	to store the total cost
disc	double	to store the discount
pay	double	to store the amount to be paid

Q30 Distance Conversion: Kilometres to Metres, Centimetres and Miles

> Write a program to accept a distance in kilometres and convert it into metres, centimetres and miles (1 mile = 1.609344 km). Display "Invalid distance" for a negative value.

Q_30.java

```
import java.util.Scanner;

public class Q_30
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the distance in km: ");
        double km = sc.nextDouble();

        if (km < 0)
        {
            System.out.println("Invalid distance");
        }
        else
        {
            double m = km * 1000;
            double cm = m * 100;
            double miles = km / 1.609344;
            System.out.println("Metres      = " + m);
            System.out.println("Centimetres = " + cm);
            System.out.println("Miles       = " + miles);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
km	double	to store the distance in kilometres
m	double	to store the distance in metres
cm	double	to store the distance in centimetres
miles	double	to store the distance in miles

Q31 Gross and Net Salary of an Employee

Write a program to accept the basic salary of an employee and calculate the gross salary and the net salary using the following:

```
Dearness Allowance (DA)   = 30% of basic
House Rent Allowance (HRA) = 15% of basic
Provident Fund (PF)        = 12.5% of basic
Gross salary = basic + DA + HRA
Net salary   = gross salary - PF
```

Display "Invalid basic salary" if the basic salary is negative.

Q_31.java

```java
import java.util.Scanner;

public class Q_31
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the basic salary: ");
        double basic = sc.nextDouble();

        if (basic < 0)
        {
            System.out.println("Invalid basic salary");
        }
        else
        {
            double da = 30.0 / 100 * basic;
            double hra = 15.0 / 100 * basic;
            double pf = 12.5 / 100 * basic;
            double gross = basic + da + hra;
            double net = gross - pf;
            System.out.println("DA           = " + da);
            System.out.println("HRA          = " + hra);
            System.out.println("PF           = " + pf);
            System.out.println("Gross salary = " + gross);
            System.out.println("Net salary   = " + net);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
basic	double	to store the basic salary
da	double	to store the dearness allowance
hra	double	to store the house rent allowance
pf	double	to store the provident fund

Name of the Variable	Data Type	Purpose / Description
`gross`	`double`	to store the gross salary
`net`	`double`	to store the net salary

CHAPTER

05



Mathematical Library Methods

The `java.lang` package · Methods of the `Math` class · Java expressions using `Math` methods

14 programs | Q32 – Q45 | 1 extra

Theory

📑 Contents

1. Library methods and the `java.lang` package
2. Methods of the `Math` class
3. Behaviour with negative numbers — `ceil`, `floor`, `round`
4. Return types — the most common exam trap
5. Generating random numbers in a range
6. Writing Java expressions using `Math` methods
7. Predict the output — solved
8. Find the error
9. One-line facts
10. Test yourself
11. Programs in this chapter

1. Library methods and the `java.lang` package

Library methods (built-in methods) are **pre-written methods** provided in the Java class library, which can be used directly in a program without writing their code.

The `Math` class belongs to the `java.lang` package, which is **imported by default** into every Java program. All its methods are `static`, so they are called using the **class name** and the dot operator — no object is needed:

```
double r = Math.sqrt(49);    // ClassName.methodName(arguments)
```

2. Methods of the `Math` class

Method	Purpose	Return type	Example	Result
`Math.pow(x, y)`	x^y	`double`	`Math.pow(2, 5)`	`32.0`
`Math.sqrt(x)`	square root of x	`double`	`Math.sqrt(81)`	`9.0`
`Math.cbrt(x)`	cube root of x	`double`	`Math.cbrt(-27)`	`-3.0`
`Math.ceil(x)`	smallest whole number **≥ x** (rounds **up**)	`double`	`Math.ceil(4.1)`	`5.0`
`Math.floor(x)`	largest whole number **≤ x** (rounds **down**)	`double`	`Math.floor(4.9)`	`4.0`
`Math.round(x)`	nearest whole number; `.5` rounds **up** (towards +∞)	`long` (for double) / `int` (for float)	`Math.round(4.5)`	`5`
`Math.abs(a)`	absolute (positive) value	same as argument	`Math.abs(-7)`	`7`
`Math.max(a, b)`	larger of two	type of the wider argument	`Math.max(3, 8.0)`	`8.0`
`Math.min(a, b)`	smaller of two	type of the wider argument	`Math.min(-2, 5)`	`-2`
`Math.random()`	random number **≥ 0.0 and < 1.0**	`double`	`Math.random()`	e.g. `0.7281…`

> **IMPORTANT**
>
> `Math.round(x)` works as **`Math.floor(x + 0.5)`**. That is why `Math.round(-2.5)` is **−2** (not −3) — `.5` always goes **up**, towards positive infinity.

Special results

Expression	Result	Reason
`Math.sqrt(-4)`	`NaN`	"Not a Number" — no real square root
`Math.cbrt(-8)`	`-2.0`	cube roots of negatives exist
`Math.pow(0, 0)`	`1.0`	defined as 1
`Math.pow(4, 0.5)`	`2.0`	same as square root
`Math.pow(-8, 1.0/3)`	`NaN`	use `cbrt` for negative cube roots
`Math.abs(-0.0)`	`0.0`	—

3. Behaviour with negative numbers

x	`Math.ceil(x)`	`Math.floor(x)`	`Math.round(x)`	`(int) x`
7.2	8.0	7.0	7	7
7.5	8.0	7.0	8	7
7.8	8.0	7.0	8	7
−7.2	−7.0	−8.0	−7	−7
−7.5	−7.0	−8.0	**−7**	−7
−7.8	−7.0	−8.0	−8	−7
−0.4	**−0.0**	−1.0	0	0
9.0	9.0	9.0	9	9

> **TIP**
>
> On a number line: **ceil moves right**, **floor moves left**, **casting moves towards zero**, **round** goes to the nearest integer (ties go right).

4. Return types — the most common exam trap

Statement	Valid?	Why
`int r = Math.sqrt(25);`	❌	`sqrt` returns `double`
`int r = (int) Math.sqrt(25);`	✅	explicit cast → 5
`int p = Math.pow(2, 3);`	❌	`pow` returns `double`
`long n = Math.round(4.6);`	✅	`round(double)` returns `long`
`int n = Math.round(4.6);`	❌	`long` cannot go into `int` without a cast

Statement	Valid?	Why
`int n = Math.round(4.6f);`	✅	`round(float)` returns `int`
`int c = Math.ceil(4.2);`	❌	`ceil` returns `double`
`int m = Math.max(4, 9);`	✅	both `int` → returns `int`
`int m = Math.max(4, 9.0);`	❌	returns `double`
`int a = Math.abs(-5);`	✅	`abs(int)` returns `int`

Printing: `System.out.println(Math.pow(2, 3));` prints **8.0**, not 8. `System.out.println(Math.round(2.6));` prints **3**. `System.out.println(Math.ceil(2.1));` prints **3.0**.

5. Generating random numbers in a range

`Math.random()` returns a `double` in the range **[0.0, 1.0)**.

Required range	Expression
0 to 9	`(int) (Math.random() * 10)`
1 to 6 (dice)	`(int) (Math.random() * 6) + 1`
1000 to 9999	`1000 + (int) (Math.random() * 9000)`
`min` to `max` (inclusive)	`min + (int) (Math.random() * (max - min + 1))`
'A' to 'Z'	`(char) ('A' + (int) (Math.random() * 26))`

> **WARNING**
>
> `(int) Math.random() * 6` is **always 0** — the cast applies to `Math.random()` first (giving 0), then 0 × 6 = 0. Use brackets: `(int) (Math.random() * 6)`.

6. Writing Java expressions using `Math` methods

Mathematical expression	Java expression
$\sqrt{a^2 + b^2}$	`Math.sqrt(a * a + b * b)` or `Math.sqrt(Math.pow(a, 2) + Math.pow(b, 2))`
$\frac{-b+\sqrt{b^2-4ac}}{2a}$	`(-b + Math.sqrt(b * b - 4 * a * c)) / (2 * a)`
$\sqrt[3]{x} + \lvert y \rvert$	`Math.cbrt(x) + Math.abs(y)`
$P\left(1 + \frac{R}{100}\right)^T$	`p * Math.pow(1 + r / 100, t)`
$\frac{4}{3}\pi r^3$	`4.0 / 3 * Math.PI * Math.pow(r, 3)`
e^x	`Math.pow(Math.E, x)` (or `Math.exp(x)`)
$\lvert a - b \rvert^3$	`Math.pow(Math.abs(a - b), 3)`
$\sqrt{\frac{x}{y}} + \frac{\sqrt{x}}{y}$	`Math.sqrt(x / y) + Math.sqrt(x) / y`
$\frac{a^5+b^5}{ab}$	`(Math.pow(a, 5) + Math.pow(b, 5)) / (a * b)`

WARNING

`-b + Math.sqrt(d) / 2 * a` is **wrong** — division and multiplication are done before addition, and `/ 2 * a` multiplies by `a`. Always put the **whole numerator** and **whole denominator** in brackets.

7. Predict the output — solved

```
System.out.println(Math.ceil(-4.7) + Math.floor(-4.2));   // (1)
System.out.println(Math.round(-6.5) + Math.round(6.5));   // (2)
System.out.println(Math.sqrt(Math.abs(-16)) * 2);         // (3)
System.out.println(Math.max(Math.min(15, 8), 10));        // (4)
System.out.println(Math.pow(Math.cbrt(27), 2));           // (5)
System.out.println((int) Math.ceil(7.01) / 2);            // (6)
System.out.println(Math.floor(Math.random()));            // (7)
System.out.println(Math.abs(Math.floor(-2.1)));           // (8)
```

Show answers with working

#	Working	Output
1	−4.0 + (−5.0)	`-9.0`
2	−6 + 7	`1`
3	`sqrt(16)` = 4.0 × 2	`8.0`
4	`min(15, 8)` = 8 → `max(8, 10)`	`10`
5	`cbrt(27)` = 3.0 → `pow(3.0, 2)`	`9.0`
6	`ceil(7.01)` = 8.0 → `(int)` 8 → 8 / 2	`4`
7	random is in [0, 1) → floor is always 0	`0.0`
8	`floor(-2.1)` = −3.0 → abs	`3.0`

8. Find the error

```
int a = Math.pow(2, 4);
long b = Math.sqrt(64);
int c = Math.round(7.8);
double d = math.cbrt(27);
int e = Math.random(1, 6);
```

Show corrected code

```
int a = (int) Math.pow(2, 4);            // pow returns double
long b = (long) Math.sqrt(64);           // sqrt returns double
int c = (int) Math.round(7.8);           // round(double) returns long
double d = Math.cbrt(27);                // class name is Math (capital M)
int e = (int) (Math.random() * 6) + 1;   // random() takes no arguments
```

9. One-line facts

- `Math` is a class of the `java.lang` package; its methods are **static**.
- `pow`, `sqrt`, `cbrt`, `ceil`, `floor` and `random` return `double`.
- `round` returns `long` for a `double` argument and `int` for a `float` argument.
- `abs`, `max` and `min` return the **same type** as their (promoted) arguments.
- `Math.sqrt` of a negative number gives `NaN`.
- `Math.random()` never returns **1.0**.
- `Math.ceil(-0.5)` is `-0.0`.
- `Math.PI` (≈ 3.141592653589793) and `Math.E` (≈ 2.718281828459045) are constants of the `Math` class.

🧠 Test Yourself

Part A — Multiple choice

1. `System.out.println(Math.round(-0.5) + Math.ceil(-0.5));` prints
   (a) `-1.0` (b) `0.0` (c) `-0.0` (d) `1.0`
2. Which returns an `int` when its argument is `-9`? (a) `Math.sqrt` (b) `Math.abs` (c) `Math.cbrt` (d) `Math.ceil`
3. `Math.max(Math.ceil(2.3), Math.floor(3.7))` evaluates to
   (a) `3.0` (b) `3` (c) `4.0` (d) `2.0`
4. *Assertion (A):* `int x = Math.round(5.5);` gives a compile-time error.
   Reason (R): `Math.round()` with a `double` argument returns a `long`.
   (a) Both true; R explains A (b) Both true; R doesn't explain A (c) A true, R false (d) A false, R true
5. Which expression gives a random integer from 5 to 15 (both inclusive)?
   (a) `(int) (Math.random() * 15) + 5` (b) `5 + (int) (Math.random() * 11)` (c) `(int) Math.random() * 11 + 5`
   (d) `5 + (int) (Math.random() * 10)`
6. `Math.pow(2, -2)` is (a) `-4.0` (b) `0.25` (c) `4.0` (d) error
7. The output of `System.out.println(Math.sqrt(-25) + 5);` is (a) `0.0` (b) `-5.0` (c) `NaN` (d) runtime error
8. For which value of `x` are `Math.ceil(x)` and `Math.floor(x)` **equal**?
   (a) 2.0001 (b) −3.0 (c) 0.5 (d) −0.5
9. What is printed? `double d = 2.5; System.out.println(Math.round(d) + Math.round(-d));`
   (a) 0 (b) 1 (c) −1 (d) 5
10. Which statement is **false**?
    (a) `Math.cbrt(-64)` returns `-4.0` (b) `Math.abs(Math.floor(-1.1))` is `2.0` (c) `Math.min(-0.0, 0.0)` is `0.0` (d) `Math.pow(9, 0.5)` is `3.0`

Part B — Short answer questions

1. Name the package of the Math class. Why is it not imported explicitly?
2. Write the return types of `Math.sqrt()`, `Math.round(double)`, `Math.abs(int)`, `Math.max(int, double)`.
3. Give the output: (i) `Math.floor(-3.6)` (ii) `Math.ceil(-3.6)` (iii) `Math.round(-3.6)` (iv) `Math.round(3.5f)`
4. Write Java expressions: (i) $\sqrt[3]{x^2 + y^2}$ (ii) $\frac{|a-b|}{\sqrt{a+b}}$ (iii) $\pi r^2 h$
5. Write a statement to generate a random two-digit number.
6. What is `NaN`? When does `Math.sqrt()` return it?
7. Why is `(int) Math.random() * 100` always 0?
8. Give the output: `System.out.println(Math.max(-5, -9) + Math.min(5.0, 9));`

✅ Answers — Part A

Q	Ans	Working
1	(b)	`round(-0.5)` = 0 (long, promoted to 0.0); `ceil(-0.5)` = −0.0; 0.0 + (−0.0) = `0.0`
2	(b)	—
3	(a)	`max(3.0, 3.0)`
4	(a)	—
5	(b)	11 possible values starting at 5
6	(b)	$2^{-2} = \frac{1}{4}$
7	(c)	NaN plus anything is NaN
8	(b)	Equal only for whole numbers
9	(b)	3 + (−2) = 1
10	(c)	`Math.min(-0.0, 0.0)` is `-0.0`

✅ Answers — Part B

1. `java.lang`; it is imported automatically into every program.
2. `double`, `long`, `int`, `double`.
3. (i) `-4.0` (ii) `-3.0` (iii) `-4` (iv) `4`
4. (i) `Math.cbrt(x * x + y * y)` (ii) `Math.abs(a - b) / Math.sqrt(a + b)` (iii) `Math.PI * r * r * h`
5. `int n = 10 + (int) (Math.random() * 90);`
6. "Not a Number" — the result of an undefined operation; `Math.sqrt()` returns it for a negative argument.
7. The cast `(int)` is applied to `Math.random()` (a value less than 1) first, giving 0; then 0 × 100 = 0.
8. −5 + 5.0 = `0.0`

💻 Programs in this chapter

No.	Program
Q32	Results of All Math Class Methods
Q33	Hypotenuse of a Right-Angled Triangle
Q34	Area of a Triangle using Heron's Formula
Q35	Roots of a Quadratic Equation
Q36	Distance between Two Points
Q37	Compound Interest using Math.pow()
Q38	Largest and Smallest of Three Numbers using Math.max() and Math.min()
Q39	Round off a Number to Two Decimal Places
Q40	Perfect Square Check using Math.sqrt()
Q41	Rolling a Pair of Dice using Math.random()

No.	Program	
Q42	Evaluating Mathematical Formulas as Java Expressions	
Q43	Volume and Surface Area of a Sphere	
Q44	Integer Part, Fractional Part, Ceiling and Floor	
Q45	Four-Digit OTP Generator	🌟 *Extra*

Programs Q32 – Q45

Q32 Results of All Math Class Methods

Write a program to accept two real numbers a and b and display the results of the following methods of the Math class:

```
Math.pow(a, b)   Math.sqrt(a)   Math.cbrt(a)   Math.ceil(a)
Math.floor(a)    Math.round(a)  Math.abs(a)    Math.max(a, b)
Math.min(a, b)   Math.random()
```

If a is negative, display "Square root not defined" instead of the square root.

Q_32.java

```java
import java.util.Scanner;

public class Q_32
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a: ");
        double a = sc.nextDouble();
        System.out.print("Enter b: ");
        double b = sc.nextDouble();

        System.out.println("Math.pow(a, b)  = " + Math.pow(a, b));
        if (a >= 0)
            System.out.println("Math.sqrt(a)    = " + Math.sqrt(a));
        else
            System.out.println("Math.sqrt(a)    : Square root not defined");
        System.out.println("Math.cbrt(a)    = " + Math.cbrt(a));
        System.out.println("Math.ceil(a)    = " + Math.ceil(a));
        System.out.println("Math.floor(a)   = " + Math.floor(a));
        System.out.println("Math.round(a)   = " + Math.round(a));
        System.out.println("Math.abs(a)     = " + Math.abs(a));
        System.out.println("Math.max(a, b)  = " + Math.max(a, b));
        System.out.println("Math.min(a, b)  = " + Math.min(a, b));
        System.out.println("Math.random()   = " + Math.random());
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	double	to store the first number
b	double	to store the second number

Q33 Hypotenuse of a Right-Angled Triangle

Write a program to accept the perpendicular and the base of a right-angled triangle and calculate its hypotenuse using the Math class.

```
Hypotenuse = square root of (perpendicular^2 + base^2)
```

Display "Invalid sides" if either side is zero or negative.

Q_33.java

```java
import java.util.Scanner;

public class Q_33
```

```
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the perpendicular: ");
        double p = sc.nextDouble();
        System.out.print("Enter the base: ");
        double b = sc.nextDouble();

        if (p <= 0 || b <= 0)
        {
            System.out.println("Invalid sides");
        }
        else
        {
            double h = Math.sqrt(Math.pow(p, 2) + Math.pow(b, 2));
            System.out.println("Hypotenuse = " + h);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
p	double	to store the perpendicular
b	double	to store the base
h	double	to store the hypotenuse

Q34 Area of a Triangle using Heron's Formula

Write a program to accept the three sides of a triangle and find its area using Heron's formula:

```
s = (a + b + c) / 2
Area = square root of [ s (s - a)(s - b)(s - c) ]
```

The three sides form a triangle only if all are positive and the sum of any two sides is greater than the third side. Otherwise display "Triangle not possible".

Q_34.java

```
import java.util.Scanner;

public class Q_34
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the three sides: ");
        double a = sc.nextDouble();
        double b = sc.nextDouble();
        double c = sc.nextDouble();

        if (a > 0 && b > 0 && c > 0 && a + b > c && b + c > a && c + a > b)
        {
            double s = (a + b + c) / 2;
            double area = Math.sqrt(s * (s - a) * (s - b) * (s - c));
            System.out.println("Area = " + area);
        }
        else
        {
            System.out.println("Triangle not possible");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	double	to store the first side
b	double	to store the second side
c	double	to store the third side
s	double	to store the semi-perimeter
area	double	to store the area

Q35 Roots of a Quadratic Equation

Write a program to accept the coefficients a, b and c of the quadratic equation ax^2 + bx + c = 0 and find its roots.

```
Discriminant d = b^2 - 4ac
Roots = (-b + square root of d) / 2a  and  (-b - square root of d) / 2a
```

If d > 0, display "Roots are real and distinct" and the two roots. If d = 0, display "Roots are real and equal" and the root. If d < 0, display "Roots are imaginary". If a is 0, display "Not a quadratic equation".

Q_35.java

```java
import java.util.Scanner;

public class Q_35
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a, b and c: ");
        double a = sc.nextDouble();
        double b = sc.nextDouble();
        double c = sc.nextDouble();

        if (a == 0)
        {
            System.out.println("Not a quadratic equation");
        }
        else
        {
            double d = b * b - 4 * a * c;
            if (d > 0)
            {
                double r1 = (-b + Math.sqrt(d)) / (2 * a);
                double r2 = (-b - Math.sqrt(d)) / (2 * a);
                System.out.println("Roots are real and distinct");
                System.out.println("Root 1 = " + r1);
                System.out.println("Root 2 = " + r2);
            }
            else if (d == 0)
            {
                double r = -b / (2 * a);
                System.out.println("Roots are real and equal");
                System.out.println("Root = " + r);
            }
            else
            {
                System.out.println("Roots are imaginary");
            }
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	double	to store the coefficient of x squared
b	double	to store the coefficient of x
c	double	to store the constant term
d	double	to store the discriminant
r1	double	to store the first root
r2	double	to store the second root
r	double	to store the equal root

Q36 Distance between Two Points

> Write a program to accept the coordinates of two points (x1, y1) and (x2, y2) and find the distance between them.
>
> `Distance = square root of [ (x2 - x1)^2 + (y2 - y1)^2 ]`

Q_36.java

```
import java.util.Scanner;

public class Q_36
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter x1 and y1: ");
        double x1 = sc.nextDouble();
        double y1 = sc.nextDouble();
        System.out.print("Enter x2 and y2: ");
        double x2 = sc.nextDouble();
        double y2 = sc.nextDouble();

        double dist = Math.sqrt(Math.pow(x2 - x1, 2) + Math.pow(y2 - y1, 2));
        System.out.println("Distance = " + dist);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
x1	double	to store x-coordinate of first point
y1	double	to store y-coordinate of first point
x2	double	to store x-coordinate of second point
y2	double	to store y-coordinate of second point
dist	double	to store the distance

Q37 Compound Interest using Math.pow()

Write a program to accept the principal, the rate of interest per annum and the time in years, and calculate the amount and the compound interest (compounded annually).

`Amount = P x (1 + R / 100)^T` `Compound Interest = Amount - P`

Display "Invalid input" if any value is negative.

Q_37.java

```java
import java.util.Scanner;

public class Q_37
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter principal, rate and time: ");
        double p = sc.nextDouble();
        double r = sc.nextDouble();
        double t = sc.nextDouble();

        if (p < 0 || r < 0 || t < 0)
        {
            System.out.println("Invalid input");
        }
        else
        {
            double amt = p * Math.pow(1 + r / 100, t);
            double ci = amt - p;
            System.out.println("Amount            = " + Math.round(amt * 100) / 100.0);
            System.out.println("Compound Interest = " + Math.round(ci * 100) / 100.0);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
p	double	to store the principal
r	double	to store the rate of interest
t	double	to store the time in years
amt	double	to store the amount
ci	double	to store the compound interest

Q38 Largest and Smallest of Three Numbers using Math.max() and Math.min()

Write a program to accept three integers and find the largest and the smallest of them using only the methods Math.max() and Math.min() (no if statement).

Q_38.java

```java
import java.util.Scanner;

public class Q_38
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter three integers: ");
        int a = sc.nextInt();
        int b = sc.nextInt();
```

```
        int c = sc.nextInt();

        int large = Math.max(Math.max(a, b), c);
        int small = Math.min(Math.min(a, b), c);
        System.out.println("Largest  = " + large);
        System.out.println("Smallest = " + small);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	int	to store the first number
b	int	to store the second number
c	int	to store the third number
large	int	to store the largest number
small	int	to store the smallest number

Q39 Round off a Number to Two Decimal Places

> Write a program to accept a real number and round it off to two decimal places using Math.round().
>
> Example: 12.3456 becomes 12.35 and 7.891 becomes 7.89

Q_39.java

```
import java.util.Scanner;

public class Q_39
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a real number: ");
        double x = sc.nextDouble();
        double r = Math.round(x * 100) / 100.0;
        System.out.println("Rounded to two decimal places = " + r);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
x	double	to store the real number
r	double	to store the rounded value

Q40 Perfect Square Check using Math.sqrt()

> Write a program to accept a number and check whether it is a perfect square or not using Math.sqrt(). A perfect square is a number that is the square of an integer, e.g. 0, 1, 4, 9, 16, 25. Display "Perfect Square" or "Not a Perfect Square".

Q_40.java

```
import java.util.Scanner;

public class Q_40
{
    public static void main(String[] args)
```

```
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();

        if (n < 0)
        {
            System.out.println("Not a Perfect Square");
        }
        else
        {
            long root = Math.round(Math.sqrt(n));
            if (root * root == n)
                System.out.println("Perfect Square");
            else
                System.out.println("Not a Perfect Square");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
root	long	to store the rounded square root

Q41 Rolling a Pair of Dice using Math.random()

> Write a program to simulate the rolling of a pair of dice. Generate two random numbers from 1 to 6 using Math.random(), display both numbers and their sum, and display "Doubles!" if both numbers are the same.

Q_41.java

```
public class Q_41
{
    public static void main(String[] args)
    {
        int d1 = (int) (Math.random() * 6) + 1;
        int d2 = (int) (Math.random() * 6) + 1;
        System.out.println("Die 1 : " + d1);
        System.out.println("Die 2 : " + d2);
        System.out.println("Sum   : " + (d1 + d2));
        if (d1 == d2)
            System.out.println("Doubles!");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
d1	int	to store the first die value
d2	int	to store the second die value

Q42 Evaluating Mathematical Formulas as Java Expressions

Write a program to accept two real numbers x and y and evaluate the following by writing them as Java expressions using the methods of the Math class:

```
(a) square root of (x^2 + y^2)
(b) cube root of (x + y)
(c) |x - y| ^ 3
(d) (x^5 + y^5) / (x y)
(e) the greater of x^y and y^x
```

If x y = 0, display "Not defined" for (d). If x or y is negative, display "Not defined" for (e).

Q_42.java

```java
import java.util.Scanner;

public class Q_42
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter x and y: ");
        double x = sc.nextDouble();
        double y = sc.nextDouble();

        double a = Math.sqrt(x * x + y * y);
        double b = Math.cbrt(x + y);
        double c = Math.pow(Math.abs(x - y), 3);
        System.out.println("(a) " + a);
        System.out.println("(b) " + b);
        System.out.println("(c) " + c);

        if (x * y != 0)
        {
            double d = (Math.pow(x, 5) + Math.pow(y, 5)) / (x * y);
            System.out.println("(d) " + d);
        }
        else
        {
            System.out.println("(d) Not defined");
        }

        if (x >= 0 && y >= 0)
        {
            double e = Math.max(Math.pow(x, y), Math.pow(y, x));
            System.out.println("(e) " + e);
        }
        else
        {
            System.out.println("(e) Not defined");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
x	double	to store the first number
y	double	to store the second number
a	double	to store the value of part (a)
b	double	to store the value of part (b)
c	double	to store the value of part (c)

Name of the Variable	Data Type	Purpose / Description
d	double	to store the value of part (d)
e	double	to store the value of part (e)

Q43 Volume and Surface Area of a Sphere

Write a program to accept the radius of a sphere and calculate its volume and surface area using Math.PI and Math.pow().

```
Volume = (4/3) x pi x r^3        Surface Area = 4 x pi x r^2
```

Display "Invalid radius" for a zero or negative radius.

Q_43.java

```java
import java.util.Scanner;

public class Q_43
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the radius: ");
        double r = sc.nextDouble();

        if (r <= 0)
        {
            System.out.println("Invalid radius");
        }
        else
        {
            double vol = 4.0 / 3 * Math.PI * Math.pow(r, 3);
            double sa = 4 * Math.PI * Math.pow(r, 2);
            System.out.println("Volume       = " + vol);
            System.out.println("Surface area = " + sa);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
r	double	to store the radius
vol	double	to store the volume
sa	double	to store the surface area

Q44 Integer Part, Fractional Part, Ceiling and Floor

Write a program to accept a real number and display:

```
(a) its integer part (using type casting)
(b) its fractional part
(c) Math.ceil() of the number
(d) Math.floor() of the number
(e) Math.round() of the number
```

Run the program for both positive and negative numbers and observe how ceil, floor and round behave.

Q_44.java

```java
import java.util.Scanner;

public class Q_44
```

```
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a real number: ");
        double x = sc.nextDouble();

        int ip = (int) x;
        double fp = x - ip;
        System.out.println("(a) Integer part    = " + ip);
        System.out.println("(b) Fractional part = " + fp);
        System.out.println("(c) Math.ceil       = " + Math.ceil(x));
        System.out.println("(d) Math.floor      = " + Math.floor(x));
        System.out.println("(e) Math.round      = " + Math.round(x));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
x	double	to store the real number
ip	int	to store the integer part
fp	double	to store the fractional part

Q45 Four-Digit OTP Generator

Extra Banks and apps send a One-Time Password (OTP) to verify a user. Write a program to generate and display a random four-digit OTP (from 1000 to 9999) using Math.random(). Then accept the OTP from the user and display "OTP verified" if it matches, otherwise "Incorrect OTP".

Q_45.java

```
import java.util.Scanner;

public class Q_45
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int otp = 1000 + (int) (Math.random() * 9000);
        System.out.println("Your OTP is " + otp);
        System.out.print("Enter the OTP: ");
        int entered = sc.nextInt();
        if (entered == otp)
            System.out.println("OTP verified");
        else
            System.out.println("Incorrect OTP");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
otp	int	to store the generated otp
entered	int	to store the otp typed by the user

CHAPTER

06



Conditional Constructs in Java

Flow of control · if · if-else · if-else-if ladder · nested if · switch-case · default · break · fall through · menu-driven programs · `System.exit(0)`

32 programs | Q46 – Q77 | 4 extra

Theory

📑 Contents

1. Flow of control
2. The `if` statement
3. The `if-else` statement
4. The `if-else-if` ladder
5. Nested `if` and the dangling-else problem
6. The `switch` statement
7. Fall through
8. Menu-driven programs and `System.exit(0)`
9. Comparisons: if-else vs switch vs ternary
10. Conversions between if-else, switch and ternary
11. Predict the output — with dry runs
12. Find the errors
13. One-line facts
14. Test yourself
15. Programs in this chapter

1. Flow of control

The **flow of control** is the order in which the statements of a program are executed.

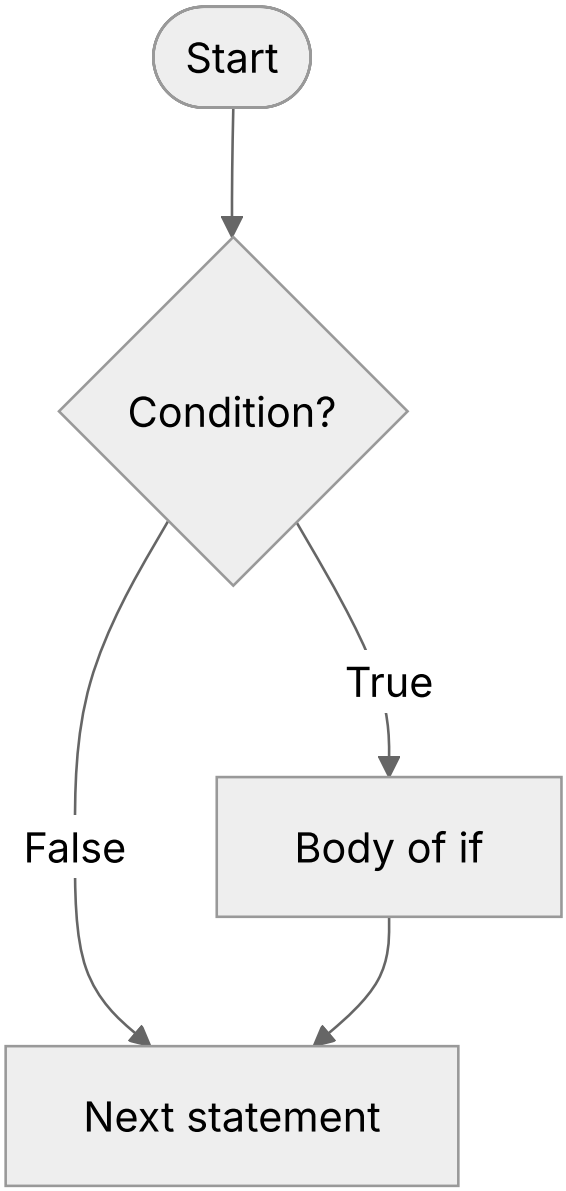
Construct	Meaning	Statements
Sequential	Statements execute one after another, top to bottom	Ordinary statements
Selection (conditional / decision-making)	One of several blocks is executed depending on a condition	`if`, `if-else`, `if-else-if`, nested `if`, `switch`, `?:`
Iteration (looping)	A block is executed repeatedly while a condition holds	`for`, `while`, `do-while` (Chapter 07)

A **compound statement (block)** is a group of statements enclosed in braces `{ }`, treated as a single statement.

2. The `if` statement

Executes a statement (or block) **only if** the condition is `true`; otherwise the statement is skipped.

```
if (condition)
{
    statements;      // executed only when condition is true
}
```




```
if (marks >= 40)
    System.out.println("Pass");
```

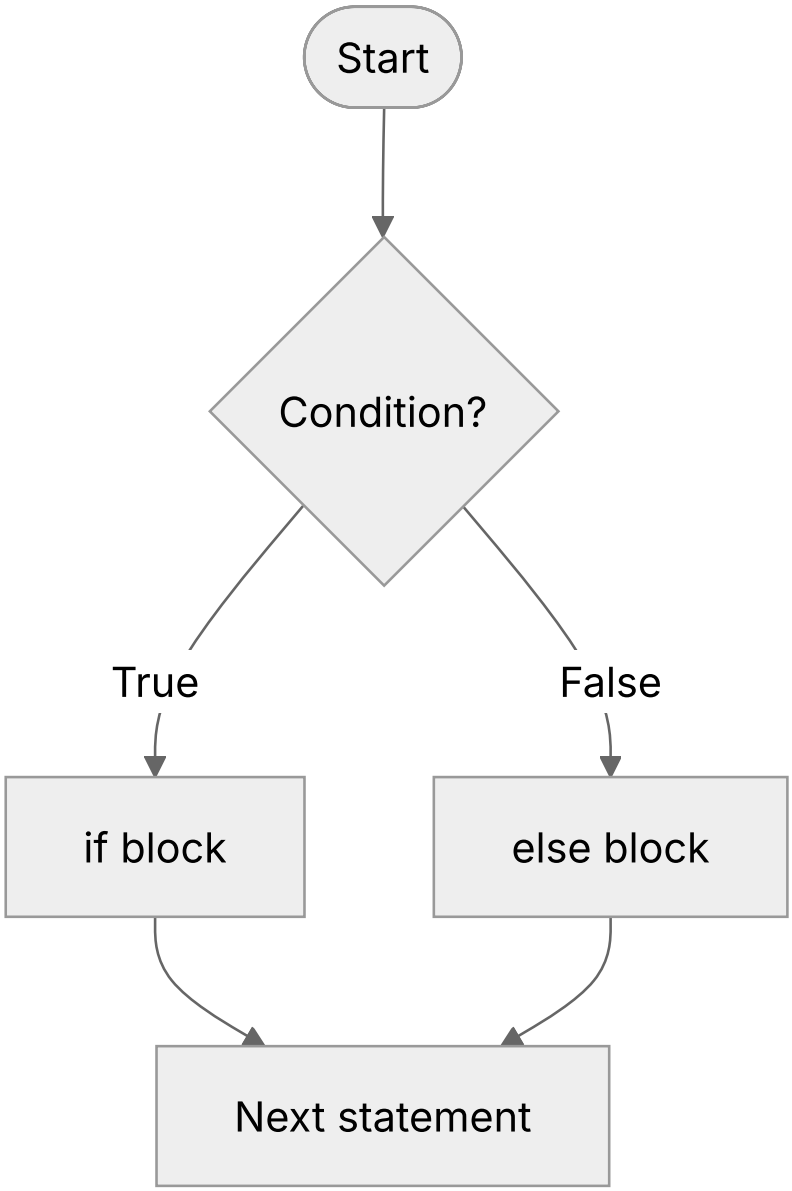
> **WARNING**
>
> `if (x > 5);` — the **semicolon** ends the `if` with an empty statement. The next line then runs **always**. This is a logical error that the compiler does not catch.

---

3. The `if-else` statement

Executes one block if the condition is `true` and **another** block if it is `false`. Exactly **one** of the two blocks is executed.

```
if (condition)
    statement1;
else
    statement2;
```

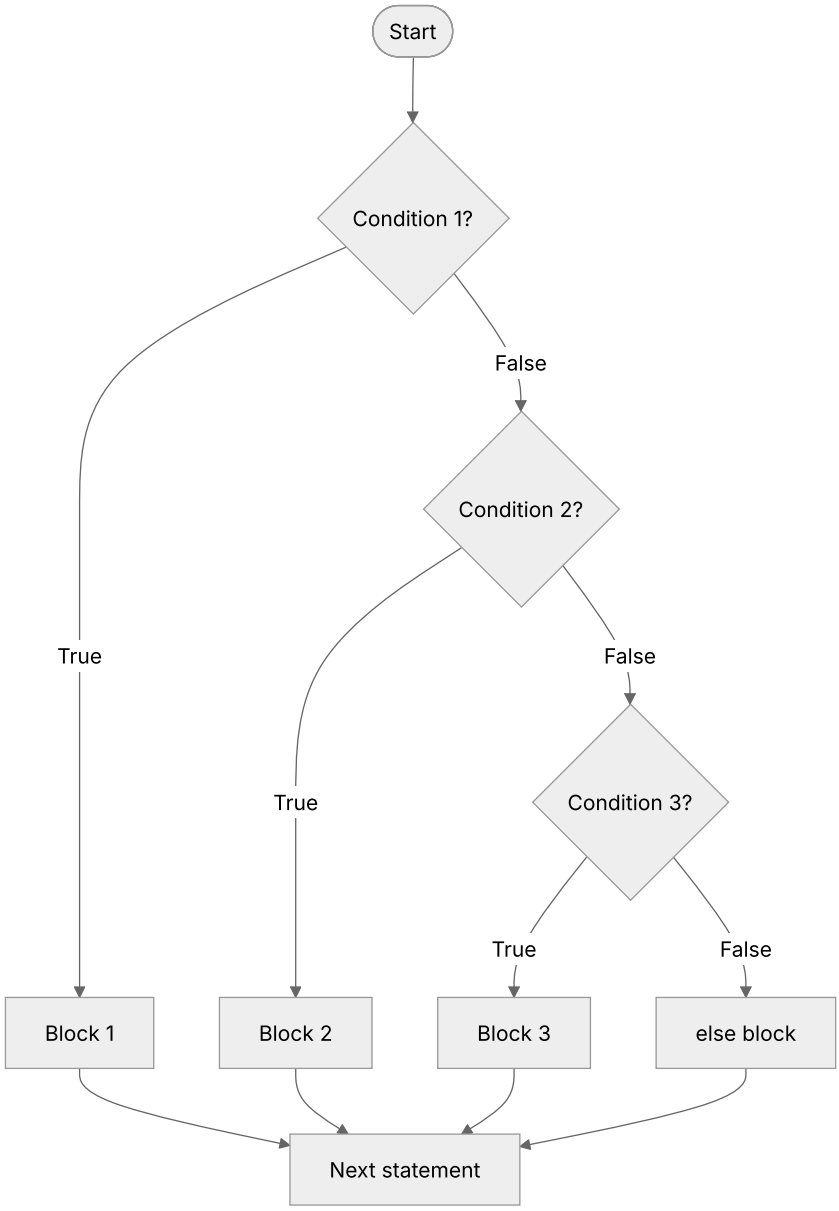



```
if (n % 2 == 0)
    System.out.println("Even");
else
    System.out.println("Odd");
```

4. The `if-else-if` ladder

Used to test **several conditions one after another**. As soon as a condition is `true`, its block is executed and the **rest of the ladder is skipped**. The final `else` (optional) runs when **no** condition is true.

```
if (condition1)
    block1;
else if (condition2)
    block2;
else if (condition3)
    block3;
else
    default_block;
```



> **TIP**
>
> In **slab programs** (grades, discounts, bills), write the conditions in increasing (or decreasing) order. Because earlier conditions have already failed, you need **only one bound** in each `else if`: `if (p >= 90) … else if (p >= 80) … else if (p >= 70) …` — no need to write `p >= 80 && p < 90`.

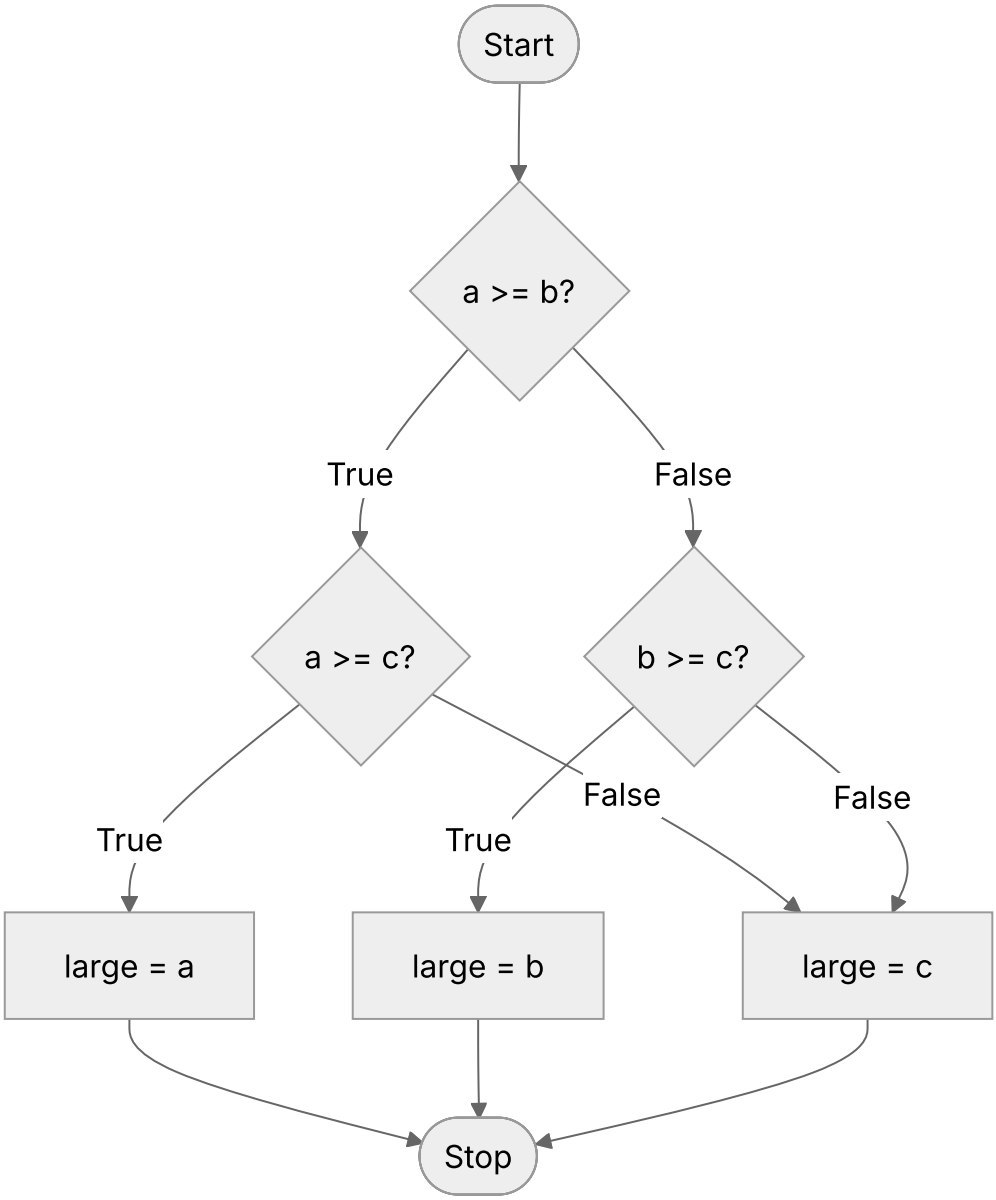
Two kinds of slab calculation

Flat rate on the whole amount	Cumulative (slab by slab)
The rate of the slab applies to the **entire** value	Each part of the value is charged at **its own** slab rate
`if (sales <= 25000) comm = sales * 5 / 100; else if …`	`if (u <= 100) bill = u * 2; else if (u <= 300) bill = 100 * 2 + (u - 100) * 3; …`
Discounts, commission	Electricity bill, telephone bill, taxi fare, income tax

5. Nested `if` and the dangling-else problem

A **nested if** is an `if` (or `if-else`) statement placed **inside** another `if` or `else` block. The inner condition is tested only when the outer condition allows it.

```
if (a >= b)
{
    if (a >= c)
        large = a;
    else
        large = c;
}
else
{
    if (b >= c)
        large = b;
    else
        large = c;
}
```



Dangling else: when there are more `if`s than `else`s and no braces, it is unclear which `if` an `else` belongs to. **Rule: an `else` is always paired with the nearest preceding unmatched `if`.**

```java
int x = 5, y = 10;
if (x > 10)
    if (y > 5)
        System.out.println("A");
else                          // belongs to if (y > 5), NOT if (x > 10)
    System.out.println("B");
// Output: nothing! (outer if is false, so the whole inner if-else is skipped)
```

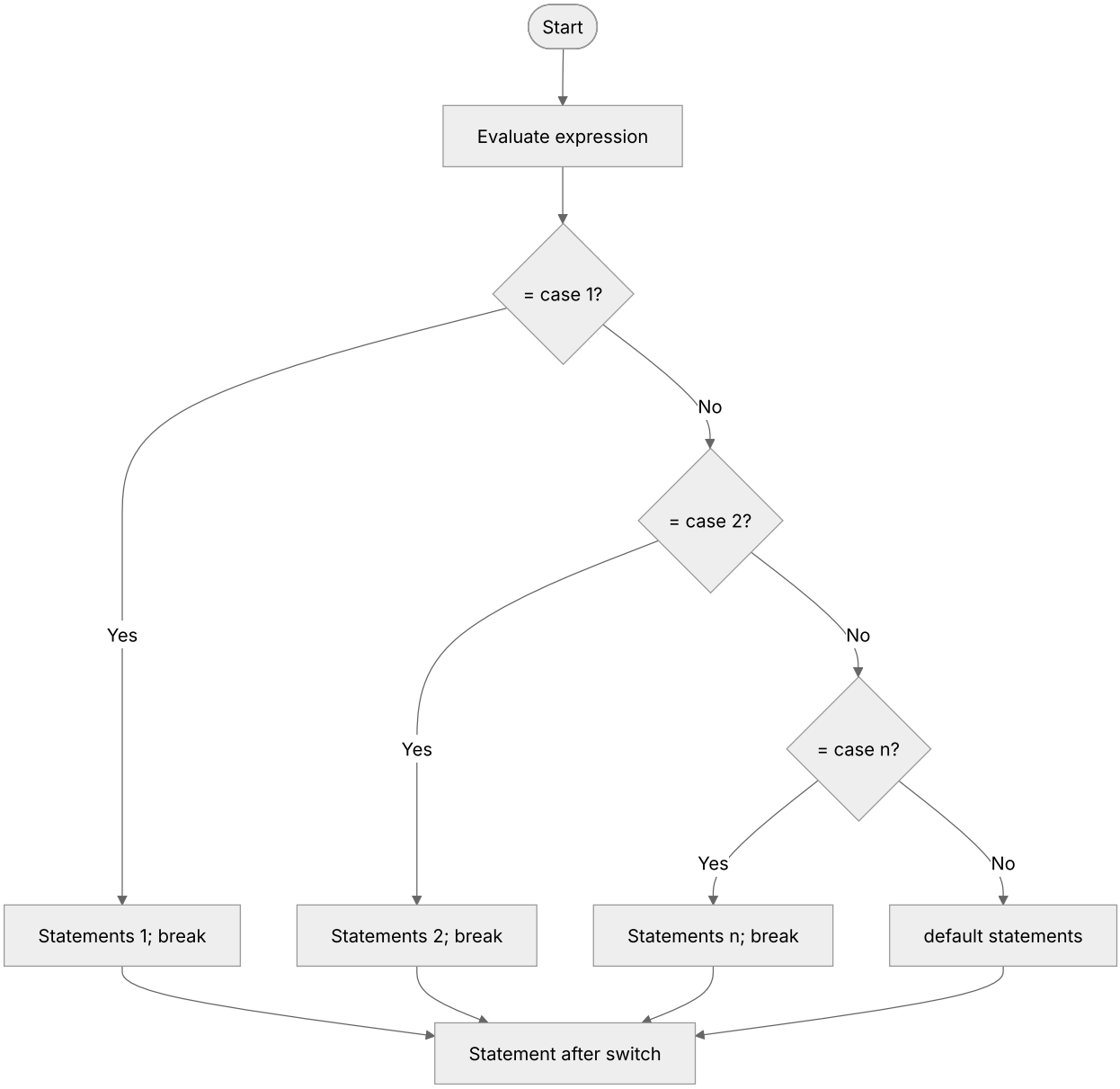
> **IMPORTANT**
>
> Indentation does **not** decide pairing — the compiler ignores it. Use braces `{ }` to make the outer `if` own the `else`.

---

6. The `switch` statement

A **multiple-branch selection** statement that compares the value of an expression with a list of **constant case labels** and executes the statements of the matching case.

```java
switch (expression)
{
    case constant1:
        statements;
        break;
    case constant2:
        statements;
        break;
    ...
    default:
        statements;
}
```



Rules of `switch`

Rule	Detail
Type of the expression	`int`, `char`, `byte`, `short`, `String` (and their wrappers). **Not** `long`, `float`, `double` or `boolean`
Case labels	Must be **constants** (literals or `final` variables) of a compatible type — **no variables, no ranges, no conditions**
Duplicates	Two case labels cannot have the **same value**
`break`	Transfers control **out of** the switch. Without it, execution **falls through** to the next case
`default`	Optional; executed when no case matches; can be placed anywhere (usually last)
Last case	`break` after the last case/default is optional

Keywords used: `switch`, `case`, `default`, `break`.

> **NOTE**
>
> A **`String`** can be used in `switch`: `switch (day) { case "Mon": … }`. The comparison is **case sensitive**.

7. Fall through

Fall through is the situation in a `switch` where, in the **absence of a** `break` statement, control **continues into the next case(s)** and executes their statements too, until a `break` or the end of the switch is reached.

Useful fall through — several cases sharing one action:

```
switch (ch)
{
    case 'a': case 'e': case 'i': case 'o': case 'u':
        System.out.println("Vowel");
        break;
    default:
        System.out.println("Not a vowel");
}
```

Accidental fall through:

```
int n = 2;
switch (n)
{
    case 1: System.out.print("One ");
    case 2: System.out.print("Two ");
    case 3: System.out.print("Three ");
            break;
    case 4: System.out.print("Four ");
}
// Output: Two Three
```

8. Menu-driven programs and `System.exit(0)`

A **menu-driven program** displays a **list of choices (menu)** to the user, accepts a choice and performs the corresponding task — usually using a `switch` statement.

`System.exit(0)` terminates the program **immediately**. The argument **0** indicates **normal termination**; a non-zero value indicates abnormal termination.

Basis	`break`	`System.exit(0)`
Effect	Exits from the **switch / loop** only	Terminates the **whole program**
Statements after the switch	Executed	Not executed

9. Comparisons

Basis	`if-else`	`switch`
Conditions tested	Any boolean expression — relational and logical	Only **equality** with constant values
Data types	Any type	`int`, `char`, `byte`, `short`, `String`
Ranges	Can test ranges (`m >= 40 && m < 60`)	Cannot test ranges
Multiple conditions	Combined with `&&`, `||`	Not possible
Readability for many fixed values	Lengthy	Neat and readable

Basis	`if-else`	`switch`
Default	`else`	`default`

Basis	Ternary operator	`if-else`
Nature	An **operator** — produces a value	A **statement**
Suitable for	Simple, single-value decisions	Complex decisions, multiple statements
Example	`max = (a > b) ? a : b;`	`if (a > b) max = a; else max = b;`

Basis	`if-else-if` ladder	Nested `if`
Structure	Conditions tested one after another at the same level	An `if` placed inside another `if` / `else`
Use	Choosing one out of many mutually exclusive options	Testing a condition only after another one is satisfied

10. Conversions between if-else, switch and ternary

(a) switch → if-else

```
switch (code)
{
    case 1: rate = 5; break;
    case 2: rate = 8; break;
    default: rate = 0;
}
```

```
if (code == 1)
    rate = 5;
else if (code == 2)
    rate = 8;
else
    rate = 0;
```

(b) if-else → switch (possible only when each condition is an equality test on one variable)

```
if (ch == 'A' || ch == 'a')
    System.out.println("Apple");
else if (ch == 'B' || ch == 'b')
    System.out.println("Ball");
else
    System.out.println("Unknown");
```

```
switch (ch)
{
    case 'A':
    case 'a':
        System.out.println("Apple");
        break;
    case 'B':
    case 'b':
        System.out.println("Ball");
        break;
    default:
        System.out.println("Unknown");
}
```

(c) if-else → ternary

```
if (age >= 18) status = "Adult"; else if (age >= 13) status = "Teen"; else status = "Child";
status = (age >= 18) ? "Adult" : (age >= 13) ? "Teen" : "Child";
```

11. Predict the output

Example 1

```
int a = 10, b = 20;
if (a > 5 && b++ < 20)
    a = a + b;
else
    a = a - b;
System.out.println(a + " " + b);
```

Show dry run

Step	Evaluation	Values
1	`a > 5` → true, so `b++ < 20` is evaluated	uses b = 20 → `20 < 20` false; b becomes 21
2	Condition false → else block	a = 10 − 21 = −11

Output: `-11 21`

Example 2

```
int x = 3;
switch (x * 2)
{
    case 4:  System.out.print("Four ");
    case 6:  System.out.print("Six ");
    case 8:  System.out.print("Eight ");
             break;
    default: System.out.print("Other ");
}
```

Show answer

`x * 2` = 6 → enters `case 6`, falls through to `case 8`, then `break`. **Output:** `Six Eight`

Example 3

```
char c = 'm';
switch (c)
{
    default:  System.out.print("D ");
    case 'a': System.out.print("A ");
              break;
    case 'b': System.out.print("B ");
}
```

Show answer

No case matches, so `default` runs. It has no `break`, so control falls through into `case 'a'`, then `break`.
Output: `D A`

Example 4

```
int n = 0;
if (n = 0)
    System.out.println("Zero");
```

Show answer

Compile-time error — `n = 0` is an assignment producing an `int`, but `if` needs a `boolean`. Correct: `if (n == 0)`.

Example 5

```
int p = 7, q = 7;
String r = (p > q) ? "P" : (p < q) ? "Q" : (p + q > 10) ? "EQ-BIG" : "EQ-SMALL";
System.out.println(r);
```

Show answer

`p > q` false → `p < q` false → `14 > 10` true → **`EQ-BIG`**

---

12. Find the errors

```
double d = 2.0;
switch (d)                        // (1)
{
    case 1.0: System.out.println("One"); break;
}

int a = 5, b = 3;
switch (a)
{
    case b:  System.out.println("B");   // (2)
    case 5:  System.out.println("Five");
    case 5:  System.out.println("Again"); // (3)
    case a > 3: System.out.println("Big"); // (4)
}
```

Show corrections

1. `switch` cannot work on `double` → use an `int`, `char`, `String` etc.
2. `b` is a variable — case labels must be **constants** (use `final int B = 3;` or a literal).
3. Duplicate case label `5`.
4. A condition cannot be a case label — use `if`.

---

13. One-line facts

- `if` and `switch` are **selection** (decision-making) statements.
- An `else` pairs with the **nearest unmatched** `if`.
- `switch` tests only for **equality**.
- `switch` does **not** accept `long`, `float`, `double` or `boolean`.

- **Fall through** occurs when `break` is missing.
- `default` is **optional** and may appear anywhere in the switch.
- `System.exit(0)` terminates the program; `0` means normal termination.
- The **ternary** operator is a compact form of `if-else` that returns a value.

🧠 Test Yourself

Part A — Multiple choice

1. What is printed? `java int k = 5; if (k++ > 5 || ++k > 6) k += 10; System.out.println(k);` (a) 5 (b) 7 (c) 17 (d) 16
2. Which can **never** be the type of a switch expression? (a) `char` (b) `String` (c) `boolean` (d) `short`
3. *Assertion (A):* In a `switch`, the `default` label must be the last label.
   Reason (R): Control goes to `default` only when no case matches.
   (a) Both true; R explains A (b) Both true; R doesn't explain A (c) A false, R true (d) Both false
4. What is printed? `java int n = 3; switch (n) { case 1: case 3: n *= 2; case 4: n += 1; break; case 7: n = 0; } System.out.println(n);` (a) 6 (b) 7 (c) 0 (d) 4
5. `java int x = 15; if (x > 10) if (x > 20) System.out.print("A"); else System.out.print("B"); else System.out.print("C");` prints (a) A (b) B (c) C (d) BC
6. Which `if` condition correctly tests that `ch` is **not** a vowel (lowercase only)? (a) `ch != 'a' || ch != 'e' || …` (b) `!(ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')` (c) `ch != "aeiou"` (d) `!ch == 'a'`
7. What happens when this runs? `int a = 4; switch (a) { case 4: System.out.print("X"); System.exit(0); default: System.out.print("Y"); } System.out.print("Z");` (a) XYZ (b) XZ (c) X (d) XY
8. *Assertion (A):* `if (a > b) ; else System.out.println(a);` compiles. *Reason (R):* A lone semicolon is a valid empty statement. (a) Both true; R explains A (b) Both true; R doesn't explain A (c) A true, R false (d) A false, R true
9. Which slab condition is **redundant** in a correctly ordered ladder `if (u <= 100) … else if (u > 100 && u <= 300) … else …`? (a) `u <= 100` (b) `u > 100` (c) `u <= 300` (d) none
10. `int m = 85; String g = m >= 90 ? "A" : m >= 80 ? "B" : "C";` gives (a) A (b) B (c) C (d) Error

Part B — Short answer questions

1. What is fall through? Write a program segment where fall through is useful.
2. Give two differences between `if-else` and `switch`.
3. Convert into `switch`: `if (n == 1) s = "Red"; else if (n == 2) s = "Green"; else s = "Blue";`
4. Convert into a ternary statement: `if (a % 2 == 0) b = a / 2; else b = 3 * a + 1;`
5. Explain the dangling-else problem with an example. How is it resolved?
6. What is the use of `System.exit(0)`? How is it different from `break`?
7. Rewrite with a single `if` and logical operators: `if (x > 0) { if (x < 100) System.out.println("OK"); }`
8. Why can't `switch` be used to check `marks >= 90`?
9. Give the output: `int a = 2; switch (a + 1) { case 1: a = 10; case 3: a = a * 3; case 6: a++; break; default: a = 0; } System.out.println(a);`

✅ Answers — Part A

Q	Ans	Working
1	(c)	`k++ > 5` → 5 > 5 false, k = 6; `++k > 6` → 7 > 6 true; k = 7 + 10 = 17

Q	Ans	Working
2	(c)	—
3	(c)	`default` can be placed anywhere; R is true
4	(b)	case 3 → n = 6, falls into case 4 → n = 7, break
5	(b)	Inner `else` pairs with `if (x > 20)`; outer `else` with `if (x > 10)`
6	(b)	Option (a) is always true
7	(c)	The program ends at `System.exit(0)`
8	(a)	—
9	(b)	It is already known that `u > 100` when the first test fails
10	(b)	—

✅ Answers — Part B

1. Execution continuing into the next case when `break` is missing. Useful for grouping: `case 'a': case 'e': case 'i': case 'o': case 'u': System.out.println("Vowel"); break;`
2. `if-else` can test any condition (ranges, logical combinations) on any type; `switch` tests only equality against constants of int/char/String types. `switch` is more readable when there are many fixed values.
3. `java switch (n) { case 1: s = "Red"; break; case 2: s = "Green"; break; default: s = "Blue"; }`
4. `b = (a % 2 == 0) ? a / 2 : 3 * a + 1;`
5. When an `if` without braces contains another `if`, an `else` might seem to belong to either. Java pairs it with the nearest unmatched `if`. Use braces to attach it to the outer `if`.
6. It terminates the program immediately. `break` exits only the switch/loop, after which the following statements still execute.
7. `if (x > 0 && x < 100) System.out.println("OK");`
8. Case labels must be constant values; `switch` cannot test ranges or relational conditions.
9. `a + 1` = 3 → `a = 2 * 3 = 6` → falls into case 6 → `a = 7` → break. Output: **7**

💻 Programs in this chapter

No.	Program
Q46	Electricity Bill (Slab System)
Q47	Income Tax by Age, Gender and Slab
Q48	Discount on Laptop or Desktop Purchase
Q49	Menu-Driven Program: Area of Circle, Square or Rectangle
Q50	Simple Calculator using switch
Q51	Menu-Driven Temperature Conversion
Q52	Day of the Week using switch
Q53	Name of the Month and Number of Days
Q54	Season of the Year using Fall-Through in switch

No.	Program	
Q55	Vowel or Consonant using switch	
Q56	Grade from Percentage	
Q57	Courier Charges by Weight and Type	
Q58	Telephone Bill with Monthly Rent	
Q59	Commission on Sales	
Q60	Fixed Deposit Interest by Period	
Q61	Taxi Fare by Distance	
Q62	Leap Year Check	
Q63	Largest of Three Numbers using Nested if	
Q64	Triangle Validity and Type by Sides	
Q65	Odd or Even Number	
Q66	Positive, Negative or Zero	
Q67	Eligibility to Vote	
Q68	Uppercase, Lowercase, Digit or Special Character	
Q69	Buzz Number	
Q70	Divisibility by 5 and 11	
Q71	Profit or Loss and its Percentage	
Q72	Quadrant of a Point	
Q73	Menu with an Exit Option using System.exit(0)	
Q74	Rock, Paper, Scissors against the Computer	🌟 *Extra*
Q75	Zodiac Sign from Date of Birth	🌟 *Extra*
Q76	Valid Date Checker	🌟 *Extra*
Q77	BMI Calculator with Category	🌟 *Extra*

Programs Q46 – Q77

Q46 Electricity Bill (Slab System)

An electricity board charges its consumers according to the following tariff. The rates apply slab by slab (the first 100 units at the first rate, the next 200 units at the second rate, and so on).

```
Units consumed              Rate per unit
First 100 units             Rs 2.00
Next 200 units (101-300)    Rs 3.00
Above 300 units             Rs 5.00
```

A surcharge of 2.5% of the bill is added if the units consumed exceed 300. Write a program to accept the name of the consumer and the number of units consumed, and display the name, the units and the bill amount. Display "Invalid units" if the units are negative.

Q_46.java

```java
import java.util.Scanner;

public class Q_46
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the consumer's name: ");
        String name = sc.nextLine();
        System.out.print("Enter the units consumed: ");
        int units = sc.nextInt();
        double bill;

        if (units < 0)
        {
            System.out.println("Invalid units");
        }
        else
        {
            if (units <= 100)
                bill = units * 2.0;
            else if (units <= 300)
                bill = 100 * 2.0 + (units - 100) * 3.0;
            else
                bill = 100 * 2.0 + 200 * 3.0 + (units - 300) * 5.0;

            if (units > 300)
                bill = bill + bill * 2.5 / 100;

            System.out.println("Name of consumer : " + name);
            System.out.println("Units consumed   : " + units);
            System.out.println("Bill amount      : Rs " + bill);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
name	String	to store the consumer's name
units	int	to store the units consumed
bill	double	to store the bill amount

Q47 Income Tax by Age, Gender and Slab

Given below is a hypothetical table showing the rates of income tax for male citizens below or of the age of 65 years:

Taxable Income (TI) in Rs	Income Tax in Rs
Does not exceed 1,60,000	Nil
1,60,001 to 5,00,000	(TI - 1,60,000) x 10%
5,00,001 to 8,00,000	[(TI - 5,00,000) x 20%] + 34,000
Above 8,00,000	[(TI - 8,00,000) x 30%] + 94,000

Write a program to input the age, the gender (M or F) and the taxable income of a person. If the age is more than 65 years or the gender is female, display "Wrong Category". Otherwise compute and display the income tax payable.

Q_47.java

```java
import java.util.Scanner;

public class Q_47
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter age: ");
        int age = sc.nextInt();
        System.out.print("Enter gender (M/F): ");
        char g = sc.next().charAt(0);
        System.out.print("Enter taxable income: ");
        double ti = sc.nextDouble();
        double tax;

        if (age > 65 || g == 'F' || g == 'f')
        {
            System.out.println("Wrong Category");
        }
        else if (age < 0 || ti < 0 || !(g == 'M' || g == 'm'))
        {
            System.out.println("Invalid input");
        }
        else
        {
            if (ti <= 160000)
                tax = 0;
            else if (ti <= 500000)
                tax = (ti - 160000) * 10 / 100;
            else if (ti <= 800000)
                tax = (ti - 500000) * 20 / 100 + 34000;
            else
                tax = (ti - 800000) * 30 / 100 + 94000;
            System.out.println("Income tax = Rs " + tax);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
age	int	to store the age
g	char	to store the gender
ti	double	to store the taxable income
tax	double	to store the income tax

Q48 Discount on Laptop or Desktop Purchase

A computer shop gives discounts on the purchase of a laptop or a desktop as follows:

Purchase amount (Rs)	Laptop	Desktop
Up to 25,000	0.0%	5.0%
25,001 to 57,000	5.0%	7.5%
57,001 to 1,00,000	7.5%	10.0%
More than 1,00,000	10.0%	15.0%

Write a program to input the purchase amount and the type of purchase ('L' for laptop, 'D' for desktop), and calculate the discount and the net amount to be paid. Display "Invalid type" for any other type and "Invalid amount" for a negative amount.

Q_48.java

```java
import java.util.Scanner;

public class Q_48
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the purchase amount: ");
        double amt = sc.nextDouble();
        System.out.print("Enter type (L/D): ");
        char type = sc.next().charAt(0);
        double rate = 0;

        if (amt < 0)
        {
            System.out.println("Invalid amount");
        }
        else if (type == 'L' || type == 'l')
        {
            if (amt <= 25000)
                rate = 0;
            else if (amt <= 57000)
                rate = 5;
            else if (amt <= 100000)
                rate = 7.5;
            else
                rate = 10;
        }
        else if (type == 'D' || type == 'd')
        {
            if (amt <= 25000)
                rate = 5;
            else if (amt <= 57000)
                rate = 7.5;
            else if (amt <= 100000)
                rate = 10;
            else
                rate = 15;
        }
        else
        {
            System.out.println("Invalid type");
            rate = -1;
        }

        if (amt >= 0 && rate >= 0)
        {
            double disc = amt * rate / 100;
            double net = amt - disc;
            System.out.println("Discount   = Rs " + disc);
            System.out.println("Net amount = Rs " + net);
        }
```

```
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
amt	double	to store the purchase amount
type	char	to store the type of purchase
rate	double	to store the discount rate
disc	double	to store the discount
net	double	to store the net amount

Q49 Menu-Driven Program: Area of Circle, Square or Rectangle

Write a menu-driven program using a switch statement to find the area of a circle, a square or a rectangle, according to the user's choice:

```
1. Area of a circle    = (22/7) x r x r
2. Area of a square    = side x side
3. Area of a rectangle = length x breadth
```

For an incorrect choice, display "Wrong choice".

Q_49.java

```java
import java.util.Scanner;

public class Q_49
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("1. Area of a circle");
        System.out.println("2. Area of a square");
        System.out.println("3. Area of a rectangle");
        System.out.print("Enter your choice: ");
        int ch = sc.nextInt();
        double area;

        switch (ch)
        {
            case 1:
                System.out.print("Enter the radius: ");
                double r = sc.nextDouble();
                area = 22.0 / 7 * r * r;
                System.out.println("Area of the circle = " + area);
                break;
            case 2:
                System.out.print("Enter the side: ");
                double s = sc.nextDouble();
                area = s * s;
                System.out.println("Area of the square = " + area);
                break;
            case 3:
                System.out.print("Enter length and breadth: ");
                double l = sc.nextDouble();
                double b = sc.nextDouble();
                area = l * b;
                System.out.println("Area of the rectangle = " + area);
                break;
            default:
                System.out.println("Wrong choice");
        }
```

```
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
ch	int	to store the user's choice
area	double	to store the area
r	double	to store the radius
s	double	to store the side
l	double	to store the length
b	double	to store the breadth

Q50 Simple Calculator using switch

Write a program to accept two numbers and an arithmetic operator (+, -, *, / or %) and perform the corresponding operation using a switch statement. If the second number is zero for / or %, display "Division by zero not allowed". For any other operator, display "Invalid operator".

Q_50.java

```java
import java.util.Scanner;

public class Q_50
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the first number: ");
        double a = sc.nextDouble();
        System.out.print("Enter the operator (+, -, *, /, %): ");
        char op = sc.next().charAt(0);
        System.out.print("Enter the second number: ");
        double b = sc.nextDouble();

        switch (op)
        {
            case '+':
                System.out.println(a + " + " + b + " = " + (a + b));
                break;
            case '-':
                System.out.println(a + " - " + b + " = " + (a - b));
                break;
            case '*':
                System.out.println(a + " * " + b + " = " + (a * b));
                break;
            case '/':
                if (b == 0)
                    System.out.println("Division by zero not allowed");
                else
                    System.out.println(a + " / " + b + " = " + (a / b));
                break;
            case '%':
                if (b == 0)
                    System.out.println("Division by zero not allowed");
                else
                    System.out.println(a + " % " + b + " = " + (a % b));
                break;
            default:
                System.out.println("Invalid operator");
        }
```

```
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	double	to store the first number
op	char	to store the operator
b	double	to store the second number

Q51 Menu-Driven Temperature Conversion

Write a menu-driven program using switch-case to convert temperatures:

1. Celsius to Fahrenheit : F = 9/5 x C + 32
2. Fahrenheit to Celsius : C = 5/9 x (F - 32)

Accept the choice and the temperature, and display the converted temperature. For an incorrect choice, display "Wrong choice".

Q_51.java

```java
import java.util.Scanner;

public class Q_51
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("1. Celsius to Fahrenheit");
        System.out.println("2. Fahrenheit to Celsius");
        System.out.print("Enter your choice: ");
        int ch = sc.nextInt();
        double t;

        switch (ch)
        {
            case 1:
                System.out.print("Enter temperature in Celsius: ");
                t = sc.nextDouble();
                System.out.println("Temperature in Fahrenheit = " + (9.0 / 5 * t + 32));
                break;
            case 2:
                System.out.print("Enter temperature in Fahrenheit: ");
                t = sc.nextDouble();
                System.out.println("Temperature in Celsius = " + (5.0 / 9 * (t - 32)));
                break;
            default:
                System.out.println("Wrong choice");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
ch	int	to store the user's choice
t	double	to store the temperature

Q52 Day of the Week using switch

> Write a program to accept a number from 1 to 7 and display the corresponding day of the week using a switch statement (1 = Sunday, 2 = Monday, ..., 7 = Saturday). For any other number, display "Invalid day number".

Q_52.java

```java
import java.util.Scanner;

public class Q_52
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a day number (1-7): ");
        int d = sc.nextInt();

        switch (d)
        {
            case 1:
                System.out.println("Sunday");
                break;
            case 2:
                System.out.println("Monday");
                break;
            case 3:
                System.out.println("Tuesday");
                break;
            case 4:
                System.out.println("Wednesday");
                break;
            case 5:
                System.out.println("Thursday");
                break;
            case 6:
                System.out.println("Friday");
                break;
            case 7:
                System.out.println("Saturday");
                break;
            default:
                System.out.println("Invalid day number");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
d	int	to store the day number

Q53 Name of the Month and Number of Days

> Write a program to accept a month number (1 to 12) and a year, and display the name of the month and the number of days in it using a switch statement. February has 29 days in a leap year and 28 days otherwise. A year is a leap year if it is divisible by 400, or divisible by 4 but not by 100. Display "Invalid month" for a wrong month number.

Q_53.java

```java
import java.util.Scanner;

public class Q_53
{
```

```
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter month number and year: ");
        int m = sc.nextInt();
        int y = sc.nextInt();
        String name = "";
        int days = 0;

        switch (m)
        {
            case 1:  name = "January";   days = 31; break;
            case 2:
                name = "February";
                if ((y % 4 == 0 && y % 100 != 0) || y % 400 == 0)
                    days = 29;
                else
                    days = 28;
                break;
            case 3:  name = "March";     days = 31; break;
            case 4:  name = "April";     days = 30; break;
            case 5:  name = "May";       days = 31; break;
            case 6:  name = "June";      days = 30; break;
            case 7:  name = "July";      days = 31; break;
            case 8:  name = "August";    days = 31; break;
            case 9:  name = "September"; days = 30; break;
            case 10: name = "October";   days = 31; break;
            case 11: name = "November";  days = 30; break;
            case 12: name = "December";  days = 31; break;
            default: name = "";
        }

        if (days == 0)
            System.out.println("Invalid month");
        else
            System.out.println(name + " " + y + " has " + days + " days");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
m	int	to store the month number
y	int	to store the year
name	String	to store the name of the month
days	int	to store the number of days

Q54 Season of the Year using Fall-Through in switch

Write a program to accept a month number (1 to 12) and display the season using the fall-through property of the switch statement:

```
December, January, February          : Winter
March, April, May                    : Summer
June, July, August, September        : Monsoon
October, November                    : Autumn
```

Display "Invalid month" for any other number.

Q_54.java

```
import java.util.Scanner;

public class Q_54
```

```
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the month number: ");
        int m = sc.nextInt();

        switch (m)
        {
            case 12:
            case 1:
            case 2:
                System.out.println("Winter");
                break;
            case 3:
            case 4:
            case 5:
                System.out.println("Summer");
                break;
            case 6:
            case 7:
            case 8:
            case 9:
                System.out.println("Monsoon");
                break;
            case 10:
            case 11:
                System.out.println("Autumn");
                break;
            default:
                System.out.println("Invalid month");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
m	int	to store the month number

Q55 Vowel or Consonant using switch

> Write a program to accept a character and check whether it is a vowel or a consonant using a switch statement with fall-through for both uppercase and lowercase vowels. If the character is not a letter, display "Not an alphabet".

Q_55.java

```
import java.util.Scanner;

public class Q_55
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a character: ");
        char ch = sc.next().charAt(0);

        switch (ch)
        {
            case 'a': case 'A':
            case 'e': case 'E':
            case 'i': case 'I':
            case 'o': case 'O':
            case 'u': case 'U':
```

```
                System.out.println("Vowel");
                break;
            default:
                if ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z'))
                    System.out.println("Consonant");
                else
                    System.out.println("Not an alphabet");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
ch	char	to store the entered character

Q56 Grade from Percentage

Write a program to accept the percentage of marks of a student and display the grade according to the following table. Display "Invalid percentage" if the percentage is not from 0 to 100.

```
Percentage              Grade
90 and above            A+
80 to less than 90      A
70 to less than 80      B
60 to less than 70      C
50 to less than 60      D
40 to less than 50      E
Below 40                F (Fail)
```

Q_56.java

```
import java.util.Scanner;

public class Q_56
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the percentage: ");
        double p = sc.nextDouble();

        if (p < 0 || p > 100)
            System.out.println("Invalid percentage");
        else if (p >= 90)
            System.out.println("Grade A+");
        else if (p >= 80)
            System.out.println("Grade A");
        else if (p >= 70)
            System.out.println("Grade B");
        else if (p >= 60)
            System.out.println("Grade C");
        else if (p >= 50)
            System.out.println("Grade D");
        else if (p >= 40)
            System.out.println("Grade E");
        else
            System.out.println("Grade F (Fail)");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
p	double	to store the percentage

Q57 Courier Charges by Weight and Type

A courier company charges for parcels as follows:

Weight of parcel	Ordinary	Express
Up to 5 kg (minimum charge)	Rs 80	Rs 150
For each kg from 5 to 10 kg	Rs 15 per kg	Rs 25 per kg
For each kg above 10 kg	Rs 10 per kg	Rs 20 per kg

Write a program to input the weight of the parcel (in kg) and the type of service ('O' for ordinary, 'E' for express) and calculate the charge. Display "Invalid input" for a zero or negative weight or a wrong type.

Q_57.java

```java
import java.util.Scanner;

public class Q_57
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter weight (kg): ");
        double w = sc.nextDouble();
        System.out.print("Enter type (O/E): ");
        char t = sc.next().charAt(0);
        double charge = 0;

        if (w <= 0)
        {
            System.out.println("Invalid input");
        }
        else if (t == 'O' || t == 'o')
        {
            if (w <= 5)
                charge = 80;
            else if (w <= 10)
                charge = 80 + (w - 5) * 15;
            else
                charge = 80 + 5 * 15 + (w - 10) * 10;
            System.out.println("Ordinary charge = Rs " + charge);
        }
        else if (t == 'E' || t == 'e')
        {
            if (w <= 5)
                charge = 150;
            else if (w <= 10)
                charge = 150 + (w - 5) * 25;
            else
                charge = 150 + 5 * 25 + (w - 10) * 20;
            System.out.println("Express charge = Rs " + charge);
        }
        else
        {
            System.out.println("Invalid input");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
w	double	to store the weight of the parcel
t	char	to store the type of service
charge	double	to store the courier charge

Q58 Telephone Bill with Monthly Rent

A telephone company charges a fixed monthly rent of Rs 180 and charges for calls as follows (slab by slab):

```
First 100 calls          Free
Next 100 calls           Rs 1.00 per call
Next 200 calls           Rs 0.80 per call
Above 400 calls          Rs 0.60 per call
```

Write a program to input the number of calls made in a month and display the total bill. Display "Invalid input" for a negative number of calls.

Q_58.java

```java
import java.util.Scanner;

public class Q_58
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of calls: ");
        int calls = sc.nextInt();
        final double RENT = 180;
        double charge;

        if (calls < 0)
        {
            System.out.println("Invalid input");
        }
        else
        {
            if (calls <= 100)
                charge = 0;
            else if (calls <= 200)
                charge = (calls - 100) * 1.0;
            else if (calls <= 400)
                charge = 100 * 1.0 + (calls - 200) * 0.80;
            else
                charge = 100 * 1.0 + 200 * 0.80 + (calls - 400) * 0.60;
            System.out.println("Call charges = Rs " + charge);
            System.out.println("Total bill   = Rs " + (RENT + charge));
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
calls	int	to store the number of calls
RENT	double	to store the fixed monthly rent
charge	double	to store the call charges

Q59 Commission on Sales

A company pays its salesmen a commission on their total sales as follows:

```
Sales (Rs)                  Commission
Up to 25,000                5% of sales
25,001 to 50,000            7.5% of sales
50,001 to 1,00,000          10% of sales
More than 1,00,000          12% of sales
```

Write a program to input the name and the sales of a salesman and display the name and the commission earned. Display "Invalid sales" for a negative value.

Q_59.java

```java
import java.util.Scanner;

public class Q_59
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the salesman's name: ");
        String name = sc.nextLine();
        System.out.print("Enter the sales: ");
        double sales = sc.nextDouble();
        double rate;

        if (sales < 0)
        {
            System.out.println("Invalid sales");
        }
        else
        {
            if (sales <= 25000)
                rate = 5;
            else if (sales <= 50000)
                rate = 7.5;
            else if (sales <= 100000)
                rate = 10;
            else
                rate = 12;
            double comm = sales * rate / 100;
            System.out.println("Name       : " + name);
            System.out.println("Commission : Rs " + comm);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
name	String	to store the salesman's name
sales	double	to store the total sales
rate	double	to store the commission rate
comm	double	to store the commission

Q60 Fixed Deposit Interest by Period

A bank pays interest on fixed deposits according to the period of the deposit:

```
Number of days              Rate of interest (per annum)
Up to 180 days              5.5%
181 to 364 days             7.5%
Exactly 365 days            9.0%
More than 365 days          8.5%
```

Write a program to input the principal and the number of days, and calculate the interest and the maturity amount using Interest = P x R x (days / 365) / 100. Display "Invalid input" if either value is zero or negative.

Q_60.java

```java
import java.util.Scanner;

public class Q_60
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the principal: ");
        double p = sc.nextDouble();
        System.out.print("Enter the number of days: ");
        int days = sc.nextInt();
        double r;

        if (p <= 0 || days <= 0)
        {
            System.out.println("Invalid input");
        }
        else
        {
            if (days <= 180)
                r = 5.5;
            else if (days <= 364)
                r = 7.5;
            else if (days == 365)
                r = 9.0;
            else
                r = 8.5;
            double interest = p * r * (days / 365.0) / 100;
            System.out.println("Rate of interest = " + r + "%");
            System.out.println("Interest         = Rs " + interest);
            System.out.println("Maturity amount  = Rs " + (p + interest));
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
p	double	to store the principal
days	int	to store the number of days
r	double	to store the rate of interest
interest	double	to store the interest earned

Q61 Taxi Fare by Distance

A taxi service charges fares as follows (slab by slab):

```
Distance                    Fare
First 1 km                  Rs 25 (minimum fare)
Next 4 km (1 to 5 km)       Rs 15 per km
Next 5 km (5 to 10 km)      Rs 12 per km
Above 10 km                 Rs 10 per km
```

Write a program to input the distance travelled (in km) and the vehicle number, and display the vehicle number, the distance and the fare. Display "Invalid distance" for a zero or negative distance.

Q_61.java

```java
import java.util.Scanner;

public class Q_61
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the vehicle number: ");
        String vno = sc.next();
        System.out.print("Enter the distance (km): ");
        double km = sc.nextDouble();
        double fare;

        if (km <= 0)
        {
            System.out.println("Invalid distance");
        }
        else
        {
            if (km <= 1)
                fare = 25;
            else if (km <= 5)
                fare = 25 + (km - 1) * 15;
            else if (km <= 10)
                fare = 25 + 4 * 15 + (km - 5) * 12;
            else
                fare = 25 + 4 * 15 + 5 * 12 + (km - 10) * 10;
            System.out.println("Vehicle number : " + vno);
            System.out.println("Distance       : " + km + " km");
            System.out.println("Fare           : Rs " + fare);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
vno	String	to store the vehicle number
km	double	to store the distance travelled
fare	double	to store the fare

Q62 Leap Year Check

Write a program to accept a year and check whether it is a leap year or not. A year is a leap year if it is divisible by 400, or if it is divisible by 4 but not divisible by 100. Display "Leap Year" or "Not a Leap Year". Display "Invalid year" for a zero or negative year.

Q_62.java

```java
import java.util.Scanner;

public class Q_62
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a year: ");
        int y = sc.nextInt();

        if (y <= 0)
            System.out.println("Invalid year");
        else if (y % 400 == 0)
            System.out.println("Leap Year");
        else if (y % 100 == 0)
            System.out.println("Not a Leap Year");
        else if (y % 4 == 0)
            System.out.println("Leap Year");
        else
            System.out.println("Not a Leap Year");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
y	int	to store the year

Q63 Largest of Three Numbers using Nested if

Write a program to accept three integers and find the largest among them using nested if statements. If all three numbers are equal, display "All numbers are equal".

Q_63.java

```java
import java.util.Scanner;

public class Q_63
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter three integers: ");
        int a = sc.nextInt();
        int b = sc.nextInt();
        int c = sc.nextInt();

        if (a == b && b == c)
        {
            System.out.println("All numbers are equal");
        }
        else
        {
            int large;
            if (a >= b)
            {
                if (a >= c)
                    large = a;
                else
                    large = c;
            }
            else
            {
                if (b >= c)
```

```
                    large = b;
                else
                    large = c;
            }
            System.out.println("Largest number = " + large);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	int	to store the first number
b	int	to store the second number
c	int	to store the third number
large	int	to store the largest number

Q64 Triangle Validity and Type by Sides

Write a program to accept the three sides of a triangle and check whether the triangle is possible. A triangle is possible only if all sides are positive and the sum of any two sides is greater than the third. If it is possible, display whether it is "Equilateral Triangle" (all sides equal), "Isosceles Triangle" (exactly two sides equal) or "Scalene Triangle" (all sides different). Otherwise display "Triangle not possible".

Q_64.java

```java
import java.util.Scanner;

public class Q_64
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the three sides: ");
        double a = sc.nextDouble();
        double b = sc.nextDouble();
        double c = sc.nextDouble();

        if (a > 0 && b > 0 && c > 0 && a + b > c && b + c > a && a + c > b)
        {
            if (a == b && b == c)
                System.out.println("Equilateral Triangle");
            else if (a == b || b == c || a == c)
                System.out.println("Isosceles Triangle");
            else
                System.out.println("Scalene Triangle");
        }
        else
        {
            System.out.println("Triangle not possible");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	double	to store the first side
b	double	to store the second side

Name of the Variable	Data Type	Purpose / Description
c	double	to store the third side

Q65 Odd or Even Number

> Write a program to accept an integer and check whether it is even or odd. The program must work correctly for negative numbers and zero. Display "Even" or "Odd".

Q_65.java

```java
import java.util.Scanner;

public class Q_65
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter an integer: ");
        int n = sc.nextInt();

        if (n % 2 == 0)
            System.out.println("Even");
        else
            System.out.println("Odd");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number

Q66 Positive, Negative or Zero

> Write a program to accept a number and check whether it is positive, negative or zero.

Q_66.java

```java
import java.util.Scanner;

public class Q_66
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        double n = sc.nextDouble();

        if (n > 0)
            System.out.println("Positive");
        else if (n < 0)
            System.out.println("Negative");
        else
            System.out.println("Zero");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	double	to store the number

Q67 Eligibility to Vote

> Write a program to accept the name and the age of a person and check whether the person is eligible to vote (age 18 years or above). Display "<name> is eligible to vote" or "<name> is not eligible to vote". Display "Invalid age" for a negative age.

Q_67.java

```java
import java.util.Scanner;

public class Q_67
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the name: ");
        String name = sc.nextLine();
        System.out.print("Enter the age: ");
        int age = sc.nextInt();

        if (age < 0)
            System.out.println("Invalid age");
        else if (age >= 18)
            System.out.println(name + " is eligible to vote");
        else
            System.out.println(name + " is not eligible to vote");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
name	String	to store the name
age	int	to store the age

Q68 Uppercase, Lowercase, Digit or Special Character

> Write a program to accept a character and check whether it is an uppercase letter, a lowercase letter, a digit or a special character, using relational operators on the character (without using any library method).

Q_68.java

```java
import java.util.Scanner;

public class Q_68
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a character: ");
        char ch = sc.next().charAt(0);

        if (ch >= 'A' && ch <= 'Z')
            System.out.println("Uppercase letter");
        else if (ch >= 'a' && ch <= 'z')
            System.out.println("Lowercase letter");
        else if (ch >= '0' && ch <= '9')
            System.out.println("Digit");
        else
            System.out.println("Special character");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
ch	char	to store the entered character

Q69 Buzz Number

Write a program to accept a number and check whether it is a Buzz number or not. A Buzz number is a number which is either divisible by 7 or ends with the digit 7.

Example: 63 (divisible by 7), 107 (ends with 7) and 147 are Buzz numbers.

Q_69.java

```
import java.util.Scanner;

public class Q_69
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        int last = Math.abs(n % 10);

        if (n != 0 && (n % 7 == 0 || last == 7))
            System.out.println("Buzz Number");
        else
            System.out.println("Not a Buzz Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number
last	int	to store the last digit

Q70 Divisibility by 5 and 11

Write a program to accept an integer and check whether it is divisible by both 5 and 11, by 5 only, by 11 only, or by neither. Display one of the messages: "Divisible by 5 and 11", "Divisible by 5 only", "Divisible by 11 only" or "Not divisible by 5 or 11".

Q_70.java

```
import java.util.Scanner;

public class Q_70
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter an integer: ");
        int n = sc.nextInt();

        if (n % 5 == 0 && n % 11 == 0)
            System.out.println("Divisible by 5 and 11");
        else if (n % 5 == 0)
            System.out.println("Divisible by 5 only");
        else if (n % 11 == 0)
            System.out.println("Divisible by 11 only");
        else
```

```
            System.out.println("Not divisible by 5 or 11");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number

Q71 Profit or Loss and its Percentage

Write a program to accept the cost price and the selling price of an article and determine whether the seller has made a profit or a loss, and the percentage of profit or loss. If there is neither, display "No profit, no loss".

`Profit% = (Profit / Cost price) x 100` `Loss% = (Loss / Cost price) x 100`

Display "Invalid input" if the cost price is zero or negative, or the selling price is negative.

Q_71.java

```java
import java.util.Scanner;

public class Q_71
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter cost price and selling price: ");
        double cp = sc.nextDouble();
        double sp = sc.nextDouble();

        if (cp <= 0 || sp < 0)
        {
            System.out.println("Invalid input");
        }
        else if (sp > cp)
        {
            double profit = sp - cp;
            System.out.println("Profit = Rs " + profit + ", Profit% = " + (profit / cp * 100));
        }
        else if (sp < cp)
        {
            double loss = cp - sp;
            System.out.println("Loss = Rs " + loss + ", Loss% = " + (loss / cp * 100));
        }
        else
        {
            System.out.println("No profit, no loss");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
cp	double	to store the cost price
sp	double	to store the selling price
profit	double	to store the profit
loss	double	to store the loss

Q72 Quadrant of a Point

Write a program to accept the coordinates (x, y) of a point and display its position: "Origin", "On the X-axis", "On the Y-axis", "First quadrant", "Second quadrant", "Third quadrant" or "Fourth quadrant".

Q_72.java

```java
import java.util.Scanner;

public class Q_72
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter x and y: ");
        double x = sc.nextDouble();
        double y = sc.nextDouble();

        if (x == 0 && y == 0)
            System.out.println("Origin");
        else if (y == 0)
            System.out.println("On the X-axis");
        else if (x == 0)
            System.out.println("On the Y-axis");
        else if (x > 0 && y > 0)
            System.out.println("First quadrant");
        else if (x < 0 && y > 0)
            System.out.println("Second quadrant");
        else if (x < 0 && y < 0)
            System.out.println("Third quadrant");
        else
            System.out.println("Fourth quadrant");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
x	double	to store the x-coordinate
y	double	to store the y-coordinate

Q73 Menu with an Exit Option using System.exit(0)

Write a menu-driven program with the following options:

```
1. Square of a number
2. Cube of a number
3. Exit
```

For choices 1 and 2, accept a number and display the result. For choice 3, display "Program terminated" and stop the program immediately using System.exit(0). For any other choice, display "Wrong choice". After the switch statement, display "Thank you" (this line must not appear when the user chooses Exit).

Q_73.java

```java
import java.util.Scanner;

public class Q_73
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("1. Square of a number");
        System.out.println("2. Cube of a number");
        System.out.println("3. Exit");
```

```
        System.out.print("Enter your choice: ");
        int ch = sc.nextInt();
        long n;

        switch (ch)
        {
            case 1:
                System.out.print("Enter a number: ");
                n = sc.nextLong();
                System.out.println("Square = " + (n * n));
                break;
            case 2:
                System.out.print("Enter a number: ");
                n = sc.nextLong();
                System.out.println("Cube = " + (n * n * n));
                break;
            case 3:
                System.out.println("Program terminated");
                System.exit(0);
                break;
            default:
                System.out.println("Wrong choice");
        }
        System.out.println("Thank you");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
ch	int	to store the user's choice
n	long	to store the number

Q74 Rock, Paper, Scissors against the Computer

Extra Write a program to play one round of Rock-Paper-Scissors against the computer. The user enters R, P or S. The computer's choice is generated randomly using Math.random() (0 = Rock, 1 = Paper, 2 = Scissors). Rock beats Scissors, Scissors beats Paper and Paper beats Rock. Display both choices and the result: "You win!", "Computer wins!" or "It's a draw!". Display "Invalid choice" for any other input.

Q_74.java

```
import java.util.Scanner;

public class Q_74
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter R (Rock), P (Paper) or S (Scissors): ");
        char u = sc.next().charAt(0);
        int user;

        switch (u)
        {
            case 'R': case 'r': user = 0; break;
            case 'P': case 'p': user = 1; break;
            case 'S': case 's': user = 2; break;
            default: user = -1;
        }

        if (user == -1)
        {
            System.out.println("Invalid choice");
```

```
        }
        else
        {
            int comp = (int) (Math.random() * 3);
            String un = (user == 0) ? "Rock" : (user == 1) ? "Paper" : "Scissors";
            String cn = (comp == 0) ? "Rock" : (comp == 1) ? "Paper" : "Scissors";
            System.out.println("You chose      : " + un);
            System.out.println("Computer chose : " + cn);

            if (user == comp)
                System.out.println("It's a draw!");
            else if ((user == 0 && comp == 2) || (user == 2 && comp == 1) || (user == 1 && comp == 0))
                System.out.println("You win!");
            else
                System.out.println("Computer wins!");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
u	char	to store the user's letter
user	int	to store the user's choice as a number
comp	int	to store the computer's choice
un	String	to store the name of the user's choice
cn	String	to store the name of the computer's choice

Q75 Zodiac Sign from Date of Birth

Extra Write a program to accept the day and month of birth and display the zodiac (sun) sign according to the table below. Display "Invalid date" if the day or month is not valid (ignore leap years; take February as 29 days).

```
Aries        21 Mar - 19 Apr    Libra        23 Sep - 22 Oct
Taurus       20 Apr - 20 May    Scorpio      23 Oct - 21 Nov
Gemini       21 May - 20 Jun    Sagittarius  22 Nov - 21 Dec
Cancer       21 Jun - 22 Jul    Capricorn    22 Dec - 19 Jan
Leo          23 Jul - 22 Aug    Aquarius     20 Jan - 18 Feb
Virgo        23 Aug - 22 Sep    Pisces       19 Feb - 20 Mar
```

Q_75.java

```
import java.util.Scanner;

public class Q_75
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter day and month of birth: ");
        int d = sc.nextInt();
        int m = sc.nextInt();
        int maxDay;

        switch (m)
        {
            case 2:
                maxDay = 29;
                break;
            case 4: case 6: case 9: case 11:
                maxDay = 30;
                break;
            case 1: case 3: case 5: case 7: case 8: case 10: case 12:
```

```
                maxDay = 31;
                break;
            default:
                maxDay = 0;
        }

        String sign = "";
        if (d < 1 || d > maxDay)
            sign = "Invalid date";
        else if ((m == 3 && d >= 21) || (m == 4 && d <= 19))
            sign = "Aries";
        else if ((m == 4 && d >= 20) || (m == 5 && d <= 20))
            sign = "Taurus";
        else if ((m == 5 && d >= 21) || (m == 6 && d <= 20))
            sign = "Gemini";
        else if ((m == 6 && d >= 21) || (m == 7 && d <= 22))
            sign = "Cancer";
        else if ((m == 7 && d >= 23) || (m == 8 && d <= 22))
            sign = "Leo";
        else if ((m == 8 && d >= 23) || (m == 9 && d <= 22))
            sign = "Virgo";
        else if ((m == 9 && d >= 23) || (m == 10 && d <= 22))
            sign = "Libra";
        else if ((m == 10 && d >= 23) || (m == 11 && d <= 21))
            sign = "Scorpio";
        else if ((m == 11 && d >= 22) || (m == 12 && d <= 21))
            sign = "Sagittarius";
        else if ((m == 12 && d >= 22) || (m == 1 && d <= 19))
            sign = "Capricorn";
        else if ((m == 1 && d >= 20) || (m == 2 && d <= 18))
            sign = "Aquarius";
        else
            sign = "Pisces";
        System.out.println(sign);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
d	int	to store the day of birth
m	int	to store the month of birth
maxDay	int	to store the days in that month
sign	String	to store the zodiac sign

Q76 Valid Date Checker

Extra Write a program to accept a date as three integers (day, month and year) and check whether it is a valid date. Take leap years into account (a year is a leap year if it is divisible by 400, or divisible by 4 but not by 100). Display "Valid Date" or "Invalid Date". Years must be positive.

Q_76.java

```
import java.util.Scanner;

public class Q_76
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter day, month and year: ");
        int d = sc.nextInt();
```

```
        int m = sc.nextInt();
        int y = sc.nextInt();
        int maxDay = 0;

        switch (m)
        {
            case 1: case 3: case 5: case 7: case 8: case 10: case 12:
                maxDay = 31;
                break;
            case 4: case 6: case 9: case 11:
                maxDay = 30;
                break;
            case 2:
                if ((y % 4 == 0 && y % 100 != 0) || y % 400 == 0)
                    maxDay = 29;
                else
                    maxDay = 28;
                break;
            default:
                maxDay = 0;
        }

        if (y > 0 && d >= 1 && d <= maxDay)
            System.out.println("Valid Date");
        else
            System.out.println("Invalid Date");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
d	int	to store the day
m	int	to store the month
y	int	to store the year
maxDay	int	to store the days in the month

Q77 BMI Calculator with Category

Extra Body Mass Index (BMI) = weight (kg) / [height (m)]^2. Write a program to accept the weight in kilograms and the height in centimetres, calculate the BMI (rounded to one decimal place) and display the category:

```
BMI below 18.5          Underweight
18.5 to below 25        Normal weight
25 to below 30          Overweight
30 and above            Obese
```

Display "Invalid input" if the weight or height is zero or negative.

Q_77.java

```
import java.util.Scanner;

public class Q_77
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter weight (kg): ");
        double w = sc.nextDouble();
        System.out.print("Enter height (cm): ");
        double hcm = sc.nextDouble();

        if (w <= 0 || hcm <= 0)
```

```
        {
            System.out.println("Invalid input");
        }
        else
        {
            double h = hcm / 100;
            double bmi = Math.round(w / (h * h) * 10) / 10.0;
            System.out.println("BMI = " + bmi);
            if (bmi < 18.5)
                System.out.println("Underweight");
            else if (bmi < 25)
                System.out.println("Normal weight");
            else if (bmi < 30)
                System.out.println("Overweight");
            else
                System.out.println("Obese");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
w	double	to store the weight in kg
hcm	double	to store the height in cm
h	double	to store the height in metres
bmi	double	to store the body mass index

CHAPTER

07



Iterative Constructs in Java

Need for loops · Entry and exit controlled loops · `for` · `while` · `do-while` · Variations · `break` and `continue` · Inter-conversion · Number programs

54 programs | Q78 – Q131 | 8 extra

Theory

📑 Contents

1. What is a loop?
2. Parts of a loop
3. The `for` loop
4. The `while` loop
5. The `do-while` loop
6. Entry-controlled vs exit-controlled loops
7. Variations in looping statements
8. Jump statements: `break` and `continue`
9. Inter-conversion of loops
10. Counting iterations and dry runs
11. Digit extraction — the heart of number programs
12. Special numbers — definitions table
13. Predict the output
14. Find the errors
15. Differentiate between
16. One-line facts
17. Test yourself
18. Programs in this chapter

1. What is a loop?

An **iterative construct (loop)** is a statement that **executes a set of statements repeatedly** as long as a given condition is true. Each execution of the loop body is called an **iteration**.

Why loops are needed: to print the numbers 1 to 1000 without a loop, 1000 `println` statements would be needed. A loop does it in three lines:

```
for (int i = 1; i <= 1000; i++)
    System.out.println(i);
```

Java has three looping statements: **`for`**, **`while`** and **`do-while`**.

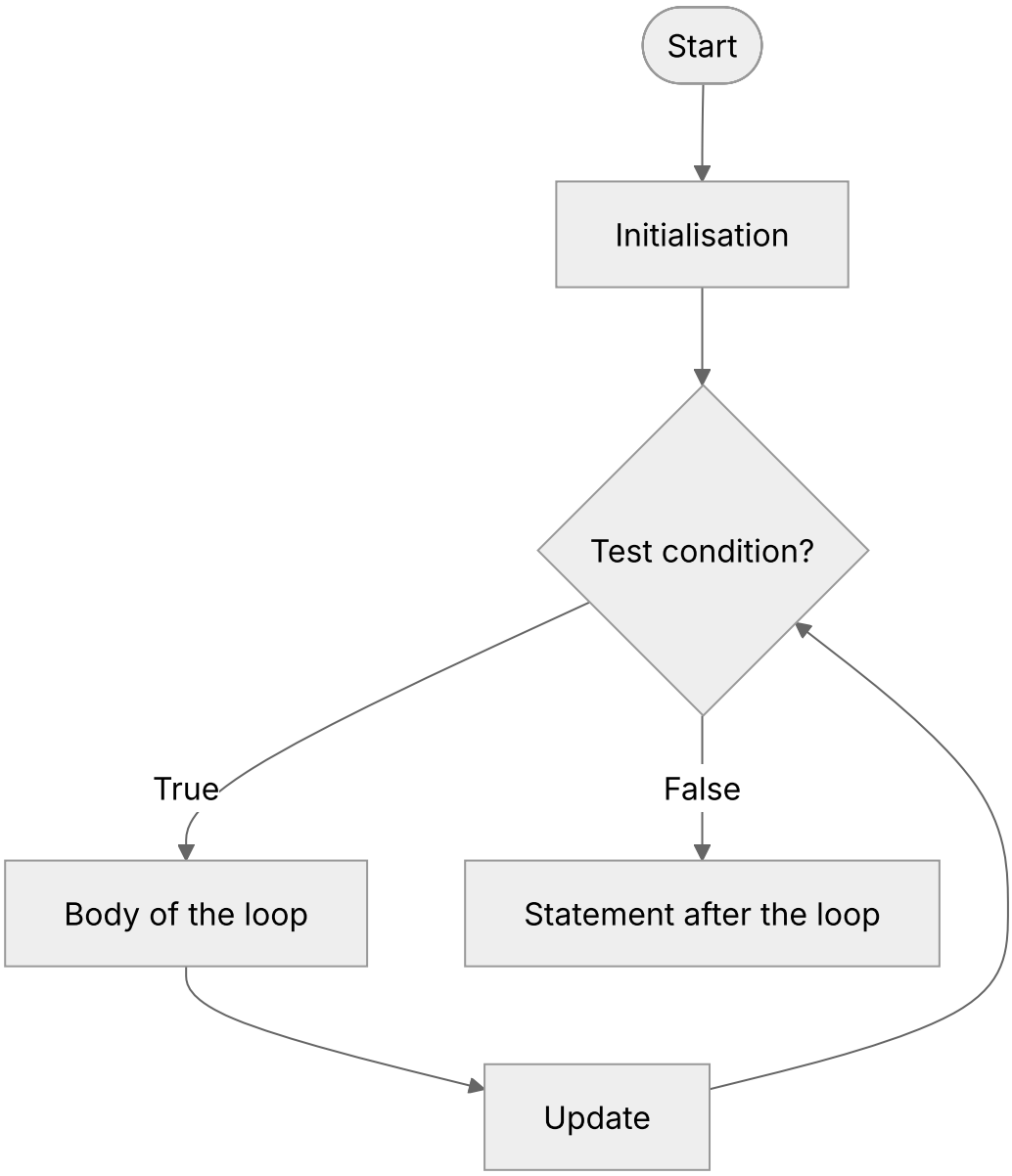
2. Parts of a loop

Part	Purpose	In `for (int i = 1; i <= 10; i++)`
Initialisation	Gives the starting value to the **control variable** (loop counter) — executed **once**	`int i = 1`
Test condition	A boolean expression checked before/after every iteration; the loop continues while it is `true`	`i <= 10`
Body of the loop	The statement(s) executed repeatedly	the statement after the `for`
Update expression	Changes the control variable so that the condition eventually becomes `false`	`i++`

3. The `for` loop

An **entry-controlled** loop in which initialisation, test condition and update are written **together in one line**. It is used when the **number of iterations is known** in advance.

```
for (initialisation; test condition; update)
{
    body;
}
```



Order of execution: initialisation (once) → condition → body → update → condition → body → update → … → condition false → exit.

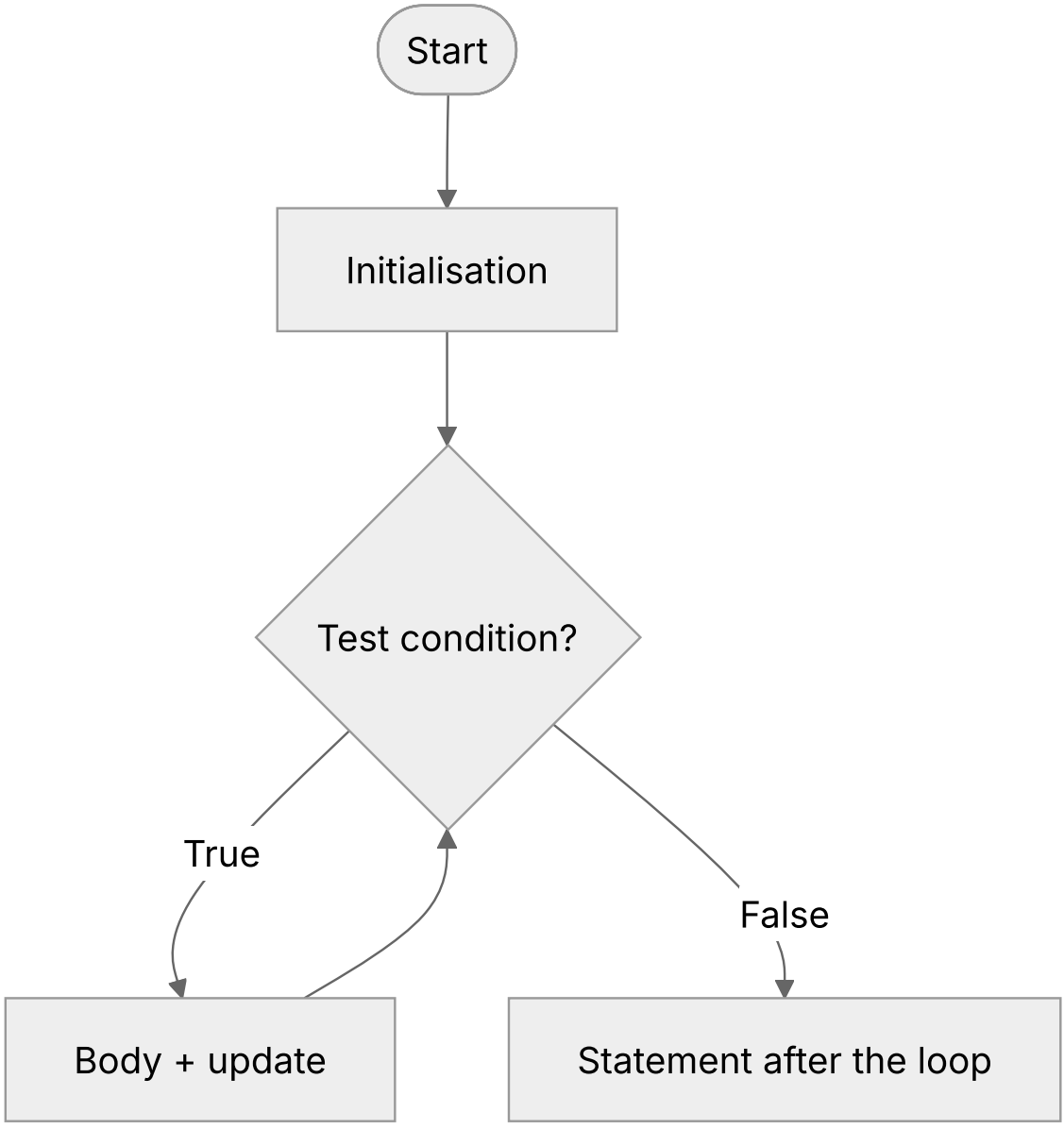
```
int sum = 0;
for (int i = 1; i <= 5; i++)
    sum = sum + i;          // sum = 15
```

> **NOTE**
>
> A variable declared inside the `for` header (`int i`) is **local to the loop** — it cannot be used after the loop ends.

4. The `while` loop

An **entry-controlled** loop that tests the condition **before** each iteration. It is generally used when the **number of iterations is not known** in advance (e.g. "repeat while the number is greater than 0").

```
initialisation;
while (test condition)
{
    body;
    update;
}
```

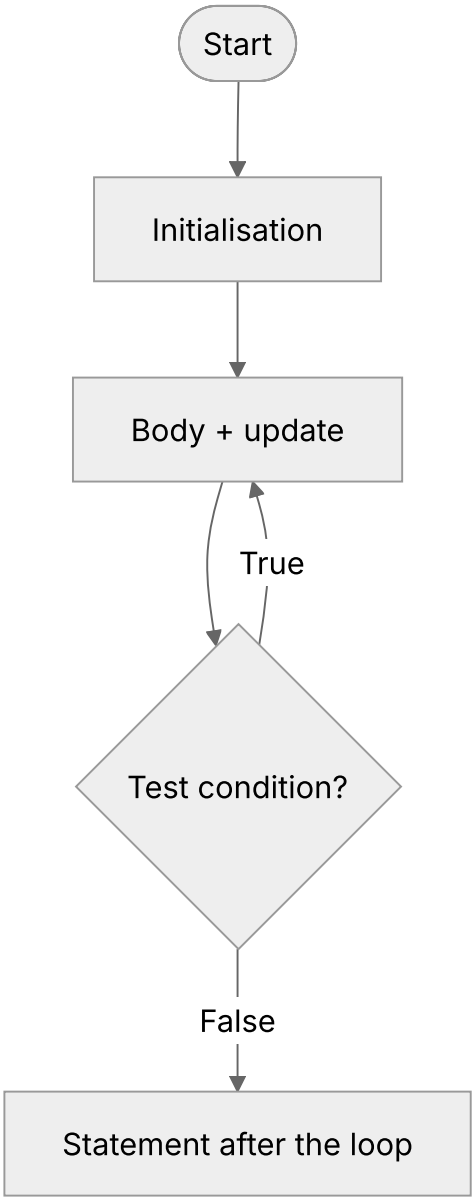



```
int n = 4072, count = 0;
while (n > 0)
{
    count++;
    n = n / 10;
}                           // count = 4
```

5. The `do-while` loop

An **exit-controlled** loop: the body is executed **first** and the condition is tested **afterwards**. Hence the body executes **at least once**, even if the condition is false at the start. Note the **semicolon** after `while (…)`.

```
initialisation;
do
{
    body;
    update;
} while (test condition);
```




```
int i = 10;
do
{
    System.out.println(i);   // prints 10 once, although 10 > 5
    i++;
} while (i <= 5);
```

> **TIP**
>
> `do-while` is the natural choice for **menu-driven programs** — the menu must be shown at least once, then repeated until the user chooses Exit.

6. Entry-controlled vs exit-controlled loops

Basis	Entry-controlled (`for`, `while`)	Exit-controlled (`do-while`)
Condition tested	**Before** executing the body	**After** executing the body
Minimum executions	**0** (if the condition is false at the start)	**1**
Semicolon after condition	Not used	Required: `while (cond);`
Example use	Counting loops, digit extraction	Menus, input validation

Basis	`for` loop	`while` loop
Structure	Initialisation, condition and update in one line	Written separately
Usually used when	The number of iterations is **known**	The number of iterations is **not known**
Missing condition	`for (;;)` is an infinite loop	`while ()` is a **syntax error**

7. Variations in looping statements

Variation	Example	Notes
Multiple initialisations and updates (comma-separated)	`for (int i = 1, j = 10; i < j; i++, j--)`	Only **one** condition allowed (combine with `&&`)
Omitting parts	`int i = 1; for ( ; i <= 5; ) { …; i++; }`	Semicolons must remain
Infinite loop	`for ( ; ; )`, `while (true)`, `do { } while (true);`	Ends only with `break`, `return` or `System.exit()`
Accidental infinite loop	`for (int i = 1; i <= 10; i--)`	Update moves away from the condition
Empty loop (null loop)	`for (i = 1; i <= 5; i++);`	Semicolon makes the body empty
Delay (time-delay) loop	`for (long k = 1; k <= 100000000L; k++);`	Used to pause the program for a while
Finite loop	`for (i = 10; i >= 1; i--)`	Terminates after a fixed number of iterations
Step value other than 1	`for (i = 2; i <= 20; i += 2)`	Even numbers
Real control variable	`for (double x = 0.0; x < 1.0; x += 0.25)`	Allowed, but beware of rounding

WARNING

```
int i;
for (i = 1; i <= 5; i++);      // ← stray semicolon: empty loop
    System.out.println(i);     // runs ONCE, prints 6
```

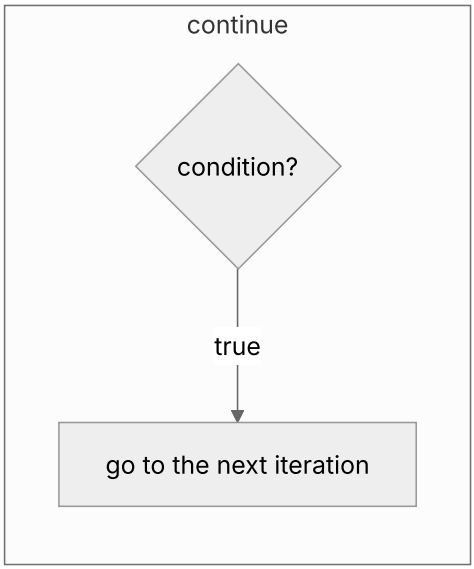
8. Jump statements: `break` and `continue`

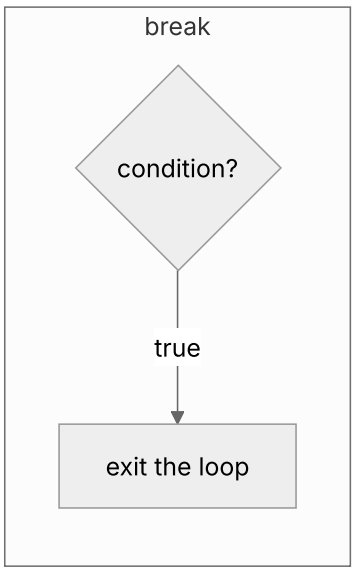
Jump statements transfer control unconditionally to another part of the program.

- `break` — **terminates the loop** (or switch) immediately; control passes to the statement **after the loop**.
- `continue` — **skips the rest of the current iteration** and moves to the **next iteration** (to the update in a `for` loop, to the condition in `while` / `do-while`).

```
for (int i = 1; i <= 10; i++)
{
    if (i == 6)
        break;
    System.out.print(i + " ");      // 1 2 3 4 5
}

for (int i = 1; i <= 10; i++)
{
    if (i % 3 == 0)
        continue;
    System.out.print(i + " ");      // 1 2 4 5 7 8 10
}
```





Basis	`break`	`continue`
Effect	Terminates the loop	Skips the remaining statements of the current iteration
Control goes to	First statement after the loop	Next iteration (update/condition)
Used in	Loops and `switch`	Loops only

WARNING

In a `while` loop, put the update **before** `continue`, otherwise the loop may become infinite:

```
int i = 0;
while (i < 10) { if (i % 2 == 0) continue; i++; }   // infinite: i stays 0
```

9. Inter-conversion of loops

Program segment: sum of the even numbers from 2 to 20.

`for`

```
int s = 0;
for (int i = 2; i <= 20; i += 2)
{
    s = s + i;
}
```

`while`

```
int s = 0, i = 2;
while (i <= 20)
{
    s = s + i;
    i += 2;
}
```

`do-while`

```
int s = 0, i = 2;
do
{
    s = s + i;
    i += 2;
} while (i <= 20);
```

> **IMPORTANT**
>
> A `while` loop and its `do-while` version give **different results** when the condition is false at the start (`do-while` still runs once). When converting, check the first iteration.

Convert into `for` :

```
int x = 100;
while (x > 0)
{
    System.out.print(x % 10);
    x /= 10;
}
```

```
for (int x = 100; x > 0; x /= 10)
    System.out.print(x % 10);      // prints 001
```

10. Counting iterations and dry runs

Loop	Iterations
`for (i = 1; i <= 10; i++)`	10
`for (i = 0; i < 10; i++)`	10
`for (i = 10; i >= 1; i -= 3)`	4 (10, 7, 4, 1)
`for (i = 1; i <= 100; i *= 2)`	7 (1, 2, 4, 8, 16, 32, 64)
`for (i = 5; i < 5; i++)`	0
`int i = 5; do { … } while (i < 5);`	1
`while (n > 0) n /= 10;` with n = 98765	5

Dry run: `int n = 1234, s = 0; while (n > 0) { s = s * 10 + n % 10; n /= 10; }`

Iteration	`n % 10`	`s`	`n` after
1	4	4	123
2	3	43	12
3	2	432	1
4	1	4321	0

11. Digit extraction — the heart of number programs

```
while (n > 0)
{
    int d = n % 10;     // extract the LAST digit
    // ... use d ...
    n = n / 10;         // remove the last digit
}
```

Task	Statement inside the loop	Initial value
Count the digits	`count++;`	`count = 0`
Sum of digits	`sum = sum + d;`	`sum = 0`
Product of digits	`prod = prod * d;`	`prod = 1`
Reverse	`rev = rev * 10 + d;`	`rev = 0`
Largest digit	`if (d > max) max = d;`	`max = 0`
Count a digit	`if (d == key) c++;`	`c = 0`

> **TIP**
>
> **Always work on a copy** (`int copy = n;`) when the original number is needed later for comparison (palindrome, Armstrong, etc.). Use `long` when squares or large reverses are involved, to avoid overflow.

Other useful techniques

- **First digit:** divide by 10 until `n < 10`. **Number of digits:** `count` loop above.
- **Factors:** `for (i = 1; i <= n; i++) if (n % i == 0) …`
- **Prime test:** count factors = 2, or check divisors from 2 to $\sqrt{n}$ with a flag and `break`.
- **Binary digits:** `n % 2` and `n / 2` repeatedly.

12. Special numbers — definitions table

Number	Definition	Example
Prime	Exactly two factors: 1 and itself	2, 13, 97
Composite	More than two factors	4, 15
Perfect	Sum of factors (excluding itself) = number	6, 28, 496
Palindrome	Same when reversed	121, 4554
Armstrong	Sum of digits each raised to the number of digits = number	153, 1634
Spy	Sum of digits = product of digits	1124, 22
Niven / Harshad	Divisible by the sum of its digits	126, 18
Neon	Sum of digits of its square = number	9 (81)
Automorphic	Square ends with the number	25 (625), 76 (5776)
Special / Krishnamurthy	Sum of factorials of digits = number	145, 40585
Special two-digit	(sum of digits) + (product of digits) = number	59

Number	Definition	Example
Duck	Contains a zero, but not as the first digit	3210, 7056
Pronic	Product of two consecutive integers	12 (3 × 4), 42
Disarium	Sum of digits raised to their positions = number	135, 175
Kaprekar	Square split into two parts that add up to the number	45 (20 + 25), 297
Tech	Even digits; (sum of the two halves)² = number	2025, 3025
Magic	Repeated digit sum ends in 1	19, 199
Happy	Repeated sum of squares of digits ends in 1	7, 19
Buzz	Divisible by 7 or ends with 7	63, 107
Emirp	Number and its reverse are both prime	13, 107
Sunny	n + 1 is a perfect square	8, 24
Evil	Binary form has an even number of 1s	9 (1001)
Unique	No digit repeats	3412
Abundant / Deficient	Sum of proper factors > / < number	12 / 8
Twin primes	Two primes differing by 2	(11, 13)
Smith *(extra)*	Composite; digit sum = digit sum of prime factors	666, 22

13. Predict the output

Example 1

```
int i, s = 0;
for (i = 1; i <= 10; i += 3)
    s += i;
System.out.println(i + " " + s);
```

Show dry run

i	s
1	1
4	5
7	12
10	22
13	loop ends

Output: `13 22`

Example 2

```
int n = 10;
do
{
    n -= 3;
```

```
    System.out.print(n + " ");
} while (n > 0);
```

Show answer

`7 4 1 -2` (the condition is tested after printing −2)

Example 3

```
int x = 5, y = 0;
while (x-- > 0)
    y += x;
System.out.println(x + " " + y);
```

Show dry run

test (`x--` value)	x after test	y
5 > 0 ✔	4	4
4 > 0 ✔	3	7
3 > 0 ✔	2	9
2 > 0 ✔	1	10
1 > 0 ✔	0	10
0 > 0 ✘	−1	—

Output: `-1 10`

Example 4

```
for (int a = 1, b = 10; a < b; a += 2, b -= 2)
    System.out.print(a * b + " ");
```

Show answer

(1,10) → 10; (3,8) → 24; (5,6) → 30; (7,4) stop. **Output:** `10 24 30`

Example 5

```
int k = 1;
while (k <= 20)
{
    k *= 3;
    if (k % 2 == 0) continue;
    System.out.print(k + " ");
}
```

Show answer

k = 3, 9, 27 (all odd). After 27 the loop ends. **Output:** `3 9 27`

14. Find the errors

```
int i = 1, sum = 0
while (i <= 10);
{
    sum = sum + i;
}
for (int j = 1, j <= 5, j++)
    System.out.println(j);
do
    System.out.println(i++);
while (i < 15)
```

Show corrected code

```
int i = 1, sum = 0;              // missing semicolon
while (i <= 10)                  // stray semicolon caused an infinite empty loop
{
    sum = sum + i;
    i++;                         // update was missing: infinite loop
}
for (int j = 1; j <= 5; j++)     // semicolons, not commas, separate the three parts
    System.out.println(j);
do
    System.out.println(i++);
while (i < 15);                  // do-while needs a semicolon
```

15. Differentiate between

Pair	Difference
`for` vs `while`	Known vs unknown number of iterations; all three parts in one line vs separate
`while` vs `do-while`	Entry vs exit controlled; minimum 0 vs 1 execution
`break` vs `continue`	Ends the loop vs skips to the next iteration
Finite vs infinite loop	Terminates after some iterations vs never terminates on its own
Counter vs accumulator	Counts by a fixed step vs adds varying values
Empty loop vs infinite loop	Has no body but ends vs has a body/no body and never ends

16. One-line facts

- `for` and `while` are **entry-controlled**; `do-while` is **exit-controlled**.
- The body of a `do-while` loop executes **at least once**.
- A `do-while` statement ends with a **semicolon**.
- `for( ; ; )` is an **infinite** loop.
- `break` takes control **out of the loop**; `continue` takes it to the **next iteration**.
- In a `for` loop, `continue` jumps to the **update** expression.
- The control variable of a `for` loop declared in its header is **local** to the loop.
- `n % 10` gives the **last digit**; `n / 10` **removes** it.

🧠 Test Yourself

Part A — Multiple choice

1. How many times does `Hello` print? `for (int i = 10; i > 0; i -= 4) System.out.println("Hello");`
   (a) 2 (b) 3 (c) 4 (d) 10
2. `java int s = 0; for (int i = 1; i <= 5; i++) { if (i == 3) continue; if (i == 5) break; s += i; }` `s` =
   (a) 15 (b) 7 (c) 10 (d) 12
3. *Assertion (A):* `int x = 9; do { x++; } while (x < 5);` leaves `x` as 10. *Reason (R):* A `do-while` loop executes its body at least once. (a) Both true; R explains A (b) Both true; R doesn't explain A (c) A true, R false (d) A false, R true
4. What is the output? `int n = 0; while (n < 5) n += 2; System.out.print(n);` (a) 4 (b) 5 (c) 6 (d) 8
5. Which loop is **infinite**? (a) `for (int i = 0; i < 5; i += 2);` (b) `for (int i = 1; i != 10; i += 2)` (c) `int i = 5; while (i > 0) i--;` (d) `int i = 1; do i *= 2; while (i < 100);`
6. What does this print? `int a = 3; for (a += 2; a < 10; a += 3) System.out.print(a);` (a) 58 (b) 3 6 9 (c) 5 8 (d) 47
7. `int c = 0; for (int i = 1; i <= 4; i++) for (int j = i; j <= 4; j++) c++;` — `c` = (a) 16 (b) 10 (c) 4 (d) 6
8. The digit loop `while (n != 0) { d = n % 10; n = n / 10; }` is run with `n = -425`. Which is correct? (a) Infinite loop (b) The digits obtained are −5, −2, −4 (c) Runtime error (d) No digit is extracted
9. *Assertion (A):* In `for (int i = 1; i <= 5; i++) { … } System.out.println(i);` the last statement prints 6. *Reason (R):* After the loop ends, `i` holds the value that made the condition false. (a) Both true; R explains A (b) Both true; R doesn't explain A (c) A false, R true (d) Both false
10. Which pair of numbers are both Armstrong numbers? (a) 153, 370 (b) 145, 153 (c) 135, 371 (d) 407, 409

Part B — Short answer questions

1. Name the three loops of Java and classify them as entry or exit controlled.
2. Convert into a `do-while` loop: `for (int i = 5; i <= 25; i += 5) System.out.print(i + " ");`
3. Give the output and the number of iterations: `int x = 1; while (x < 50) x = x * 3 + 1; System.out.println(x);`
4. What is an infinite loop? Write one using each of the three loops.
5. What is the difference between `break` and `continue`?
6. Write a statement to extract the first digit of a positive integer `n` using a loop.
7. Give the output: `int i = 0; do { i += 2; System.out.print(i * i + " "); } while (i <= 4);`
8. What is a delay loop? Give an example.
9. Why is `long` used for the reverse of a large number?

✅ Answers — Part A

Q	Ans	Working
1	(b)	i = 10, 6, 2
2	(b)	1 + 2 + 4 = 7 (3 skipped, loop broken at 5)
3	(a)	—
4	(c)	0 → 2 → 4 → 6
5	(b)	`i` is always odd, so it can never be equal to 10
6	(a)	a = 5, 8 → prints `58`

Q	Ans	Working
7	(b)	4 + 3 + 2 + 1
8	(b)	`-425 % 10` = −5, etc.; the loop ends at 0
9	(c)	**A is false** — `i` is declared inside the `for` header, so `System.out.println(i)` is a compile-time error. R is true in general
10	(a)	153 and 370; 145 is a Special number; 135 is Disarium

✅ Answers — Part B

1. `for` and `while` — entry controlled; `do-while` — exit controlled.
2. `java int i = 5; do { System.out.print(i + " "); i += 5; } while (i <= 25);`
3. x: 1 → 4 → 13 → 40 → 121. Output `121`; 4 iterations.
4. A loop whose condition never becomes false. `for (;;) { }`, `while (true) { }`, `do { } while (true);`
5. `break` terminates the loop; `continue` skips the rest of the current iteration and starts the next one.
6. `while (n >= 10) n = n / 10;` — `n` is now the first digit.
7. `4 16 36` (i = 2, 4, 6; the condition is checked after printing).
8. A loop with an empty body used to create a time gap: `for (long k = 1; k <= 500000000L; k++);`
9. Because the reverse (or square) of a large `int` may exceed the range of `int`, causing overflow.

💻 Programs in this chapter

No.	Program	
Q78	Prime Number	
Q79	Palindrome Number	
Q80	Armstrong Number	
Q81	Perfect Number	
Q82	Spy Number	📝 ICSE 2017
Q83	Niven (Harshad) Number	
Q84	Neon Number	
Q85	Automorphic Number	
Q86	Special Number (Krishnamurthy Number)	
Q87	Special Two-Digit Number	📝 ICSE 2014
Q88	Duck Number	
Q89	Pronic Number	
Q90	Disarium Number	
Q91	Kaprekar Number	
Q92	Tech Number	
Q93	Magic Number	

No.	Program	
Q94	Happy Number	
Q95	Composite Number	
Q96	Emirp Number	
Q97	Sunny Number	
Q98	Evil Number	
Q99	Unique Number	
Q100	Abundant, Deficient or Perfect Number	
Q101	Fibonacci Series	
Q102	HCF and LCM of Two Numbers	
Q103	Factorial of a Number	
Q104	Sum and Product of Digits	
Q105	Reverse a Number and Count its Digits	
Q106	Largest and Smallest Digit of a Number	
Q107	Frequency of a Given Digit in a Number	
Q108	Count of Even and Odd Digits	
Q109	Factors of a Number and their Count	
Q110	Prime Factors of a Number	
Q111	Largest and Smallest of n Numbers	
Q112	Sum of Numbers until 0 is Entered	
Q113	Numbers from 1 to 50 not Divisible by 3 or 5 (continue)	
Q114	Sum of First n Natural Numbers and their Squares	
Q115	Sum of the Series 1 + 1/2 + 1/3 + ... + 1/n	
Q116	Sum of the Series x/1 + x^2/2 + x^3/3 + ... + x^n/n	
Q117	Sum of the Series 1 - 2 + 3 - 4 + ... n	
Q118	Sum of the Series a/2 + a/5 + a/8 + ... n Terms	
Q119	Series 1, 4, 9, 16, ... and 2, 5, 10, 17, 26, ...	
Q120	Series 0, 7, 26, 63, ...	
Q121	Series 1, 11, 111, 1111, ... and its Sum	
Q122	Series 1, -3, 5, -7, 9, ... and its Sum	
Q123	Looping Menu: Sum of Digits, Reverse, Count of Digits	
Q124	Decimal to Binary Conversion	🌟 *Extra*
Q125	Decimal to Octal Conversion	🌟 *Extra*
Q126	Decimal to Hexadecimal Conversion	🌟 *Extra*
Q127	Binary to Decimal Conversion	🌟 *Extra*
Q128	Collatz (Hailstone) Sequence	🌟 *Extra*

No.	Program	
Q129	Number Guessing Game	🌟 *Extra*
Q130	Reverse-and-Add until a Palindrome	🌟 *Extra*
Q131	Smith Number	🌟 *Extra*

Programs Q78 – Q131

Q78 Prime Number

> Write a program to accept a number and check whether it is a Prime number or not. A Prime number is a number that has exactly two factors, 1 and the number itself.
>
> Example: 2, 3, 5, 7, 11, 13 are Prime numbers. (1 is not prime.)

Q_78.java

```java
import java.util.Scanner;

public class Q_78
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        boolean isPrime = n > 1;

        for (long i = 2; i * i <= n; i++)
        {
            if (n % i == 0)
            {
                isPrime = false;
                break;
            }
        }

        if (isPrime)
            System.out.println("Prime Number");
        else
            System.out.println("Not a Prime Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
isPrime	boolean	to record whether n is prime
i	long	to act as the trial divisor

Q79 Palindrome Number

> Write a program to accept a number and check whether it is a Palindrome number or not. A Palindrome number is a number that remains the same when its digits are reversed.
>
> Example: 121, 4554, 7 are Palindrome numbers.

Q_79.java

```java
import java.util.Scanner;

public class Q_79
{
    public static void main(String[] args)
    {
```

```
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        long copy = n, rev = 0;

        while (copy > 0)
        {
            int d = (int) (copy % 10);
            rev = rev * 10 + d;
            copy = copy / 10;
        }

        if (n >= 0 && rev == n)
            System.out.println("Palindrome Number");
        else
            System.out.println("Not a Palindrome Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
copy	long	to store a working copy of n
rev	long	to store the reversed number
d	int	to store the extracted digit

Q80 Armstrong Number

> Write a program to accept a number and check whether it is an Armstrong number or not. An Armstrong number is a number that is equal to the sum of its digits, each raised to the power of the number of digits.
>
> Example: 153 = 1^3 + 5^3 + 3^3 and 1634 = 1^4 + 6^4 + 3^4 + 4^4

Q_80.java

```
import java.util.Scanner;

public class Q_80
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        long copy = n;
        int count = 0;

        while (copy > 0)
        {
            count++;
            copy = copy / 10;
        }

        long sum = 0;
        copy = n;
        while (copy > 0)
        {
            int d = (int) (copy % 10);
            sum = sum + (long) Math.pow(d, count);
            copy = copy / 10;
        }
```

```
        if (n > 0 && sum == n)
            System.out.println("Armstrong Number");
        else
            System.out.println("Not an Armstrong Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
copy	long	to store a working copy of n
count	int	to count the digits
sum	long	to store the sum of powers
d	int	to store the extracted digit

Q81 Perfect Number

> Write a program to accept a number and check whether it is a Perfect number or not. A Perfect number is a number that is equal to the sum of its factors excluding the number itself.
>
> Example: 6 = 1 + 2 + 3 and 28 = 1 + 2 + 4 + 7 + 14

Q_81.java

```
import java.util.Scanner;

public class Q_81
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        int sum = 0;

        for (int i = 1; i <= n / 2; i++)
        {
            if (n % i == 0)
                sum = sum + i;
        }

        if (n > 1 && sum == n)
            System.out.println("Perfect Number");
        else
            System.out.println("Not a Perfect Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number
sum	int	to store the sum of factors
i	int	to act as the possible factor

Q82 Spy Number

ICSE 2017 Write a program to accept a number and check whether it is a Spy number or not. A Spy number is a number in which the sum of its digits is equal to the product of its digits.

Example: 1124 → sum = 1 + 1 + 2 + 4 = 8, product = 1 × 1 × 2 × 4 = 8

Q_82.java

```java
import java.util.Scanner;

public class Q_82
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        long copy = n, sum = 0, prod = 1;

        while (copy > 0)
        {
            int d = (int) (copy % 10);
            sum = sum + d;
            prod = prod * d;
            copy = copy / 10;
        }

        if (n > 0 && sum == prod)
            System.out.println("Spy Number");
        else
            System.out.println("Not a Spy Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
copy	long	to store a working copy of n
sum	long	to store the sum of digits
prod	long	to store the product of digits
d	int	to store the extracted digit

Q83 Niven (Harshad) Number

Write a program to accept a number and check whether it is a Niven (Harshad) number or not. A Niven number is a number that is divisible by the sum of its digits.

Example: 126 → sum of digits = 1 + 2 + 6 = 9, and 126 is divisible by 9.

Q_83.java

```java
import java.util.Scanner;

public class Q_83
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
```

```
        long copy = n, sum = 0;

        while (copy > 0)
        {
            sum = sum + copy % 10;
            copy = copy / 10;
        }

        if (n > 0 && n % sum == 0)
            System.out.println("Niven Number");
        else
            System.out.println("Not a Niven Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
copy	long	to store a working copy of n
sum	long	to store the sum of digits

Q84 Neon Number

> Write a program to accept a number and check whether it is a Neon number or not. A Neon number is a number in which the sum of the digits of its square is equal to the number itself.
>
> Example: 9 → 9^2 = 81, and 8 + 1 = 9

Q_84.java

```
import java.util.Scanner;

public class Q_84
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        long sq = n * n, sum = 0;

        while (sq > 0)
        {
            sum = sum + sq % 10;
            sq = sq / 10;
        }

        if (n >= 0 && sum == n)
            System.out.println("Neon Number");
        else
            System.out.println("Not a Neon Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
sq	long	to store the square of n
sum	long	to store the sum of digits of the square

Q85 Automorphic Number

Write a program to accept a number and check whether it is an Automorphic number or not. An Automorphic number is a number whose square ends with the number itself.

Example: 25 → 625, 76 → 5776, 6 → 36

Q_85.java

```java
import java.util.Scanner;

public class Q_85
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        long sq = n * n;
        long copy = n, p = 1;

        while (copy > 0)
        {
            p = p * 10;
            copy = copy / 10;
        }

        if (n > 0 && sq % p == n)
            System.out.println("Automorphic Number");
        else
            System.out.println("Not an Automorphic Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
sq	long	to store the square of n
copy	long	to store a working copy of n
p	long	to store 10 raised to the digit count

Q86 Special Number (Krishnamurthy Number)

Write a program to accept a number and check whether it is a Special number (also called a Krishnamurthy number) or not. A Special number is a number in which the sum of the factorials of its digits is equal to the number itself.

Example: 145 = 1! + 4! + 5! = 1 + 24 + 120

Q_86.java

```java
import java.util.Scanner;

public class Q_86
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        long copy = n, sum = 0;
```

```
        while (copy > 0)
        {
            int d = (int) (copy % 10);
            long f = 1;
            for (int i = 1; i <= d; i++)
                f = f * i;
            sum = sum + f;
            copy = copy / 10;
        }

        if (n > 0 && sum == n)
            System.out.println("Special Number");
        else
            System.out.println("Not a Special Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
copy	long	to store a working copy of n
sum	long	to store the sum of factorials
d	int	to store the extracted digit
f	long	to store the factorial of a digit
i	int	to act as the loop counter

Q87 Special Two-Digit Number

ICSE 2014 Write a program to accept a two-digit number and check whether it is a Special two-digit number or not. A Special two-digit number is one in which the sum of the sum of its digits and the product of its digits is equal to the number itself. If the number is not a two-digit number, display "Invalid input".

Example: 59 → (5 + 9) + (5 × 9) = 14 + 45 = 59

Q_87.java

```
import java.util.Scanner;

public class Q_87
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a two-digit number: ");
        int n = sc.nextInt();

        if (n < 10 || n > 99)
        {
            System.out.println("Invalid input");
        }
        else
        {
            int first = n / 10;
            int last = n % 10;
            int sum = first + last;
            int prod = first * last;
            if (sum + prod == n)
                System.out.println("Special 2-digit Number");
            else
                System.out.println("Not a Special 2-digit Number");
```

```
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number
first	int	to store the tens digit
last	int	to store the units digit
sum	int	to store the sum of digits
prod	int	to store the product of digits

Q88 Duck Number

> Write a program to accept a positive number and check whether it is a Duck number or not. A Duck number is a positive number that contains at least one zero, but not as the first digit.
>
> Example: 3210, 7056, 50001 are Duck numbers.

Q_88.java

```java
import java.util.Scanner;

public class Q_88
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        long copy = n;
        boolean hasZero = false;

        while (copy > 0)
        {
            if (copy % 10 == 0)
            {
                hasZero = true;
                break;
            }
            copy = copy / 10;
        }

        if (n > 0 && hasZero)
            System.out.println("Duck Number");
        else
            System.out.println("Not a Duck Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
copy	long	to store a working copy of n
hasZero	boolean	to record whether a zero digit exists

Q89 Pronic Number

Write a program to accept a number and check whether it is a Pronic number or not. A Pronic (oblong / heteromecic) number is a number that is the product of two consecutive integers.

Example: 12 = 3 × 4, 20 = 4 × 5, 42 = 6 × 7

Q_89.java

```java
import java.util.Scanner;

public class Q_89
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        boolean found = false;

        for (long i = 1; i * (i + 1) <= n; i++)
        {
            if (i * (i + 1) == n)
            {
                found = true;
                break;
            }
        }

        if (found)
            System.out.println("Pronic Number");
        else
            System.out.println("Not a Pronic Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
found	boolean	to record whether n is pronic
i	long	to act as the smaller factor

Q90 Disarium Number

Write a program to accept a number and check whether it is a Disarium number or not. A Disarium number is a number in which the sum of its digits, each raised to the power of its position (counted from the left, starting at 1), is equal to the number itself.

Example: 135 = 1^1 + 3^2 + 5^3 = 1 + 9 + 125

Q_90.java

```java
import java.util.Scanner;

public class Q_90
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        long copy = n;
        int pos = 0;
```

```
        while (copy > 0)
        {
            pos++;
            copy = copy / 10;
        }

        long sum = 0;
        copy = n;
        while (copy > 0)
        {
            int d = (int) (copy % 10);
            sum = sum + (long) Math.pow(d, pos);
            pos--;
            copy = copy / 10;
        }

        if (n > 0 && sum == n)
            System.out.println("Disarium Number");
        else
            System.out.println("Not a Disarium Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
copy	long	to store a working copy of n
pos	int	to store the position of the digit
sum	long	to store the sum of powers
d	int	to store the extracted digit

Q91 Kaprekar Number

Write a program to accept a number and check whether it is a Kaprekar number or not. A number n having d digits is a Kaprekar number if its square, split into a right part having d digits and a left part having the remaining digits, gives two parts whose sum is equal to n. The right part must not be zero.

Example: 45 → 45^2 = 2025 → 20 + 25 = 45 297 → 297^2 = 88209 → 88 + 209 = 297

Q_91.java

```
import java.util.Scanner;

public class Q_91
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        long sq = n * n;
        long copy = n, p = 1;

        while (copy > 0)
        {
            p = p * 10;
            copy = copy / 10;
        }

        long right = sq % p;
```

```
        long left = sq / p;

        if (n > 0 && right > 0 && left + right == n)
            System.out.println("Kaprekar Number");
        else
            System.out.println("Not a Kaprekar Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
sq	long	to store the square of n
copy	long	to store a working copy of n
p	long	to store 10 raised to the digit count
right	long	to store the right part of the square
left	long	to store the left part of the square

Q92 Tech Number

> Write a program to accept a number and check whether it is a Tech number or not. A Tech number has an even number of digits; when it is divided into two equal halves, the square of the sum of the two halves is equal to the number itself.
>
> Example: 2025 → 20 + 25 = 45 → 45^2 = 2025

Q_92.java

```
import java.util.Scanner;

public class Q_92
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        long copy = n;
        int count = 0;

        while (copy > 0)
        {
            count++;
            copy = copy / 10;
        }

        boolean tech = false;
        if (n > 0 && count % 2 == 0)
        {
            long p = (long) Math.pow(10, count / 2);
            long first = n / p;
            long second = n % p;
            long s = first + second;
            tech = (s * s == n);
        }

        if (tech)
            System.out.println("Tech Number");
        else
            System.out.println("Not a Tech Number");
```

```
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
copy	long	to store a working copy of n
count	int	to count the digits
tech	boolean	to record whether n is a tech number
p	long	to store 10 raised to half the digits
first	long	to store the first half
second	long	to store the second half
s	long	to store the sum of the halves

Q93 Magic Number

> Write a program to accept a number and check whether it is a Magic number or not. A Magic number is a number in which the eventual sum of the digits is equal to 1, i.e. the sum of digits is found repeatedly until a single-digit number is obtained, and that digit is 1.
>
> Example: 19 → 1 + 9 = 10 → 1 + 0 = 1

Q_93.java

```java
import java.util.Scanner;

public class Q_93
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        long num = n;

        while (num > 9)
        {
            long sum = 0;
            while (num > 0)
            {
                sum = sum + num % 10;
                num = num / 10;
            }
            num = sum;
        }

        if (n > 0 && num == 1)
            System.out.println("Magic Number");
        else
            System.out.println("Not a Magic Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number

Name of the Variable	Data Type	Purpose / Description
num	long	to store the number being reduced
sum	long	to store the sum of digits

Q94 Happy Number

Write a program to accept a number and check whether it is a Happy number or not. A number is Happy if, on repeatedly replacing it by the sum of the squares of its digits, it finally becomes 1. If it never reaches 1, it enters a cycle containing 4 and the number is not happy.

Example: 7 → 49 → 97 → 130 → 10 → 1

Q_94.java

```java
import java.util.Scanner;

public class Q_94
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        long num = n;

        while (num != 1 && num != 4 && num > 0)
        {
            long sum = 0;
            while (num > 0)
            {
                long d = num % 10;
                sum = sum + d * d;
                num = num / 10;
            }
            num = sum;
        }

        if (num == 1)
            System.out.println("Happy Number");
        else
            System.out.println("Not a Happy Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
num	long	to store the number being transformed
sum	long	to store the sum of squares of digits
d	long	to store the extracted digit

Q95 Composite Number

Write a program to accept a number and check whether it is a Composite number or not. A Composite number is a number greater than 1 that has more than two factors.

Example: 4, 6, 9, 15 are Composite numbers. (1 is neither prime nor composite.)

Q_95.java

```java
import java.util.Scanner;

public class Q_95
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        int count = 0;

        for (int i = 1; i <= n; i++)
        {
            if (n % i == 0)
                count++;
        }

        if (count > 2)
            System.out.println("Composite Number");
        else
            System.out.println("Not a Composite Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number
count	int	to count the factors
i	int	to act as the possible factor

Q96 Emirp Number

Write a program to accept a number and check whether it is an Emirp number or not. An Emirp number is a number which is prime both forwards and backwards, i.e. the number and its reverse are both prime.

Example: 13 and its reverse 31 are both prime, so 13 is an Emirp number.

Q_96.java

```java
import java.util.Scanner;

public class Q_96
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();

        long copy = n, rev = 0;
        while (copy > 0)
        {
            rev = rev * 10 + copy % 10;
            copy = copy / 10;
        }

        boolean p1 = n > 1;
        for (long i = 2; i * i <= n; i++)
        {
            if (n % i == 0)
            {
                p1 = false;
```

```
                break;
            }
        }

        boolean p2 = rev > 1;
        for (long i = 2; i * i <= rev; i++)
        {
            if (rev % i == 0)
            {
                p2 = false;
                break;
            }
        }

        if (p1 && p2)
            System.out.println("Emirp Number");
        else
            System.out.println("Not an Emirp Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
copy	long	to store a working copy of n
rev	long	to store the reverse of n
p1	boolean	to record whether n is prime
p2	boolean	to record whether rev is prime
i	long	to act as the trial divisor

Q97 Sunny Number

> Write a program to accept a number and check whether it is a Sunny number or not. A number n is a Sunny number if the next number (n + 1) is a perfect square.
>
> Example: 8 is a Sunny number since 8 + 1 = 9 = 3^2

Q_97.java

```
import java.util.Scanner;

public class Q_97
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        long next = n + 1;
        long root = Math.round(Math.sqrt(next));

        if (n > 0 && root * root == next)
            System.out.println("Sunny Number");
        else
            System.out.println("Not a Sunny Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
next	long	to store n + 1
root	long	to store the square root of next

Q98 Evil Number

Write a program to accept a positive number and check whether it is an Evil number or not. An Evil number is a positive number whose binary equivalent contains an even number of 1s.

Example: 9 → binary 1001 has two 1s, so 9 is an Evil number.

Q_98.java

```
import java.util.Scanner;

public class Q_98
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        long copy = n;
        int ones = 0;

        while (copy > 0)
        {
            if (copy % 2 == 1)
                ones++;
            copy = copy / 2;
        }

        if (n > 0 && ones % 2 == 0)
            System.out.println("Evil Number");
        else
            System.out.println("Not an Evil Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
copy	long	to store a working copy of n
ones	int	to count the 1s in binary

Q99 Unique Number

Write a program to accept a number and check whether it is a Unique number or not. A Unique number is a positive number in which no digit is repeated.

Example: 3412 is Unique, but 3121 is not (1 is repeated).

Q_99.java

```
import java.util.Scanner;

public class Q_99
```

```
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        boolean unique = n > 0;

        for (int d = 0; d <= 9 && unique; d++)
        {
            int count = 0;
            long copy = n;
            while (copy > 0)
            {
                if (copy % 10 == d)
                    count++;
                copy = copy / 10;
            }
            if (count > 1)
                unique = false;
        }

        if (unique)
            System.out.println("Unique Number");
        else
            System.out.println("Not a Unique Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
unique	boolean	to record whether all digits differ
d	int	to act as the digit being counted
count	int	to count occurrences of d
copy	long	to store a working copy of n

Q100 Abundant, Deficient or Perfect Number

Write a program to accept a positive number and find the sum of its proper factors (all factors except the number itself). Then display:

```
"Abundant Number"  if the sum is greater than the number
"Deficient Number" if the sum is less than the number
"Perfect Number"   if the sum is equal to the number
```

Example: 12 (1+2+3+4+6 = 16) is Abundant, 8 (1+2+4 = 7) is Deficient, 6 is Perfect. Display "Invalid input" for zero or a negative number.

Q_100.java

```
import java.util.Scanner;

public class Q_100
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();

        if (n <= 0)
```

```
        {
            System.out.println("Invalid input");
        }
        else
        {
            int sum = 0;
            for (int i = 1; i <= n / 2; i++)
            {
                if (n % i == 0)
                    sum = sum + i;
            }
            if (sum > n)
                System.out.println("Abundant Number");
            else if (sum < n)
                System.out.println("Deficient Number");
            else
                System.out.println("Perfect Number");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number
sum	int	to store the sum of proper factors
i	int	to act as the possible factor

Q101 Fibonacci Series

Write a program to display the first n terms of the Fibonacci series, where each term is the sum of the previous two terms:

`0, 1, 1, 2, 3, 5, 8, 13, 21, ...`

Use long so that the terms do not overflow for n up to 92. Display "Invalid input" if n is less than 1 or more than 92.

Q_101.java

```
import java.util.Scanner;

public class Q_101
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of terms: ");
        int n = sc.nextInt();

        if (n < 1 || n > 92)
        {
            System.out.println("Invalid input");
        }
        else
        {
            long a = 0, b = 1;
            for (int i = 1; i <= n; i++)
            {
                System.out.print(a + " ");
                long c = a + b;
                a = b;
                b = c;
            }
```

```
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of terms
a	long	to store the current term
b	long	to store the next term
c	long	to store the sum of a and b
i	int	to act as the loop counter

Q102 HCF and LCM of Two Numbers

> Write a program to accept two integers and find their HCF (Highest Common Factor) using the division (Euclid's) method and their LCM (Lowest Common Multiple) using the relation LCM = (a x b) / HCF. Work with the absolute values of the numbers. Display "Not defined" if both numbers are zero.

Q_102.java

```java
import java.util.Scanner;

public class Q_102
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter two integers: ");
        long a = Math.abs(sc.nextLong());
        long b = Math.abs(sc.nextLong());

        if (a == 0 && b == 0)
        {
            System.out.println("Not defined");
        }
        else
        {
            long x = a, y = b;
            while (y != 0)
            {
                long r = x % y;
                x = y;
                y = r;
            }
            long hcf = x;
            long lcm = a / hcf * b;
            System.out.println("HCF = " + hcf);
            System.out.println("LCM = " + lcm);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	long	to store the first number
b	long	to store the second number

Name of the Variable	Data Type	Purpose / Description
x	long	to store the dividend in each step
y	long	to store the divisor in each step
r	long	to store the remainder
hcf	long	to store the hcf
lcm	long	to store the lcm

Q103 Factorial of a Number

Write a program to accept a number and find its factorial. n! = 1 × 2 × 3 x ... x n, and 0! = 1. Use long; since 21! exceeds the range of long, display "Too large" for n greater than 20 and "Invalid input" for a negative number.

Q_103.java

```java
import java.util.Scanner;

public class Q_103
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();

        if (n < 0)
        {
            System.out.println("Invalid input");
        }
        else if (n > 20)
        {
            System.out.println("Too large");
        }
        else
        {
            long f = 1;
            for (int i = 1; i <= n; i++)
                f = f * i;
            System.out.println(n + "! = " + f);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number
f	long	to store the factorial
i	int	to act as the loop counter

Q104 Sum and Product of Digits

Write a program to accept a number and find the sum and the product of its digits. For a negative number, use its absolute value.

Example: 2537 → Sum = 17, Product = 210

Q_104.java

```java
import java.util.Scanner;

public class Q_104
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = Math.abs(sc.nextLong());
        long sum = 0, prod = 1;

        do
        {
            long d = n % 10;
            sum = sum + d;
            prod = prod * d;
            n = n / 10;
        } while (n > 0);

        System.out.println("Sum of digits     = " + sum);
        System.out.println("Product of digits = " + prod);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
sum	long	to store the sum of digits
prod	long	to store the product of digits
d	long	to store the extracted digit

Q105 Reverse a Number and Count its Digits

> Write a program to accept a number, display its reverse and the number of digits in it. For a negative number, the reverse should keep the minus sign (e.g. -123 gives -321). The number 0 has one digit.

Q_105.java

```java
import java.util.Scanner;

public class Q_105
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        long copy = Math.abs(n), rev = 0;
        int count = 0;

        do
        {
            rev = rev * 10 + copy % 10;
            count++;
            copy = copy / 10;
        } while (copy > 0);

        if (n < 0)
            rev = -rev;
        System.out.println("Reverse          = " + rev);
        System.out.println("Number of digits = " + count);
```

```
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
copy	long	to store a working copy of n
rev	long	to store the reversed number
count	int	to count the digits

Q106 Largest and Smallest Digit of a Number

> Write a program to accept a number and find the largest and the smallest digit in it.
>
> Example: 59217 → Largest digit = 9, Smallest digit = 1

Q_106.java

```java
import java.util.Scanner;

public class Q_106
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = Math.abs(sc.nextLong());
        long large = 0, small = 9;

        do
        {
            long d = n % 10;
            if (d > large)
                large = d;
            if (d < small)
                small = d;
            n = n / 10;
        } while (n > 0);

        System.out.println("Largest digit  = " + large);
        System.out.println("Smallest digit = " + small);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
large	long	to store the largest digit
small	long	to store the smallest digit
d	long	to store the extracted digit

Q107 Frequency of a Given Digit in a Number

Write a program to accept a number and a digit, and count how many times the digit occurs in the number. Display "Invalid digit" if the digit is not from 0 to 9.

Example: Number = 1223242, Digit = 2 → Frequency = 4

Q_107.java

```java
import java.util.Scanner;

public class Q_107
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = Math.abs(sc.nextLong());
        System.out.print("Enter the digit to count: ");
        int dig = sc.nextInt();

        if (dig < 0 || dig > 9)
        {
            System.out.println("Invalid digit");
        }
        else
        {
            int freq = 0;
            do
            {
                if (n % 10 == dig)
                    freq++;
                n = n / 10;
            } while (n > 0);
            System.out.println("Frequency = " + freq);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
dig	int	to store the digit to count
freq	int	to store the frequency

Q108 Count of Even and Odd Digits

Write a program to accept a number and count the number of even digits and odd digits in it.

Example: 45672 → Even digits = 3 (4, 6, 2), Odd digits = 2 (5, 7)

Q_108.java

```java
import java.util.Scanner;

public class Q_108
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = Math.abs(sc.nextLong());
        int even = 0, odd = 0;
```

```
        do
        {
            long d = n % 10;
            if (d % 2 == 0)
                even++;
            else
                odd++;
            n = n / 10;
        } while (n > 0);

        System.out.println("Even digits = " + even);
        System.out.println("Odd digits  = " + odd);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
even	int	to count the even digits
odd	int	to count the odd digits
d	long	to store the extracted digit

Q109 Factors of a Number and their Count

> Write a program to accept a positive number and display all its factors and the number of factors.
>
> Example: 12 → Factors: 1 2 3 4 6 12, Number of factors = 6

Q_109.java

```
import java.util.Scanner;

public class Q_109
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();

        if (n <= 0)
        {
            System.out.println("Invalid input");
        }
        else
        {
            int count = 0;
            System.out.print("Factors: ");
            for (int i = 1; i <= n; i++)
            {
                if (n % i == 0)
                {
                    System.out.print(i + " ");
                    count++;
                }
            }
            System.out.println();
            System.out.println("Number of factors = " + count);
        }
```

```
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number
count	int	to count the factors
i	int	to act as the possible factor

Q110 Prime Factors of a Number

> Write a program to accept a number greater than 1 and display its prime factorisation.
>
> Example: 360 = 2 × 2 × 2 × 3 × 3 × 5

Q_110.java

```java
import java.util.Scanner;

public class Q_110
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();

        if (n <= 1)
        {
            System.out.println("Invalid input");
        }
        else
        {
            System.out.print(n + " = ");
            long f = 2;
            boolean first = true;
            while (n > 1)
            {
                if (f * f > n)
                    f = n;
                if (n % f == 0)
                {
                    if (!first)
                        System.out.print(" x ");
                    System.out.print(f);
                    first = false;
                    n = n / f;
                }
                else
                {
                    f++;
                }
            }
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number

Name of the Variable	Data Type	Purpose / Description
f	long	to act as the trial factor
first	boolean	to check if it is the first factor

Q111 Largest and Smallest of n Numbers

> Write a program to accept n numbers from the user and find the largest and the smallest of them. Display "Invalid input" if n is less than 1.

Q_111.java

```java
import java.util.Scanner;

public class Q_111
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("How many numbers? ");
        int n = sc.nextInt();

        if (n < 1)
        {
            System.out.println("Invalid input");
        }
        else
        {
            System.out.println("Enter " + n + " numbers:");
            double x = sc.nextDouble();
            double large = x, small = x;
            for (int i = 2; i <= n; i++)
            {
                x = sc.nextDouble();
                if (x > large)
                    large = x;
                if (x < small)
                    small = x;
            }
            System.out.println("Largest  = " + large);
            System.out.println("Smallest = " + small);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store how many numbers
x	double	to store each number entered
large	double	to store the largest number
small	double	to store the smallest number
i	int	to act as the loop counter

Q112 Sum of Numbers until 0 is Entered

> Write a program using a do-while loop to keep accepting numbers from the user until 0 is entered. Display the sum, the count and the average of the numbers entered (excluding the 0). If the first number itself is 0, display "No numbers entered".

Q_112.java

```java
import java.util.Scanner;

public class Q_112
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        double sum = 0;
        int count = 0;
        double n;

        System.out.println("Enter numbers (0 to stop):");
        do
        {
            n = sc.nextDouble();
            if (n != 0)
            {
                sum = sum + n;
                count++;
            }
        } while (n != 0);

        if (count == 0)
        {
            System.out.println("No numbers entered");
        }
        else
        {
            System.out.println("Sum     = " + sum);
            System.out.println("Count   = " + count);
            System.out.println("Average = " + (sum / count));
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
sum	double	to store the sum of the numbers
count	int	to count the numbers entered
n	double	to store each number entered

Q113 Numbers from 1 to 50 not Divisible by 3 or 5 (continue)

> Write a program to display all the numbers from 1 to 50 that are divisible neither by 3 nor by 5, using the continue statement to skip the other numbers. Also display how many such numbers there are.

Q_113.java

```java
public class Q_113
{
    public static void main(String[] args)
    {
        int count = 0;
        for (int i = 1; i <= 50; i++)
```

```
        {
            if (i % 3 == 0 || i % 5 == 0)
                continue;
            System.out.print(i + " ");
            count++;
        }
        System.out.println();
        System.out.println("Count = " + count);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
count	int	to count the numbers displayed
i	int	to act as the loop counter

Q114 Sum of First n Natural Numbers and their Squares

Write a program to accept n and find:

S1 = 1 + 2 + 3 + ... + n
S2 = 1^2 + 2^2 + 3^2 + ... + n^2

using a loop. Verify the results with the formulas n(n + 1)/2 and n(n + 1)(2n + 1)/6. Display "Invalid input" if n is less than 1.

Q_114.java

```
import java.util.Scanner;

public class Q_114
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        long n = sc.nextLong();

        if (n < 1)
        {
            System.out.println("Invalid input");
        }
        else
        {
            long s1 = 0, s2 = 0;
            for (long i = 1; i <= n; i++)
            {
                s1 = s1 + i;
                s2 = s2 + i * i;
            }
            System.out.println("S1 = " + s1 + "   (formula: " + (n * (n + 1) / 2) + ")");
            System.out.println("S2 = " + s2 + "   (formula: " + (n * (n + 1) * (2 * n + 1) / 6) + ")");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number of terms
s1	long	to store the sum of natural numbers

Name of the Variable	Data Type	Purpose / Description
s2	long	to store the sum of squares
i	long	to act as the loop counter

Q115 Sum of the Series 1 + 1/2 + 1/3 + ... + 1/n

Write a program to accept n and find the sum of the series:

`S = 1 + 1/2 + 1/3 + 1/4 + ... + 1/n`

Q_115.java

```java
import java.util.Scanner;

public class Q_115
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();
        double s = 0;

        for (int i = 1; i <= n; i++)
            s = s + 1.0 / i;

        System.out.println("Sum = " + s);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of terms
s	double	to store the sum
i	int	to act as the loop counter

Q116 Sum of the Series x/1 + x^2/2 + x^3/3 + ... + x^n/n

Write a program to accept x and n, and find the sum of the series:

`S = x/1 + x^2/2 + x^3/3 + ... + x^n/n`

Q_116.java

```java
import java.util.Scanner;

public class Q_116
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter x and n: ");
        double x = sc.nextDouble();
        int n = sc.nextInt();
        double s = 0;

        for (int i = 1; i <= n; i++)
            s = s + Math.pow(x, i) / i;

        System.out.println("Sum = " + s);
```

```
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
x	double	to store the value of x
n	int	to store the number of terms
s	double	to store the sum
i	int	to act as the loop counter

Q117 Sum of the Series 1 - 2 + 3 - 4 + ... n

> Write a program to accept n and find the sum of the series:
> `S = 1 - 2 + 3 - 4 + 5 - 6 + ... up to n terms`

Q_117.java

```java
import java.util.Scanner;

public class Q_117
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        long n = sc.nextLong();
        long s = 0;

        for (long i = 1; i <= n; i++)
        {
            if (i % 2 == 1)
                s = s + i;
            else
                s = s - i;
        }
        System.out.println("Sum = " + s);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number of terms
s	long	to store the sum
i	long	to act as the loop counter

Q118 Sum of the Series a/2 + a/5 + a/8 + ... n Terms

> Write a program to accept a and n, and find the sum of the series:
> `S = a/2 + a/5 + a/8 + a/11 + ... up to n terms`

Q_118.java

```java
import java.util.Scanner;

public class Q_118
{
    public static void main(String[] args)
```

```
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a and n: ");
        double a = sc.nextDouble();
        int n = sc.nextInt();
        double s = 0;
        int den = 2;

        for (int i = 1; i <= n; i++)
        {
            s = s + a / den;
            den = den + 3;
        }
        System.out.println("Sum = " + s);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	double	to store the value of a
n	int	to store the number of terms
s	double	to store the sum
den	int	to store the denominator of each term
i	int	to act as the loop counter

Q119 Series 1, 4, 9, 16, ... and 2, 5, 10, 17, 26, ...

Write a program to accept n and display the first n terms of each of the following series:

(a) 1, 4, 9, 16, 25, ...
(b) 2, 5, 10, 17, 26, ...

Q_119.java

```
import java.util.Scanner;

public class Q_119
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();

        System.out.print("(a) ");
        for (int i = 1; i <= n; i++)
            System.out.print((long) i * i + " ");
        System.out.println();

        System.out.print("(b) ");
        for (int i = 1; i <= n; i++)
            System.out.print(((long) i * i + 1) + " ");
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of terms

Name of the Variable	Data Type	Purpose / Description
i	int	to act as the loop counter

Q120 Series 0, 7, 26, 63, ...

> Write a program to accept n and display the first n terms of the series:
>
> `0, 7, 26, 63, 124, ...`
>
> (Hint: each term is one less than the cube of its position.)

Q_120.java

```java
import java.util.Scanner;

public class Q_120
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();

        for (long i = 1; i <= n; i++)
            System.out.print((i * i * i - 1) + " ");
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of terms
i	long	to act as the loop counter

Q121 Series 1, 11, 111, 1111, ... and its Sum

> Write a program to accept n and display the first n terms of the series 1, 11, 111, 1111, ... and their sum. Use long, and accept n only from 1 to 18 (otherwise display "Invalid input").

Q_121.java

```java
import java.util.Scanner;

public class Q_121
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();

        if (n < 1 || n > 18)
        {
            System.out.println("Invalid input");
        }
        else
        {
            long term = 0, sum = 0;
            for (int i = 1; i <= n; i++)
            {
                term = term * 10 + 1;
                sum = sum + term;
```

```
                System.out.print(term + " ");
            }
            System.out.println();
            System.out.println("Sum = " + sum);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of terms
term	long	to store the current term
sum	long	to store the sum of the terms
i	int	to act as the loop counter

Q122 Series 1, -3, 5, -7, 9, ... and its Sum

> Write a program to accept n and display the first n terms of the series 1, -3, 5, -7, 9, -11, ... and find their sum.

Q_122.java

```java
import java.util.Scanner;

public class Q_122
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();
        long sum = 0;
        int sign = 1;

        for (int i = 1; i <= n; i++)
        {
            long term = sign * (2L * i - 1);
            System.out.print(term + " ");
            sum = sum + term;
            sign = -sign;
        }
        System.out.println();
        System.out.println("Sum = " + sum);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of terms
sum	long	to store the sum
sign	int	to store the sign of the term
i	int	to act as the loop counter
term	long	to store the current term

Q123 Looping Menu: Sum of Digits, Reverse, Count of Digits

Write a menu-driven program that repeatedly displays the following menu until the user chooses Exit:

```
1. Sum of digits of a number
2. Reverse of a number
3. Number of digits in a number
4. Exit
```

For choices 1 to 3, accept a number and display the result. Display "Wrong choice" for any other choice. Use a do-while loop with a switch statement.

Q_123.java

```java
import java.util.Scanner;

public class Q_123
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int ch;
        long n, res;

        do
        {
            System.out.println("1. Sum of digits");
            System.out.println("2. Reverse of a number");
            System.out.println("3. Number of digits");
            System.out.println("4. Exit");
            System.out.print("Enter your choice: ");
            ch = sc.nextInt();

            switch (ch)
            {
                case 1:
                    System.out.print("Enter a number: ");
                    n = Math.abs(sc.nextLong());
                    res = 0;
                    while (n > 0)
                    {
                        res = res + n % 10;
                        n = n / 10;
                    }
                    System.out.println("Sum of digits = " + res);
                    break;
                case 2:
                    System.out.print("Enter a number: ");
                    n = Math.abs(sc.nextLong());
                    res = 0;
                    while (n > 0)
                    {
                        res = res * 10 + n % 10;
                        n = n / 10;
                    }
                    System.out.println("Reverse = " + res);
                    break;
                case 3:
                    System.out.print("Enter a number: ");
                    n = Math.abs(sc.nextLong());
                    res = 0;
                    do
                    {
                        res++;
                        n = n / 10;
                    } while (n > 0);
                    System.out.println("Number of digits = " + res);
                    break;
```

```
                case 4:
                    System.out.println("Goodbye!");
                    break;
                default:
                    System.out.println("Wrong choice");
            }
            System.out.println();
        } while (ch != 4);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
ch	int	to store the user's choice
n	long	to store the number entered
res	long	to store the result

Q124 Decimal to Binary Conversion

Extra Write a program to accept a non-negative decimal (base 10) integer and convert it into its binary (base 2) equivalent by repeatedly dividing by 2 and collecting the remainders in reverse order.

Example: 25 → 11001

Q_124.java

```
import java.util.Scanner;

public class Q_124
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a decimal number: ");
        long n = sc.nextLong();

        if (n < 0)
        {
            System.out.println("Invalid input");
        }
        else
        {
            String bin = "";
            long copy = n;
            do
            {
                bin = (copy % 2) + bin;
                copy = copy / 2;
            } while (copy > 0);
            System.out.println("Binary of " + n + " = " + bin);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the decimal number
bin	String	to store the binary digits
copy	long	to store a working copy of n

Q125 Decimal to Octal Conversion

Extra Write a program to accept a non-negative decimal integer and convert it into its octal (base 8) equivalent by repeatedly dividing by 8.

Example: 100 → 144

Q_125.java

```java
import java.util.Scanner;

public class Q_125
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a decimal number: ");
        long n = sc.nextLong();

        if (n < 0)
        {
            System.out.println("Invalid input");
        }
        else
        {
            String oct = "";
            long copy = n;
            do
            {
                oct = (copy % 8) + oct;
                copy = copy / 8;
            } while (copy > 0);
            System.out.println("Octal of " + n + " = " + oct);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the decimal number
oct	String	to store the octal digits
copy	long	to store a working copy of n

Q126 Decimal to Hexadecimal Conversion

Extra Write a program to accept a non-negative decimal integer and convert it into its hexadecimal (base 16) equivalent. The remainders 10 to 15 are written as A to F.

Example: 255 → FF, 4096 → 1000

Q_126.java

```java
import java.util.Scanner;

public class Q_126
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a decimal number: ");
        long n = sc.nextLong();

        if (n < 0)
```

```
        {
            System.out.println("Invalid input");
        }
        else
        {
            String hex = "";
            long copy = n;
            do
            {
                int r = (int) (copy % 16);
                char digit;
                if (r < 10)
                    digit = (char) ('0' + r);
                else
                    digit = (char) ('A' + r - 10);
                hex = digit + hex;
                copy = copy / 16;
            } while (copy > 0);
            System.out.println("Hexadecimal of " + n + " = " + hex);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the decimal number
hex	String	to store the hexadecimal digits
copy	long	to store a working copy of n
r	int	to store the remainder
digit	char	to store the hexadecimal digit

Q127 Binary to Decimal Conversion

Extra Write a program to accept a binary number (made of the digits 0 and 1 only, up to 18 digits) and convert it into its decimal equivalent. If the number contains any digit other than 0 or 1, display "Invalid binary number".

Example: 11001 → 25

Q_127.java

```
import java.util.Scanner;

public class Q_127
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a binary number: ");
        long bin = sc.nextLong();
        long copy = bin, dec = 0, place = 1;
        boolean valid = bin >= 0;

        while (copy > 0)
        {
            long d = copy % 10;
            if (d > 1)
            {
                valid = false;
                break;
            }
            dec = dec + d * place;
```

```
            place = place * 2;
            copy = copy / 10;
        }

        if (valid)
            System.out.println("Decimal = " + dec);
        else
            System.out.println("Invalid binary number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
bin	long	to store the binary number
copy	long	to store a working copy of bin
dec	long	to store the decimal equivalent
place	long	to store the place value
valid	boolean	to record whether the input is binary
d	long	to store the extracted digit

Q128 Collatz (Hailstone) Sequence

> Extra The Collatz sequence of a positive number n is formed as follows: if n is even, the next term is n / 2; if n is odd, the next term is 3n + 1. The sequence stops when it reaches 1. Write a program to accept a positive number, display its Collatz sequence and the number of steps taken to reach 1.
>
> Example: 6 → 6 3 10 5 16 8 4 2 1 (8 steps)

Q_128.java

```
import java.util.Scanner;

public class Q_128
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a positive number: ");
        long n = sc.nextLong();

        if (n <= 0)
        {
            System.out.println("Invalid input");
        }
        else
        {
            int steps = 0;
            System.out.print(n + " ");
            while (n != 1)
            {
                if (n % 2 == 0)
                    n = n / 2;
                else
                    n = 3 * n + 1;
                System.out.print(n + " ");
                steps++;
            }
            System.out.println();
            System.out.println("Steps = " + steps);
        }
```

```
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the current term
steps	int	to count the steps

Q129 Number Guessing Game

Extra Write a program for a number guessing game. The computer chooses a secret number from 1 to 100 using Math.random(). The user gets at most 7 attempts. After each wrong guess, display "Too high!" or "Too low!". If the user guesses correctly, display "Correct! You guessed it in <k> attempts." and stop. If all attempts are used up, display "Out of attempts! The number was <secret>."

Q_129.java

```java
import java.util.Scanner;

public class Q_129
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int secret = (int) (Math.random() * 100) + 1;
        boolean won = false;
        int attempt;

        for (attempt = 1; attempt <= 7; attempt++)
        {
            System.out.print("Attempt " + attempt + " - enter your guess: ");
            int guess = sc.nextInt();
            if (guess == secret)
            {
                won = true;
                break;
            }
            else if (guess > secret)
            {
                System.out.println("Too high!");
            }
            else
            {
                System.out.println("Too low!");
            }
        }

        if (won)
            System.out.println("Correct! You guessed it in " + attempt + " attempts.");
        else
            System.out.println("Out of attempts! The number was " + secret + ".");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
secret	int	to store the secret number
won	boolean	to record whether the user won
attempt	int	to count the attempts

Name of the Variable	Data Type	Purpose / Description
guess	int	to store the user's guess

Q130 Reverse-and-Add until a Palindrome

Extra Many numbers become palindromes when they are repeatedly added to their reverse. For example, 87 → 87 + 78 = 165 → 165 + 561 = 726 → 726 + 627 = 1353 → 1353 + 3531 = 4884 (a palindrome after 4 steps). Write a program to accept a positive number and apply this process at least once, displaying each sum, until a palindrome is obtained. Stop with the message "No palindrome found within the limit" if 30 steps are over or the sum exceeds 10^16 (196 is a famous number that never seems to give a palindrome).

Q_130.java

```java
import java.util.Scanner;

public class Q_130
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a positive number: ");
        long n = sc.nextLong();

        if (n <= 0)
        {
            System.out.println("Invalid input");
        }
        else
        {
            final long LIMIT = 10000000000000000L;
            int steps = 0;
            boolean pal = false;
            do
            {
                long copy = n, rev = 0;
                while (copy > 0)
                {
                    rev = rev * 10 + copy % 10;
                    copy = copy / 10;
                }
                n = n + rev;
                steps++;
                System.out.println("Step " + steps + ": " + n);

                copy = n;
                rev = 0;
                while (copy > 0)
                {
                    rev = rev * 10 + copy % 10;
                    copy = copy / 10;
                }
                pal = (rev == n);
            } while (!pal && steps < 30 && n <= LIMIT);

            if (pal)
                System.out.println("Palindrome " + n + " obtained in " + steps + " steps");
            else
                System.out.println("No palindrome found within the limit");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the current number
LIMIT	long	to store the largest allowed sum
steps	int	to count the steps
pal	boolean	to record whether n is a palindrome
copy	long	to store a working copy of n
rev	long	to store the reverse of n

Q131 Smith Number

Extra Write a program to accept a number and check whether it is a Smith number or not. A Smith number is a composite number whose sum of digits is equal to the sum of the digits of its prime factors (counted with repetition).

Example: 666 = 2 × 3 × 3 × 37 Sum of digits of 666 = 6 + 6 + 6 = 18 Sum of digits of prime factors = 2 + 3 + 3 + (3 + 7) = 18

Q_131.java

```
import java.util.Scanner;

public class Q_131
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();

        long copy = n, sumDigits = 0;
        while (copy > 0)
        {
            sumDigits = sumDigits + copy % 10;
            copy = copy / 10;
        }

        long num = n, sumFactors = 0;
        int factorCount = 0;
        for (long f = 2; num > 1 && f * f <= num; f++)
        {
            while (num % f == 0)
            {
                long t = f;
                while (t > 0)
                {
                    sumFactors = sumFactors + t % 10;
                    t = t / 10;
                }
                factorCount++;
                num = num / f;
            }
        }
        if (num > 1 && factorCount > 0)
        {
            long t = num;
            while (t > 0)
            {
                sumFactors = sumFactors + t % 10;
                t = t / 10;
```

```
            }
            factorCount++;
        }

        if (n > 3 && factorCount > 1 && sumDigits == sumFactors)
            System.out.println("Smith Number");
        else
            System.out.println("Not a Smith Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
copy	long	to store a working copy of n
sumDigits	long	to store the sum of digits of n
num	long	to store the part left to factorise
sumFactors	long	to store digit sum of prime factors
factorCount	int	to count the prime factors
f	long	to act as the trial factor
t	long	to store a copy of a factor

CHAPTER

08



Nested for Loops

Nested loops · Working and number of iterations · break and continue in nested loops · Rectangular and right-angled triangle patterns · Series using nested loops

44 programs | Q132 – Q175 | 6 extra

Theory

📑 Contents

1. What is a nested loop?
2. How a nested `for` loop works
3. Counting iterations
4. `break` and `continue` in nested loops
5. Building patterns — the method
6. Pattern relationships at a glance
7. Series using nested loops
8. Predict the output
9. Find the errors
10. One-line facts
11. Test yourself
12. Programs in this chapter

1. What is a nested loop?

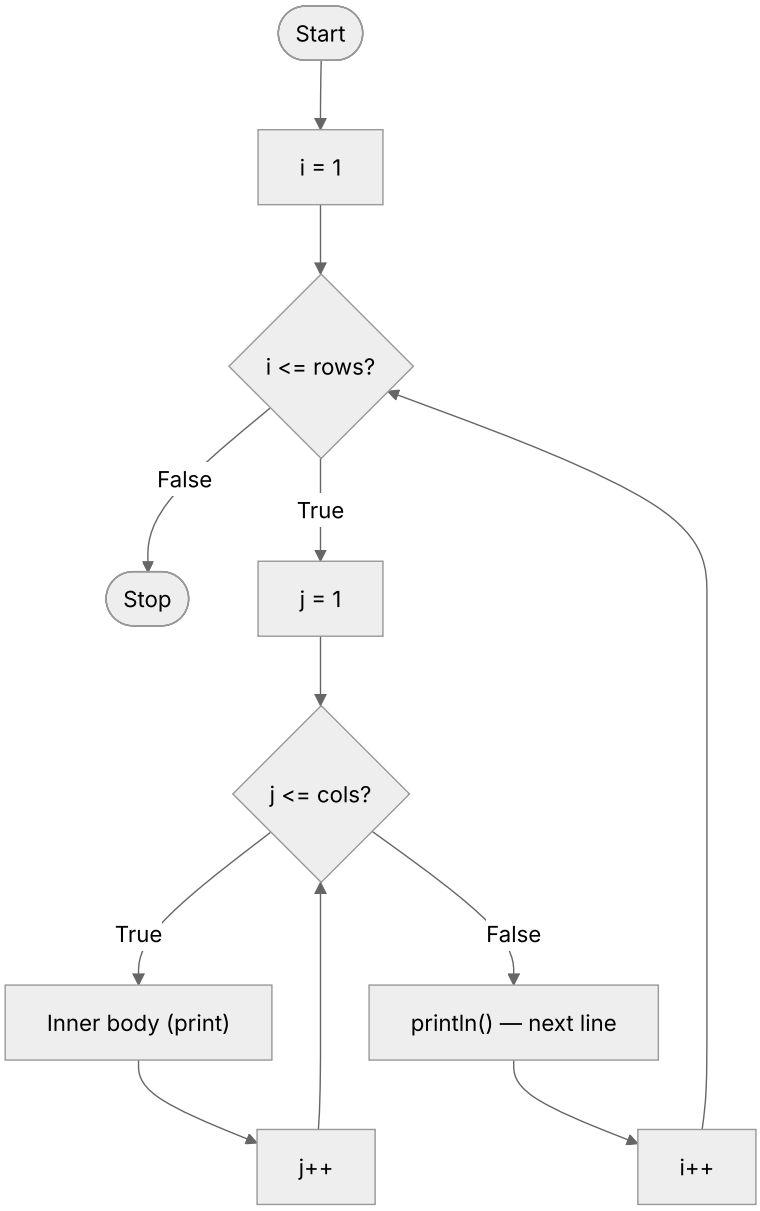
A **nested loop** is a loop placed **inside the body of another loop**. The enclosing loop is the **outer loop** and the enclosed one is the **inner loop**.

```
for (int i = 1; i <= 3; i++)          // outer loop
{
    for (int j = 1; j <= 4; j++)      // inner loop
    {
        System.out.print("* ");
    }
    System.out.println();
}
```

> **NOTE**
>
> The ICSE syllabus covers **nested `for` loops** only. Nested `while` and nested `do-while` loops are **not included**.

2. How a nested `for` loop works

For **every single iteration** of the outer loop, the inner loop runs **completely** — from its initialisation until its condition becomes false. Then the outer loop's update is performed and the process repeats.



Analogy: a clock — the minute hand (inner loop) completes a full round of 60 for every single step of the hour hand (outer loop).

Trace of the example above

i	j values	Printed
1	1, 2, 3, 4	`* * * *`
2	1, 2, 3, 4	`* * * *`
3	1, 2, 3, 4	`* * * *`

3. Counting iterations

Outer	Inner	Total executions of the inner body
`for (i = 1; i <= 3; i++)`	`for (j = 1; j <= 4; j++)`	3 × 4 = **12**
`for (i = 1; i <= 5; i++)`	`for (j = 1; j <= i; j++)`	1 + 2 + 3 + 4 + 5 = **15**
`for (i = 5; i >= 1; i--)`	`for (j = 1; j <= i; j++)`	5 + 4 + 3 + 2 + 1 = **15**

Outer	Inner	Total executions of the inner body
`for (i = 1; i <= 4; i++)`	`for (j = i; j <= 4; j++)`	4 + 3 + 2 + 1 = **10**
`for (i = 1; i <= n; i++)`	`for (j = 1; j <= n; j++)`	n^2
`for (i = 1; i <= n; i++)`	`for (j = 1; j <= i; j++)`	$\frac{n(n+1)}{2}$

4. `break` and `continue` in nested loops

A `break` or `continue` affects **only the loop in which it is written (the innermost enclosing loop)**.

- `break` inside the inner loop ends **only the inner loop**; the outer loop continues with its next iteration.
- `continue` inside the inner loop skips to the **next iteration of the inner loop**.

```
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 3; j++)
    {
        if (j == 2)
            break;               // ends only the inner loop
        System.out.print(i + "" + j + " ");
    }
}
// Output: 11 21 31
```

```
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 3; j++)
    {
        if (j == 2)
            continue;            // skips j = 2 only
        System.out.print(i + "" + j + " ");
    }
}
// Output: 11 13 21 23 31 33
```

> **TIP**
>
> To stop **both** loops at once, use a boolean **flag**: set it in the inner loop, `break`, and test it in the outer loop to `break` again.

5. Building patterns — the method

1. **Outer loop = rows.** Count the rows; decide whether the row number increases or decreases.
2. **Inner loop = columns.** Find how many items are printed in row `i` — e.g. `i` items, `5 - i + 1` items, or a fixed number (rectangle).
3. **What is printed?** Write the item in terms of `i` (row), `j` (column) or a separate counter (`k`). Make a small table of `i`, `j` and the printed value.
4. **After the inner loop**, call `System.out.println()` to move to the next row.

Worked example:

```
1
2 4
```

```
3 6 9
4 8 12 16
```

Row i	Items in row	Item in column j
1	1	1 × 1
2	2	2 × j
3	3	3 × j
i	i	**i × j**

```java
for (int i = 1; i <= 4; i++)
{
    for (int j = 1; j <= i; j++)
        System.out.print(i * j + " ");
    System.out.println();
}
```

6. Pattern relationships at a glance

Pattern (first rows)	Outer loop	Inner loop	Print
`1` / `1 2` / `1 2 3`	`i = 1 → 5`	`j = 1 → i`	`j`
`1` / `2 2` / `3 3 3`	`i = 1 → 5`	`j = 1 → i`	`i`
`1 2 3 4 5` / `1 2 3 4`	`i = 5 → 1`	`j = 1 → i`	`j`
`5` / `5 4` / `5 4 3`	`i = 5 → 1`	`j = 5 → i` (decreasing)	`j`
`1` / `2 1` / `3 2 1`	`i = 1 → 5`	`j = i → 1` (decreasing)	`j`
Floyd: `1` / `2 3` / `4 5 6`	`i = 1 → 5`	`j = 1 → i`	counter `k++`
`1` / `0 1` / `1 0 1`	`i = 1 → 5`	`j = 1 → i`	`(i + j) % 2 == 0 ? 1 : 0`
`A` / `A B` / `A B C`	`i = 1 → 5`	`j = 1 → i`	`(char) ('A' + j - 1)`
`A` / `B B` / `C C C`	`i = 1 → 5`	`j = 1 → i`	`(char) ('A' + i - 1)`
Rectangle `1 2 3 4 5` ×4	`i = 1 → 4`	`j = 1 → 5`	`j`
Hollow rectangle	`i = 1 → r`	`j = 1 → c`	`*` on the border (`i == 1 || i == r || j == 1 || j == c`), else space

WARNING

`System.out.print('A' + j)` prints a **number** (e.g. 66), because `char + int` is `int`. Cast it: `System.out.print((char) ('A' + j))`. Similarly `System.out.print('*' + ' ')` prints `74`, not `*` — use the String `"* "`.

7. Series using nested loops

When each **term** of a series itself needs a loop (a factorial or a running sum), the **outer loop** moves from term to term and the **inner loop** computes the term.

$S = 1! + 2! + 3! + \cdots + n!$

```
long s = 0;
for (int i = 1; i <= n; i++)
{
    long f = 1;                     // reset for every term!
    for (int j = 1; j <= i; j++)
        f = f * j;
    s = s + f;
}
```

$S = \frac{x}{1!} + \frac{x^2}{2!} + \cdots + \frac{x^n}{n!}$ → inner loop for the factorial, `Math.pow(x, i)` for the numerator.

IMPORTANT

The most common mistake in nested series is **forgetting to reset** the inner accumulator (`f = 1` or `term = 0`) inside the outer loop.

8. Predict the output

Example 1

```
for (int i = 1; i <= 3; i++)
{
    for (int j = 3; j >= i; j--)
        System.out.print(j);
    System.out.println();
}
```

Show answer

i	j runs	Printed
1	3, 2, 1	`321`
2	3, 2	`32`
3	3	`3`

Example 2

```
int c = 0;
for (int i = 1; i <= 4; i++)
```

```
    for (int j = 1; j <= 4; j++)
        if ((i + j) % 3 == 0)
            c++;
System.out.println(c);
```

Show answer

Pairs with `i + j` = 3 or 6: (1,2), (2,1), (2,4), (3,3), (4,2) → **5**

Example 3

```
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 3; j++)
    {
        if (i == j) continue;
        if (i + j == 5) break;
        System.out.print(i * j + " ");
    }
}
```

Show dry run

i	j	Action
1	1	continue
1	2	print 2
1	3	print 3
2	1	print 2
2	2	continue
2	3	2 + 3 = 5 → break
3	1	print 3
3	2	3 + 2 = 5 → break

Output: `2 3 2 3`

Example 4

```
for (char ch = 'a'; ch <= 'c'; ch++)
{
    for (int k = 1; k <= 2; k++)
        System.out.print(ch);
    System.out.print(" ");
}
```

Show answer

`aa bb cc`

9. Find the errors

The program should print

```
* * *
* * *
```

```
for (int i = 1; i <= 2; i++);
{
    for (int j = 1, j <= 3, j++)
        System.out.print('*' + ' ');
    System.out.println;
}
```

Show corrected code

```
for (int i = 1; i <= 2; i++)           // stray semicolon removed (and i was out of scope in the
block)
{
    for (int j = 1; j <= 3; j++)       // semicolons, not commas
        System.out.print("* ");        // '*' + ' ' adds the codes (42 + 32 = 74)
    System.out.println();              // method call needs ()
}
```

10. One-line facts

- In a nested loop, the inner loop completes **all** its iterations for **each** iteration of the outer loop.
- Total inner iterations of an `m × n` rectangle pattern = **m × n**.
- `break` in an inner loop terminates **only the inner loop**.
- The outer loop usually decides the **rows**; the inner loop decides the **columns**.
- `System.out.println()` after the inner loop moves the cursor to the **next row**.
- Reset inner accumulators **inside** the outer loop for series.

🧠 Test Yourself

Part A — Multiple choice

1. How many times is `x++` executed? `for (i = 0; i < 5; i++) for (j = i; j < 5; j++) x++;`
   (a) 25 (b) 15 (c) 10 (d) 5
2. Which code prints `1 1 2 1 2 3` (a) `for (i=1;i<=3;i++){for(j=1;j<=3;j++) print(j+" ");println();}` (b) `for (i=1;i<=3;i++){for(j=1;j<=i;j++) print(j+" ");println();}` (c) `for (i=1;i<=3;i++){for(j=1;j<=i;j++) print(i+" ");println();}` (d) `for (i=3;i>=1;i--){for(j=1;j<=i;j++) print(j+" ");println();}`
3. *Assertion (A):* A `break` in the inner loop of a nested loop terminates both loops.
   Reason (R): `break` transfers control out of the innermost loop that contains it.
   (a) Both true; R explains A (b) Both true; R doesn't explain A (c) A false, R true (d) Both false
4. What is printed? `for (int i = 1; i <= 2; i++) for (int j = 1; j <= 2; j++) System.out.print(i + j);`
   (a) 2334 (b) 1212 (c) 1122 (d) 2343
5. `System.out.print((char) (65 + 2) + "" + (char) 97);` prints (a) 67a (b) Ca (c) C97 (d) 6797

6. A hollow 5 × 5 square of `*` is printed. How many stars appear? (a) 25 (b) 20 (c) 16 (d) 9
7. In the code for $S = 1! + 2! + \cdots + n!$, `f = 1` was written **before** the outer loop instead of inside it. For n = 3 the output becomes
   (a) 9 (b) 15 (c) 33 (d) 6
8. What is printed? `java for (int i = 3; i >= 1; i--) { for (int j = 1; j <= i; j++) System.out.print((char) ('A' + i - 1)); System.out.print(" "); }` (a) `AAA BB C` (b) `CCC BB A` (c) `ABC AB A` (d) `CBA CB C`

Part B — Short answer questions

1. What is a nested loop? Give a real-life analogy.
2. How many times will the inner body run? `for (i = 2; i <= 10; i += 2) for (j = 1; j < i; j++)`
3. Write the code for the pattern: `5 4 3 2 1 5 4 3 2 5 4 3 5 4 5`
4. Write the output: `for (int i = 1; i <= 3; i++) { for (int j = 1; j <= i; j++) System.out.print(j * j + " "); System.out.println(); }`
5. Explain how `continue` behaves inside the inner loop of a nested loop.
6. Why must the factorial variable be reset inside the outer loop in a series program?

✅ Answers — Part A

Q	Ans	Working
1	(b)	5 + 4 + 3 + 2 + 1
2	(b)	—
3	(c)	Only the inner loop ends
4	(a)	(1,1)=2, (1,2)=3, (2,1)=3, (2,2)=4
5	(b)	—
6	(c)	25 − 9 inner positions
7	(b)	f = 1·1 = 1, then 1·1·2 = 2, then 2·1·2·3 = 12 → 1 + 2 + 12 = 15
8	(b)	—

✅ Answers — Part B

1. A loop inside another loop. Analogy: a clock — the minute hand completes a full round for each step of the hour hand.
2. 1 + 3 + 5 + 7 + 9 = **25** (for i = 2, 4, 6, 8, 10 the inner loop runs i − 1 times).
3. `java for (int i = 1; i <= 5; i++) { for (int j = 5; j >= i; j--) System.out.print(j + " "); System.out.println(); }`
4. `1 1 4 1 4 9`
5. It skips the remaining statements of the current inner iteration and moves to the next value of the inner loop variable; the outer loop is unaffected.
6. Each term needs its own factorial starting from 1; otherwise the previous factorial keeps being multiplied, giving wrong terms.

💻 Programs in this chapter

No.	Program	
Q132	Pattern: 1, 1 2, 1 2 3, ...	
Q133	Pattern: 1, 2 2, 3 3 3, ...	
Q134	Pattern: 1 2 3 4 5, 1 2 3 4, ...	
Q135	Pattern: 5, 5 4, 5 4 3, ...	
Q136	Pattern: 5 4 3 2 1, 4 3 2 1, ...	
Q137	Pattern: 1, 2 1, 3 2 1, ...	
Q138	Pattern: 5 5 5 5 5, 4 4 4 4, ...	
Q139	Floyd's Triangle	
Q140	Pattern: 1, 0 1, 1 0 1, ...	
Q141	Pattern: 1, 2 4, 3 6 9, ...	
Q142	Pattern: 1, 1 3, 1 3 5, ...	
Q143	Pattern: Right-Angled Triangle of Stars	
Q144	Pattern: Inverted Triangle of Stars	
Q145	Pattern: Alternating # and * Triangle	
Q146	Pattern: A, A B, A B C, ...	
Q147	Pattern: A, B B, C C C, ...	
Q148	Pattern: A B C D E, A B C D, ...	
Q149	Pattern: E, D E, C D E, ...	
Q150	Floyd's Triangle of Letters	
Q151	Pattern: Rows of Numbers and Letters Alternately	
Q152	Rectangular Pattern: 1 2 3 4 5 in Every Row	
Q153	Rectangular Pattern: Row Number Repeated	
Q154	Rectangular Pattern: 1 2 3 4 5, 2 3 4 5 6, ...	
Q155	Hollow Rectangle of Stars	
Q156	Rectangle with Alternate Rows of * and #	
Q157	Pyramid of Stars	🌟 *Extra*
Q158	Diamond of Stars	🌟 *Extra*
Q159	Number Pyramid: 1, 1 2 1, 1 2 3 2 1, ...	🌟 *Extra*
Q160	Pascal's Triangle	🌟 *Extra*
Q161	Hourglass of Stars	🌟 *Extra*
Q162	First n Multiples of Each Number from 1 to a Limit	
Q163	Multiplication Tables from 1 to 10 in a Grid	
Q164	Prime Numbers between Two Limits	

No.	Program	
Q165	Perfect Numbers from 1 to n	
Q166	Armstrong Numbers in a Range	
Q167	Palindrome Numbers in a Range	
Q168	Twin Primes up to n	
Q169	Sum of the Series 1! + 2! + 3! + ... + n!	
Q170	Sum of the Series 1 + (1+2) + (1+2+3) + ...	
Q171	Sum of the Series x/1! + x^2/2! + ... + x^n/n!	
Q172	Sum of the Series 1 - x^2/2! + x^4/4! - ...	
Q173	Disarium Numbers between Two Limits	
Q174	break and continue in Nested Loops	
Q175	Pythagorean Triplets up to n	🌟 *Extra*

Programs Q132 – Q175

Q132 Pattern: 1, 1 2, 1 2 3, ...

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Q_132.java

```
public class Q_132
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 5; i++)
        {
            for (int j = 1; j <= i; j++)
                System.out.print(j + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q133 Pattern: 1, 2 2, 3 3 3, ...

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

Q_133.java

```
public class Q_133
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 5; i++)
        {
            for (int j = 1; j <= i; j++)
                System.out.print(i + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q134 Pattern: 1 2 3 4 5, 1 2 3 4, ...

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

Q_134.java

```java
public class Q_134
{
    public static void main(String[] args)
    {
        for (int i = 5; i >= 1; i--)
        {
            for (int j = 1; j <= i; j++)
                System.out.print(j + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q135 Pattern: 5, 5 4, 5 4 3, ...

```
5
5 4
5 4 3
5 4 3 2
5 4 3 2 1
```

Q_135.java

```java
public class Q_135
{
    public static void main(String[] args)
    {
        for (int i = 5; i >= 1; i--)
        {
            for (int j = 5; j >= i; j--)
                System.out.print(j + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q136 Pattern: 5 4 3 2 1, 4 3 2 1, ...

```
5 4 3 2 1
4 3 2 1
3 2 1
2 1
1
```

Q_136.java

```java
public class Q_136
{
    public static void main(String[] args)
    {
        for (int i = 5; i >= 1; i--)
        {
            for (int j = i; j >= 1; j--)
                System.out.print(j + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q137 Pattern: 1, 2 1, 3 2 1, ...

```
1
2 1
3 2 1
4 3 2 1
5 4 3 2 1
```

Q_137.java

```java
public class Q_137
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 5; i++)
        {
            for (int j = i; j >= 1; j--)
                System.out.print(j + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q138 Pattern: 5 5 5 5 5, 4 4 4 4, ...

```
5 5 5 5 5
4 4 4 4
3 3 3
2 2
1
```

Q_138.java

```java
public class Q_138
{
    public static void main(String[] args)
    {
        for (int i = 5; i >= 1; i--)
        {
            for (int j = 1; j <= i; j++)
                System.out.print(i + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q139 Floyd's Triangle

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Q_139.java

```java
public class Q_139
{
    public static void main(String[] args)
    {
        int k = 1;
        for (int i = 1; i <= 5; i++)
        {
            for (int j = 1; j <= i; j++)
            {
                System.out.print(k + " ");
                k++;
            }
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
k	int	to store the number to print
i	int	to control the rows
j	int	to control the columns

Q140 Pattern: 1, 0 1, 1 0 1, ...

```
1
0 1
1 0 1
0 1 0 1
1 0 1 0 1
```

Q_140.java

```java
public class Q_140
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 5; i++)
        {
            for (int j = 1; j <= i; j++)
            {
                if ((i + j) % 2 == 0)
                    System.out.print("1 ");
                else
                    System.out.print("0 ");
            }
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q141 Pattern: 1, 2 4, 3 6 9, ...

```
1
2 4
3 6 9
4 8 12 16
5 10 15 20 25
```

Q_141.java

```java
public class Q_141
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 5; i++)
        {
            for (int j = 1; j <= i; j++)
                System.out.print(i * j + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q142 Pattern: 1, 1 3, 1 3 5, ...

```
1
1 3
1 3 5
1 3 5 7
1 3 5 7 9
```

Q_142.java

```java
public class Q_142
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 5; i++)
        {
            for (int j = 1; j <= i; j++)
                System.out.print((2 * j - 1) + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q143 Pattern: Right-Angled Triangle of Stars

```
*
* *
* * *
* * * *
* * * * *
```

Q_143.java

```java
public class Q_143
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 5; i++)
        {
            for (int j = 1; j <= i; j++)
                System.out.print("* ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q144 Pattern: Inverted Triangle of Stars

```
* * * * *
* * * *
* * *
* *
*
```

Q_144.java

```java
public class Q_144
{
    public static void main(String[] args)
    {
        for (int i = 5; i >= 1; i--)
        {
            for (int j = 1; j <= i; j++)
                System.out.print("* ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q145 Pattern: Alternating # and * Triangle

```
#
* #
# * #
* # * #
# * # * #
```

Q_145.java

```java
public class Q_145
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 5; i++)
        {
            for (int j = 1; j <= i; j++)
            {
                if ((i + j) % 2 == 0)
                    System.out.print("# ");
                else
                    System.out.print("* ");
            }
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q146 Pattern: A, A B, A B C, ...

```
A
A B
A B C
A B C D
A B C D E
```

Q_146.java

```java
public class Q_146
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 5; i++)
        {
            for (int j = 1; j <= i; j++)
                System.out.print((char) ('A' + j - 1) + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q147 Pattern: A, B B, C C C, ...

```
A
B B
C C C
D D D D
E E E E E
```

Q_147.java

```java
public class Q_147
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 5; i++)
        {
            char ch = (char) ('A' + i - 1);
            for (int j = 1; j <= i; j++)
                System.out.print(ch + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
ch	char	to store the letter of the row
j	int	to control the columns

Q148 Pattern: A B C D E, A B C D, ...

```
A B C D E
A B C D
A B C
A B
A
```

Q_148.java

```java
public class Q_148
{
    public static void main(String[] args)
    {
        for (int i = 5; i >= 1; i--)
        {
            for (char ch = 'A'; ch < 'A' + i; ch++)
                System.out.print(ch + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
ch	char	to store the letter to print

Q149 Pattern: E, D E, C D E, ...

```
E
D E
C D E
B C D E
A B C D E
```

Q_149.java

```java
public class Q_149
{
    public static void main(String[] args)
    {
        for (char start = 'E'; start >= 'A'; start--)
        {
            for (char ch = start; ch <= 'E'; ch++)
                System.out.print(ch + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
start	char	to store the first letter of the row
ch	char	to store the letter to print

Q150 Floyd's Triangle of Letters

```
A
B C
D E F
G H I J
K L M N O
```

Q_150.java

```java
public class Q_150
{
    public static void main(String[] args)
    {
        char ch = 'A';
        for (int i = 1; i <= 5; i++)
        {
            for (int j = 1; j <= i; j++)
            {
                System.out.print(ch + " ");
                ch++;
            }
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
ch	char	to store the letter to print
i	int	to control the rows
j	int	to control the columns

Q151 Pattern: Rows of Numbers and Letters Alternately

```
1
A B
1 2 3
A B C D
1 2 3 4 5
```

Q_151.java

```java
public class Q_151
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 5; i++)
        {
            for (int j = 1; j <= i; j++)
            {
                if (i % 2 == 1)
                    System.out.print(j + " ");
                else
                    System.out.print((char) ('A' + j - 1) + " ");
            }
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q152 Rectangular Pattern: 1 2 3 4 5 in Every Row

```
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
```

Q_152.java

```java
public class Q_152
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 4; i++)
        {
            for (int j = 1; j <= 5; j++)
                System.out.print(j + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q153 Rectangular Pattern: Row Number Repeated

```
1 1 1 1 1
2 2 2 2 2
3 3 3 3 3
4 4 4 4 4
```

Q_153.java

```java
public class Q_153
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 4; i++)
        {
            for (int j = 1; j <= 5; j++)
                System.out.print(i + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q154 Rectangular Pattern: 1 2 3 4 5, 2 3 4 5 6, ...

```
1 2 3 4 5
2 3 4 5 6
3 4 5 6 7
4 5 6 7 8
5 6 7 8 9
```

Q_154.java

```java
public class Q_154
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 5; i++)
        {
            for (int j = i; j <= i + 4; j++)
                System.out.print(j + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to store the number to print

Q155 Hollow Rectangle of Stars

```
* * * * *
*       *
*       *
* * * * *
```

Q_155.java

```java
public class Q_155
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 4; i++)
        {
            for (int j = 1; j <= 5; j++)
            {
                if (i == 1 || i == 4 || j == 1 || j == 5)
                    System.out.print("* ");
                else
                    System.out.print("  ");
            }
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q156 Rectangle with Alternate Rows of * and

```
* * * * *
# # # # #
* * * * *
# # # # #
```

Q_156.java

```
public class Q_156
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 4; i++)
        {
            for (int j = 1; j <= 5; j++)
            {
                if (i % 2 == 1)
                    System.out.print("* ");
                else
                    System.out.print("# ");
            }
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q157 Pyramid of Stars

Extra Write a program to display the following pattern:

```
    *
   * *
  * * *
 * * * *
* * * * *
```

Q_157.java

```
public class Q_157
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 5; i++)
        {
            for (int s = 1; s <= 5 - i; s++)
                System.out.print(" ");
            for (int j = 1; j <= i; j++)
                System.out.print("* ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows

Name of the Variable	Data Type	Purpose / Description
s	int	to print the leading spaces
j	int	to print the stars

Q158 Diamond of Stars

Extra Write a program to display the following pattern:

```
    *
   * *
  * * *
 * * * *
* * * * *
 * * * *
  * * *
   * *
    *
```

Q_158.java

```java
public class Q_158
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 5; i++)
        {
            for (int s = 1; s <= 5 - i; s++)
                System.out.print(" ");
            for (int j = 1; j <= i; j++)
                System.out.print("* ");
            System.out.println();
        }
        for (int i = 4; i >= 1; i--)
        {
            for (int s = 1; s <= 5 - i; s++)
                System.out.print(" ");
            for (int j = 1; j <= i; j++)
                System.out.print("* ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
s	int	to print the leading spaces
j	int	to print the stars

Q159 Number Pyramid: 1, 1 2 1, 1 2 3 2 1, ...

Extra Write a program to display the following pattern:

```
        1
      1 2 1
    1 2 3 2 1
  1 2 3 4 3 2 1
1 2 3 4 5 4 3 2 1
```

Q_159.java

```java
public class Q_159
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 5; i++)
        {
            for (int s = 1; s <= 2 * (5 - i); s++)
                System.out.print(" ");
            for (int j = 1; j <= i; j++)
                System.out.print(j + " ");
            for (int j = i - 1; j >= 1; j--)
                System.out.print(j + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
s	int	to print the leading spaces
j	int	to print the numbers

Q160 Pascal's Triangle

Extra Write a program to display the following pattern:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
```

Q_160.java

```java
public class Q_160
{
    public static void main(String[] args)
    {
        for (int i = 0; i <= 5; i++)
        {
            int c = 1;
            for (int j = 0; j <= i; j++)
            {
                System.out.print(c + " ");
                c = c * (i - j) / (j + 1);
            }
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
c	int	to store the current coefficient
j	int	to control the columns

Q161 Hourglass of Stars

Extra Write a program to display the following pattern:

```
* * * * *
 * * * *
  * * *
   * *
    *
   * *
  * * *
 * * * *
* * * * *
```

Q_161.java

```java
public class Q_161
{
    public static void main(String[] args)
    {
        for (int i = 5; i >= 1; i--)
        {
            for (int s = 1; s <= 5 - i; s++)
                System.out.print(" ");
            for (int j = 1; j <= i; j++)
                System.out.print("* ");
            System.out.println();
        }
        for (int i = 2; i <= 5; i++)
        {
            for (int s = 1; s <= 5 - i; s++)
                System.out.print(" ");
            for (int j = 1; j <= i; j++)
                System.out.print("* ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
s	int	to print the leading spaces
j	int	to print the stars

Q162 First n Multiples of Each Number from 1 to a Limit

Write a program to accept a limit L and a number n, and generate the first n multiples of every number from 1 to L, each number's multiples on a separate line, using nested for loops.

Example: L = 3, n = 4 1 : 1 2 3 4 2 : 2 4 6 8 3 : 3 6 9 12

Q_162.java

```java
import java.util.Scanner;

public class Q_162
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the limit L: ");
        int lim = sc.nextInt();
        System.out.print("Enter the number of multiples n: ");
```

```
        int n = sc.nextInt();

        if (lim < 1 || n < 1)
        {
            System.out.println("Invalid input");
        }
        else
        {
            for (int i = 1; i <= lim; i++)
            {
                System.out.print(i + " : ");
                for (int j = 1; j <= n; j++)
                    System.out.print(i * j + " ");
                System.out.println();
            }
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
lim	int	to store the limit
n	int	to store the number of multiples
i	int	to store the current number
j	int	to act as the multiplier

Q163 Multiplication Tables from 1 to 10 in a Grid

> Write a program to display the multiplication tables of 1 to 10 in the form of a 10 × 10 grid, where the entry in row i and column j is i x j. Separate the columns with tab spaces.

Q_163.java

```
public class Q_163
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 10; i++)
        {
            for (int j = 1; j <= 10; j++)
                System.out.print(i * j + "\t");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the rows
j	int	to control the columns

Q164 Prime Numbers between Two Limits

> Write a program to accept two limits and display all the prime numbers between them (both inclusive), along with their count. If the first limit is greater than the second, interchange them.

Q_164.java

```java
import java.util.Scanner;

public class Q_164
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the two limits: ");
        int a = sc.nextInt();
        int b = sc.nextInt();
        if (a > b)
        {
            int t = a;
            a = b;
            b = t;
        }

        int count = 0;
        for (int n = a; n <= b; n++)
        {
            boolean prime = n > 1;
            for (int i = 2; i * i <= n; i++)
            {
                if (n % i == 0)
                {
                    prime = false;
                    break;
                }
            }
            if (prime)
            {
                System.out.print(n + " ");
                count++;
            }
        }
        System.out.println();
        System.out.println("Count = " + count);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	int	to store the lower limit
b	int	to store the upper limit
t	int	to hold a value while swapping
count	int	to count the primes
n	int	to store the number being tested
prime	boolean	to record whether n is prime
i	int	to act as the trial divisor

Q165 Perfect Numbers from 1 to n

> Write a program to accept n and display all the Perfect numbers from 1 to n. A Perfect number is equal to the sum of its factors excluding itself (e.g. 6 = 1 + 2 + 3).

Q_165.java

```java
import java.util.Scanner;

public class Q_165
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();

        for (int num = 2; num <= n; num++)
        {
            int sum = 0;
            for (int i = 1; i <= num / 2; i++)
            {
                if (num % i == 0)
                    sum = sum + i;
            }
            if (sum == num)
                System.out.print(num + " ");
        }
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the upper limit
num	int	to store the number being tested
sum	int	to store the sum of factors
i	int	to act as the possible factor

Q166 Armstrong Numbers in a Range

> Write a program to accept two limits and display all the Armstrong numbers between them. An Armstrong number is equal to the sum of its digits each raised to the power of the number of digits (e.g. 153, 370, 371, 407, 1634).

Q_166.java

```java
import java.util.Scanner;

public class Q_166
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the two limits: ");
        int a = sc.nextInt();
        int b = sc.nextInt();

        for (int n = Math.max(a, 1); n <= b; n++)
        {
            int count = 0;
            for (int t = n; t > 0; t = t / 10)
                count++;
            int sum = 0;
            for (int t = n; t > 0; t = t / 10)
                sum = sum + (int) Math.pow(t % 10, count);
            if (sum == n)
                System.out.print(n + " ");
```

```
        }
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	int	to store the lower limit
b	int	to store the upper limit
n	int	to store the number being tested
count	int	to count the digits
t	int	to store a working copy of n
sum	int	to store the sum of powers

Q167 Palindrome Numbers in a Range

> Write a program to accept two limits and display all the palindrome numbers between them and their count. A palindrome number reads the same when reversed (e.g. 121, 1331).

Q_167.java

```java
import java.util.Scanner;

public class Q_167
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the two limits: ");
        int a = sc.nextInt();
        int b = sc.nextInt();
        int count = 0;

        for (int n = Math.max(a, 0); n <= b; n++)
        {
            int rev = 0;
            for (int t = n; t > 0; t = t / 10)
                rev = rev * 10 + t % 10;
            if (rev == n)
            {
                System.out.print(n + " ");
                count++;
            }
        }
        System.out.println();
        System.out.println("Count = " + count);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	int	to store the lower limit
b	int	to store the upper limit
count	int	to count the palindromes
n	int	to store the number being tested

Name of the Variable	Data Type	Purpose / Description
rev	int	to store the reverse of n
t	int	to store a working copy of n

Q168 Twin Primes up to n

> Twin primes are pairs of prime numbers that differ by 2, e.g. (3, 5), (5, 7), (11, 13). Write a program to accept n and display all the pairs of twin primes up to n.

Q_168.java

```java
import java.util.Scanner;

public class Q_168
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();

        for (int a = 3; a + 2 <= n; a += 2)
        {
            boolean p1 = true, p2 = true;
            for (int i = 2; i * i <= a; i++)
            {
                if (a % i == 0)
                {
                    p1 = false;
                    break;
                }
            }
            for (int i = 2; i * i <= a + 2; i++)
            {
                if ((a + 2) % i == 0)
                {
                    p2 = false;
                    break;
                }
            }
            if (p1 && p2)
                System.out.print("(" + a + ", " + (a + 2) + ") ");
        }
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the upper limit
a	int	to store the first number of a pair
p1	boolean	to record whether a is prime
p2	boolean	to record whether a + 2 is prime
i	int	to act as the trial divisor

Q169 Sum of the Series 1! + 2! + 3! + ... + n!

Write a program to accept n and find the sum of the series:

`S = 1! + 2! + 3! + ... + n!`

using nested loops (the inner loop calculates each factorial). Use long, and accept n only from 1 to 20.

Q_169.java

```java
import java.util.Scanner;

public class Q_169
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();

        if (n < 1 || n > 20)
        {
            System.out.println("Invalid input");
        }
        else
        {
            long s = 0;
            for (int i = 1; i <= n; i++)
            {
                long f = 1;
                for (int j = 1; j <= i; j++)
                    f = f * j;
                s = s + f;
            }
            System.out.println("Sum = " + s);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of terms
s	long	to store the sum
i	int	to control the terms
f	long	to store the factorial of i
j	int	to act as the inner loop counter

Q170 Sum of the Series 1 + (1+2) + (1+2+3) + ...

Write a program to accept n and find the sum of the series using nested loops:

`S = 1 + (1 + 2) + (1 + 2 + 3) + ... + (1 + 2 + 3 + ... + n)`

Q_170.java

```java
import java.util.Scanner;

public class Q_170
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
```

```
        int n = sc.nextInt();
        long s = 0;

        for (int i = 1; i <= n; i++)
        {
            long term = 0;
            for (int j = 1; j <= i; j++)
                term = term + j;
            s = s + term;
        }
        System.out.println("Sum = " + s);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of terms
s	long	to store the sum
i	int	to control the terms
term	long	to store the current term
j	int	to act as the inner loop counter

Q171 Sum of the Series x/1! + x^2/2! + ... + x^n/n!

Write a program to accept x and n and find the sum of the series using nested loops:

`S = x/1! + x^2/2! + x^3/3! + ... + x^n/n!`

Q_171.java

```
import java.util.Scanner;

public class Q_171
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter x and n: ");
        double x = sc.nextDouble();
        int n = sc.nextInt();
        double s = 0;

        for (int i = 1; i <= n; i++)
        {
            double f = 1;
            for (int j = 1; j <= i; j++)
                f = f * j;
            s = s + Math.pow(x, i) / f;
        }
        System.out.println("Sum = " + s);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
x	double	to store the value of x
n	int	to store the number of terms
s	double	to store the sum

Name of the Variable	Data Type	Purpose / Description
i	int	to control the terms
f	double	to store the factorial of i
j	int	to act as the inner loop counter

Q172 Sum of the Series 1 - x^2/2! + x^4/4! - ...

Write a program to accept x and n and find the sum of the first n terms of the series using nested loops:

`S = 1 - x^2/2! + x^4/4! - x^6/6! + ...`

Q_172.java

```java
import java.util.Scanner;

public class Q_172
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter x and n: ");
        double x = sc.nextDouble();
        int n = sc.nextInt();
        double s = 0;
        int sign = 1;

        for (int i = 0; i < n; i++)
        {
            int p = 2 * i;
            double f = 1;
            for (int j = 1; j <= p; j++)
                f = f * j;
            s = s + sign * Math.pow(x, p) / f;
            sign = -sign;
        }
        System.out.println("Sum = " + s);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
x	double	to store the value of x
n	int	to store the number of terms
s	double	to store the sum
sign	int	to store the sign of the term
i	int	to control the terms
p	int	to store the power of the term
f	double	to store the factorial of p
j	int	to act as the inner loop counter

Q173 Disarium Numbers between Two Limits

Write a program to accept two limits and display all the Disarium numbers between them. A Disarium number is equal to the sum of its digits raised to the power of their respective positions (e.g. 135 = 1^1 + 3^2 + 5^3).

Q_173.java

```java
import java.util.Scanner;

public class Q_173
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the two limits: ");
        int a = sc.nextInt();
        int b = sc.nextInt();

        for (int n = Math.max(a, 1); n <= b; n++)
        {
            int pos = 0;
            for (int t = n; t > 0; t = t / 10)
                pos++;
            int sum = 0;
            for (int t = n; t > 0; t = t / 10)
            {
                sum = sum + (int) Math.pow(t % 10, pos);
                pos--;
            }
            if (sum == n)
                System.out.print(n + " ");
        }
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	int	to store the lower limit
b	int	to store the upper limit
n	int	to store the number being tested
pos	int	to store the position of a digit
t	int	to store a working copy of n
sum	int	to store the sum of powers

Q174 break and continue in Nested Loops

Write a program to show how break and continue work in nested loops. For i = 1 to 4 and j = 1 to 4, display the pairs (i, j), but

- skip the pair when i is equal to j (use continue), and
- stop the inner loop as soon as i + j becomes greater than 5 (use break).

Display the pairs of each value of i on a separate line.

Q_174.java

```java
public class Q_174
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 4; i++)
        {
            System.out.print("i = " + i + " : ");
            for (int j = 1; j <= 4; j++)
            {
```

```
                if (i + j > 5)
                    break;
                if (i == j)
                    continue;
                System.out.print("(" + i + ", " + j + ") ");
            }
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
i	int	to control the outer loop
j	int	to control the inner loop

Q175 Pythagorean Triplets up to n

> **Extra** A Pythagorean triplet is a set of three positive integers a < b < c such that a^2 + b^2 = c^2, e.g. (3, 4, 5). Write a program to accept n and display all the Pythagorean triplets with c not greater than n.

Q_175.java

```
import java.util.Scanner;

public class Q_175
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();
        int count = 0;

        for (int a = 1; a <= n; a++)
        {
            for (int b = a + 1; b <= n; b++)
            {
                int csq = a * a + b * b;
                int c = (int) Math.round(Math.sqrt(csq));
                if (c <= n && c * c == csq)
                {
                    System.out.println("(" + a + ", " + b + ", " + c + ")");
                    count++;
                }
            }
        }
        System.out.println("Number of triplets = " + count);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the upper limit for c
count	int	to count the triplets
a	int	to store the first number
b	int	to store the second number
csq	int	to store a squared plus b squared

Name of the Variable	Data Type	Purpose / Description
c	int	to store the possible third number

CHAPTER

09



Class as the Basis of all Computation

Objects encapsulate state and behaviour · Member variables and methods · Class as an abstraction and object factory · Primitive and composite data types · Objects as instances

6 programs

Q176 – Q181

Theory

📑 Contents

1. Objects: state and behaviour
2. Class: the basis of all computation
3. Members of a class
4. Creating objects — instances of a class
5. Primitive vs composite data types
6. Reference variables: `=`, `==` and `null`
7. Writing a class — a complete example
8. Predict the output
9. Find the errors
10. Differentiate between
11. One-line facts
12. Test yourself
13. Programs in this chapter

1. Objects: state and behaviour

An **object** is an entity that has a **state** and a **behaviour**.

- The **state** of an object is given by the **values of its attributes (characteristics / features)**. In Java, attributes are represented by **member variables** (data members / instance variables). *Variables define state.*
- The **behaviour** of an object is what it can **do**. In Java, behaviour is represented by **member methods** (functions / operations / messages). *Methods define behaviour.*

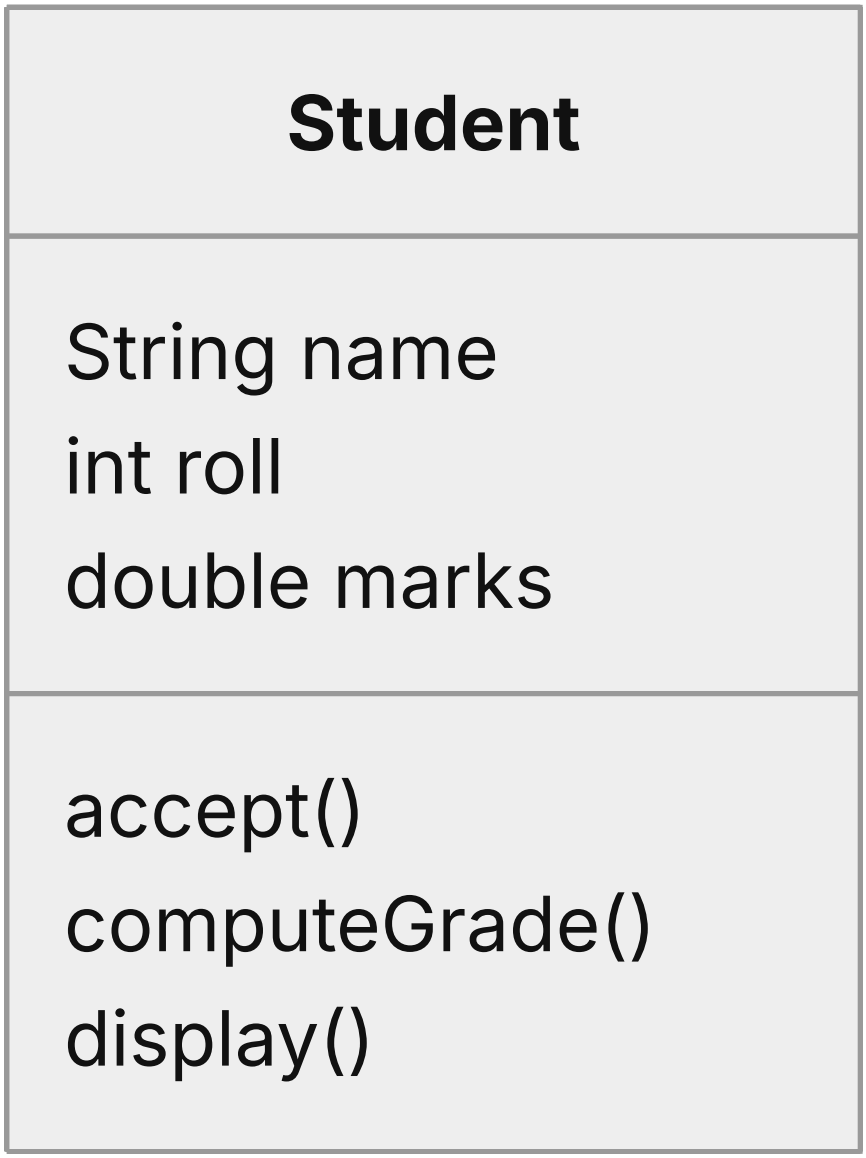
Object	Attributes → member variables	Behaviour → member methods
Fan	speed, colour, isOn	switchOn(), switchOff(), changeSpeed()
Student	name, roll, marks	readBook(), writeExam(), computeGrade()
Mobile phone	brand, model, battery	call(), charge(), sendSMS()
Bank account	accNo, holder, balance	deposit(), withdraw(), showBalance()

An object **encapsulates** (wraps together) its state and behaviour, and the methods operate on the object's own data.

2. Class: the basis of all computation

A **class** is a **template / blueprint** that describes the **common attributes and behaviour** of a group of similar objects. In Java **every** computation happens inside methods of classes — even `main()` lives in a class — hence the class is called the **basis of all computation**.

View of a class	Meaning
Class as an abstraction	A class captures only the **essential** features of a set of objects (e.g. every Car has a brand and speed) and leaves out details of any particular object
Class as an object factory	Any number of objects with similar features can be **produced** from one class
Class as a user-defined data type	Once defined, a class name can be used to declare variables, just like `int`: `Student s1;`



Objects `s1`, `s2`, `s3` created from this class each have their **own** `name`, `roll` and `marks`, but share the **same** methods.

---

3. Members of a class

```
class Student                       // class header
{
    String name;                    // ┐
    int roll;                       //  ├─ data members (instance variables) → STATE
    double marks;                   // ┘

    void accept() { … }             // ┐
    char grade() { … }              //  ├─ member methods → BEHAVIOUR
    void display() { … }            // ┘
}
```

Term	Meaning
Data members / instance variables / attributes	Variables declared inside the class but outside every method; each object gets its own copy

Term	Meaning
Member methods / functions	Methods defined inside the class that describe its behaviour
Members	Data members + member methods

> **NOTE**
>
> Instance variables get **default values** automatically (`0` , `0.0` , `false` , `'\u0000'` , `null`) when an object is created; local variables inside methods do not.

4. Creating objects — instances of a class

An object is an **instance** of a class. Creating an object is called **instantiation**.

```
Student s1;                 // (1) declaration of a reference variable
s1 = new Student();         // (2) instantiation using new + (3) initialisation by the constructor
Student s2 = new Student(); // all three steps in one statement
```

Step	What happens
Declaration	A reference variable `s1` of type `Student` is created; it does not yet point to any object (`null` if it is a data member)
Instantiation	The `new` operator allocates memory for a new object (in the **heap**) and returns its address
Initialisation	The **constructor** `Student()` initialises the object; the address is stored in `s1`

Accessing members — use the **dot operator**:

```
s1.name = "Riya";       // data member
s1.display();           // member method → "sending the message display() to s1"
```

> **IMPORTANT**
>
> **Computation as message passing:** calling a method of an object is called **sending a message** to it. `acc1.withdraw(500)` sends the message *withdraw 500* to the object `acc1` , which changes only **its own** balance.

5. Primitive vs composite data types

Primitive (intrinsic) data types are built into Java and hold a **single value**: `byte` , `short` , `int` , `long` , `float` , `double` , `char` , `boolean` .

Composite (non-primitive / reference / user-defined) data types are made up of **one or more primitive and/or other composite types**. A class is a composite type. Arrays and `String` are also composite types.

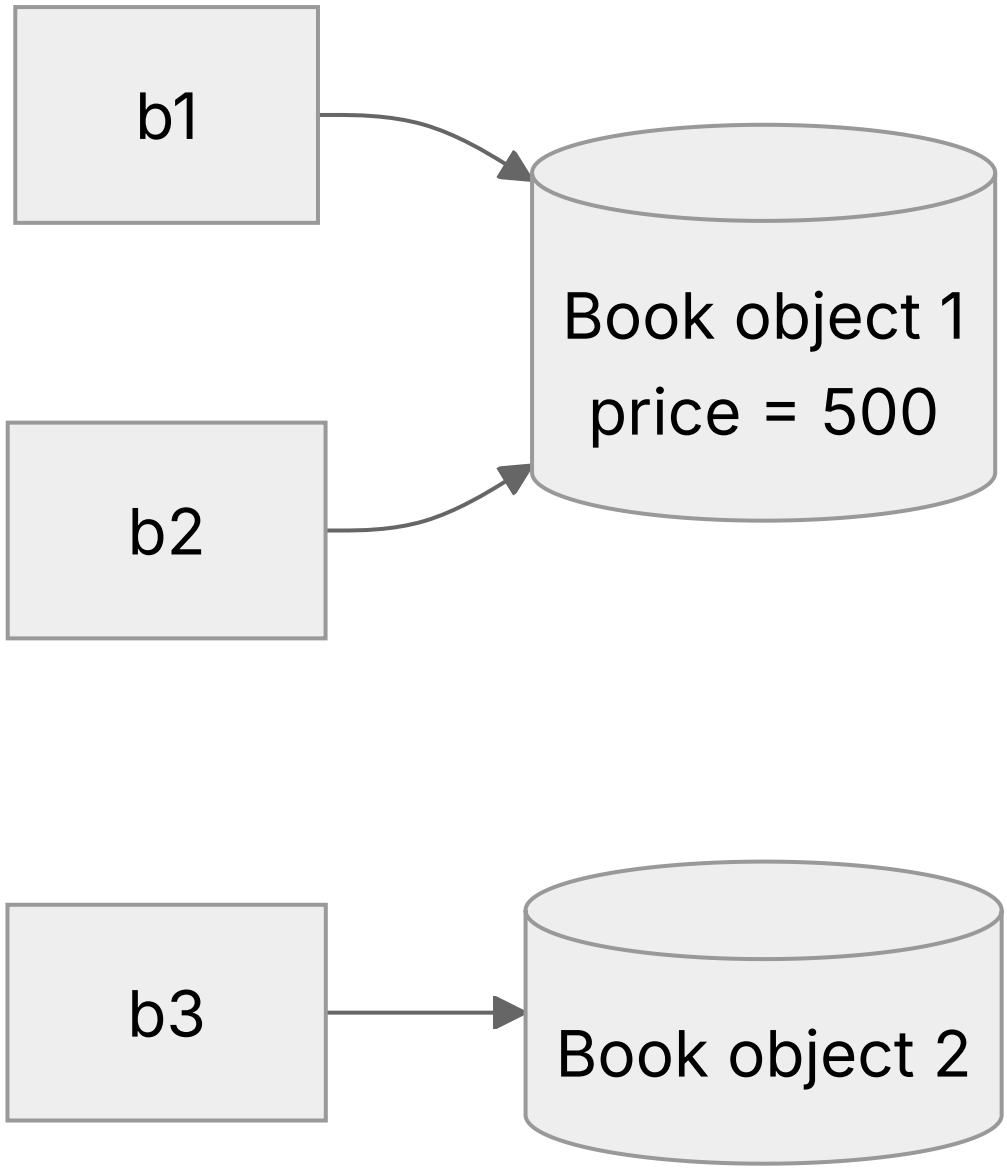
```
int age = 15;               // primitive variable – stores the value 15 itself
Student s = new Student();  // composite variable – stores a reference (address) of an object
```

Basis	Primitive data type	Composite data type (class)
Defined by	Java language	The programmer / Java library
Contains	A single value	Several values (data members) and methods

Basis	Primitive data type	Composite data type (class)
Variable stores	The **value**	The **reference (address)** of an object
Size	Fixed (e.g. `int` = 4 bytes)	Depends on its members
Memory	Allocated when declared	Object allocated when created using `new`
Default value (as a data member)	`0` , `0.0` , `false` , `'\u0000'`	`null`
Example	`int` , `double` , `char`	`String` , arrays, `Student` , `Scanner`

6. Reference variables: `=` , `==` and `null`

```
Book b1 = new Book();
Book b2 = b1;           // b2 now refers to the SAME object as b1 — no new object is created
b2.price = 500;         // changes the object, so b1.price is also 500
Book b3 = new Book();   // a different object
```



- `b1 == b2` is `true` — both references hold the **same address**.

- `b1 == b3` is `false` even if all their data members are equal — `==` **compares references, not contents**.
- A reference that does not refer to any object has the value `null`. Calling a method through a `null` reference causes a `NullPointerException` at run time.

7. Writing a class — a complete example

```
public class Circle
{
    double radius;                          // state

    void setRadius(double r)                // behaviour
    {
        radius = r;
    }

    double area()
    {
        return Math.PI * radius * radius;
    }

    public static void main(String[] args)
    {
        Circle c1 = new Circle();           // object 1
        Circle c2 = new Circle();           // object 2
        c1.setRadius(7);
        c2.setRadius(3.5);
        System.out.println(c1.area());      // 153.93804002589985
        System.out.println(c2.area());      // 38.48451000647496
    }
}
```

> **TIP**
>
> In ICSE class-based questions, follow the specification **exactly**: same class name, same member names and types, same method signatures. Marks are given for each correctly defined member.

8. Predict the output

```
class Counter
{
    int count;
    void up() { count++; }

    public static void main(String[] args)
    {
        Counter a = new Counter();
        Counter b = new Counter();
        Counter c = a;
        a.up(); a.up(); b.up(); c.up();
        System.out.println(a.count + " " + b.count + " " + c.count);
    }
}
```

Show answer

`a` and `c` refer to the same object, which is incremented three times (`a.up()` twice and `c.up()` once). `b` is a separate object incremented once. **Output:** `3 1 3`

9. Find the errors

```
class Box
{
    double side;
    double volume()
    {
        return side * side * side;
    }
    public static void main(String[] args)
    {
        Box b;
        b.side = 4;
        System.out.println(volume());
    }
}
```

Show corrected code

```
Box b = new Box();                  // a local reference must refer to an object before use
b.side = 4;
System.out.println(b.volume());     // a non-static method must be called through an object
```

10. Differentiate between

Pair	Difference
Class vs object	Blueprint (logical) vs instance (physical, occupies memory)
Data member vs member method	Represents state vs represents behaviour
Primitive vs composite type	Single built-in value vs user-defined type made of other types
Reference vs object	Variable holding an address vs the actual memory block created by `new`
`b1 == b2` vs equal data	Same object vs different objects with the same values

11. One-line facts

- An object's **state** is given by its **data members**; its **behaviour** by its **member methods**.
- A class is called an **object factory** because many objects can be created from it.
- A class is a **user-defined / composite** data type.
- An object is an **instance** of a class; creating it is **instantiation**.
- The `new` operator allocates memory for an object and returns its **reference**.
- Assigning one reference to another (`b2 = b1`) does **not** create a new object.
- The default value of a reference data member is `null`.

🧠 Test Yourself

Part A — Multiple choice

1. `Student s1 = new Student(), s2 = new Student(); s2 = s1;` How many `Student` objects exist and how many can still be reached?
   (a) 2 created, 2 reachable (b) 2 created, 1 reachable (c) 1 created (d) 3 created
2. Which statement about a class is **false**?
   (a) It is a user-defined data type (b) It occupies memory for its data members as soon as it is defined (c) It is a blueprint for objects (d) It can be used to declare variables
3. *Assertion (A):* Two objects of the same class can have different states.
   Reason (R): Each object gets its own copy of the instance variables.
   (a) Both true; R explains A (b) Both true; R doesn't explain A (c) A true, R false (d) A false, R true
4. `Point p = new Point(); Point q = new Point(); p.x = 5; q.x = 5; System.out.println(p == q);` prints
   (a) true (b) false (c) 5 (d) Error
5. What happens? `Point p = null; p.x = 3;`
   (a) Compile-time error (b) `NullPointerException` at run time (c) p.x becomes 3 (d) A new object is created
6. Which is a composite data type? (a) `char` (b) `boolean` (c) `int[]` (d) `long`
7. The behaviour of an object is defined by its (a) data members (b) constructor only (c) member methods (d) class name

Part B — Short answer questions

1. Why is a class called the basis of all computation?
2. "A class is an abstraction for a set of objects." Explain.
3. Give two differences between primitive and composite data types.
4. Write a statement to create an object `m1` of class `Mobile`. Name the operator used and state its function.
5. Identify the state and behaviour: a `Lamp` that can be switched on or off and has a wattage and colour.
6. What is meant by "objects as instances of a class"?
7. What is printed? `Lamp a = new Lamp(); Lamp b = a; b.watt = 60; a.watt = a.watt + 40; System.out.println(b.watt);`

✅ Answers — Part A

Q	Ans	Reason
1	(b)	The object first referred to by `s2` has no reference now
2	(b)	Memory is allocated only when an object is created
3	(a)	—
4	(b)	Different objects, so different references
5	(b)	—
6	(c)	Arrays are composite types
7	(c)	—

✅ Answers — Part B

1. Because in Java all data and all operations (methods) must be written inside classes; every computation is performed by objects of classes calling methods.

2. A class captures only the essential, common attributes and behaviour of all its objects, leaving out the specific details of individual objects.
3. A primitive type is built-in and stores a single value; a composite type is user-defined, can contain many values and methods, and its variable stores a reference.
4. `Mobile m1 = new Mobile();` — the `new` operator allocates memory for the object and returns its reference.
5. State: wattage, colour, on/off status. Behaviour: switchOn(), switchOff().
6. Each object is a concrete example (instance) created from the class template, having its own values for the attributes defined by the class.
7. `100` — `a` and `b` refer to the same object.

💻 Programs in this chapter

No.	Program
Q176	Class Rectangle with input(), area() and display()
Q177	Class Student: Average and Grade
Q178	Class BankAccount: Two Objects with Deposit and Withdraw
Q179	Class Point: Distance from the Origin
Q180	Class Temperature with Conversion Methods
Q181	Class Book as a Composite Data Type: References and Objects

Programs Q176 – Q181

Q176 Class Rectangle with input(), area() and display()

Define a class Rectangle (written here as class Q_176) with the following description:

```
Data members  : double length, breadth
void input()   : to accept the length and breadth from the user
double area()  : to calculate and return the area
double perimeter() : to calculate and return the perimeter
void display() : to display the length, breadth, area and
                 perimeter
```

Write a main method to create an object of the class and call the member methods. If a dimension entered is zero or negative, display "Invalid dimensions" instead of the results.

Q_176.java

```java
import java.util.Scanner;

public class Q_176
{
    double length, breadth;

    void input()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter length and breadth: ");
        length = sc.nextDouble();
        breadth = sc.nextDouble();
    }

    double area()
    {
        return length * breadth;
    }

    double perimeter()
    {
        return 2 * (length + breadth);
    }

    void display()
    {
        if (length <= 0 || breadth <= 0)
        {
            System.out.println("Invalid dimensions");
        }
        else
        {
            System.out.println("Length    = " + length);
            System.out.println("Breadth   = " + breadth);
            System.out.println("Area      = " + area());
            System.out.println("Perimeter = " + perimeter());
        }
    }

    public static void main(String[] args)
    {
        Q_176 r = new Q_176();
        r.input();
        r.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
length	double	to store the length
breadth	double	to store the breadth
r	Q_176	to refer to the rectangle object

Q177 Class Student: Average and Grade

Define a class Student (written here as class Q_177) with the following description:

```
Data members : String name; int m1, m2, m3; double avg; char grade
void accept()  : to accept the name and marks in three subjects
void compute() : to calculate the average and assign the grade:
                   average >= 80  : 'A'
                   60 to below 80 : 'B'
                   40 to below 60 : 'C'
                   below 40       : 'D'
void display() : to display the name, marks, average and grade
```

Write a main method to create an object and call the methods.

Q_177.java

```java
import java.util.Scanner;

public class Q_177
{
    String name;
    int m1, m2, m3;
    double avg;
    char grade;

    void accept()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter name: ");
        name = sc.nextLine();
        System.out.print("Enter marks in three subjects: ");
        m1 = sc.nextInt();
        m2 = sc.nextInt();
        m3 = sc.nextInt();
    }

    void compute()
    {
        avg = (m1 + m2 + m3) / 3.0;
        if (avg >= 80)
            grade = 'A';
        else if (avg >= 60)
            grade = 'B';
        else if (avg >= 40)
            grade = 'C';
        else
            grade = 'D';
    }

    void display()
    {
        System.out.println("Name    : " + name);
        System.out.println("Marks   : " + m1 + ", " + m2 + ", " + m3);
        System.out.println("Average : " + Math.round(avg * 100) / 100.0);
        System.out.println("Grade   : " + grade);
    }
```

```
    public static void main(String[] args)
    {
        Q_177 s = new Q_177();
        s.accept();
        s.compute();
        s.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
name	String	to store the student's name
m1	int	to store marks in subject 1
m2	int	to store marks in subject 2
m3	int	to store marks in subject 3
avg	double	to store the average marks
grade	char	to store the grade
s	Q_177	to refer to the student object

Q178 Class BankAccount: Two Objects with Deposit and Withdraw

Define a class BankAccount (written here as class Q_178) with the following description:

```
Data members : String holder; long accNo; double balance
void open(String h, long a, double amt) : to store the details;
               a negative opening amount is stored as 0
void deposit(double amt)  : to add amt to the balance (only if
                            amt is positive)
void withdraw(double amt) : to subtract amt from the balance if
               enough balance is available, otherwise display
               "Insufficient balance"
void display() : to display the account details
```

In main(), create two account objects and show that each object keeps its own balance: open account 1 with Rs 5000 and account 2 with Rs 1200, deposit Rs 2500 in account 1, withdraw Rs 2000 from account 2 and withdraw Rs 3000 from account 1. Display both accounts at the end.

Q_178.java

```
public class Q_178
{
    String holder;
    long accNo;
    double balance;

    void open(String h, long a, double amt)
    {
        holder = h;
        accNo = a;
        if (amt > 0)
            balance = amt;
        else
            balance = 0;
    }

    void deposit(double amt)
    {
        if (amt > 0)
            balance = balance + amt;
    }
```

```
    void withdraw(double amt)
    {
        if (amt <= balance)
            balance = balance - amt;
        else
            System.out.println("Insufficient balance in account " + accNo);
    }

    void display()
    {
        System.out.println(accNo + "  " + holder + "  Balance: Rs " + balance);
    }

    public static void main(String[] args)
    {
        Q_178 acc1 = new Q_178();
        Q_178 acc2 = new Q_178();
        acc1.open("Priya Sharma", 1001L, 5000);
        acc2.open("Arjun Mehta", 1002L, 1200);

        acc1.deposit(2500);
        acc2.withdraw(2000);
        acc1.withdraw(3000);

        acc1.display();
        acc2.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
holder	String	to store the account holder's name
accNo	long	to store the account number
balance	double	to store the balance
h	String	to receive the holder's name
a	long	to receive the account number
amt	double	to receive an amount
acc1	Q_178	to refer to the first account
acc2	Q_178	to refer to the second account

Q179 Class Point: Distance from the Origin

Define a class Point (written here as class Q_179) with the following description:

```
Data members : double x, y
void setPoint(double a, double b) : to assign the coordinates
double distance() : to return the distance of the point from
                    the origin, i.e. square root of (x^2 + y^2)
void display()    : to display the point as (x, y)
```

In main(), accept the coordinates of two points, create two objects, and display which point is farther from the origin (or "Both are equally far").

Q_179.java

```
import java.util.Scanner;

public class Q_179
{
```

```
    double x, y;

    void setPoint(double a, double b)
    {
        x = a;
        y = b;
    }

    double distance()
    {
        return Math.sqrt(x * x + y * y);
    }

    void display()
    {
        System.out.print("(" + x + ", " + y + ")");
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_179 p1 = new Q_179();
        Q_179 p2 = new Q_179();
        System.out.print("Enter x and y of the first point: ");
        p1.setPoint(sc.nextDouble(), sc.nextDouble());
        System.out.print("Enter x and y of the second point: ");
        p2.setPoint(sc.nextDouble(), sc.nextDouble());

        double d1 = p1.distance();
        double d2 = p2.distance();
        if (d1 > d2)
        {
            p1.display();
            System.out.println(" is farther from the origin");
        }
        else if (d2 > d1)
        {
            p2.display();
            System.out.println(" is farther from the origin");
        }
        else
        {
            System.out.println("Both are equally far");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
x	double	to store the x-coordinate
y	double	to store the y-coordinate
a	double	to receive the x-coordinate
b	double	to receive the y-coordinate
p1	Q_179	to refer to the first point
p2	Q_179	to refer to the second point
d1	double	to store the distance of p1
d2	double	to store the distance of p2

Q180 Class Temperature with Conversion Methods

Define a class Temperature (written here as class Q_180) with the following description:

```
Data member : double celsius
void setCelsius(double c)  : to assign the temperature
double toFahrenheit()      : to return the temperature in
                             Fahrenheit (9/5 x C + 32)
double toKelvin()          : to return the temperature in
                             Kelvin (C + 273.15)
void show()                : to display the temperature in all
                             three scales
```

In main(), accept a temperature in Celsius, create an object and call the methods. If the temperature is below absolute zero (-273.15 C), display "Temperature below absolute zero is not possible".

Q_180.java

```java
import java.util.Scanner;

public class Q_180
{
    double celsius;

    void setCelsius(double c)
    {
        celsius = c;
    }

    double toFahrenheit()
    {
        return 9.0 / 5 * celsius + 32;
    }

    double toKelvin()
    {
        return celsius + 273.15;
    }

    void show()
    {
        System.out.println("Celsius    : " + celsius);
        System.out.println("Fahrenheit : " + toFahrenheit());
        System.out.println("Kelvin     : " + toKelvin());
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter temperature in Celsius: ");
        double c = sc.nextDouble();

        if (c < -273.15)
        {
            System.out.println("Temperature below absolute zero is not possible");
        }
        else
        {
            Q_180 t = new Q_180();
            t.setCelsius(c);
            t.show();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
celsius	double	to store the temperature in celsius
c	double	to store/receive the celsius value
t	Q_180	to refer to the temperature object

Q181 Class Book as a Composite Data Type: References and Objects

Define a class Book (written here as class Q_181) with data members String title and double price, and a method void show() to display them. In main(), demonstrate that a class is a composite (reference) data type:

```
(a) create an object b1 and assign its title and price
(b) declare another reference b2 and assign b1 to it (b2 = b1);
    change the price through b2 and display b1 to show that both
    refer to the same object
(c) create a separate object b3 with the same title and price,
    and use == to show that b1 and b3 are different objects
(d) copy an int variable into another, change the copy and show
    that the original is unaffected (primitive behaviour)
```

Q_181.java

```java
public class Q_181
{
    String title;
    double price;

    void show()
    {
        System.out.println("  " + title + " : Rs " + price);
    }

    public static void main(String[] args)
    {
        Q_181 b1 = new Q_181();
        b1.title = "Computer Applications";
        b1.price = 450.0;
        System.out.println("(a) b1 :");
        b1.show();

        Q_181 b2 = b1;
        b2.price = 500.0;
        System.out.println("(b) after b2 = b1 and changing price through b2, b1 is :");
        b1.show();
        System.out.println("    b1 == b2 is " + (b1 == b2));

        Q_181 b3 = new Q_181();
        b3.title = "Computer Applications";
        b3.price = 500.0;
        System.out.println("(c) b3 has the same values, but b1 == b3 is " + (b1 == b3));

        int p = 10;
        int q = p;
        q = 99;
        System.out.println("(d) int p = 10, q = p, q = 99, but p is still " + p);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
title	String	to store the title of the book

Name of the Variable	Data Type	Purpose / Description
`price`	`double`	to store the price of the book
`b1`	`Q_181`	to refer to the first book object
`b2`	`Q_181`	to refer to the same object as b1
`b3`	`Q_181`	to refer to a separate book object
`p`	`int`	to store a primitive value
`q`	`int`	to store a copy of p

CHAPTER

10



User-Defined Methods

Need and syntax of methods · Prototype and signature · Calling methods · Actual and formal parameters · Call by value and reference · Pure and impure methods · Static and non-static methods · Method overloading

30 programs | Q182 – Q211 | 6 extra

Theory

📑 Contents

1. What is a method and why do we need it?
2. Syntax of a method
3. Method prototype and method signature
4. The `return` statement
5. Forms of methods
6. Calling (invoking) a method
7. Actual and formal parameters
8. Call by value and call by reference
9. Pure and impure methods
10. Static and non-static methods
11. Method overloading (polymorphism)
12. Predict the output
13. Find the errors
14. Differentiate between
15. One-line facts
16. Test yourself
17. Programs in this chapter

1. What is a method and why do we need it?

A **method** (function) is a **named block of code** inside a class that performs a **specific task**, and can be **called (invoked)** whenever that task is needed.

Need for methods (advantages):

1. **Modular programming** — a large program is divided into small, manageable modules.
2. **Reusability** — a method written once can be called many times; code is not repeated.
3. **Easier debugging and maintenance** — errors are confined to one small module.
4. **Abstraction** — the caller only needs to know *what* the method does, not *how*.
5. **Reduced program size.**

2. Syntax of a method

```
access_specifier  modifier  return_type  method_name ( parameter list )   // header
{
    // body: local variables and statements
    return value;     // if return_type is not void
}
```

```
public static int sum(int a, int b)
{
    int s = a + b;
    return s;
}
```

Part	Meaning	In the example
Access specifier	Where the method can be accessed from (`public` , `private` , `protected` , default)	`public`
Modifier	Optional keyword such as `static` , `final`	`static`
Return type	Data type of the value returned; **`void`** if nothing is returned	`int`
Method name	An identifier (verb-like, camelCase)	`sum`

Part	Meaning	In the example
Parameter list	Variables (with types) that receive values; may be empty `()`	`int a, int b`
Body	Statements enclosed in `{ }`	—

3. Method prototype and method signature

- **Method prototype** = the **first line (header)** of the method: access specifier, modifier, return type, name and parameter list. `public static int sum(int a, int b)`
- **Method signature** = the **method name + the number, types and order of its parameters**. The return type and access specifier are **not** part of the signature. `sum(int, int)`

> **IMPORTANT**
>
> The compiler distinguishes overloaded methods **only by their signatures**. Two methods with the same name and the same parameter list but different return types **cannot** coexist.

4. The `return` statement

`return` sends control (and a value, if any) **back to the calling statement** and **terminates** the method.

- A method can return **at most one value**.
- A `void` method may use `return;` (without a value) to exit early.
- Any statement written directly after `return` in the same block is **unreachable** — a compile-time error.
- A non-void method must return a value on **every** path:

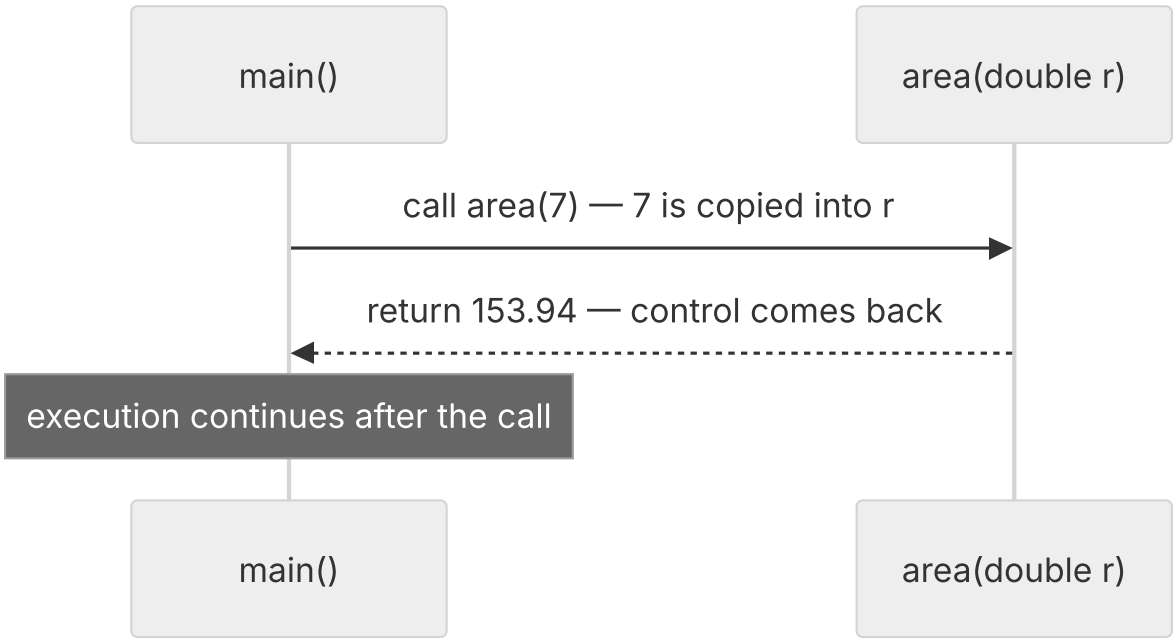
```
int sign(int n)
{
    if (n > 0) return 1;
    else if (n < 0) return -1;
    return 0;                 // without this line → "missing return statement"
}
```

5. Forms of methods

Form	Example prototype	Typical call
No arguments, no return value	`void line()`	`line();`
Arguments, no return value	`void table(int n)`	`table(7);`
No arguments, returns a value	`double readRadius()`	`double r = readRadius();`
Arguments and a return value	`int max(int a, int b)`	`int m = max(x, y);`

6. Calling (invoking) a method

Method type	Called from	Syntax
Non-static (instance)	`main()` or another static method	`objectName.method(args);`
Non-static	another non-static method of the same class	`method(args);`
Static	anywhere in the same class	`method(args);` or `ClassName.method(args);`
Static of another class	—	`ClassName.method(args);` e.g. `Math.sqrt(9)`



7. Actual and formal parameters

- **Actual parameters (arguments):** the values/variables written in the **method call**. They can be literals, variables or expressions.
- **Formal parameters:** the variables written in the **method header** that **receive** the values of the actual parameters.

```
double si(double p, double r, double t)   // p, r, t → formal parameters
{
    return p * r * t / 100;
}
...
double interest = si(5000, rate, 2 * time);   // 5000, rate, 2 * time → actual parameters
```

Basis	Actual parameters	Formal parameters
Appear in	The calling statement	The method header
Can be	Literals, variables, expressions	Only variables (with types)
Data type written?	No	Yes
Memory	Belong to the caller	Created when the method is called, destroyed when it ends

> **NOTE**
>
> The actual and formal parameters must match in **number, type (or be convertible) and order**. Their **names need not** be the same.

8. Call by value and call by reference

Call by value (pass by value): a **copy** of the actual parameter's value is passed to the formal parameter. Changes made to the formal parameter inside the method **do not affect** the actual parameter. In Java, **all primitive types** (`int`, `double`, `char`, ...) are passed by value.

```
void change(int x) { x = 100; }
...
int a = 5;
change(a);
System.out.println(a);      // 5 — unchanged
```

Call by reference (pass by reference): the **reference (address)** of the actual parameter is passed, so the method works on the **same** memory location; changes made inside the method **are reflected** in the caller. In Java, **objects and arrays** behave this way, since a copy of their **reference** is passed.

```
void doubleAll(int arr[])
{
    for (int i = 0; i < arr.length; i++)
        arr[i] = arr[i] * 2;
}
...
int a[] = {1, 2, 3};
doubleAll(a);               // a is now {2, 4, 6}
```

Basis	Call by value	Call by reference
What is passed	A copy of the value	The reference (address)
Changes inside method	Not reflected in the caller	Reflected in the caller
Used for (Java)	Primitive types	Objects, arrays
Memory	Separate copies	Same memory shared

> **WARNING**
>
> Strictly speaking, Java passes **everything by value** — for objects, the *value* passed is a copy of the reference. That is why the object's contents can be changed, but **making the formal parameter refer to a new object** does not affect the caller. For ICSE, write the definitions above.

9. Pure and impure methods

Basis	Pure method (accessor / getter)	Impure method (mutator / modifier)
Changes the state of the object (data members)?	**No**	**Yes**
Side effects	None — same arguments always give the same result	Has side effects

Basis	Pure method (accessor / getter)	Impure method (mutator / modifier)
Usually	Returns a value	May or may not return a value
Example	`int getMarks() { return marks; }`	`void setMarks(int m) { marks = m; }`

10. Static and non-static methods

Basis	Static method (class method)	Non-static method (instance method)
Keyword	Declared with `static`	No `static` keyword
Belongs to	The **class**	Each **object**
Called using	Class name or directly (no object needed)	An **object**
Can access	Only static members directly	Both static and instance members
Example	`Math.sqrt()`, `main()`	`sc.nextInt()`, `s1.display()`

> **IMPORTANT**
>
> `main()` is static, so it **cannot call a non-static method directly** — it must first create an object: `Q ob = new Q(); ob.display();`. Calling `display();` directly from `main()` gives "non-static method cannot be referenced from a static context".

11. Method overloading (polymorphism)

Method overloading is defining **two or more methods with the same name in the same class** that differ in their **parameter lists** (number, type or order of parameters). It is an example of **polymorphism** (compile-time / static polymorphism).

```
double area(double r)              { return Math.PI * r * r; }   // circle
double area(double l, double b)    { return l * b; }             // rectangle
double area(double a, double b, double c) { … }                  // triangle
```

Rules

Valid overloading	Not valid overloading
Different **number** of parameters: `f(int)`, `f(int, int)`	Only the **return type** differs: `int f(int)` and `double f(int)`
Different **types**: `f(int)`, `f(double)`	Only **parameter names** differ: `f(int a)` and `f(int b)`
Different **order** of types: `f(int, char)`, `f(char, int)`	Only the **access specifier** differs

How Java chooses the method (overload resolution):

1. An **exact match** of parameter types is chosen first.
2. If none, Java uses **automatic widening** (`byte → short → int → long → float → double`, `char → int`) and picks the closest match.
3. If two methods match equally well → **ambiguity error** at compile time.

```
void show(int x)    { System.out.println("int"); }
void show(double x) { System.out.println("double"); }
...
```

```
show(5);      // int     (exact)
show(5.0);    // double  (exact)
show('A');    // int     (char widened to int — closer than double)
show(5L);     // double  (long widened to double; no long version)
```

Advantages: one meaningful name for related tasks; programs are easier to read; it implements polymorphism.

---

12. Predict the output

Example 1

```
static int fun(int a, int b)
{
    a = a + b;
    b = a - b;
    return a * b;
}
public static void main(String[] args)
{
    int x = 3, y = 4;
    int z = fun(x, y);
    System.out.println(x + " " + y + " " + z);
}
```

Show answer

Inside: a = 7, b = 3, returns 21. `x` and `y` are unchanged (call by value). **Output:** `3 4 21`

Example 2

```
static void f(int a[], int n)
{
    a[0] = a[0] + 10;
    n = n + 10;
}
public static void main(String[] args)
{
    int arr[] = {1, 2};
    int n = 1;
    f(arr, n);
    System.out.println(arr[0] + " " + n);
}
```

Show answer

The array reference is shared, so `arr[0]` becomes 11; `n` is a primitive copy. **Output:** `11 1`

Example 3

```
static void p(long x)   { System.out.print("L "); }
static void p(double x) { System.out.print("D "); }
public static void main(String[] args)
{
    p(7);  p('c');  p(7.0f);
}
```

Show answer

`7` (int) → widens to `long` (closer than double) → L; `'c'` → int → long → L; `7.0f` (float) → double → D. **Output:** `L L D`

13. Find the errors

```
class Calc
{
    int add(int a, int b) { return a + b; }
    double add(int x, int y) { return x + y; }        // (1)
    void show(int n) { return n * 2; }                // (2)
    int square(int n) { return n * n; System.out.println(n); }   // (3)
    public static void main(String args[])
    {
        System.out.println(square(5));                // (4)
    }
}
```

Show corrections

1. Same signature `add(int, int)` — differing only in return type is not overloading. Remove it or change its parameters.
2. A `void` method cannot return a value → `int show(int n)` or remove the value.
3. Unreachable statement after `return` → move the `println` before `return`.
4. A non-static method cannot be called from static `main()` directly → `Calc ob = new Calc();` `System.out.println(ob.square(5));`

14. Differentiate between

Pair	Difference
Prototype vs signature	Full header vs name + parameter types only
Actual vs formal parameters	In the call vs in the header
Call by value vs call by reference	Copy passed, original safe vs reference passed, original changes
Pure vs impure method	Doesn't change object state vs changes it
Static vs non-static method	Called with class name vs called with an object
`void` vs non-void method	Returns nothing vs returns one value
Method overloading vs a different method	Same name, different parameters vs different name

15. One-line facts

- A method can return **only one value**.
- The keyword for "no return value" is **`void`**.
- Method signature = **name + parameter list** (not the return type).
- Primitive arguments are passed **by value**; objects and arrays **by reference**.
- An **impure** method changes the state of the object.

- A **static** method is called using the **class name**.
- Method overloading is **compile-time polymorphism**.
- Recursion (a method calling itself) is **not** in the ICSE Class 10 syllabus.

🧠 Test Yourself

Part A — Multiple choice

1. Which pair of methods can **legally** exist together in one class?
   (a) `int f(int a)` and `int f(int b)` (b) `void f(int a, double b)` and `void f(double a, int b)` (c) `int f()` and `double f()` (d) `public void f(int a)` and `private void f(int a)`
2. Given `void m(int x)` and `void m(double x)`, the call `m('A')` invokes
   (a) `m(int)` (b) `m(double)` (c) compile-time ambiguity error (d) runtime error
3. *Assertion (A):* A method that only returns `n * n` for its parameter `n` is a pure method.
   Reason (R): It does not change the state of any object.
   (a) Both true; R explains A (b) Both true; R doesn't explain A (c) A true, R false (d) A false, R true
4. What is printed? `java static int f(int n) { n = n * 2; return n + 1; } ... int k = 4; int r = f(f(k)); System.out.println(k + " " + r);` (a) 4 19 (b) 19 19 (c) 4 9 (d) 9 19
5. The signature of `public static double power(double b, int e)` is
   (a) `public static double power` (b) `power(double, int)` (c) `double power(double, int)` (d) `power(b, e)`
6. Which statement is **true**?
   (a) Formal parameters can be literals (b) A `void` method can contain `return;` (c) A method can return two values using `return a, b;` (d) The return type is part of the signature
7. *Assertion (A):* Calling `display()` directly inside `main()` is an error if `display()` is non-static.
   Reason (R): Static methods cannot access instance members without an object.
   (a) Both true; R explains A (b) Both true; R doesn't explain A (c) A true, R false (d) A false, R true
8. Which overloaded method is chosen for `f(10L)` if only `f(int)` and `f(float)` exist?
   (a) `f(int)` (b) `f(float)` (c) Error (d) Runtime decision

Part B — Short answer questions

1. State two advantages of using methods.
2. Write the prototype of a method `check` that takes a `String` and a `char` and returns `true` / `false`.
3. Differentiate between actual and formal parameters with an example.
4. What is method overloading? Why is it called polymorphism?
5. What is the output? `static void s(int a, int b) { int t = a; a = b; b = t; } … int x = 1, y = 2; s(x, y); System.out.println(x + "," + y);`
6. Give one example each of a pure and an impure method.
7. Why can't two methods differ only in their return type?
8. What is the difference between `return` and `System.exit(0)`?

✅ Answers — Part A

Q	Ans	Reason
1	(b)	Order of parameter types differs
2	(a)	`char` widens to `int` first
3	(a)	—

Q	Ans	Reason
4	(a)	f(4) = 9, f(9) = 19; k unchanged
5	(b)	—
6	(b)	—
7	(a)	—
8	(b)	`long` cannot narrow to `int`, but widens to `float`

✅ Answers — Part B

1. Reusability of code; modular programs that are easier to debug and maintain.
2. `boolean check(String s, char ch)`
3. Actual parameters are in the call, formal parameters in the header. In `area(5)` → `double area(double r)`, 5 is actual and `r` is formal.
4. Defining several methods with the same name but different parameter lists in one class. The same name behaves in many forms depending on the arguments — "one name, many forms".
5. `1,2` — primitives are passed by value.
6. Pure: `int getAge() { return age; }`. Impure: `void setAge(int a) { age = a; }`.
7. The compiler chooses a method from the call using only the name and arguments; the return type is not visible in a call statement, so such methods would be indistinguishable.
8. `return` ends only the current method and returns control to the caller; `System.exit(0)` terminates the whole program.

💻 Programs in this chapter

No.	Program	
Q182	Overloading volume(): Sphere, Cylinder and Cuboid	
Q183	Overloading area(): Circle, Rectangle and Triangle	
Q184	Overloading series(): Three Different Series	📝 ICSE 2019
Q185	Overloading polygon(): Square, Rectangle and Triangle Patterns	
Q186	Overloading display() for Patterns	
Q187	Overloading sumSeries()	
Q188	Overloading largest()	
Q189	Overloading count(): Digits and Occurrences of a Digit	
Q190	Overloading perimeter(): Square, Rectangle and Triangle	
Q191	Prime Palindrome (PalPrime) Numbers in a Range	
Q192	Twin Primes in a Range using isPrime()	
Q193	Twisted Prime using isPrime() and reverse()	
Q194	Armstrong Numbers in a Range using isArmstrong()	
Q195	Perfect Number using a boolean Method	

No.	Program	
Q196	HCF, LCM and Co-prime Check using Methods	
Q197	nCr using a factorial() Method	
Q198	Disarium Number using countDigits()	
Q199	Goldbach's Conjecture	🌟 *Extra*
Q200	Pure and Impure Methods	
Q201	Static and Non-Static Methods	
Q202	Call by Value: Swapping inside a Method	
Q203	Actual and Formal Parameters with Return Values	
Q204	Menu-Driven Number Checks using Methods	
Q205	Class Number with Data Member and Member Methods	
Q206	Class Series with Non-Static Methods	
Q207	Date Helper Methods: isLeap(), daysInMonth(), isValid()	🌟 *Extra*
Q208	Day Number of the Year	🌟 *Extra*
Q209	Day of the Week for any Date	🌟 *Extra*
Q210	Calendar of a Month	🌟 *Extra*
Q211	Age in Years, Months and Days	🌟 *Extra*

Programs Q182 – Q211

Q182 Overloading volume(): Sphere, Cylinder and Cuboid

Design a class (written here as class Q_182) to overload a method volume() as follows:

```
(i)   double volume(double r)
      returns the volume of a sphere = (4/3) x pi x r^3
(ii)  double volume(double h, double r)
      returns the volume of a cylinder = pi x r^2 x h
(iii) double volume(double l, double b, double h)
      returns the volume of a cuboid = l x b x h
```

Take pi = 3.14. Write a main method to create an object of the class and call the above methods with suitable arguments.

Q_182.java

```
public class Q_182
{
    double volume(double r)
    {
        return 4.0 / 3 * 3.14 * r * r * r;
    }

    double volume(double h, double r)
    {
        return 3.14 * r * r * h;
    }

    double volume(double l, double b, double h)
    {
        return l * b * h;
    }

    public static void main(String[] args)
    {
        Q_182 ob = new Q_182();
        System.out.println("Volume of sphere (r = 3)          = " + ob.volume(3));
        System.out.println("Volume of cylinder (h = 10, r = 2) = " + ob.volume(10, 2));
        System.out.println("Volume of cuboid (5 x 4 x 2.5)     = " + ob.volume(5, 4, 2.5));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
r	double	to receive the radius
h	double	to receive the height
l	double	to receive the length
b	double	to receive the breadth
ob	Q_182	to refer to the object

Q183 Overloading area(): Circle, Rectangle and Triangle

Design a class (written here as class Q_183) to overload a method area() as follows:

```
(i)   double area(double r)
      returns the area of a circle = pi x r x r
(ii)  double area(double l, double b)
      returns the area of a rectangle = l x b
(iii) double area(double a, double b, double c)
      returns the area of a triangle using Heron's formula,
      s = (a + b + c) / 2, area = sqrt(s(s-a)(s-b)(s-c));
      return -1 if the sides cannot form a triangle
```

Write a main method to accept the values from the user and call the methods.

Q_183.java

```java
import java.util.Scanner;

public class Q_183
{
    double area(double r)
    {
        return Math.PI * r * r;
    }

    double area(double l, double b)
    {
        return l * b;
    }

    double area(double a, double b, double c)
    {
        if (a <= 0 || b <= 0 || c <= 0 || a + b <= c || b + c <= a || a + c <= b)
            return -1;
        double s = (a + b + c) / 2;
        return Math.sqrt(s * (s - a) * (s - b) * (s - c));
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_183 ob = new Q_183();
        System.out.print("Enter radius of the circle: ");
        double r = sc.nextDouble();
        System.out.println("Area of circle = " + ob.area(r));

        System.out.print("Enter length and breadth of the rectangle: ");
        double l = sc.nextDouble();
        double b = sc.nextDouble();
        System.out.println("Area of rectangle = " + ob.area(l, b));

        System.out.print("Enter the three sides of the triangle: ");
        double x = sc.nextDouble();
        double y = sc.nextDouble();
        double z = sc.nextDouble();
        double t = ob.area(x, y, z);
        if (t < 0)
            System.out.println("Triangle not possible");
        else
            System.out.println("Area of triangle = " + t);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
r	double	to store/receive the radius
l	double	to store/receive the length
b	double	to store/receive the breadth or a side
a	double	to receive the first side
c	double	to receive the third side
s	double	to store the semi-perimeter
ob	Q_183	to refer to the object
x	double	to store the first side
y	double	to store the second side
z	double	to store the third side
t	double	to store the triangle's area

Q184 Overloading series(): Three Different Series

ICSE 2019 Design a class (written here as class Q_184) to overload a method series() as follows:

```
(i)   void series(int x, int n)
      to display the sum of the series
      x^1 + x^2 + x^3 + ... + x^n
(ii)  void series(int p)
      to display the first p terms of the series
      0, 7, 26, 63, ...  (term = i^3 - 1)
(iii) void series()
      to display the sum of the series
      1/2 + 1/3 + 1/4 + ... + 1/10
```

Write a main method to create an object and call the methods.

Q_184.java

```java
import java.util.Scanner;

public class Q_184
{
    void series(int x, int n)
    {
        long sum = 0;
        for (int i = 1; i <= n; i++)
            sum = sum + (long) Math.pow(x, i);
        System.out.println("Sum = " + sum);
    }

    void series(int p)
    {
        for (long i = 1; i <= p; i++)
            System.out.print((i * i * i - 1) + " ");
        System.out.println();
    }

    void series()
    {
        double sum = 0;
        for (int i = 2; i <= 10; i++)
            sum = sum + 1.0 / i;
        System.out.println("Sum = " + sum);
    }
```

```
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_184 ob = new Q_184();
        System.out.print("Enter x and n: ");
        int x = sc.nextInt();
        int n = sc.nextInt();
        ob.series(x, n);
        System.out.print("Enter p: ");
        int p = sc.nextInt();
        ob.series(p);
        ob.series();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
x	int	to store/receive the base x
n	int	to store/receive the number of terms
p	int	to store/receive the number of terms
sum	long	to store the sum in series(int, int)
sum	double	to store the sum in series()
i	int	to act as the loop counter
i	long	to act as the counter in series(int)
ob	Q_184	to refer to the object

Q185 Overloading polygon(): Square, Rectangle and Triangle Patterns

Design a class (written here as class Q_185) to overload a method polygon() as follows:

```
(i)   void polygon(int n, char ch)
      to print a filled square of side n using the character ch
(ii)  void polygon(int x, int y)
      to print a filled rectangle of x rows and y columns using
      the symbol @
(iii) void polygon()
      to print the filled triangle shown below:
          *
          * *
          * * *
```

Example: polygon(2, 'o') gives polygon(2, 5) gives o o @ @ @ @ o o @ @ @ @ @

Q_185.java

```
public class Q_185
{
    void polygon(int n, char ch)
    {
        for (int i = 1; i <= n; i++)
        {
            for (int j = 1; j <= n; j++)
                System.out.print(ch + " ");
            System.out.println();
        }
    }

    void polygon(int x, int y)
    {
```

```
        for (int i = 1; i <= x; i++)
        {
            for (int j = 1; j <= y; j++)
                System.out.print("@ ");
            System.out.println();
        }
    }

    void polygon()
    {
        for (int i = 1; i <= 3; i++)
        {
            for (int j = 1; j <= i; j++)
                System.out.print("* ");
            System.out.println();
        }
    }

    public static void main(String[] args)
    {
        Q_185 ob = new Q_185();
        ob.polygon(2, 'o');
        System.out.println();
        ob.polygon(2, 5);
        System.out.println();
        ob.polygon();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to receive the side of the square
ch	char	to receive the character to print
x	int	to receive the number of rows
y	int	to receive the number of columns
i	int	to control the rows
j	int	to control the columns
ob	Q_185	to refer to the object

Q186 Overloading display() for Patterns

Design a class (written here as class Q_186) to overload a method display() as follows:

```
(i)   void display()          : to display the pattern
                                1
                                2 2
                                3 3 3
                                4 4 4 4
(ii)  void display(int n)     : to display n rows of the pattern
                                1
                                1 2
                                1 2 3   ... up to n rows
(iii) void display(char ch, int n) : to display n rows of the
                                character ch in a triangle
                                ch
                                ch ch
                                ch ch ch ...
```

Write a main method to call the methods with n = 5 and ch = '#'.

Q_186.java

```
public class Q_186
{
    void display()
    {
        for (int i = 1; i <= 4; i++)
        {
            for (int j = 1; j <= i; j++)
                System.out.print(i + " ");
            System.out.println();
        }
    }

    void display(int n)
    {
        for (int i = 1; i <= n; i++)
        {
            for (int j = 1; j <= i; j++)
                System.out.print(j + " ");
            System.out.println();
        }
    }

    void display(char ch, int n)
    {
        for (int i = 1; i <= n; i++)
        {
            for (int j = 1; j <= i; j++)
                System.out.print(ch + " ");
            System.out.println();
        }
    }

    public static void main(String[] args)
    {
        Q_186 ob = new Q_186();
        ob.display();
        ob.display(5);
        ob.display('#', 5);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to receive the number of rows
ch	char	to receive the character to print
i	int	to control the rows
j	int	to control the columns
ob	Q_186	to refer to the object

Q187 Overloading sumSeries()

Design a class (written here as class Q_187) to overload a method sumSeries() as follows:

```
(i)  double sumSeries(int n)
     returns the sum of 1 + 1/2 + 1/3 + ... + 1/n
(ii) double sumSeries(double x, int n)
     returns the sum of x/1 + x^2/2 + x^3/3 + ... + x^n/n
```

Write a main method to accept n and x, and display both sums.

Q_187.java

```java
import java.util.Scanner;

public class Q_187
{
    double sumSeries(int n)
    {
        double s = 0;
        for (int i = 1; i <= n; i++)
            s = s + 1.0 / i;
        return s;
    }

    double sumSeries(double x, int n)
    {
        double s = 0;
        for (int i = 1; i <= n; i++)
            s = s + Math.pow(x, i) / i;
        return s;
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_187 ob = new Q_187();
        System.out.print("Enter n: ");
        int n = sc.nextInt();
        System.out.print("Enter x: ");
        double x = sc.nextDouble();
        System.out.println("1 + 1/2 + ... + 1/n   = " + ob.sumSeries(n));
        System.out.println("x/1 + x^2/2 + ... + x^n/n = " + ob.sumSeries(x, n));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store/receive the number of terms
x	double	to store/receive the value of x
s	double	to store the sum
i	int	to act as the loop counter
ob	Q_187	to refer to the object

Q188 Overloading largest()

Design a class (written here as class Q_188) to overload a method largest() as follows:

```
(i)   int largest(int a, int b)          : returns the larger of two
                                           integers
(ii)  int largest(int a, int b, int c) : returns the largest of
                                           three integers (use (i))
(iii) double largest(double a, double b) : returns the larger of
                                           two real numbers
```

Write a main method to accept values and call the methods.

Q_188.java

```java
import java.util.Scanner;

public class Q_188
{
    int largest(int a, int b)
```

```
    {
        if (a > b)
            return a;
        else
            return b;
    }

    int largest(int a, int b, int c)
    {
        return largest(largest(a, b), c);
    }

    double largest(double a, double b)
    {
        if (a > b)
            return a;
        else
            return b;
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_188 ob = new Q_188();
        System.out.print("Enter three integers: ");
        int p = sc.nextInt();
        int q = sc.nextInt();
        int r = sc.nextInt();
        System.out.print("Enter two real numbers: ");
        double x = sc.nextDouble();
        double y = sc.nextDouble();

        System.out.println("Larger of first two integers = " + ob.largest(p, q));
        System.out.println("Largest of three integers    = " + ob.largest(p, q, r));
        System.out.println("Larger real number           = " + ob.largest(x, y));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	int	to receive the first integer
b	int	to receive the second integer
c	int	to receive the third integer
a	double	to receive the first real number
b	double	to receive the second real number
p	int	to store the first integer
q	int	to store the second integer
r	int	to store the third integer
x	double	to store the first real number
y	double	to store the second real number
ob	Q_188	to refer to the object

Q189 Overloading count(): Digits and Occurrences of a Digit

Design a class (written here as class Q_189) to overload a method count() as follows:

```
(i)  int count(long n)        : returns the number of digits in n
(ii) int count(long n, int d) : returns the number of times the
                                digit d occurs in n
```

Work with the absolute value of n; the number 0 has one digit. Write a main method to accept a number and a digit and display both results.

Q_189.java

```java
import java.util.Scanner;

public class Q_189
{
    int count(long n)
    {
        n = Math.abs(n);
        int c = 0;
        do
        {
            c++;
            n = n / 10;
        } while (n > 0);
        return c;
    }

    int count(long n, int d)
    {
        n = Math.abs(n);
        int c = 0;
        do
        {
            if (n % 10 == d)
                c++;
            n = n / 10;
        } while (n > 0);
        return c;
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_189 ob = new Q_189();
        System.out.print("Enter a number: ");
        long num = sc.nextLong();
        System.out.print("Enter a digit: ");
        int dig = sc.nextInt();
        System.out.println("Number of digits = " + ob.count(num));
        System.out.println("Occurrences of " + dig + " = " + ob.count(num, dig));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to receive the number
d	int	to receive the digit to count
c	int	to store the count
num	long	to store the number entered
dig	int	to store the digit entered

Name of the Variable	Data Type	Purpose / Description
ob	Q_189	to refer to the object

Q190 Overloading perimeter(): Square, Rectangle and Triangle

Design a class (written here as class Q_190) to overload a method perimeter() as follows:

```
(i)   double perimeter(double s)                : square, 4 x s
(ii)  double perimeter(double l, double b)      : rectangle,
                                                  2 x (l + b)
(iii) double perimeter(double a, double b, double c) : triangle,
                                                  a + b + c
```

Write a main method to create an object and display the perimeters of a square of side 6.5, a rectangle 8 × 3 and a triangle with sides 3, 4 and 5.

Q_190.java

```java
public class Q_190
{
    double perimeter(double s)
    {
        return 4 * s;
    }

    double perimeter(double l, double b)
    {
        return 2 * (l + b);
    }

    double perimeter(double a, double b, double c)
    {
        return a + b + c;
    }

    public static void main(String[] args)
    {
        Q_190 ob = new Q_190();
        System.out.println("Perimeter of square    = " + ob.perimeter(6.5));
        System.out.println("Perimeter of rectangle = " + ob.perimeter(8, 3));
        System.out.println("Perimeter of triangle  = " + ob.perimeter(3, 4, 5));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	double	to receive the side of the square
l	double	to receive the length
b	double	to receive the breadth or a side
a	double	to receive the first side
c	double	to receive the third side
ob	Q_190	to refer to the object

Q191 Prime Palindrome (PalPrime) Numbers in a Range

A PalPrime number is a number that is both prime and a palindrome (e.g. 11, 101, 131). Write a program with the following methods:

```
boolean isPrime(int n)      : returns true if n is prime
boolean isPalindrome(int n) : returns true if n is a palindrome
void main()                 : accepts two limits m and n and
                              displays all PalPrime numbers
                              between them, using the above
                              methods
```

If m > n, display "Invalid range".

Q_191.java

```java
import java.util.Scanner;

public class Q_191
{
    boolean isPrime(int n)
    {
        if (n < 2)
            return false;
        for (int i = 2; i * i <= n; i++)
        {
            if (n % i == 0)
                return false;
        }
        return true;
    }

    boolean isPalindrome(int n)
    {
        int copy = n, rev = 0;
        while (copy > 0)
        {
            rev = rev * 10 + copy % 10;
            copy = copy / 10;
        }
        return rev == n;
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_191 ob = new Q_191();
        System.out.print("Enter the limits m and n: ");
        int m = sc.nextInt();
        int n = sc.nextInt();

        if (m > n)
        {
            System.out.println("Invalid range");
        }
        else
        {
            for (int k = m; k <= n; k++)
            {
                if (ob.isPrime(k) && ob.isPalindrome(k))
                    System.out.print(k + " ");
            }
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to receive a number / store the upper limit
i	int	to act as the trial divisor
copy	int	to store a working copy of n
rev	int	to store the reverse of n
m	int	to store the lower limit
k	int	to store the number being tested
ob	Q_191	to refer to the object

Q192 Twin Primes in a Range using isPrime()

> Twin primes are two prime numbers that differ by 2, e.g. (5, 7), (17, 19). Write a program with a method boolean isPrime(int n) that returns true if n is prime. In main(), accept two limits and display all the twin prime pairs that lie completely within the limits, using the method isPrime().

Q_192.java

```
import java.util.Scanner;

public class Q_192
{
    boolean isPrime(int n)
    {
        if (n < 2)
            return false;
        for (int i = 2; i * i <= n; i++)
        {
            if (n % i == 0)
                return false;
        }
        return true;
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_192 ob = new Q_192();
        System.out.print("Enter the two limits: ");
        int a = sc.nextInt();
        int b = sc.nextInt();
        int count = 0;

        for (int k = a; k + 2 <= b; k++)
        {
            if (ob.isPrime(k) && ob.isPrime(k + 2))
            {
                System.out.println("(" + k + ", " + (k + 2) + ")");
                count++;
            }
        }
        if (count == 0)
            System.out.println("No twin primes in this range");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to receive the number to test
i	int	to act as the trial divisor
a	int	to store the lower limit
b	int	to store the upper limit
count	int	to count the twin prime pairs
k	int	to store the first number of a pair
ob	Q_192	to refer to the object

Q193 Twisted Prime using isPrime() and reverse()

A number is called a Twisted Prime if the number is prime and the number obtained by reversing its digits is also prime (e.g. 97 and 79 are both prime, so 97 is a Twisted Prime). Write a program with the methods:

```
int reverse(int n)     : returns the reverse of n
boolean isPrime(int n) : returns true if n is prime
```

In main(), accept a number and check whether it is a Twisted Prime using the above methods. Display "Twisted Prime" or "Not a Twisted Prime".

Q_193.java

```java
import java.util.Scanner;

public class Q_193
{
    int reverse(int n)
    {
        int rev = 0;
        while (n > 0)
        {
            rev = rev * 10 + n % 10;
            n = n / 10;
        }
        return rev;
    }

    boolean isPrime(int n)
    {
        if (n < 2)
            return false;
        for (int i = 2; i * i <= n; i++)
        {
            if (n % i == 0)
                return false;
        }
        return true;
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_193 ob = new Q_193();
        System.out.print("Enter a number: ");
        int num = sc.nextInt();

        if (ob.isPrime(num) && ob.isPrime(ob.reverse(num)))
            System.out.println("Twisted Prime");
        else
```

```
            System.out.println("Not a Twisted Prime");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to receive the number
rev	int	to store the reversed number
i	int	to act as the trial divisor
num	int	to store the number entered
ob	Q_193	to refer to the object

Q194 Armstrong Numbers in a Range using isArmstrong()

Write a program with the methods:

```
int countDigits(int n)     : returns the number of digits in n
boolean isArmstrong(int n) : returns true if n is an Armstrong
                             number, i.e. n is equal to the sum of
                             its digits each raised to the power of
                             the number of digits
```

In main(), accept two limits and display all the Armstrong numbers between them.

Q_194.java

```
import java.util.Scanner;

public class Q_194
{
    int countDigits(int n)
    {
        int c = 0;
        while (n > 0)
        {
            c++;
            n = n / 10;
        }
        return c;
    }

    boolean isArmstrong(int n)
    {
        int p = countDigits(n);
        int sum = 0, copy = n;
        while (copy > 0)
        {
            sum = sum + (int) Math.pow(copy % 10, p);
            copy = copy / 10;
        }
        return n > 0 && sum == n;
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_194 ob = new Q_194();
        System.out.print("Enter the two limits: ");
        int a = sc.nextInt();
        int b = sc.nextInt();

        for (int k = a; k <= b; k++)
```

```
        {
            if (ob.isArmstrong(k))
                System.out.print(k + " ");
        }
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to receive the number
c	int	to count the digits
p	int	to store the number of digits
sum	int	to store the sum of powers
copy	int	to store a working copy of n
a	int	to store the lower limit
b	int	to store the upper limit
k	int	to store the number being tested
ob	Q_194	to refer to the object

Q195 Perfect Number using a boolean Method

> Write a program with a method boolean isPerfect(int n) that returns true if n is a Perfect number (equal to the sum of its factors excluding itself) and false otherwise. In main(), accept a number and display "Perfect Number" or "Not a Perfect Number" using the method. Also display all the perfect numbers below 10000 using the same method.

Q_195.java

```java
import java.util.Scanner;

public class Q_195
{
    boolean isPerfect(int n)
    {
        if (n < 2)
            return false;
        int sum = 1;
        for (int i = 2; i * i <= n; i++)
        {
            if (n % i == 0)
            {
                sum = sum + i;
                if (i != n / i)
                    sum = sum + n / i;
            }
        }
        return sum == n;
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_195 ob = new Q_195();
        System.out.print("Enter a number: ");
        int num = sc.nextInt();
        if (ob.isPerfect(num))
```

```
            System.out.println("Perfect Number");
        else
            System.out.println("Not a Perfect Number");

        System.out.print("Perfect numbers below 10000: ");
        for (int k = 1; k < 10000; k++)
        {
            if (ob.isPerfect(k))
                System.out.print(k + " ");
        }
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to receive the number
sum	int	to store the sum of factors
i	int	to act as the possible factor
num	int	to store the number entered
k	int	to store the number being tested
ob	Q_195	to refer to the object

Q196 HCF, LCM and Co-prime Check using Methods

Write a program with the methods:

```
long hcf(long a, long b) : returns the HCF of a and b using the
                           Euclidean (division) method
long lcm(long a, long b) : returns the LCM using hcf()
```

In main(), accept two positive integers and display their HCF and LCM, and whether they are co-prime (two numbers are co-prime if their HCF is 1). Display "Invalid input" if either number is not positive.

Q_196.java

```
import java.util.Scanner;

public class Q_196
{
    long hcf(long a, long b)
    {
        while (b != 0)
        {
            long r = a % b;
            a = b;
            b = r;
        }
        return a;
    }

    long lcm(long a, long b)
    {
        return a / hcf(a, b) * b;
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_196 ob = new Q_196();
        System.out.print("Enter two positive integers: ");
```

```
        long x = sc.nextLong();
        long y = sc.nextLong();

        if (x <= 0 || y <= 0)
        {
            System.out.println("Invalid input");
        }
        else
        {
            long h = ob.hcf(x, y);
            System.out.println("HCF = " + h);
            System.out.println("LCM = " + ob.lcm(x, y));
            if (h == 1)
                System.out.println("Co-prime numbers");
            else
                System.out.println("Not co-prime numbers");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	long	to receive the first number
b	long	to receive the second number
r	long	to store the remainder
x	long	to store the first number entered
y	long	to store the second number entered
h	long	to store the hcf
ob	Q_196	to refer to the object

Q197 nCr using a factorial() Method

The number of ways of choosing r objects from n objects is

`nCr = n! / ( r! x (n - r)! )`

Write a program with a method long factorial(int n) that returns n!. In main(), accept n and r and display nCr using the method. Accept only 0 <= r <= n <= 20 (otherwise display "Invalid input").

Q_197.java

```
import java.util.Scanner;

public class Q_197
{
    long factorial(int n)
    {
        long f = 1;
        for (int i = 2; i <= n; i++)
            f = f * i;
        return f;
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_197 ob = new Q_197();
        System.out.print("Enter n and r: ");
        int n = sc.nextInt();
        int r = sc.nextInt();
```

```
        if (r < 0 || n < r || n > 20)
        {
            System.out.println("Invalid input");
        }
        else
        {
            long ncr = ob.factorial(n) / (ob.factorial(r) * ob.factorial(n - r));
            System.out.println(n + "C" + r + " = " + ncr);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store/receive n
f	long	to store the factorial
i	int	to act as the loop counter
r	int	to store r
ncr	long	to store the value of ncr
ob	Q_197	to refer to the object

Q198 Disarium Number using countDigits()

A Disarium number is a number whose digits, raised to the power of their respective positions (from the left), add up to the number itself (e.g. 175 = 1^1 + 7^2 + 5^3). Write a program with the methods:

```
int countDigits(int n)    : returns the number of digits in n
boolean isDisarium(int n) : returns true if n is a Disarium
                            number, using countDigits()
```

In main(), accept a number and display "Disarium Number" or "Not a Disarium Number".

Q_198.java

```
import java.util.Scanner;

public class Q_198
{
    int countDigits(int n)
    {
        int c = 0;
        while (n > 0)
        {
            c++;
            n = n / 10;
        }
        return c;
    }

    boolean isDisarium(int n)
    {
        int pos = countDigits(n);
        int sum = 0, copy = n;
        while (copy > 0)
        {
            sum = sum + (int) Math.pow(copy % 10, pos);
            pos--;
            copy = copy / 10;
        }
        return n > 0 && sum == n;
```

```
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_198 ob = new Q_198();
        System.out.print("Enter a number: ");
        int num = sc.nextInt();
        if (ob.isDisarium(num))
            System.out.println("Disarium Number");
        else
            System.out.println("Not a Disarium Number");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to receive the number
c	int	to count the digits
pos	int	to store the position of a digit
sum	int	to store the sum of powers
copy	int	to store a working copy of n
num	int	to store the number entered
ob	Q_198	to refer to the object

Q199 Goldbach's Conjecture

Extra Goldbach's conjecture states that every even number greater than 2 can be written as the sum of two prime numbers. Write a program with a method boolean isPrime(int n). In main(), accept an even number greater than 2 and display all the ways of writing it as the sum of two primes p + q with p <= q. Display "Invalid input" for an odd number or a number not greater than 2.

Example: 30 = 7 + 23 = 11 + 19 = 13 + 17

Q_199.java

```
import java.util.Scanner;

public class Q_199
{
    boolean isPrime(int n)
    {
        if (n < 2)
            return false;
        for (int i = 2; i * i <= n; i++)
        {
            if (n % i == 0)
                return false;
        }
        return true;
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_199 ob = new Q_199();
        System.out.print("Enter an even number greater than 2: ");
        int n = sc.nextInt();
```

```
        if (n <= 2 || n % 2 != 0)
        {
            System.out.println("Invalid input");
        }
        else
        {
            for (int p = 2; p <= n / 2; p++)
            {
                int q = n - p;
                if (ob.isPrime(p) && ob.isPrime(q))
                    System.out.println(n + " = " + p + " + " + q);
            }
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store/receive the number
i	int	to act as the trial divisor
p	int	to store the first prime
q	int	to store the second prime
ob	Q_199	to refer to the object

Q200 Pure and Impure Methods

Write a class (written here as class Q_200) to show the difference between pure and impure methods. The class has a data member int marks.

```
int addBonus(int m, int b) : a pure method that returns m + b
                             without changing any data member
void updateMarks(int b)    : an impure method that adds b to the
                             data member marks (changes the state
                             of the object)
```

In main(), create an object with marks = 70. Call addBonus(marks, 5) and display its result and the value of marks; then call updateMarks(5) and display marks again.

Q_200.java

```
public class Q_200
{
    int marks;

    int addBonus(int m, int b)
    {
        return m + b;
    }

    void updateMarks(int b)
    {
        marks = marks + b;
    }

    public static void main(String[] args)
    {
        Q_200 ob = new Q_200();
        ob.marks = 70;

        int r = ob.addBonus(ob.marks, 5);
        System.out.println("Pure method returned " + r + ", marks is still " + ob.marks);
```

```
        ob.updateMarks(5);
        System.out.println("After impure method, marks is now " + ob.marks);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
marks	int	to store the marks
m	int	to receive the marks
b	int	to receive the bonus
r	int	to store the returned value
ob	Q_200	to refer to the object

Q201 Static and Non-Static Methods

Write a class (written here as class Q_201) with:

```
a data member double radius,
a static method double circleArea(double r) that returns the area
    of a circle of radius r,
a non-static method void show() that displays the radius of the
    object and its area (by calling circleArea()).
```

In main(), call the static method directly without creating an object (circleArea(7)), then create an object with radius 3.5 and call show() through the object.

Q_201.java

```java
public class Q_201
{
    double radius;

    static double circleArea(double r)
    {
        return Math.PI * r * r;
    }

    void show()
    {
        System.out.println("Radius = " + radius + ", Area = " + circleArea(radius));
    }

    public static void main(String[] args)
    {
        System.out.println("Static call: circleArea(7) = " + circleArea(7));

        Q_201 c = new Q_201();
        c.radius = 3.5;
        System.out.print("Non-static call: ");
        c.show();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
radius	double	to store the radius of the object
r	double	to receive the radius
c	Q_201	to refer to the object

Q202 Call by Value: Swapping inside a Method

Write a program to show that primitive arguments are passed by value in Java. Define a method void swap(int a, int b) that interchanges a and b and displays them. In main(), accept two numbers x and y, display them, call swap(x, y), and display x and y again to show that the original values are unchanged.

Q_202.java

```java
import java.util.Scanner;

public class Q_202
{
    void swap(int a, int b)
    {
        int t = a;
        a = b;
        b = t;
        System.out.println("Inside swap : a = " + a + ", b = " + b);
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_202 ob = new Q_202();
        System.out.print("Enter x and y: ");
        int x = sc.nextInt();
        int y = sc.nextInt();

        System.out.println("Before call : x = " + x + ", y = " + y);
        ob.swap(x, y);
        System.out.println("After call  : x = " + x + ", y = " + y);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	int	to receive a copy of x
b	int	to receive a copy of y
t	int	to hold a value while swapping
x	int	to store the first number
y	int	to store the second number
ob	Q_202	to refer to the object

Q203 Actual and Formal Parameters with Return Values

Write a program with a method double simpleInterest(double p, double r, double t) that returns the simple interest (p x r x t / 100). In main(), call the method three times with different actual parameters:

```
(i)   principal 5000, rate 8, time 3 (literal arguments)
(ii)  values entered by the user (variable arguments)
(iii) principal = twice the principal entered, same rate and
      time (expression argument)
```

Display the interest returned each time and the total of the three.

Q_203.java

```java
import java.util.Scanner;

public class Q_203
{
```

```
    double simpleInterest(double p, double r, double t)
    {
        return p * r * t / 100;
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_203 ob = new Q_203();

        double i1 = ob.simpleInterest(5000, 8, 3);
        System.out.println("(i)   Interest = " + i1);

        System.out.print("Enter principal, rate and time: ");
        double pr = sc.nextDouble();
        double rt = sc.nextDouble();
        double tm = sc.nextDouble();
        double i2 = ob.simpleInterest(pr, rt, tm);
        System.out.println("(ii)  Interest = " + i2);

        double i3 = ob.simpleInterest(2 * pr, rt, tm);
        System.out.println("(iii) Interest = " + i3);
        System.out.println("Total interest = " + (i1 + i2 + i3));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
p	double	to receive the principal (formal)
r	double	to receive the rate (formal)
t	double	to receive the time (formal)
i1	double	to store the first interest
pr	double	to store the principal entered
rt	double	to store the rate entered
tm	double	to store the time entered
i2	double	to store the second interest
i3	double	to store the third interest
ob	Q_203	to refer to the object

Q204 Menu-Driven Number Checks using Methods

Write a menu-driven program using methods to check a number:

```
1. Prime number      - boolean isPrime(int n)
2. Palindrome number - boolean isPalindrome(int n)
3. Perfect number    - boolean isPerfect(int n)
```

Accept the choice and a number, call the corresponding method and display the result (e.g. "Prime Number" or "Not a Prime Number"). Display "Wrong choice" for any other choice.

Q_204.java

```
import java.util.Scanner;

public class Q_204
{
    boolean isPrime(int n)
    {
```

```
        if (n < 2)
            return false;
        for (int i = 2; i * i <= n; i++)
        {
            if (n % i == 0)
                return false;
        }
        return true;
    }

    boolean isPalindrome(int n)
    {
        int copy = n, rev = 0;
        while (copy > 0)
        {
            rev = rev * 10 + copy % 10;
            copy = copy / 10;
        }
        return n >= 0 && rev == n;
    }

    boolean isPerfect(int n)
    {
        int sum = 0;
        for (int i = 1; i <= n / 2; i++)
        {
            if (n % i == 0)
                sum = sum + i;
        }
        return n > 1 && sum == n;
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_204 ob = new Q_204();
        System.out.println("1. Prime  2. Palindrome  3. Perfect");
        System.out.print("Enter your choice: ");
        int ch = sc.nextInt();
        System.out.print("Enter a number: ");
        int num = sc.nextInt();

        switch (ch)
        {
            case 1:
                if (ob.isPrime(num))
                    System.out.println("Prime Number");
                else
                    System.out.println("Not a Prime Number");
                break;
            case 2:
                if (ob.isPalindrome(num))
                    System.out.println("Palindrome Number");
                else
                    System.out.println("Not a Palindrome Number");
                break;
            case 3:
                if (ob.isPerfect(num))
                    System.out.println("Perfect Number");
                else
                    System.out.println("Not a Perfect Number");
                break;
            default:
                System.out.println("Wrong choice");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to receive the number
i	int	to act as the loop counter
copy	int	to store a working copy of n
rev	int	to store the reverse of n
sum	int	to store the sum of factors
ch	int	to store the user's choice
num	int	to store the number entered
ob	Q_204	to refer to the object

Q205 Class Number with Data Member and Member Methods

Define a class Number (written here as class Q_205) with the following description:

```
Data member : long n
void input()        : to accept the number
int sumDigits()     : to return the sum of the digits of n
long reverse()      : to return the reverse of n
boolean isPalindrome() : to return true if n is a palindrome
                         (using reverse())
void display()      : to display n, the sum of its digits, its
                      reverse and whether it is a palindrome
```

Write a main method to create an object and invoke the methods. Treat a negative number by its absolute value.

Q_205.java

```java
import java.util.Scanner;

public class Q_205
{
    long n;

    void input()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        n = Math.abs(sc.nextLong());
    }

    int sumDigits()
    {
        int s = 0;
        long t = n;
        while (t > 0)
        {
            s = s + (int) (t % 10);
            t = t / 10;
        }
        return s;
    }

    long reverse()
    {
        long rev = 0, t = n;
        while (t > 0)
        {
            rev = rev * 10 + t % 10;
```

```
            t = t / 10;
        }
        return rev;
    }

    boolean isPalindrome()
    {
        return reverse() == n;
    }

    void display()
    {
        System.out.println("Number        : " + n);
        System.out.println("Sum of digits : " + sumDigits());
        System.out.println("Reverse       : " + reverse());
        if (isPalindrome())
            System.out.println("Palindrome Number");
        else
            System.out.println("Not a Palindrome Number");
    }

    public static void main(String[] args)
    {
        Q_205 ob = new Q_205();
        ob.input();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
s	int	to store the sum of digits
t	long	to store a working copy of n
rev	long	to store the reverse
ob	Q_205	to refer to the object

Q206 Class Series with Non-Static Methods

Define a class Series (written here as class Q_206) with the following description:

```
Data member : int n (number of terms)
void accept()     : to accept n
void fibonacci()  : to display the first n Fibonacci terms
                    0, 1, 1, 2, 3, 5, ...
void oddSquares() : to display the squares of the first n odd
                    numbers 1, 9, 25, 49, ...
long sumCubes()   : to return 1^3 + 2^3 + ... + n^3
```

Write a main method to create an object and call all the methods. Display "Invalid input" if n is less than 1 or more than 90.

Q_206.java

```
import java.util.Scanner;

public class Q_206
{
    int n;

    void accept()
    {
```

```
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of terms: ");
        n = sc.nextInt();
    }

    void fibonacci()
    {
        long a = 0, b = 1;
        for (int i = 1; i <= n; i++)
        {
            System.out.print(a + " ");
            long c = a + b;
            a = b;
            b = c;
        }
        System.out.println();
    }

    void oddSquares()
    {
        for (long i = 1; i <= n; i++)
        {
            long odd = 2 * i - 1;
            System.out.print(odd * odd + " ");
        }
        System.out.println();
    }

    long sumCubes()
    {
        long s = 0;
        for (long i = 1; i <= n; i++)
            s = s + i * i * i;
        return s;
    }

    public static void main(String[] args)
    {
        Q_206 ob = new Q_206();
        ob.accept();
        if (ob.n < 1 || ob.n > 90)
        {
            System.out.println("Invalid input");
        }
        else
        {
            System.out.print("Fibonacci   : ");
            ob.fibonacci();
            System.out.print("Odd squares : ");
            ob.oddSquares();
            System.out.println("Sum of cubes: " + ob.sumCubes());
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of terms
a	long	to store the current fibonacci term
b	long	to store the next fibonacci term
c	long	to store the sum of a and b

Name of the Variable	Data Type	Purpose / Description
i	int	to act as the counter in fibonacci()
i	long	to act as the counter in other methods
odd	long	to store the current odd number
s	long	to store the sum of cubes
ob	Q_206	to refer to the object

Q207 Date Helper Methods: isLeap(), daysInMonth(), isValid()

Extra Write a program with the following methods:

```
boolean isLeap(int y)            : returns true for a leap year
int daysInMonth(int m, int y)    : returns the number of days in
                                   month m of year y (0 for an
                                   invalid month)
boolean isValid(int d, int m, int y) : returns true if d/m/y is a
                                   valid date (y > 0)
```

In main(), accept a date and display whether it is valid; if it is, also display whether its year is a leap year and how many days its month has.

Q_207.java

```java
import java.util.Scanner;

public class Q_207
{
    boolean isLeap(int y)
    {
        return (y % 4 == 0 && y % 100 != 0) || y % 400 == 0;
    }

    int daysInMonth(int m, int y)
    {
        switch (m)
        {
            case 1: case 3: case 5: case 7: case 8: case 10: case 12:
                return 31;
            case 4: case 6: case 9: case 11:
                return 30;
            case 2:
                if (isLeap(y))
                    return 29;
                else
                    return 28;
            default:
                return 0;
        }
    }

    boolean isValid(int d, int m, int y)
    {
        return y > 0 && d >= 1 && d <= daysInMonth(m, y);
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_207 ob = new Q_207();
        System.out.print("Enter day, month and year: ");
        int d = sc.nextInt();
        int m = sc.nextInt();
```

```
        int y = sc.nextInt();

        if (ob.isValid(d, m, y))
        {
            System.out.println("Valid date");
            if (ob.isLeap(y))
                System.out.println(y + " is a leap year");
            else
                System.out.println(y + " is not a leap year");
            System.out.println("Days in this month = " + ob.daysInMonth(m, y));
        }
        else
        {
            System.out.println("Invalid date");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
y	int	to store/receive the year
m	int	to store/receive the month
d	int	to store/receive the day
ob	Q_207	to refer to the object

Q208 Day Number of the Year

> **Extra** Write a program to accept a date (day, month and year) and find the day number of that date in the year, i.e. how many days have passed from 1 January up to and including that date (e.g. 1 March 2024 is day 61 because 2024 is a leap year). Use the methods isLeap(int y) and daysInMonth(int m, int y). Display "Invalid date" for a wrong date.

Q_208.java

```
import java.util.Scanner;

public class Q_208
{
    boolean isLeap(int y)
    {
        return (y % 4 == 0 && y % 100 != 0) || y % 400 == 0;
    }

    int daysInMonth(int m, int y)
    {
        if (m == 2)
        {
            if (isLeap(y))
                return 29;
            else
                return 28;
        }
        else if (m == 4 || m == 6 || m == 9 || m == 11)
            return 30;
        else if (m >= 1 && m <= 12)
            return 31;
        else
            return 0;
    }

    public static void main(String[] args)
```

```
    {
        Scanner sc = new Scanner(System.in);
        Q_208 ob = new Q_208();
        System.out.print("Enter day, month and year: ");
        int d = sc.nextInt();
        int m = sc.nextInt();
        int y = sc.nextInt();

        if (y <= 0 || d < 1 || d > ob.daysInMonth(m, y))
        {
            System.out.println("Invalid date");
        }
        else
        {
            int dayNo = d;
            for (int i = 1; i < m; i++)
                dayNo = dayNo + ob.daysInMonth(i, y);
            System.out.println("Day number = " + dayNo);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
y	int	to store/receive the year
m	int	to store/receive the month
d	int	to store the day
dayNo	int	to store the day number
i	int	to act as the month counter
ob	Q_208	to refer to the object

Q209 Day of the Week for any Date

Extra Write a program to accept a date (day, month and year, year from 1600 onwards) and find the day of the week on which it falls, using Zeller's congruence:

```
If the month is January or February, treat it as month 13 or 14
of the previous year.
K = year % 100,  J = year / 100
h = (d + (13 (m + 1)) / 5 + K + K/4 + J/4 + 5J) % 7
h = 0 : Saturday, 1 : Sunday, 2 : Monday, ..., 6 : Friday
```

Write a method int zeller(int d, int m, int y) that returns h, and a method String dayName(int h). Display "Invalid date" for an invalid date.

Q_209.java

```
import java.util.Scanner;

public class Q_209
{
    int zeller(int d, int m, int y)
    {
        if (m < 3)
        {
            m = m + 12;
            y = y - 1;
        }
        int k = y % 100;
        int j = y / 100;
        return (d + 13 * (m + 1) / 5 + k + k / 4 + j / 4 + 5 * j) % 7;
```

```
    }

    String dayName(int h)
    {
        switch (h)
        {
            case 0: return "Saturday";
            case 1: return "Sunday";
            case 2: return "Monday";
            case 3: return "Tuesday";
            case 4: return "Wednesday";
            case 5: return "Thursday";
            default: return "Friday";
        }
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_209 ob = new Q_209();
        System.out.print("Enter day, month and year: ");
        int d = sc.nextInt();
        int m = sc.nextInt();
        int y = sc.nextInt();

        int maxDay;
        if (m == 2)
            maxDay = ((y % 4 == 0 && y % 100 != 0) || y % 400 == 0) ? 29 : 28;
        else if (m == 4 || m == 6 || m == 9 || m == 11)
            maxDay = 30;
        else if (m >= 1 && m <= 12)
            maxDay = 31;
        else
            maxDay = 0;

        if (y < 1600 || d < 1 || d > maxDay)
            System.out.println("Invalid date");
        else
            System.out.println(d + "/" + m + "/" + y + " is a " + ob.dayName(ob.zeller(d, m, y)));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
d	int	to store/receive the day
m	int	to store/receive the month
y	int	to store/receive the year
k	int	to store the year of the century
j	int	to store the century
h	int	to receive the day code
maxDay	int	to store the days in the month
ob	Q_209	to refer to the object

Q210 Calendar of a Month

Extra Write a program to accept a month and a year (from 1600 onwards) and print the calendar of that month in the format shown below. Use a method int firstDay(int m, int y) to find the day of the week of the 1st of the month (using Zeller's congruence, converted so that 0 = Sunday ... 6 = Saturday) and a method int daysInMonth(int m, int y). Display "Invalid input" for a wrong month or a year before 1600.

Example: Input 11 2026

```
 Su  Mo  Tu  We  Th  Fr  Sa
  1   2   3   4   5   6   7
  8   9  10  11  12  13  14
 15  16  17  18  19  20  21
 22  23  24  25  26  27  28
 29  30
```

Q_210.java

```java
import java.util.Scanner;

public class Q_210
{
    int firstDay(int m, int y)
    {
        int d = 1;
        if (m < 3)
        {
            m = m + 12;
            y = y - 1;
        }
        int k = y % 100;
        int j = y / 100;
        int h = (d + 13 * (m + 1) / 5 + k + k / 4 + j / 4 + 5 * j) % 7;
        return (h + 6) % 7;
    }

    int daysInMonth(int m, int y)
    {
        if (m == 2)
            return ((y % 4 == 0 && y % 100 != 0) || y % 400 == 0) ? 29 : 28;
        else if (m == 4 || m == 6 || m == 9 || m == 11)
            return 30;
        else
            return 31;
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_210 ob = new Q_210();
        System.out.print("Enter month and year: ");
        int m = sc.nextInt();
        int y = sc.nextInt();

        if (m < 1 || m > 12 || y < 1600)
        {
            System.out.println("Invalid input");
        }
        else
        {
            int start = ob.firstDay(m, y);
            int days = ob.daysInMonth(m, y);
            System.out.println(" Su  Mo  Tu  We  Th  Fr  Sa");
            for (int i = 0; i < start; i++)
                System.out.print("    ");
            for (int d = 1; d <= days; d++)
            {
```

```
                if (d < 10)
                    System.out.print("  " + d + " ");
                else
                    System.out.print(" " + d + " ");
                if ((start + d) % 7 == 0)
                    System.out.println();
            }
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
m	int	to store/receive the month
y	int	to store/receive the year
d	int	to store the day of the month
k	int	to store the year of the century
j	int	to store the century
h	int	to store zeller's day code
ob	Q_210	to refer to the object
start	int	to store the column of the 1st
days	int	to store the days in the month
i	int	to print the leading blanks

Q211 Age in Years, Months and Days

Extra Write a program to accept a person's date of birth and today's date (each as day, month, year) and calculate the person's exact age in years, months and days. While the day of today is smaller than the day of birth, borrow the number of days of the previous month (going back one month each time); if the month is then smaller, borrow 12 months. Use a method int daysInMonth(int m, int y). Display "Invalid input" if the date of birth is after today's date.

Example: DOB 15/8/2011, Today 10/11/2026 → 15 years 2 months 26 days

Q_211.java

```
import java.util.Scanner;

public class Q_211
{
    int daysInMonth(int m, int y)
    {
        if (m == 2)
            return ((y % 4 == 0 && y % 100 != 0) || y % 400 == 0) ? 29 : 28;
        else if (m == 4 || m == 6 || m == 9 || m == 11)
            return 30;
        else
            return 31;
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_211 ob = new Q_211();
        System.out.print("Enter date of birth (d m y): ");
```

```
        int bd = sc.nextInt();
        int bm = sc.nextInt();
        int by = sc.nextInt();
        System.out.print("Enter today's date (d m y): ");
        int td = sc.nextInt();
        int tm = sc.nextInt();
        int ty = sc.nextInt();

        long birth = by * 10000L + bm * 100 + bd;
        long today = ty * 10000L + tm * 100 + td;
        if (birth > today)
        {
            System.out.println("Invalid input");
        }
        else
        {
            while (td < bd)
            {
                int pm = tm - 1;
                int py = ty;
                if (pm == 0)
                {
                    pm = 12;
                    py = ty - 1;
                }
                td = td + ob.daysInMonth(pm, py);
                tm = tm - 1;
                if (tm == 0)
                {
                    tm = 12;
                    ty = ty - 1;
                }
            }
            if (tm < bm)
            {
                tm = tm + 12;
                ty = ty - 1;
            }
            System.out.println("Age: " + (ty - by) + " years " + (tm - bm) + " months " + (td - bd) + "
days");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
m	int	to receive the month
y	int	to receive the year
bd	int	to store the day of birth
bm	int	to store the month of birth
by	int	to store the year of birth
td	int	to store today's day
tm	int	to store today's month
ty	int	to store today's year
birth	long	to store the birth date as yyyymmdd
today	long	to store today's date as yyyymmdd
pm	int	to store the previous month

Name of the Variable	Data Type	Purpose / Description
`py`	`int`	to store the year of the previous month
`ob`	`Q_211`	to refer to the object

CHAPTER

11



Constructors

Definition and characteristics · Default and parameterised constructors · Constructor overloading · The `this` keyword · Constructor vs method

26 programs | Q212 – Q237 | 1 extra

Theory

📑 Contents

1. What is a constructor?
2. Characteristics of a constructor
3. Types of constructors
4. Constructor overloading
5. The `this` keyword
6. Constructor vs method
7. How to answer a class-based (constructor) question
8. Predict the output
9. Find the errors
10. One-line facts
11. Test yourself
12. Programs in this chapter

1. What is a constructor?

A **constructor** is a **special member method** of a class that has the **same name as the class** and **no return type**, and is **invoked automatically** when an object of the class is created. It is used to **initialise the data members** of the object.

```
class Student
{
    String name;
    int roll;

    Student()                // constructor
    {
        name = "";
        roll = 0;
    }
}
...
Student s = new Student();    // the constructor runs automatically here
```

Use of constructors: to give initial values to the data members of every object as soon as it is created, so that no object is left in an undefined state.

2. Characteristics of a constructor

1. It has the **same name as the class**.
2. It has **no return type**, not even `void`.
3. It is **called automatically** when an object is created using `new`; it cannot be called like an ordinary method (`s.Student()` is invalid).
4. It is used to **initialise** the data members.
5. It is generally declared `public` so that objects can be created from anywhere.
6. It can be **overloaded** (a class can have several constructors with different parameter lists).
7. If no constructor is written, the compiler supplies a **default constructor** automatically.
8. A constructor **cannot be** `static` and is not inherited.

> **WARNING**
>
> If you write `void Student() { … }`, it is **no longer a constructor** — it becomes an ordinary method that happens to have the class name, and it is not called automatically.

3. Types of constructors

3.1 Default (non-parameterised) constructor

A constructor that takes **no parameters**. It initialises every object with the **same** values.

```
Student()
{
    name = "Not assigned";
    roll = 0;
}
```

Compiler-supplied default constructor: if a class has **no constructor at all**, Java automatically provides one with no parameters and an empty body. It leaves the data members with their **default values** (`0`, `0.0`, `false`, `'\u0000'`, `null`).

> **IMPORTANT**
>
> The compiler supplies the default constructor **only if the class has no constructor of its own**. Once you write *any* constructor (say a parameterised one), `new Student()` gives a compile-time error unless you also write `Student()` yourself.

3.2 Parameterised constructor

A constructor that **takes parameters**, so that different objects can be initialised with **different values** at the time of creation.

```
Student(String n, int r)
{
    name = n;
    roll = r;
}
...
Student s1 = new Student("Riya", 7);
Student s2 = new Student("Aman", 12);
```

Basis	Default (non-parameterised) constructor	Parameterised constructor
Parameters	None	One or more
Initial values	Same for every object	Can differ for each object
Supplied by compiler?	Yes, if no constructor is written	Never
Object creation	`new Student()`	`new Student("Riya", 7)`

> **NOTE**
>
> The **copy constructor** (a constructor that takes an object of the same class) is **not** in the ICSE syllabus.

4. Constructor overloading

Constructor overloading means defining **more than one constructor in the same class**, each with a **different parameter list** (number, type or order). The appropriate constructor is chosen automatically according to the arguments given with `new`.

```
class Rectangle
{
    double length, breadth;

    Rectangle()                    { length = 1; breadth = 1; }
    Rectangle(double s)            { length = s; breadth = s; }
    Rectangle(double l, double b)  { length = l; breadth = b; }
}
...
Rectangle r1 = new Rectangle();         // 1 x 1
Rectangle r2 = new Rectangle(5);        // 5 x 5
Rectangle r3 = new Rectangle(8, 3);     // 8 x 3
```

Advantage: objects can be created and initialised in different ways, according to the information available.

5. The `this` keyword

`this` is a reference to the **current object** — the object whose method or constructor is being executed.

Use 1 — resolving name conflicts (shadowing): when a parameter (local variable) has the **same name as a data member**, the parameter hides the data member inside that method. `this.member` refers to the data member.

```
Book(String title, double price)
{
    this.title = title;      // data member = parameter
    this.price = price;
}
```

> **WARNING**
>
> Without `this`, `title = title;` simply assigns the parameter to itself — the data member stays `null`. The program compiles but gives a **logical error**.

6. Constructor vs method

Basis	Constructor	Method
Name	Same as the class	Any valid identifier
Return type	None (not even `void`)	Must have one (`void`, `int`, ...)
Called	Automatically, when the object is created with `new`	Explicitly, through an object or class name
Number of calls per object	Exactly once, at creation	Any number of times
Purpose	To initialise the object	To perform operations / define behaviour
Supplied by compiler	Yes (default), if none is written	Never
`static`	Not allowed	Allowed

7. How to answer a class-based question

ICSE Section B always has a question of the form *"Define a class ... with the following specifications"*. To score full marks:

1. Use the **exact class name**, **data member names and types** given.
2. Write the **constructor** exactly as asked (default → initialise all members; parameterised → assign the parameters).
3. Write each method with the **exact prototype** given (`void accept()`, `void calculate()`, ...).
4. Use the **correct slab logic** (flat-rate vs slab-by-slab) — read the table carefully.
5. Write `main()` that creates an object and calls the methods **in the right order** (accept → calculate → display).
6. Add the **variable description table** if required.



8. Predict the output

Example 1

```
class Demo
{
    int x;
    Demo()       { x = 5; System.out.print("A "); }
    Demo(int n)  { x = n; System.out.print("B "); }
    public static void main(String[] args)
    {
        Demo d1 = new Demo();
        Demo d2 = new Demo(9);
        Demo d3 = d1;
        d3.x += d2.x;
        System.out.println(d1.x + " " + d2.x);
    }
}
```

Show answer

Constructors print `A` then `B`. `d3` refers to the same object as `d1`, so `d1.x` = 5 + 9 = 14. **Output:** `A B 14 9`

Example 2

```
class P
{
    int a, b;
    P(int a, int b)
    {
        a = a;
        this.b = b;
    }
    public static void main(String[] args)
    {
        P p = new P(3, 4);
        System.out.println(p.a + " " + p.b);
    }
}
```

Show answer

`a = a;` assigns the parameter to itself; the data member `a` keeps its default 0. **Output:** `0 4`

9. Find the errors

```
class Item
{
    String name;
    double price;
    void Item(String n, double p) { name = n; price = p; }   // (1)
    Item(String n) { name = n; return 0; }                    // (2)
    public static void main(String[] args)
    {
        Item a = new Item();                                  // (3)
        Item b = new Item("Pen");
        b.Item("Pencil");                                     // (4)
    }
}
```

Show corrections

1. With `void`, this is a method, not a constructor — remove `void`: `Item(String n, double p)`.
2. A constructor cannot return a value — remove `return 0;`.
3. The class already has a constructor, so no default constructor is supplied — add `Item() { … }` or pass arguments.
4. A constructor cannot be called through an object — use `new Item("Pencil")`.

10. One-line facts

- A constructor has the **same name as the class** and **no return type**.
- It is invoked **automatically** when an object is created with `new`.
- The compiler provides a **default constructor** only when the class has **no** constructor.
- A constructor with parameters is a **parameterised constructor**.
- Several constructors with different parameter lists = **constructor overloading**.
- `this` refers to the **current object**.
- A constructor **cannot be static** and cannot return a value.

🧠 Test Yourself

Part A — Multiple choice

1. Which is a valid constructor of class `Car`?
   (a) `void Car() { }` (b) `Car() { }` (c) `int Car() { return 0; }` (d) `static Car() { }`
2. A class has only `Car(String model)`. Which statement compiles?
   (a) `Car c = new Car();` (b) `Car c = new Car("i20");` (c) both (d) neither
3. *Assertion (A):* Every class in Java has at least one constructor.
   Reason (R): If no constructor is defined, the compiler supplies a default constructor.

(a) Both true; R explains A (b) Both true; R doesn't explain A (c) A true, R false (d) A false, R true

4. What is printed? `java class T { int n; T() { n = 10; } T(int x) { n = x * 2; } public static void main(String[] a) { T t1 = new T(), t2 = new T(t1.n); System.out.println(t1.n + t2.n); } }` (a) 20 (b) 30 (c) 1020 (d) 40
5. How many times is a constructor called for `Box b1 = new Box(); Box b2 = b1; Box b3 = new Box();`? (a) 1 (b) 2 (c) 3 (d) 0
6. The keyword used to refer to the current object is (a) `new` (b) `super` (c) `this` (d) `self`
7. *Assertion (A):* Constructors can be overloaded.
   Reason (R): Constructors have no return type.
   (a) Both true; R explains A (b) Both true; R doesn't explain A (c) A true, R false (d) A false, R true

Part B — Short answer questions

1. Define a constructor. State two of its characteristics.
2. Differentiate between a default and a parameterised constructor.
3. What is constructor overloading? Give an example.
4. Give two differences between a constructor and a method.
5. When does the compiler **not** provide a default constructor?
6. What is the use of the keyword `this`? Give an example.
7. Name the data member values set by the compiler-supplied default constructor for `int`, `double`, `String`, `boolean`.

✅ Answers — Part A

Q	Ans	Reason
1	(b)	No return type, not static
2	(b)	No default constructor is supplied once a constructor is defined
3	(a)	—
4	(b)	10 + 20
5	(b)	`b2 = b1` creates no object
6	(c)	—
7	(b)	Both are true, but overloading is possible because parameter lists differ, not because there is no return type

✅ Answers — Part B

1. A member method with the same name as the class and no return type, called automatically when an object is created, to initialise it. It has no return type; it is invoked automatically with `new`.
2. A default constructor has no parameters and gives the same initial values to every object; a parameterised constructor takes parameters so that each object can be initialised with different values.
3. Having more than one constructor in a class with different parameter lists, e.g. `Rectangle()`, `Rectangle(double s)`, `Rectangle(double l, double b)`.
4. A constructor has the class name and no return type, and is called automatically once per object; a method has any name, a return type, and is called explicitly any number of times.
5. When the class already defines at least one constructor.
6. `this` refers to the current object; it is used to distinguish data members from parameters with the same name: `this.price = price;`

7. `0`, `0.0`, `null`, `false`.

💻 Programs in this chapter

No.	Program	
Q212	Non-Parameterised (Default) Constructor	
Q213	Parameterised Constructor	
Q214	Constructor Overloading: Rectangle with Three Constructors	
Q215	The this Keyword in a Constructor	
Q216	Constructor Overloading: Class Time	
Q217	Class ElectricBill	📝 ICSE 2017
Q218	Class ShowRoom	📝 ICSE 2019
Q219	Class RailwayTicket	📝 ICSE 2018
Q220	Class BookFair	📝 ICSE 2016
Q221	Class ParkingLot	📝 ICSE 2015
Q222	Class movieMagic	📝 ICSE 2014
Q223	Class FruitJuice	
Q224	Class Mobike	
Q225	Class CabService	
Q226	Class Employee: Annual Income Tax	
Q227	Class Student: Stream Allocation	
Q228	Class Library: Fine for Late Return	
Q229	Class Hotel: Room Bill	
Q230	Class Telephone: Bill from Meter Readings	
Q231	Class Courier: Charges per 500 g	
Q232	Class CloudStorage	
Q233	Class Eshop: Discount on Online Purchase	
Q234	Class BankDeposit: Maturity with Compound Interest	
Q235	Class BusConveyance	
Q236	Class CinemaTicket	
Q237	Class Quiz: A Small Quiz Game	🌟 *Extra*

Programs Q212 – Q237

Q212 Non-Parameterised (Default) Constructor

Define a class Student (written here as class Q_212) with data members String name, int roll and double marks. Write a non-parameterised (default) constructor that initialises name to "Not assigned", roll to 0 and marks to 0.0, and a method void show() to display the data members. In main(), create two objects and display them to show that the constructor is invoked automatically for every object created. Then assign new values to the second object and display it again.

Q_212.java

```
public class Q_212
{
    String name;
    int roll;
    double marks;

    Q_212()
    {
        name = "Not assigned";
        roll = 0;
        marks = 0.0;
        System.out.println("Constructor invoked");
    }

    void show()
    {
        System.out.println("Name: " + name + ", Roll: " + roll + ", Marks: " + marks);
    }

    public static void main(String[] args)
    {
        Q_212 s1 = new Q_212();
        Q_212 s2 = new Q_212();
        s1.show();
        s2.show();
        s2.name = "Neha Bose";
        s2.roll = 12;
        s2.marks = 91.5;
        s2.show();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
name	String	to store the student's name
roll	int	to store the roll number
marks	double	to store the marks
s1	Q_212	to refer to the first student
s2	Q_212	to refer to the second student

Q213 Parameterised Constructor

Define a class Employee (written here as class Q_213) with the following description:

```
Data members : String name; double basic, da, hra, gross
Employee(String n, double b) : parameterised constructor to
               initialise name and basic, and calculate
               da = 25% of basic, hra = 12% of basic and
               gross = basic + da + hra
void display() : to display all the details
```

In main(), accept the name and basic salary, create an object using the parameterised constructor and display the details.

Q_213.java

```java
import java.util.Scanner;

public class Q_213
{
    String name;
    double basic, da, hra, gross;

    Q_213(String n, double b)
    {
        name = n;
        basic = b;
        da = basic * 25 / 100;
        hra = basic * 12 / 100;
        gross = basic + da + hra;
    }

    void display()
    {
        System.out.println("Name   : " + name);
        System.out.println("Basic  : " + basic);
        System.out.println("DA     : " + da);
        System.out.println("HRA    : " + hra);
        System.out.println("Gross  : " + gross);
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter name: ");
        String nm = sc.nextLine();
        System.out.print("Enter basic salary: ");
        double bs = sc.nextDouble();
        Q_213 e = new Q_213(nm, bs);
        e.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
name	String	to store the employee's name
basic	double	to store the basic salary
da	double	to store the dearness allowance
hra	double	to store the house rent allowance
gross	double	to store the gross salary
n	String	to receive the name

Name of the Variable	Data Type	Purpose / Description
b	double	to receive the basic salary
nm	String	to store the name entered
bs	double	to store the basic entered
e	Q_213	to refer to the employee object

Q214 Constructor Overloading: Rectangle with Three Constructors

Define a class Rectangle (written here as class Q_214) with data members double length and breadth, and the following overloaded constructors:

```
Rectangle()                   : sets length and breadth to 1
Rectangle(double s)           : creates a square of side s
Rectangle(double l, double b) : sets length = l, breadth = b
```

Also define double area() and void show(). In main(), create one object using each constructor and display their dimensions and areas.

Q_214.java

```java
public class Q_214
{
    double length, breadth;

    Q_214()
    {
        length = 1;
        breadth = 1;
    }

    Q_214(double s)
    {
        length = s;
        breadth = s;
    }

    Q_214(double l, double b)
    {
        length = l;
        breadth = b;
    }

    double area()
    {
        return length * breadth;
    }

    void show()
    {
        System.out.println(length + " x " + breadth + ", area = " + area());
    }

    public static void main(String[] args)
    {
        Q_214 r1 = new Q_214();
        Q_214 r2 = new Q_214(4.5);
        Q_214 r3 = new Q_214(8, 2.5);
        r1.show();
        r2.show();
        r3.show();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
length	double	to store the length
breadth	double	to store the breadth
s	double	to receive the side of a square
l	double	to receive the length
b	double	to receive the breadth
r1	Q_214	to refer to the default rectangle
r2	Q_214	to refer to the square
r3	Q_214	to refer to the third rectangle

Q215 The this Keyword in a Constructor

Define a class Book (written here as class Q_215) with data members String title, String author and double price. Write a parameterised constructor whose parameters have the same names as the data members (title, author, price) and use the keyword this to assign them. Also define void show(). In main(), create an object using values entered by the user and display it. If the price entered is negative, store 0.

Q_215.java

```
import java.util.Scanner;

public class Q_215
{
    String title, author;
    double price;

    Q_215(String title, String author, double price)
    {
        this.title = title;
        this.author = author;
        if (price < 0)
            this.price = 0;
        else
            this.price = price;
    }

    void show()
    {
        System.out.println("Title  : " + title);
        System.out.println("Author : " + author);
        System.out.println("Price  : Rs " + price);
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter title: ");
        String t = sc.nextLine();
        System.out.print("Enter author: ");
        String a = sc.nextLine();
        System.out.print("Enter price: ");
        double p = sc.nextDouble();
        Q_215 bk = new Q_215(t, a, p);
        bk.show();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
title	String	to store/receive the title
author	String	to store/receive the author
price	double	to store/receive the price
t	String	to store the title entered
a	String	to store the author entered
p	double	to store the price entered
bk	Q_215	to refer to the book object

Q216 Constructor Overloading: Class Time

Define a class Time (written here as class Q_216) with data members int hh, mm, ss and the following overloaded constructors:

```
Time()                    : sets the time to 00:00:00
Time(int h)               : sets hours to h, minutes and
                            seconds to 0
Time(int h, int m)        : sets hours and minutes
Time(int h, int m, int s) : sets hours, minutes and seconds
```

Every constructor must store a valid time: any value out of range (hours 0-23, minutes and seconds 0-59) is stored as 0. Also write void display() to show the time as hh:mm:ss with two digits each. In main(), create one object with each constructor, including one with invalid values, and display them.

Q_216.java

```
public class Q_216
{
    int hh, mm, ss;

    Q_216()
    {
        hh = 0;
        mm = 0;
        ss = 0;
    }

    Q_216(int h)
    {
        hh = (h >= 0 && h <= 23) ? h : 0;
        mm = 0;
        ss = 0;
    }

    Q_216(int h, int m)
    {
        hh = (h >= 0 && h <= 23) ? h : 0;
        mm = (m >= 0 && m <= 59) ? m : 0;
        ss = 0;
    }

    Q_216(int h, int m, int s)
    {
        hh = (h >= 0 && h <= 23) ? h : 0;
        mm = (m >= 0 && m <= 59) ? m : 0;
        ss = (s >= 0 && s <= 59) ? s : 0;
    }

    void display()
```

```
    {
        String t = "";
        if (hh < 10) t = t + "0";
        t = t + hh + ":";
        if (mm < 10) t = t + "0";
        t = t + mm + ":";
        if (ss < 10) t = t + "0";
        t = t + ss;
        System.out.println(t);
    }

    public static void main(String[] args)
    {
        Q_216 t1 = new Q_216();
        Q_216 t2 = new Q_216(9);
        Q_216 t3 = new Q_216(14, 5);
        Q_216 t4 = new Q_216(23, 59, 7);
        Q_216 t5 = new Q_216(25, 70, 30);
        t1.display();
        t2.display();
        t3.display();
        t4.display();
        t5.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
hh	int	to store the hours
mm	int	to store the minutes
ss	int	to store the seconds
h	int	to receive the hours
m	int	to receive the minutes
s	int	to receive the seconds
t	String	to build the formatted time
t1	Q_216	to refer to the first time object
t2	Q_216	to refer to the second time object
t3	Q_216	to refer to the third time object
t4	Q_216	to refer to the fourth time object
t5	Q_216	to refer to the fifth time object

Q217 Class ElectricBill

ICSE 2017 Define a class ElectricBill (written here as class Q_217) with the following specifications:

```
Instance variables:
String n      : to store the name of the customer
int units     : to store the number of units consumed
double bill   : to store the amount to be paid

Member methods:
ElectricBill()   : default constructor to initialise the data
                   members (n = "", units = 0, bill = 0.0)
void accept()    : to accept the name and the units consumed
void calculate() : to calculate the bill as per the tariff:
                     First 100 units        Rs 2.00 per unit
                     Next 200 units         Rs 3.00 per unit
                     Above 300 units        Rs 5.00 per unit
                   A surcharge of 2.5% is charged if the number
                   of units consumed is above 300 units.
void print()     : to print the details as follows:
                     Name of the customer : ...
                     Number of units consumed : ...
                     Bill amount : ...
```

Write a main method to create an object of the class and call the above member methods.

Q_217.java

```java
import java.util.Scanner;

public class Q_217
{
    String n;
    int units;
    double bill;

    Q_217()
    {
        n = "";
        units = 0;
        bill = 0.0;
    }

    void accept()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the name of the customer: ");
        n = sc.nextLine();
        System.out.print("Enter the units consumed: ");
        units = sc.nextInt();
        if (units < 0)
            units = 0;
    }

    void calculate()
    {
        if (units <= 100)
            bill = units * 2.0;
        else if (units <= 300)
            bill = 100 * 2.0 + (units - 100) * 3.0;
        else
        {
            bill = 100 * 2.0 + 200 * 3.0 + (units - 300) * 5.0;
            bill = bill + bill * 2.5 / 100;
        }
    }

    void print()
    {
```

```
        System.out.println("Name of the customer     : " + n);
        System.out.println("Number of units consumed : " + units);
        System.out.println("Bill amount              : " + bill);
    }

    public static void main(String[] args)
    {
        Q_217 ob = new Q_217();
        ob.accept();
        ob.calculate();
        ob.print();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	String	to store the customer's name
units	int	to store the units consumed
bill	double	to store the bill amount
ob	Q_217	to refer to the object

Q218 Class ShowRoom

ICSE 2019 Define a class ShowRoom (written here as class Q_218) with the following description:

```
Instance variables:
String name   : to store the name of the customer
long mobno    : to store the mobile number of the customer
double cost   : to store the cost of the items purchased
double dis    : to store the discount amount
double amount : to store the amount to be paid after discount

Member methods:
ShowRoom()       : default constructor to initialise the data
                   members
void input()     : to input the customer name, mobile number and
                   cost
void calculate() : to calculate the discount on the cost of the
                   purchased items, based on the following:
                     Cost                        Discount (%)
                     Less than or equal to 10000      5%
                     10001 to 20000                  10%
                     20001 to 35000                  15%
                     More than 35000                 20%
void display()   : to display the customer name, mobile number
                   and the amount to be paid after discount
```

Write a main method to create an object and call the member methods.

Q_218.java

```
import java.util.Scanner;

public class Q_218
{
    String name;
    long mobno;
    double cost, dis, amount;

    Q_218()
    {
        name = "";
        mobno = 0;
        cost = 0;
```

```
        dis = 0;
        amount = 0;
    }

    void input()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter customer name: ");
        name = sc.nextLine();
        System.out.print("Enter mobile number: ");
        mobno = sc.nextLong();
        System.out.print("Enter cost: ");
        cost = sc.nextDouble();
    }

    void calculate()
    {
        int rate;
        if (cost <= 10000)
            rate = 5;
        else if (cost <= 20000)
            rate = 10;
        else if (cost <= 35000)
            rate = 15;
        else
            rate = 20;
        dis = cost * rate / 100;
        amount = cost - dis;
    }

    void display()
    {
        System.out.println("Customer name : " + name);
        System.out.println("Mobile number : " + mobno);
        System.out.println("Amount to pay : " + amount);
    }

    public static void main(String[] args)
    {
        Q_218 ob = new Q_218();
        ob.input();
        ob.calculate();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
name	String	to store the customer's name
mobno	long	to store the mobile number
cost	double	to store the cost of items
dis	double	to store the discount
amount	double	to store the amount to be paid
rate	int	to store the discount percentage
ob	Q_218	to refer to the object

Q219 Class RailwayTicket

ICSE 2018 Design a class RailwayTicket (written here as class Q_219) with the following description:

```
Instance variables:
String name     : to store the name of the customer
String coach    : to store the type of coach the customer wants
                  to travel in
long mobno      : to store the customer's mobile number
int amt         : to store the basic amount of the ticket
int totalamt    : to store the amount to be paid after updating
                  the original amount

Member methods:
RailwayTicket() : default constructor
void accept()   : to take input for name, coach, mobile number
                  and amount
void update()   : to update the amount as per the coach selected
                    Type of coach      Extra amount
                    First_AC           700
                    Second_AC          500
                    Third_AC           250
                    sleeper            None
void display()  : to display all the details of the customer,
                  such as name, coach, total amount and mobile
                  number
```

Write a main method to create an object and call the member methods. (The coach type is compared ignoring case.)

Q_219.java

```java
import java.util.Scanner;

public class Q_219
{
    String name, coach;
    long mobno;
    int amt, totalamt;

    Q_219()
    {
        name = "";
        coach = "";
        mobno = 0;
        amt = 0;
        totalamt = 0;
    }

    void accept()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter name: ");
        name = sc.nextLine();
        System.out.print("Enter coach (First_AC/Second_AC/Third_AC/sleeper): ");
        coach = sc.next();
        System.out.print("Enter mobile number: ");
        mobno = sc.nextLong();
        System.out.print("Enter basic amount: ");
        amt = sc.nextInt();
    }

    void update()
    {
        if (coach.equalsIgnoreCase("First_AC"))
            totalamt = amt + 700;
        else if (coach.equalsIgnoreCase("Second_AC"))
            totalamt = amt + 500;
        else if (coach.equalsIgnoreCase("Third_AC"))
```

```
            totalamt = amt + 250;
        else
            totalamt = amt;
    }

    void display()
    {
        System.out.println("Name          : " + name);
        System.out.println("Coach         : " + coach);
        System.out.println("Total amount  : " + totalamt);
        System.out.println("Mobile number : " + mobno);
    }

    public static void main(String[] args)
    {
        Q_219 ob = new Q_219();
        ob.accept();
        ob.update();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
name	String	to store the customer's name
coach	String	to store the type of coach
mobno	long	to store the mobile number
amt	int	to store the basic amount
totalamt	int	to store the total amount
ob	Q_219	to refer to the object

Q220 Class BookFair

ICSE 2016 Define a class BookFair (written here as class Q_220) with the following description:

```
Instance variables:
String Bname  : to store the name of the book
double price  : to store the price of the book

Member methods:
BookFair()       : default constructor to initialise the data
                   members
void Input()     : to input and store the name and the price of
                   the book
void calculate() : to calculate the price after discount.
                   Discount is calculated based on the following:
                     Price                          Discount
                     Less than or equal to Rs 1000    2% of price
                     More than Rs 1000 and less
                       than or equal to Rs 3000      10% of price
                     More than Rs 3000              15% of price
void display()   : to display the name and the price of the book
                   after discount
```

Write a main method to create an object and call the member methods.

Q_220.java

```
import java.util.Scanner;

public class Q_220
{
```

```
    String Bname;
    double price;

    Q_220()
    {
        Bname = "";
        price = 0.0;
    }

    void Input()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the name of the book: ");
        Bname = sc.nextLine();
        System.out.print("Enter the price: ");
        price = sc.nextDouble();
    }

    void calculate()
    {
        double disc;
        if (price <= 1000)
            disc = price * 2 / 100;
        else if (price <= 3000)
            disc = price * 10 / 100;
        else
            disc = price * 15 / 100;
        price = price - disc;
    }

    void display()
    {
        System.out.println("Name of the book       : " + Bname);
        System.out.println("Price after discount   : " + price);
    }

    public static void main(String[] args)
    {
        Q_220 ob = new Q_220();
        ob.Input();
        ob.calculate();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
Bname	String	to store the name of the book
price	double	to store the price of the book
disc	double	to store the discount
ob	Q_220	to refer to the object

Q221 Class ParkingLot

ICSE 2015 Define a class ParkingLot (written here as class Q_221) with the following description:

```
Instance variables:
int vno       : to store the vehicle number
int hours     : to store the number of hours the vehicle is parked
double bill   : to store the bill amount

Member methods:
ParkingLot()     : default constructor
void input()     : to input and store the vehicle number and hours
void calculate() : to compute the parking charge at the rate of
                   Rs 3 for the first hour or part thereof, and
                   Rs 1.50 for each additional hour or part thereof
void display()   : to display the details
```

Write a main method to create an object and call the member methods. (Treat hours less than 1 as 1 hour.)

Q_221.java

```java
import java.util.Scanner;

public class Q_221
{
    int vno, hours;
    double bill;

    Q_221()
    {
        vno = 0;
        hours = 0;
        bill = 0.0;
    }

    void input()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter vehicle number: ");
        vno = sc.nextInt();
        System.out.print("Enter hours parked: ");
        hours = sc.nextInt();
        if (hours < 1)
            hours = 1;
    }

    void calculate()
    {
        bill = 3 + (hours - 1) * 1.50;
    }

    void display()
    {
        System.out.println("Vehicle number : " + vno);
        System.out.println("Hours parked   : " + hours);
        System.out.println("Bill           : Rs " + bill);
    }

    public static void main(String[] args)
    {
        Q_221 ob = new Q_221();
        ob.input();
        ob.calculate();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
vno	int	to store the vehicle number
hours	int	to store the hours parked
bill	double	to store the bill amount
ob	Q_221	to refer to the object

Q222 Class movieMagic

ICSE 2014 Define a class named movieMagic (written here as class Q_222) with the following description:

```
Instance variables:
int year       : to store the year of release of a movie
String title   : to store the title of the movie
float rating   : to store the popularity rating of the movie
                 (minimum rating = 0.0 and maximum rating = 5.0)

Member methods:
movieMagic()   : default constructor to initialise numeric data
                 members to 0 and String data member to ""
void accept()  : to input and store year, title and rating
void display() : to display the title of the movie and a message
                 based on the rating as per the table below:
                   Rating              Message to be displayed
                   0.0 to 2.0          Flop
                   2.1 to 3.4          Semi-hit
                   3.5 to 4.5          Hit
                   4.6 to 5.0          Super Hit
```

Write a main method to create an object of the class and call the above member methods. (Display "Invalid rating" for a rating outside 0.0 to 5.0.)

Q_222.java

```java
import java.util.Scanner;

public class Q_222
{
    int year;
    String title;
    float rating;

    Q_222()
    {
        year = 0;
        title = "";
        rating = 0.0f;
    }

    void accept()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the title: ");
        title = sc.nextLine();
        System.out.print("Enter the year of release: ");
        year = sc.nextInt();
        System.out.print("Enter the rating: ");
        rating = sc.nextFloat();
    }

    void display()
    {
        System.out.println("Title : " + title);
        if (rating < 0.0f || rating > 5.0f)
```

```
            System.out.println("Invalid rating");
        else if (rating <= 2.0f)
            System.out.println("Flop");
        else if (rating <= 3.4f)
            System.out.println("Semi-hit");
        else if (rating <= 4.5f)
            System.out.println("Hit");
        else
            System.out.println("Super Hit");
    }

    public static void main(String[] args)
    {
        Q_222 ob = new Q_222();
        ob.accept();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
year	int	to store the year of release
title	String	to store the title
rating	float	to store the rating
ob	Q_222	to refer to the object

Q223 Class FruitJuice

Define a class called FruitJuice (written here as class Q_223) with the following description:

```
Instance variables:
int product_code     : stores the product code number
String flavour       : stores the flavour of the juice
                       (e.g. orange, apple)
String pack_type     : stores the type of packaging (e.g. tetra
                       pack, PET bottle)
int pack_size        : stores the package size (e.g. 200 ml)
int product_price    : stores the price of the product

Member methods:
FruitJuice()   : default constructor to initialise integer data
                 members to 0 and String data members to ""
void input()   : to input and store the product code, flavour,
                 pack type, pack size and product price
void discount(): to reduce the product price by 10
void display() : to display the product code, flavour, pack type,
                 pack size and product price
```

Write a main method to create an object and call the member methods.

Q_223.java

```
import java.util.Scanner;

public class Q_223
{
    int product_code;
    String flavour;
    String pack_type;
    int pack_size;
    int product_price;

    Q_223()
    {
```

```
        product_code = 0;
        flavour = "";
        pack_type = "";
        pack_size = 0;
        product_price = 0;
    }

    void input()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter product code: ");
        product_code = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter flavour: ");
        flavour = sc.nextLine();
        System.out.print("Enter pack type: ");
        pack_type = sc.nextLine();
        System.out.print("Enter pack size (ml): ");
        pack_size = sc.nextInt();
        System.out.print("Enter product price: ");
        product_price = sc.nextInt();
    }

    void discount()
    {
        product_price = product_price - 10;
        if (product_price < 0)
            product_price = 0;
    }

    void display()
    {
        System.out.println("Product code  : " + product_code);
        System.out.println("Flavour       : " + flavour);
        System.out.println("Pack type     : " + pack_type);
        System.out.println("Pack size     : " + pack_size + " ml");
        System.out.println("Product price : Rs " + product_price);
    }

    public static void main(String[] args)
    {
        Q_223 ob = new Q_223();
        ob.input();
        ob.discount();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
product_code	int	to store the product code
flavour	String	to store the flavour
pack_type	String	to store the type of packaging
pack_size	int	to store the package size
product_price	int	to store the price
ob	Q_223	to refer to the object

Q224 Class Mobike

Define a class Mobike (written here as class Q_224) with the following description:

```
Instance variables:
int bno      : to store the bike's number
long phno    : to store the phone number of the customer
String name  : to store the name of the customer
int days     : to store the number of days the bike is taken on
               rent
int charge   : to calculate and store the rental charge

Member methods:
Mobike()       : default constructor
void input()   : to input and store the details of the customer
void compute() : to compute the rental charge. The rent for a
                 Mobike is charged on the following basis:
                   First five days          Rs 500 per day
                   Next five days           Rs 400 per day
                   Rest of the days         Rs 200 per day
void display() : to display the details in the following format:
                   Bike No.  Phone No.  Name  No. of days  Charge
```

Write a main method to create an object and call the member methods.

Q_224.java

```java
import java.util.Scanner;

public class Q_224
{
    int bno;
    long phno;
    String name;
    int days, charge;

    Q_224()
    {
        bno = 0;
        phno = 0;
        name = "";
        days = 0;
        charge = 0;
    }

    void input()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter bike number: ");
        bno = sc.nextInt();
        System.out.print("Enter phone number: ");
        phno = sc.nextLong();
        sc.nextLine();
        System.out.print("Enter name: ");
        name = sc.nextLine();
        System.out.print("Enter number of days: ");
        days = sc.nextInt();
        if (days < 0)
            days = 0;
    }

    void compute()
    {
        if (days <= 5)
            charge = days * 500;
        else if (days <= 10)
            charge = 5 * 500 + (days - 5) * 400;
        else
            charge = 5 * 500 + 5 * 400 + (days - 10) * 200;
```

```
    }

    void display()
    {
        System.out.println("Bike No.\tPhone No.\tName\tNo. of days\tCharge");
        System.out.println(bno + "\t\t" + phno + "\t" + name + "\t" + days + "\t\t" + charge);
    }

    public static void main(String[] args)
    {
        Q_224 ob = new Q_224();
        ob.input();
        ob.compute();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
bno	int	to store the bike number
phno	long	to store the phone number
name	String	to store the customer's name
days	int	to store the number of days
charge	int	to store the rental charge
ob	Q_224	to refer to the object

Q225 Class CabService

Define a class CabService (written here as class Q_225) with the following description:

```
Instance variables:
String car_type : to store the type of car ("AC" or "NON AC")
double km       : to store the kilometres travelled
double bill     : to calculate and store the bill amount

Member methods:
CabService()     : default constructor to initialise the data
                   members (String to "" and double to 0.0)
void accept()    : to accept car_type and km
void calculate() : to calculate the bill as per the rules:
                      Car type    Up to 5 km    Beyond 5 km
                      AC          Rs 150        Rs 10 per km extra
                      NON AC      Rs 120        Rs 8 per km extra
void display()   : to display the bill in the format
                      CAR TYPE  : ...
                      KILOMETRE TRAVELLED : ...
                      TOTAL BILL : ...
```

Write a main method to create an object and call the member methods. Display "Invalid car type" for any other type.

Q_225.java

```
import java.util.Scanner;

public class Q_225
{
    String car_type;
    double km, bill;

    Q_225()
    {
```

```
        car_type = "";
        km = 0.0;
        bill = 0.0;
    }

    void accept()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter car type (AC / NON AC): ");
        car_type = sc.nextLine().trim().toUpperCase();
        System.out.print("Enter kilometres travelled: ");
        km = sc.nextDouble();
    }

    void calculate()
    {
        if (car_type.equals("AC"))
            bill = (km <= 5) ? 150 : 150 + (km - 5) * 10;
        else if (car_type.equals("NON AC"))
            bill = (km <= 5) ? 120 : 120 + (km - 5) * 8;
        else
            bill = -1;
    }

    void display()
    {
        if (bill < 0)
        {
            System.out.println("Invalid car type");
        }
        else
        {
            System.out.println("CAR TYPE            : " + car_type);
            System.out.println("KILOMETRE TRAVELLED : " + km);
            System.out.println("TOTAL BILL          : " + bill);
        }
    }

    public static void main(String[] args)
    {
        Q_225 ob = new Q_225();
        ob.accept();
        ob.calculate();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
car_type	String	to store the type of car
km	double	to store the kilometres travelled
bill	double	to store the bill amount
ob	Q_225	to refer to the object

Q226 Class Employee: Annual Income Tax

Define a class Employee (written here as class Q_226) with the following description:

```
Instance variables:
String pan     : to store the PAN number
String name    : to store the name
double income  : to store the annual taxable income
double tax     : to store the tax calculated

Member methods:
Employee()   : default constructor
void input() : to accept the PAN number, name and annual income
void calc()  : to calculate the tax as per the following slabs:
                 Annual income (Rs)        Tax
                 Up to 2,50,000            Nil
                 2,50,001 to 5,00,000      5% of the amount above
                                           2,50,000
                 5,00,001 to 10,00,000     12,500 + 20% of the
                                           amount above 5,00,000
                 Above 10,00,000           1,12,500 + 30% of the
                                           amount above 10,00,000
void display() : to display PAN, name, income and tax
```

Write a main method to create an object and call the member methods.

Q_226.java

```java
import java.util.Scanner;

public class Q_226
{
    String pan, name;
    double income, tax;

    Q_226()
    {
        pan = "";
        name = "";
        income = 0;
        tax = 0;
    }

    void input()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter PAN: ");
        pan = sc.next();
        sc.nextLine();
        System.out.print("Enter name: ");
        name = sc.nextLine();
        System.out.print("Enter annual income: ");
        income = sc.nextDouble();
    }

    void calc()
    {
        if (income <= 250000)
            tax = 0;
        else if (income <= 500000)
            tax = (income - 250000) * 5 / 100;
        else if (income <= 1000000)
            tax = 12500 + (income - 500000) * 20 / 100;
        else
            tax = 112500 + (income - 1000000) * 30 / 100;
    }

    void display()
    {
```

```
        System.out.println("PAN    : " + pan);
        System.out.println("Name   : " + name);
        System.out.println("Income : " + income);
        System.out.println("Tax    : " + tax);
    }

    public static void main(String[] args)
    {
        Q_226 ob = new Q_226();
        ob.input();
        ob.calc();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
pan	String	to store the pan number
name	String	to store the name
income	double	to store the annual income
tax	double	to store the tax
ob	Q_226	to refer to the object

Q227 Class Student: Stream Allocation

Define a class Student (written here as class Q_227) with the following description:

```
Instance variables:
String name; int age; int m1, m2, m3; int maximum; double average

Member methods:
Student()    : default constructor
void acceptDetails() : to accept the name, age and marks in three
                       subjects
void compute() : to compute the average and the maximum of the
                 three marks
void display() : to display the name, age, marks, maximum and
                 average, and the stream allotted as per the table:
                   Average marks         Stream
                   90 and above          Science with Computers
                   80 to below 90        Science with Biology
                   70 to below 80        Commerce with Mathematics
                   60 to below 70        Commerce
                   Below 60              Humanities
```

Write a main method to create an object and call the member methods.

Q_227.java

```
import java.util.Scanner;

public class Q_227
{
    String name;
    int age, m1, m2, m3, maximum;
    double average;

    Q_227()
    {
        name = "";
        age = 0;
        m1 = 0;
        m2 = 0;
```

```
        m3 = 0;
        maximum = 0;
        average = 0.0;
    }

    void acceptDetails()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter name: ");
        name = sc.nextLine();
        System.out.print("Enter age: ");
        age = sc.nextInt();
        System.out.print("Enter marks in three subjects: ");
        m1 = sc.nextInt();
        m2 = sc.nextInt();
        m3 = sc.nextInt();
    }

    void compute()
    {
        average = (m1 + m2 + m3) / 3.0;
        maximum = Math.max(m1, Math.max(m2, m3));
    }

    void display()
    {
        String stream;
        if (average >= 90)
            stream = "Science with Computers";
        else if (average >= 80)
            stream = "Science with Biology";
        else if (average >= 70)
            stream = "Commerce with Mathematics";
        else if (average >= 60)
            stream = "Commerce";
        else
            stream = "Humanities";

        System.out.println("Name    : " + name);
        System.out.println("Age     : " + age);
        System.out.println("Marks   : " + m1 + ", " + m2 + ", " + m3);
        System.out.println("Maximum : " + maximum);
        System.out.println("Average : " + Math.round(average * 100) / 100.0);
        System.out.println("Stream  : " + stream);
    }

    public static void main(String[] args)
    {
        Q_227 ob = new Q_227();
        ob.acceptDetails();
        ob.compute();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
name	String	to store the student's name
age	int	to store the age
m1	int	to store marks in subject 1
m2	int	to store marks in subject 2

Name of the Variable	Data Type	Purpose / Description
m3	int	to store marks in subject 3
maximum	int	to store the highest marks
average	double	to store the average marks
stream	String	to store the stream allotted
ob	Q_227	to refer to the object

Q228 Class Library: Fine for Late Return

Define a class Library (written here as class Q_228) with the following description:

```
Instance variables:
String title, author; int daysLate; double fine

Member methods:
Library(String t, String a, int d) : parameterised constructor to
               initialise the title, author and number of days the
               book is returned late (a negative value is stored
               as 0)
void computeFine() : to compute the fine:
                       First 5 days late       Rs 2.00 per day
                       Next 5 days (6 to 10)   Rs 3.50 per day
                       Beyond 10 days          Rs 5.00 per day
void display()     : to display the details and the fine
```

In main(), accept the details, create an object using the constructor and call the methods.

Q_228.java

```java
import java.util.Scanner;

public class Q_228
{
    String title, author;
    int daysLate;
    double fine;

    Q_228(String t, String a, int d)
    {
        title = t;
        author = a;
        daysLate = (d < 0) ? 0 : d;
        fine = 0;
    }

    void computeFine()
    {
        if (daysLate <= 5)
            fine = daysLate * 2.0;
        else if (daysLate <= 10)
            fine = 5 * 2.0 + (daysLate - 5) * 3.5;
        else
            fine = 5 * 2.0 + 5 * 3.5 + (daysLate - 10) * 5.0;
    }

    void display()
    {
        System.out.println("Title     : " + title);
        System.out.println("Author    : " + author);
        System.out.println("Days late : " + daysLate);
        System.out.println("Fine      : Rs " + fine);
    }

    public static void main(String[] args)
```

```
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter title: ");
        String t = sc.nextLine();
        System.out.print("Enter author: ");
        String a = sc.nextLine();
        System.out.print("Enter days late: ");
        int d = sc.nextInt();
        Q_228 ob = new Q_228(t, a, d);
        ob.computeFine();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
title	String	to store the title
author	String	to store the author
daysLate	int	to store the days returned late
fine	double	to store the fine
t	String	to store/receive the title
a	String	to store/receive the author
d	int	to store/receive the days late
ob	Q_228	to refer to the object

Q229 Class Hotel: Room Bill

Define a class Hotel (written here as class Q_229) with the following description:

```
Instance variables:
String guest; char roomType; int days; double bill

Member methods:
Hotel()        : default constructor
void input()   : to accept the guest's name, the room type
                 ('S' single, 'D' double, 'U' suite) and the
                 number of days of stay
void compute() : to calculate the bill using the daily tariff
                   Single Rs 1500, Double Rs 2500, Suite Rs 4000
                 A GST of 12% is added if the bill exceeds
                 Rs 7500; otherwise 5% is added.
void display() : to display the guest name, room type, days and
                 the bill
```

Write a main method to create an object and call the methods. For a wrong room type display "Invalid room type".

Q_229.java

```
import java.util.Scanner;

public class Q_229
{
    String guest;
    char roomType;
    int days;
    double bill;

    Q_229()
    {
```

```
        guest = "";
        roomType = ' ';
        days = 0;
        bill = 0;
    }

    void input()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter guest name: ");
        guest = sc.nextLine();
        System.out.print("Enter room type (S/D/U): ");
        roomType = sc.next().charAt(0);
        System.out.print("Enter number of days: ");
        days = sc.nextInt();
    }

    void compute()
    {
        int rate;
        switch (roomType)
        {
            case 'S': case 's': rate = 1500; break;
            case 'D': case 'd': rate = 2500; break;
            case 'U': case 'u': rate = 4000; break;
            default: rate = -1;
        }
        if (rate < 0 || days < 1)
        {
            bill = -1;
        }
        else
        {
            bill = rate * days;
            if (bill > 7500)
                bill = bill + bill * 12 / 100;
            else
                bill = bill + bill * 5 / 100;
        }
    }

    void display()
    {
        if (bill < 0)
        {
            System.out.println("Invalid room type");
        }
        else
        {
            System.out.println("Guest     : " + guest);
            System.out.println("Room type : " + roomType);
            System.out.println("Days      : " + days);
            System.out.println("Bill      : Rs " + bill);
        }
    }

    public static void main(String[] args)
    {
        Q_229 ob = new Q_229();
        ob.input();
        ob.compute();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
guest	String	to store the guest's name
roomType	char	to store the room type
days	int	to store the days of stay
bill	double	to store the bill
rate	int	to store the daily tariff
ob	Q_229	to refer to the object

Q230 Class Telephone: Bill from Meter Readings

Define a class Telephone (written here as class Q_230) with the following description:

```
Instance variables:
int prv, pre   : to store the previous and present meter readings
int call       : to store the calls made (pre - prv)
String name    : to store the name of the consumer
double amt     : to store the call charges
double total   : to store the total amount to be paid

Member methods:
Telephone()     : default constructor
void input()    : to accept the readings and the name; display
                  "Invalid readings" (and treat calls as 0) if
                  the present reading is less than the previous
void cal()      : to calculate the charges as follows (slab by
                  slab):
                    First 100 calls        Rs 0.60 per call
                    Next 100 calls         Rs 0.80 per call
                    Next 100 calls         Rs 1.20 per call
                    Above 300 calls        Rs 1.50 per call
                  Every consumer pays a monthly rental of Rs 180.
void display()  : to display the name, calls, amount and total
```

Write a main method to create an object and call the methods.

Q_230.java

```java
import java.util.Scanner;

public class Q_230
{
    int prv, pre, call;
    String name;
    double amt, total;

    Q_230()
    {
        prv = 0;
        pre = 0;
        call = 0;
        name = "";
        amt = 0;
        total = 0;
    }

    void input()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter name: ");
        name = sc.nextLine();
        System.out.print("Enter previous and present readings: ");
        prv = sc.nextInt();
```

```
        pre = sc.nextInt();
        if (pre < prv)
        {
            System.out.println("Invalid readings");
            call = 0;
        }
        else
        {
            call = pre - prv;
        }
    }

    void cal()
    {
        if (call <= 100)
            amt = call * 0.60;
        else if (call <= 200)
            amt = 100 * 0.60 + (call - 100) * 0.80;
        else if (call <= 300)
            amt = 100 * 0.60 + 100 * 0.80 + (call - 200) * 1.20;
        else
            amt = 100 * 0.60 + 100 * 0.80 + 100 * 1.20 + (call - 300) * 1.50;
        total = amt + 180;
    }

    void display()
    {
        System.out.println("Name          : " + name);
        System.out.println("Calls made    : " + call);
        System.out.println("Call charges  : Rs " + amt);
        System.out.println("Total payable : Rs " + total);
    }

    public static void main(String[] args)
    {
        Q_230 ob = new Q_230();
        ob.input();
        ob.cal();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
prv	int	to store the previous reading
pre	int	to store the present reading
call	int	to store the calls made
name	String	to store the consumer's name
amt	double	to store the call charges
total	double	to store the total amount
ob	Q_230	to refer to the object

Q231 Class Courier: Charges per 500 g

Define a class Courier (written here as class Q_231) with the following description:

```
Instance variables:
String sender; double weight (in kg); char type; double charge

Member methods:
Courier()       : default constructor
void accept()   : to accept the sender's name, the weight of the
                  parcel and the type ('D' domestic,
                  'I' international)
void compute()  : to calculate the charge for every 500 g or part
                  thereof:
                    Domestic        Rs 50 per 500 g or part
                    International   Rs 200 per 500 g or part
                  (e.g. a 1.2 kg parcel counts as 3 parts of 500 g)
void display()  : to display the details and the charge
```

Write a main method to create an object and call the member methods. Display "Invalid input" for a weight that is not positive or a wrong type.

Q_231.java

```java
import java.util.Scanner;

public class Q_231
{
    String sender;
    double weight, charge;
    char type;

    Q_231()
    {
        sender = "";
        weight = 0;
        type = ' ';
        charge = 0;
    }

    void accept()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter sender's name: ");
        sender = sc.nextLine();
        System.out.print("Enter weight (kg): ");
        weight = sc.nextDouble();
        System.out.print("Enter type (D/I): ");
        type = sc.next().charAt(0);
    }

    void compute()
    {
        int parts = (int) Math.ceil(weight / 0.5);
        if (weight <= 0)
            charge = -1;
        else if (type == 'D' || type == 'd')
            charge = parts * 50;
        else if (type == 'I' || type == 'i')
            charge = parts * 200;
        else
            charge = -1;
    }

    void display()
    {
        if (charge < 0)
        {
            System.out.println("Invalid input");
```

```
        }
        else
        {
            System.out.println("Sender : " + sender);
            System.out.println("Weight : " + weight + " kg");
            System.out.println("Type   : " + type);
            System.out.println("Charge : Rs " + charge);
        }
    }

    public static void main(String[] args)
    {
        Q_231 ob = new Q_231();
        ob.accept();
        ob.compute();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
sender	String	to store the sender's name
weight	double	to store the weight in kg
charge	double	to store the charge
type	char	to store the type of service
parts	int	to store the number of 500 g parts
ob	Q_231	to refer to the object

Q232 Class CloudStorage

Define a class CloudStorage (written here as class Q_232) with the following description:

```
Instance variables:
int acno       : to store the account number
int storage    : to store the storage space used in GB
double bill    : to store the total bill

Member methods:
CloudStorage() : default constructor
void accept()  : to accept the account number and storage space
void calculate() : to calculate the bill (slab by slab):
                     First 15 GB          Rs 15 per GB
                     Next 15 GB           Rs 13 per GB
                     Above 30 GB          Rs 11 per GB
                   If the bill exceeds Rs 400, a surcharge of 5%
                   is added.
void display() : to display the account number, storage and bill
```

Write a main method to create an object and call the methods.

Q_232.java

```
import java.util.Scanner;

public class Q_232
{
    int acno, storage;
    double bill;

    Q_232()
    {
        acno = 0;
```

```
        storage = 0;
        bill = 0.0;
    }

    void accept()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter account number: ");
        acno = sc.nextInt();
        System.out.print("Enter storage used (GB): ");
        storage = sc.nextInt();
        if (storage < 0)
            storage = 0;
    }

    void calculate()
    {
        if (storage <= 15)
            bill = storage * 15;
        else if (storage <= 30)
            bill = 15 * 15 + (storage - 15) * 13;
        else
            bill = 15 * 15 + 15 * 13 + (storage - 30) * 11;
        if (bill > 400)
            bill = bill + bill * 5 / 100;
    }

    void display()
    {
        System.out.println("Account number : " + acno);
        System.out.println("Storage used   : " + storage + " GB");
        System.out.println("Total bill     : Rs " + bill);
    }

    public static void main(String[] args)
    {
        Q_232 ob = new Q_232();
        ob.accept();
        ob.calculate();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
acno	int	to store the account number
storage	int	to store the storage used
bill	double	to store the bill
ob	Q_232	to refer to the object

Q233 Class Eshop: Discount on Online Purchase

Define a class Eshop (written here as class Q_233) with the following description:

```
Instance variables:
String name   : to store the name of the item purchased
double price  : to store the price of the item

Member methods:
Eshop()          : default constructor
void accept()    : to accept the name and the price of the item
void calculate() : to calculate the net amount to be paid after
                   discount:
                     Price                   Discount
                     1000 to 25000           5.0%
                     25001 to 57000          7.5%
                     57001 to 100000         10.0%
                     More than 100000        15.0%
                     (below 1000: no discount)
void display()   : to display the name of the item and the net
                   amount
```

Write a main method to create an object and call the member methods.

Q_233.java

```java
import java.util.Scanner;

public class Q_233
{
    String name;
    double price;
    double net;

    Q_233()
    {
        name = "";
        price = 0.0;
        net = 0.0;
    }

    void accept()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter item name: ");
        name = sc.nextLine();
        System.out.print("Enter price: ");
        price = sc.nextDouble();
    }

    void calculate()
    {
        double rate;
        if (price < 1000)
            rate = 0;
        else if (price <= 25000)
            rate = 5.0;
        else if (price <= 57000)
            rate = 7.5;
        else if (price <= 100000)
            rate = 10.0;
        else
            rate = 15.0;
        net = price - price * rate / 100;
    }

    void display()
    {
        System.out.println("Item       : " + name);
```

```
        System.out.println("Net amount : Rs " + net);
    }

    public static void main(String[] args)
    {
        Q_233 ob = new Q_233();
        ob.accept();
        ob.calculate();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
name	String	to store the item name
price	double	to store the price
net	double	to store the net amount
rate	double	to store the discount rate
ob	Q_233	to refer to the object

Q234 Class BankDeposit: Maturity with Compound Interest

Define a class BankDeposit (written here as class Q_234) with the following description:

```
Instance variables:
String name; double principal; int years; double rate, maturity

Member methods:
BankDeposit(String n, double p, int y) : parameterised constructor
void setRate() : to set the rate of interest:
                   Up to 1 year          6.0%
                   2 to 3 years          7.0%
                   More than 3 years     7.5%
void compute() : to compute the maturity amount using compound
                 interest, maturity = principal x (1 + rate/100)^years
void display() : to display all the details (maturity rounded to
                 two decimal places)
```

In main(), accept the details, create an object and call the methods. Display "Invalid input" if the principal or years is not positive.

Q_234.java

```
import java.util.Scanner;

public class Q_234
{
    String name;
    double principal;
    int years;
    double rate, maturity;

    Q_234(String n, double p, int y)
    {
        name = n;
        principal = p;
        years = y;
        rate = 0;
        maturity = 0;
    }

    void setRate()
    {
```

```
        if (years <= 1)
            rate = 6.0;
        else if (years <= 3)
            rate = 7.0;
        else
            rate = 7.5;
    }

    void compute()
    {
        maturity = principal * Math.pow(1 + rate / 100, years);
    }

    void display()
    {
        System.out.println("Name      : " + name);
        System.out.println("Principal : " + principal);
        System.out.println("Years     : " + years);
        System.out.println("Rate      : " + rate + "%");
        System.out.println("Maturity  : " + Math.round(maturity * 100) / 100.0);
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter name: ");
        String n = sc.nextLine();
        System.out.print("Enter principal and years: ");
        double p = sc.nextDouble();
        int y = sc.nextInt();

        if (p <= 0 || y <= 0)
        {
            System.out.println("Invalid input");
        }
        else
        {
            Q_234 ob = new Q_234(n, p, y);
            ob.setRate();
            ob.compute();
            ob.display();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
name	String	to store the depositor's name
principal	double	to store the principal
years	int	to store the period in years
rate	double	to store the rate of interest
maturity	double	to store the maturity amount
n	String	to store/receive the name
p	double	to store/receive the principal
y	int	to store/receive the years
ob	Q_234	to refer to the object

Q235 Class BusConveyance

Define a class BusConveyance (written here as class Q_235) with the following description:

```
Instance variables:
String name; int age; double distance, fare

Member methods:
BusConveyance() : default constructor
void input()    : to accept the passenger's name, age and distance
void calculate(): to calculate the fare (slab by slab):
                    First 10 km          Rs 5 per km
                    Next 20 km           Rs 4 per km
                    Above 30 km          Rs 3 per km
                  Senior citizens (age 60 and above) get a 30%
                  concession on the fare.
void display()  : to display the details and the fare
```

Write a main method to create an object and call the member methods.

Q_235.java

```java
import java.util.Scanner;

public class Q_235
{
    String name;
    int age;
    double distance, fare;

    Q_235()
    {
        name = "";
        age = 0;
        distance = 0;
        fare = 0;
    }

    void input()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter name: ");
        name = sc.nextLine();
        System.out.print("Enter age and distance (km): ");
        age = sc.nextInt();
        distance = sc.nextDouble();
        if (distance < 0)
            distance = 0;
    }

    void calculate()
    {
        if (distance <= 10)
            fare = distance * 5;
        else if (distance <= 30)
            fare = 10 * 5 + (distance - 10) * 4;
        else
            fare = 10 * 5 + 20 * 4 + (distance - 30) * 3;
        if (age >= 60)
            fare = fare - fare * 30 / 100;
    }

    void display()
    {
        System.out.println("Name     : " + name);
        System.out.println("Age      : " + age);
        System.out.println("Distance : " + distance + " km");
        System.out.println("Fare     : Rs " + fare);
    }
```

```
    public static void main(String[] args)
    {
        Q_235 ob = new Q_235();
        ob.input();
        ob.calculate();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
name	String	to store the passenger's name
age	int	to store the age
distance	double	to store the distance
fare	double	to store the fare
ob	Q_235	to refer to the object

Q236 Class CinemaTicket

Define a class CinemaTicket (written here as class Q_236) with the following description:

```
Instance variables:
String name; char seat; int tickets; double amount

Member methods:
CinemaTicket() : default constructor
void accept()  : to accept the customer's name, the seat class
                 ('G' Gold, 'S' Silver, 'R' Regular) and the
                 number of tickets
void compute() : to calculate the amount using a switch statement:
                   Gold Rs 350, Silver Rs 250, Regular Rs 150 per
                   ticket, plus a convenience fee of 5% of the
                   ticket cost. Booking more than 5 tickets gives a
                   discount of 10% on the ticket cost (before the
                   fee is added).
void display() : to display the details and the amount
```

Write a main method to create an object and call the methods. Display "Invalid booking" for a wrong seat class or fewer than 1 ticket.

Q_236.java

```
import java.util.Scanner;

public class Q_236
{
    String name;
    char seat;
    int tickets;
    double amount;

    Q_236()
    {
        name = "";
        seat = ' ';
        tickets = 0;
        amount = 0;
    }

    void accept()
    {
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter name: ");
        name = sc.nextLine();
        System.out.print("Enter seat class (G/S/R): ");
        seat = sc.next().charAt(0);
        System.out.print("Enter number of tickets: ");
        tickets = sc.nextInt();
    }

    void compute()
    {
        int rate;
        switch (seat)
        {
            case 'G': case 'g': rate = 350; break;
            case 'S': case 's': rate = 250; break;
            case 'R': case 'r': rate = 150; break;
            default: rate = 0;
        }
        if (rate == 0 || tickets < 1)
        {
            amount = -1;
        }
        else
        {
            double cost = rate * tickets;
            if (tickets > 5)
                cost = cost - cost * 10 / 100;
            amount = cost + cost * 5 / 100;
        }
    }

    void display()
    {
        if (amount < 0)
        {
            System.out.println("Invalid booking");
        }
        else
        {
            System.out.println("Name    : " + name);
            System.out.println("Seat    : " + seat);
            System.out.println("Tickets : " + tickets);
            System.out.println("Amount  : Rs " + amount);
        }
    }

    public static void main(String[] args)
    {
        Q_236 ob = new Q_236();
        ob.accept();
        ob.compute();
        ob.display();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
name	String	to store the customer's name
seat	char	to store the seat class
tickets	int	to store the number of tickets
amount	double	to store the amount payable

Name of the Variable	Data Type	Purpose / Description
rate	int	to store the rate per ticket
cost	double	to store the ticket cost
ob	Q_236	to refer to the object

Q237 Class Quiz: A Small Quiz Game

Extra Define a class Quiz (written here as class Q_237) to conduct a short multiple-choice quiz:

```
Instance variables:
String player; int score, asked

Member methods:
Quiz(String p) : parameterised constructor to store the player's
                 name and set score and asked to 0
void ask(String question, String options, char answer) : to
                 display the question and options, accept the
                 player's answer (a, b, c or d) and add 1 to the
                 score if it is correct (ignoring case); it also
                 increments asked
void result()  : to display the score out of the questions asked,
                 the percentage and a remark: "Excellent!" (80%
                 or more), "Good" (50% or more) or "Keep practising"
```

Use a single Scanner object shared by the whole class (a static data member). In main(), accept the player's name, create the object and ask four Java questions.

Q_237.java

```java
import java.util.Scanner;

public class Q_237
{
    static Scanner sc = new Scanner(System.in);
    String player;
    int score, asked;

    Q_237(String p)
    {
        player = p;
        score = 0;
        asked = 0;
    }

    void ask(String question, String options, char answer)
    {
        asked++;
        System.out.println("Q" + asked + ". " + question);
        System.out.println(options);
        System.out.print("Your answer: ");
        char reply = sc.next().charAt(0);
        if (Character.toLowerCase(reply) == answer)
        {
            score++;
            System.out.println("Correct!");
        }
        else
        {
            System.out.println("Wrong! The answer is " + answer);
        }
    }

    void result()
    {
        double per = score * 100.0 / asked;
```

```
        System.out.println(player + ", you scored " + score + " out of " + asked + " (" + per + "%)");
        if (per >= 80)
            System.out.println("Excellent!");
        else if (per >= 50)
            System.out.println("Good");
        else
            System.out.println("Keep practising");
    }

    public static void main(String[] args)
    {
        System.out.print("Enter your name: ");
        String nm = sc.nextLine();
        Q_237 q = new Q_237(nm);
        q.ask("Which keyword creates an object?", "(a) class  (b) new  (c) this  (d) void", 'b');
        q.ask("What is the size of a char in Java?", "(a) 1 byte  (b) 4 bytes  (c) 2 bytes  (d) 8
bytes", 'c');
        q.ask("Which loop runs at least once?", "(a) for  (b) while  (c) nested for  (d) do-while",
'd');
        q.ask("Math.round(-2.5) returns", "(a) -2  (b) -3  (c) -2.0  (d) 2", 'a');
        q.result();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
player	String	to store the player's name
score	int	to store the score
asked	int	to count the questions asked
p	String	to receive the player's name
question	String	to receive the question text
options	String	to receive the options text
answer	char	to receive the correct option
reply	char	to store the player's answer
per	double	to store the percentage
nm	String	to store the name entered
q	Q_237	to refer to the quiz object

CHAPTER

12



Library Classes

Wrapper classes · Class as a composite type · Primitive vs composite data types · Autoboxing and unboxing · Methods of the wrapper classes

12 programs | Q238 – Q249 | 1 extra

Theory

📑 Contents

1. Library classes and packages
2. Class as a composite type
3. Wrapper classes
4. Autoboxing and unboxing
5. String-to-number conversion methods
6. Methods of the `Character` class
7. Predict the output
8. Find the errors
9. Differentiate between
10. One-line facts
11. Test yourself
12. Programs in this chapter

1. Library classes and packages

Library classes are the **pre-defined classes** supplied with Java (the Java Class Library / API). They are grouped into **packages**:

Package	Example classes
`java.lang` (imported by default)	`String`, `Math`, `System`, `Integer`, `Long`, `Float`, `Double`, `Character`, `Boolean`
`java.util`	`Scanner`
`java.io`	`BufferedReader`, `InputStreamReader`

2. Class as a composite type

A class groups together data members of various types (primitive and/or other classes) and methods. Hence a class is a **composite (user-defined / reference) data type**. Once a class is defined, its name can be used to declare variables, exactly like `int` or `double`.

```
class Date
{
    int dd, mm, yy;           // composite type made from three ints
}
...
Date birthday = new Date();   // Date is used as a data type
```

Basis	Primitive data type	Composite data type
Defined by	Java language (built-in)	Programmer or Java library (class)
Number of values	One	Many (one for each data member)
Size	Fixed	Depends on its members
Contains methods?	No	Yes
Stores	The value	A reference to an object
Created with `new`?	No	Yes (objects)
Example	`int`, `char`, `boolean`	`String`, `Scanner`, `Date`, arrays

3. Wrapper classes

A **wrapper class** is a library class that **wraps (encloses) a value of a primitive data type inside an object**. Every primitive type has a corresponding wrapper class in `java.lang`.

Primitive type	Wrapper class
`byte`	`Byte`
`short`	`Short`
`int`	**`Integer`**
`long`	`Long`
`float`	`Float`
`double`	`Double`
`char`	**`Character`**
`boolean`	`Boolean`

> **TIP**
>
> Only two wrapper names differ from their primitive names by more than a capital letter: **`Integer`** (for `int`) and **`Character`** (for `char`).

Need for wrapper classes:

1. To convert **strings into numbers** (`Integer.parseInt("25")`) — needed when numbers arrive as text.
2. To provide useful **utility methods** (`Character.isDigit()`, `Character.toUpperCase()`).
3. Some parts of Java work only with **objects**, not with primitive values; wrappers let primitives be treated as objects.

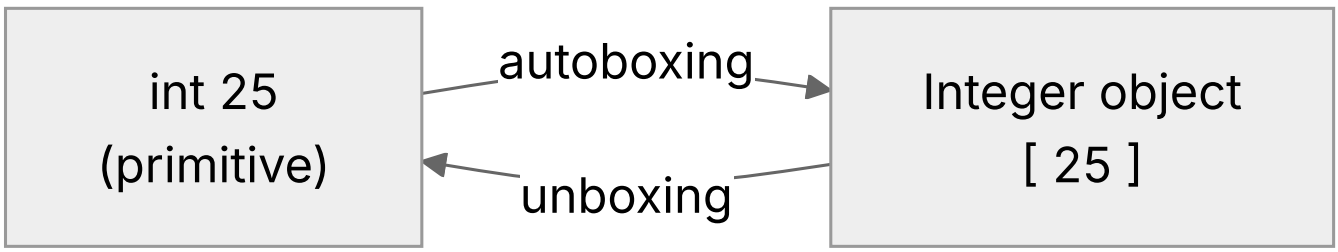
---

4. Autoboxing and unboxing

- **Autoboxing:** the **automatic conversion of a primitive value into an object of its wrapper class** by the compiler.
- **Unboxing:** the **automatic conversion of a wrapper class object into its primitive value**.

```
Integer obj = 25;          // autoboxing: int → Integer
Double d = 3.5;            // autoboxing: double → Double
Character c = 'J';         // autoboxing: char → Character

int n = obj;               // unboxing: Integer → int
double sum = obj + d;      // unboxing inside an expression → 28.5
```



Basis	Autoboxing	Unboxing
Converts	Primitive → wrapper object	Wrapper object → primitive
Example	`Integer x = 10;`	`int y = x;`

5. String-to-number conversion methods

Method	Return type	Example	Result
`Integer.parseInt(String s)`	`int`	`Integer.parseInt("452") + 8`	`460`
`Long.parseLong(String s)`	`long`	`Long.parseLong("9830012345")`	`9830012345`
`Float.parseFloat(String s)`	`float`	`Float.parseFloat("2.5")`	`2.5`
`Double.parseDouble(String s)`	`double`	`Double.parseDouble("3.75") * 2`	`7.5`

```
String a = "20", b = "30";
System.out.println(a + b);                                  // 2030  (string joining)
System.out.println(Integer.parseInt(a) + Integer.parseInt(b)); // 50  (numeric addition)
```

> **WARNING**
>
> If the string is not a valid number, these methods throw a **`NumberFormatException`** at run time: `Integer.parseInt("12a")`, `Integer.parseInt("4.5")`, `Integer.parseInt(" 45")` (space!), `Integer.parseInt("")`. `Double.parseDouble("45")` is fine → `45.0`.

Converting a number to a string: `String.valueOf(25)` → `"25"` (String class, Chapter 15), or `"" + 25`.

6. Methods of the `Character` class

Method	Return type	Returns **`true`** / result when…	Example → result
`Character.isDigit(char ch)`	`boolean`	`ch` is a digit `0`–`9`	`isDigit('7')` → `true`
`Character.isLetter(char ch)`	`boolean`	`ch` is a letter	`isLetter('$')` → `false`
`Character.isLetterOrDigit(char ch)`	`boolean`	`ch` is a letter or a digit	`isLetterOrDigit('_')` → `false`
`Character.isLowerCase(char ch)`	`boolean`	`ch` is a lowercase letter	`isLowerCase('a')` → `true`
`Character.isUpperCase(char ch)`	`boolean`	`ch` is an uppercase letter	`isUpperCase('5')` → `false`
`Character.isWhitespace(char ch)`	`boolean`	`ch` is a space, tab, new line, etc.	`isWhitespace('\t')` → `true`
`Character.toLowerCase(char ch)`	`char`	lowercase form of `ch` (unchanged if not an uppercase letter)	`toLowerCase('Q')` → `q`
`Character.toUpperCase(char ch)`	`char`	uppercase form of `ch` (unchanged if not a lowercase letter)	`toUpperCase('7')` → `7`

> **IMPORTANT**
>
> `toUpperCase()` / `toLowerCase()` return a **`char`**, so `int x = Character.toUpperCase('a');` stores **65**, and `System.out.println(Character.toUpperCase('a') + 1);` prints **66** (char + int → int).

Common uses

```
if (Character.isUpperCase(ch)) up++;                    // counting
int value = ch - '0';                                   // digit character → its value (when isDigit(ch))
char t = Character.isUpperCase(ch) ? Character.toLowerCase(ch) : Character.toUpperCase(ch);   // toggle case
```

7. Predict the output

```
System.out.println(Integer.parseInt("15") + Integer.parseInt("5") + "5");    // (1)
System.out.println("15" + 5 + Double.parseDouble("2.5"));                    // (2)
System.out.println(Character.isLetter('9') || Character.isDigit('9'));      // (3)
System.out.println(Character.toUpperCase('b') + 'a');                        // (4)
System.out.println((char) (Character.toLowerCase('Z') - 1));                 // (5)
Integer x = 7; int y = x * x + Long.parseLong("3") > 50 ? 1 : 0;             // (6)
System.out.println(y);
```

Show answers with working

#	Working	Output
1	15 + 5 = 20, then `20 + "5"`	`205`
2	`"15" + 5 = "155"`, then + 2.5	`1552.5`
3	false || true	`true`
4	`'B'` (66) + `'a'` (97)	`163`
5	`'z'` (122) − 1 = 121 → `'y'`	`y`
6	49 + 3 = 52 > 50 → 1	`1`

8. Find the errors

```
int a = Integer.parseInt(45);                // (1)
double d = Double.parseDouble("3.5", "2");   // (2)
boolean b = Character.isDigit("7");          // (3)
char c = Character.toUpperCase("x");         // (4)
String s = Character.toLowerCase('A');       // (5)
```

Show corrections

1. `parseInt` takes a String → `Integer.parseInt("45")`
2. `parseDouble` takes one argument → `Double.parseDouble("3.5")`
3. `isDigit` takes a char → `Character.isDigit('7')`
4. `toUpperCase(char)` takes a char → `Character.toUpperCase('x')`

5. It returns a `char` → `char s = Character.toLowerCase('A');` (or `String s = "" + Character.toLowerCase('A');`)

9. Differentiate between

Pair	Difference
`int` vs `Integer`	Primitive type storing a value vs wrapper class whose object wraps an int
Autoboxing vs unboxing	Primitive → object vs object → primitive
`parseInt()` vs `String.valueOf()`	String → int vs number → String
`isLetter()` vs `isLetterOrDigit()`	Letters only vs letters and digits
`isUpperCase()` vs `toUpperCase()`	Tests and returns `boolean` vs converts and returns `char`
Primitive vs composite type	Built-in single value vs class grouping several values and methods

10. One-line facts

- Wrapper classes are in the **`java.lang`** package.
- The wrapper class of `char` is **`Character`**; of `int` is **`Integer`**.
- `Integer.parseInt()` returns an **`int`**; `Double.parseDouble()` returns a **`double`**.
- An invalid number string causes **`NumberFormatException`**.
- The `isXxx()` methods of `Character` return **`boolean`**; `toUpperCase()` / `toLowerCase()` return **`char`**.
- Converting a primitive to its wrapper object automatically is **autoboxing**.
- A class is a **composite** data type.

🧠 Test Yourself

Part A — Multiple choice

1. What is printed? `System.out.println(Integer.parseInt("10") + Integer.parseInt("10") + "10");`
   (a) 101010 (b) 30 (c) 2010 (d) 1020
2. Which call causes a `NumberFormatException`?
   (a) `Double.parseDouble("7")` (b) `Integer.parseInt("-7")` (c) `Integer.parseInt("7.0")` (d) `Long.parseLong("70")`
3. *Assertion (A):* `Character.isLetterOrDigit('_')` returns false.
   Reason (R): The underscore is neither a letter nor a digit.
   (a) Both true; R explains A (b) Both true; R doesn't explain A (c) A true, R false (d) A false, R true
4. `char ch = 'm'; System.out.print(Character.toUpperCase(ch) + 1);` prints
   (a) N (b) M1 (c) 78 (d) m1
5. Which statement shows **unboxing**?
   (a) `Integer a = 5;` (b) `Integer a = 5; int b = a;` (c) `Double d = 2.0;` (d) `Character c = 'x';`
6. Which returns `true`? (a) `Character.isUpperCase('7')` (b) `Character.isWhitespace('\n')` (c) `Character.isDigit('A')` (d) `Character.isLetter(' ')`
7. The return type of `Character.toLowerCase('A')` is (a) String (b) int (c) char (d) boolean

8. `String s = "3" + 4; int k = Integer.parseInt(s) + 6;` gives `k` = (a) 13 (b) 40 (c) 346 (d) 7

Part B — Short answer questions

1. What is a wrapper class? Name the wrapper classes of `char` and `double`.
2. Define autoboxing and unboxing with one example each.
3. Give the output: `System.out.println(Character.isLetter('A') + " " + Character.toLowerCase('A'));`
4. Write a statement to convert the string `"2025"` into an integer and add 1 to it.
5. What is the difference between `Character.isUpperCase()` and `Character.toUpperCase()`?
6. Why is a class called a composite data type?
7. Name the exception thrown by `Integer.parseInt("1,000")`.

✅ Answers — Part A

Q	Ans	Reason
1	(c)	10 + 10 = 20, then `"20" + "10"`
2	(c)	`"7.0"` is not an integer string
3	(a)	—
4	(c)	`'M'` (77) + 1 = 78
5	(b)	Assigning a wrapper object to a primitive unboxes it
6	(b)	—
7	(c)	—
8	(b)	`"34"` → 34 + 6 = 40

✅ Answers — Part B

1. A library class that wraps a primitive value in an object. `Character`, `Double`.
2. Autoboxing: automatic conversion of a primitive into its wrapper object — `Integer i = 5;`. Unboxing: automatic conversion of a wrapper object into a primitive — `int k = i;`.
3. `true a`
4. `int y = Integer.parseInt("2025") + 1;`
5. `isUpperCase()` checks whether a character is an uppercase letter and returns `boolean`; `toUpperCase()` converts a character to uppercase and returns `char`.
6. Because it is made up of (composed of) several data members of primitive and/or other class types, and it can be used to create variables like a data type.
7. `NumberFormatException`.

💻 Programs in this chapter

No.	Program
Q238	Type of a Character using Character Methods
Q239	Count Uppercase, Lowercase, Digits, Spaces and Special Characters
Q240	Change the Case of a Character

No.	Program	
Q241	Adding Numbers Entered as Strings	
Q242	Output of the Wrapper Class Methods	
Q243	Autoboxing and Unboxing	
Q244	Sum of the Digits Present in an Alphanumeric String	
Q245	Build a Number from the Digits in a String	
Q246	Remove Characters that are not Letters or Digits	
Q247	Validate a 10-Digit Mobile Number	
Q248	Percentage from Marks Entered as Strings	
Q249	Password Strength Checker	🌟 *Extra*

Programs Q238 – Q249

Q238 Type of a Character using Character Methods

Write a program to accept a character and check whether it is an uppercase letter, a lowercase letter, a digit, a whitespace character or a special character, using the methods of the Character wrapper class: isUpperCase(), isLowerCase(), isDigit(), isWhitespace() and isLetterOrDigit(). (Read a whole line so that a blank space can also be entered; if nothing is entered, display "No character entered".)

Q_238.java

```
import java.util.Scanner;

public class Q_238
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a character: ");
        String line = sc.nextLine();

        if (line.length() == 0)
        {
            System.out.println("No character entered");
        }
        else
        {
            char ch = line.charAt(0);
            if (Character.isUpperCase(ch))
                System.out.println("Uppercase letter");
            else if (Character.isLowerCase(ch))
                System.out.println("Lowercase letter");
            else if (Character.isDigit(ch))
                System.out.println("Digit");
            else if (Character.isWhitespace(ch))
                System.out.println("Whitespace character");
            else if (!Character.isLetterOrDigit(ch))
                System.out.println("Special character");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
line	String	to store the line entered
ch	char	to store the first character

Q239 Count Uppercase, Lowercase, Digits, Spaces and Special Characters

Write a program to accept a sentence and count the number of uppercase letters, lowercase letters, digits, whitespace characters and special characters in it, using the methods of the Character class.

Example: "ICSE 2027 Exam!" → Uppercase: 5, Lowercase: 3, Digits: 4, Spaces: 2, Special: 1

Q_239.java

```
import java.util.Scanner;

public class Q_239
```

```
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine();
        int up = 0, low = 0, dig = 0, sp = 0, spl = 0;

        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (Character.isUpperCase(ch))
                up++;
            else if (Character.isLowerCase(ch))
                low++;
            else if (Character.isDigit(ch))
                dig++;
            else if (Character.isWhitespace(ch))
                sp++;
            else
                spl++;
        }

        System.out.println("Uppercase letters  : " + up);
        System.out.println("Lowercase letters  : " + low);
        System.out.println("Digits             : " + dig);
        System.out.println("Spaces             : " + sp);
        System.out.println("Special characters : " + spl);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
up	int	to count uppercase letters
low	int	to count lowercase letters
dig	int	to count digits
sp	int	to count whitespace characters
spl	int	to count special characters
i	int	to act as the index
ch	char	to store each character

Q240 Change the Case of a Character

> Write a program to accept a character. If it is an uppercase letter, convert it to lowercase; if it is a lowercase letter, convert it to uppercase, using Character.toLowerCase() and Character.toUpperCase(). Otherwise display the message "<ch> is not a letter".

Q_240.java

```
import java.util.Scanner;

public class Q_240
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a character: ");
```

```
        char ch = sc.next().charAt(0);

        if (Character.isUpperCase(ch))
            System.out.println(ch + " becomes " + Character.toLowerCase(ch));
        else if (Character.isLowerCase(ch))
            System.out.println(ch + " becomes " + Character.toUpperCase(ch));
        else
            System.out.println(ch + " is not a letter");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
ch	char	to store the character

Q241 Adding Numbers Entered as Strings

Write a program to accept two numbers as strings (each may be a whole number or a decimal number, and may begin with a minus sign). Display:

```
(a) the result of joining the two strings with +
(b) their sum, using Double.parseDouble()
(c) their sum, using Integer.parseInt(), only if both are whole
    numbers
```

Use a method boolean isNumber(String s) to check that a string is a valid number (digits with at most one decimal point and an optional leading minus sign) before converting it; if not, display "Invalid number".

Q_241.java

```
import java.util.Scanner;

public class Q_241
{
    static boolean isNumber(String s)
    {
        int start = 0, dots = 0, digits = 0;
        if (s.length() > 0 && s.charAt(0) == '-')
            start = 1;
        for (int i = start; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (ch == '.')
                dots++;
            else if (Character.isDigit(ch))
                digits++;
            else
                return false;
        }
        return digits > 0 && dots <= 1;
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter two numbers: ");
        String a = sc.next();
        String b = sc.next();

        if (!isNumber(a) || !isNumber(b))
        {
            System.out.println("Invalid number");
        }
        else
        {
```

```
            System.out.println("(a) a + b as strings : " + a + b);
            double sum = Double.parseDouble(a) + Double.parseDouble(b);
            System.out.println("(b) sum as double    : " + sum);
            if (a.indexOf('.') == -1 && b.indexOf('.') == -1 && a.length() <= 9 && b.length() <= 9)
                System.out.println("(c) sum as int       : " + (Integer.parseInt(a) +
Integer.parseInt(b)));
            else
                System.out.println("(c) not both whole numbers (int range)");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to receive the string to check
start	int	to store the index to start checking
dots	int	to count decimal points
digits	int	to count digits
i	int	to act as the index
ch	char	to store each character
a	String	to store the first number
b	String	to store the second number
sum	double	to store the sum

Q242 Output of the Wrapper Class Methods

Write a program to store the strings "123", "9876543210", "2.5" and "3.75" and the characters '7', 'x', '@', 'a', '\t' (tab), 'Q', 'q' and '5' in variables, and display the results of the following wrapper class methods:

```
Integer.parseInt("123") + 1        Long.parseLong("9876543210")
Float.parseFloat("2.5")            Double.parseDouble("3.75") * 2
Character.isDigit('7')             Character.isLetter('x')
Character.isLetterOrDigit('@')     Character.isLowerCase('a')
Character.isUpperCase('a')         Character.isWhitespace('\t')
Character.toLowerCase('Q')         Character.toUpperCase('q')
Character.toUpperCase('5')
```

Q_242.java

```
public class Q_242
{
    public static void main(String[] args)
    {
        String si = "123", sl = "9876543210", sf = "2.5", sd = "3.75";
        char c1 = '7', c2 = 'x', c3 = '@', c4 = 'a', c5 = '\t', c6 = 'Q', c7 = 'q', c8 = '5';

        System.out.println("Integer.parseInt(\"123\") + 1    = " + (Integer.parseInt(si) + 1));
        System.out.println("Long.parseLong(\"9876543210\")   = " + Long.parseLong(sl));
        System.out.println("Float.parseFloat(\"2.5\")        = " + Float.parseFloat(sf));
        System.out.println("Double.parseDouble(\"3.75\") * 2 = " + (Double.parseDouble(sd) * 2));
        System.out.println("Character.isDigit('7')         = " + Character.isDigit(c1));
        System.out.println("Character.isLetter('x')        = " + Character.isLetter(c2));
        System.out.println("Character.isLetterOrDigit('@') = " + Character.isLetterOrDigit(c3));
        System.out.println("Character.isLowerCase('a')     = " + Character.isLowerCase(c4));
        System.out.println("Character.isUpperCase('a')     = " + Character.isUpperCase(c4));
        System.out.println("Character.isWhitespace('\\t')   = " + Character.isWhitespace(c5));
        System.out.println("Character.toLowerCase('Q')     = " + Character.toLowerCase(c6));
        System.out.println("Character.toUpperCase('q')     = " + Character.toUpperCase(c7));
```

```
        System.out.println("Character.toUpperCase('5')     = " + Character.toUpperCase(c8));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
si	String	to store an integer in text form
sl	String	to store a long in text form
sf	String	to store a float in text form
sd	String	to store a double in text form
c1	char	to store the digit character '7'
c2	char	to store the letter 'x'
c3	char	to store the symbol '@'
c4	char	to store the letter 'a'
c5	char	to store the tab character
c6	char	to store the letter 'Q'
c7	char	to store the letter 'q'
c8	char	to store the digit character '5'

Q243 Autoboxing and Unboxing

Write a program to demonstrate autoboxing and unboxing:

```
(a) store the int value 25 in an Integer object (autoboxing)
(b) store the double value 3.5 in a Double object (autoboxing)
(c) store the char 'J' in a Character object (autoboxing)
(d) assign the Integer object to an int variable (unboxing)
(e) use the Integer and Double objects in an arithmetic expression
    (automatic unboxing) and display the result
(f) pass an int to a method that takes an Integer parameter and
    return an int from it
```

Q_243.java

```
public class Q_243
{
    static int twice(Integer x)
    {
        return x * 2;
    }

    public static void main(String[] args)
    {
        Integer iObj = 25;
        Double dObj = 3.5;
        Character cObj = 'J';
        System.out.println("(a) Integer object : " + iObj);
        System.out.println("(b) Double object  : " + dObj);
        System.out.println("(c) Character obj  : " + cObj);

        int n = iObj;
        System.out.println("(d) unboxed int    : " + n);

        double r = iObj + dObj * 2;
        System.out.println("(e) iObj + dObj * 2 = " + r);
```

```
        int t = twice(10);
        System.out.println("(f) twice(10) = " + t);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
x	Integer	to receive an integer object
iObj	Integer	to store a boxed integer
dObj	Double	to store a boxed double
cObj	Character	to store a boxed character
n	int	to store the unboxed integer
r	double	to store the result of the expression
t	int	to store the value returned

Q244 Sum of the Digits Present in an Alphanumeric String

Write a program to accept an alphanumeric string and find the sum of all the digits present in it, using Character.isDigit(). Also display the count of digits.

Example: "Room 4B, Floor 12" → digits 4, 1, 2 → Sum = 7

Q_244.java

```java
import java.util.Scanner;

public class Q_244
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String s = sc.nextLine();
        int sum = 0, count = 0;

        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (Character.isDigit(ch))
            {
                sum = sum + (ch - '0');
                count++;
            }
        }
        System.out.println("Number of digits = " + count);
        System.out.println("Sum of digits    = " + sum);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the string
sum	int	to store the sum of digits
count	int	to count the digits
i	int	to act as the index

Name of the Variable	Data Type	Purpose / Description
ch	char	to store each character

Q245 Build a Number from the Digits in a String

> Write a program to accept a string such as "Price: Rs 4,550 only" and form a number from all the digits in it, in the order in which they appear (here 4550), using Character.isDigit(). Display the number and the number increased by 10% to show that it is a real number and not a string. Display "No digits found" if there are none, and "Number too large" if there are more than 18 digits.

Q_245.java

```
import java.util.Scanner;

public class Q_245
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String s = sc.nextLine();
        long num = 0;
        int count = 0;

        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (Character.isDigit(ch))
            {
                num = num * 10 + (ch - '0');
                count++;
                if (count > 18)
                    break;
            }
        }

        if (count == 0)
            System.out.println("No digits found");
        else if (count > 18)
            System.out.println("Number too large");
        else
        {
            System.out.println("Number formed = " + num);
            System.out.println("After 10% increase = " + (num * 1.1));
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the string
num	long	to store the number formed
count	int	to count the digits
i	int	to act as the index
ch	char	to store each character

Q246 Remove Characters that are not Letters or Digits

Write a program to accept a sentence and remove all characters that are neither letters nor digits (keep the spaces), using Character.isLetterOrDigit() and Character.isWhitespace(). Display the cleaned sentence and the number of characters removed.

Example: "Hi!! I'm #1 @ ICSE." → "Hi Im 1 ICSE"

Q_246.java

```java
import java.util.Scanner;

public class Q_246
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine();
        String res = "";
        int removed = 0;

        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (Character.isLetterOrDigit(ch) || Character.isWhitespace(ch))
                res = res + ch;
            else
                removed++;
        }
        System.out.println("Cleaned sentence : " + res);
        System.out.println("Characters removed : " + removed);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
res	String	to store the cleaned sentence
removed	int	to count the characters removed
i	int	to act as the index
ch	char	to store each character

Q247 Validate a 10-Digit Mobile Number

Write a program to accept a mobile number as a string and check whether it is a valid Indian mobile number: it must have exactly 10 characters, all of them digits, and must begin with 6, 7, 8 or 9. If valid, convert it to a long using Long.parseLong() and display "Valid mobile number" and the number; otherwise display "Invalid mobile number". Leading and trailing spaces must be ignored.

Q_247.java

```java
import java.util.Scanner;

public class Q_247
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a mobile number: ");
```

```
        String s = sc.nextLine().trim();
        boolean valid = s.length() == 10;

        for (int i = 0; i < s.length() && valid; i++)
        {
            if (!Character.isDigit(s.charAt(i)))
                valid = false;
        }
        if (valid)
        {
            char first = s.charAt(0);
            if (first < '6')
                valid = false;
        }

        if (valid)
        {
            long num = Long.parseLong(s);
            System.out.println("Valid mobile number");
            System.out.println("As a long: " + num);
        }
        else
        {
            System.out.println("Invalid mobile number");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the number entered
valid	boolean	to record whether it is valid
i	int	to act as the index
first	char	to store the first digit
num	long	to store the number as a long

Q248 Percentage from Marks Entered as Strings

> Write a program to accept the marks of five subjects as strings (one word each). Convert each into an integer using Integer.parseInt() after checking with Character.isDigit() that it contains only digits and is from 0 to 100. Display the total and the percentage. If any entry is invalid, display "Invalid marks: <entry>" and stop.

Q_248.java

```
import java.util.Scanner;

public class Q_248
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter marks in five subjects:");
        int total = 0;

        for (int k = 1; k <= 5; k++)
        {
            String m = sc.next();
            boolean ok = m.length() <= 3;
            for (int i = 0; i < m.length() && ok; i++)
            {
```

```
                if (!Character.isDigit(m.charAt(i)))
                    ok = false;
            }
            if (ok && Integer.parseInt(m) > 100)
                ok = false;
            if (!ok)
            {
                System.out.println("Invalid marks: " + m);
                return;
            }
            total = total + Integer.parseInt(m);
        }

        System.out.println("Total      = " + total);
        System.out.println("Percentage = " + (total * 100.0 / 500) + "%");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
total	int	to store the total marks
k	int	to count the subjects
m	String	to store the marks as a string
ok	boolean	to record whether the entry is valid
i	int	to act as the index

Q249 Password Strength Checker

Extra Write a program to accept a password and check its strength using the methods of the Character class. Award one point each for: length of at least 8, at least one uppercase letter, at least one lowercase letter, at least one digit, and at least one special character (not a letter, digit or space). Display "Strong" for 5 points, "Medium" for 3 or 4 points and "Weak" otherwise, along with suggestions for every missing condition. A password containing a space is "Invalid".

Q_249.java

```
import java.util.Scanner;

public class Q_249
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a password: ");
        String p = sc.nextLine();
        boolean up = false, low = false, dig = false, spl = false, space = false;

        for (int i = 0; i < p.length(); i++)
        {
            char ch = p.charAt(i);
            if (Character.isUpperCase(ch))
                up = true;
            else if (Character.isLowerCase(ch))
                low = true;
            else if (Character.isDigit(ch))
                dig = true;
            else if (Character.isWhitespace(ch))
                space = true;
            else
                spl = true;
```

```
        }

        if (space || p.length() == 0)
        {
            System.out.println("Invalid");
        }
        else
        {
            int points = 0;
            if (p.length() >= 8) points++; else System.out.println("- use at least 8 characters");
            if (up) points++; else System.out.println("- add an uppercase letter");
            if (low) points++; else System.out.println("- add a lowercase letter");
            if (dig) points++; else System.out.println("- add a digit");
            if (spl) points++; else System.out.println("- add a special character");

            if (points == 5)
                System.out.println("Strong");
            else if (points >= 3)
                System.out.println("Medium");
            else
                System.out.println("Weak");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
p	String	to store the password
up	boolean	to record an uppercase letter
low	boolean	to record a lowercase letter
dig	boolean	to record a digit
spl	boolean	to record a special character
space	boolean	to record a space
i	int	to act as the index
ch	char	to store each character
points	int	to store the strength points

CHAPTER

13



Encapsulation

Encapsulation and data hiding · Access specifiers and their visibility · Scope of variables: class, instance, argument and local variables

5 programs

Q250 – Q254

Theory

📑 Contents

1. Encapsulation
2. Access specifiers
3. Visibility rules
4. Scope of variables
5. Class variables vs instance variables
6. Identify the variables — solved
7. Predict the output
8. Find the errors
9. One-line facts
10. Test yourself
11. Programs in this chapter

1. Encapsulation

Encapsulation is the **wrapping up of data and the methods that operate on that data into a single unit — the class**, while **restricting direct access** to the data from outside the class.

Data hiding: declaring data members `private` so that they can be read or changed **only through the public methods** of the class. These methods can **validate** the data before changing it.

```
class Student
{
    private int marks;                  // hidden data

    public void setMarks(int m)         // controlled write access (setter / mutator)
    {
        if (m >= 0 && m <= 100)
            marks = m;
    }

    public int getMarks()               // controlled read access (getter / accessor)
    {
        return marks;
    }
}
```

Advantages of encapsulation

1. **Data security** — data cannot be corrupted from outside (e.g. marks cannot be set to −5).
2. **Control** — the class decides how its data is read or changed.
3. **Flexibility and maintainability** — the internal code can change without affecting other classes.
4. **Modularity** — each class is a self-contained unit.

> **NOTE**
>
> **Encapsulation vs abstraction:** encapsulation *binds and hides* the data (implementation level); abstraction *shows only the essential features* (design level). Encapsulation is one way of achieving abstraction.

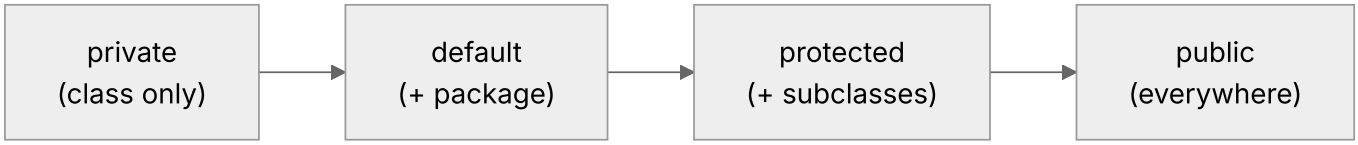
2. Access specifiers

Access specifiers (access modifiers) are keywords that control the **visibility (accessibility)** of a class and its members from other classes.

Access specifier	Keyword	Meaning
private	`private`	Accessible **only within the same class**
default (package-private)	*(no keyword)*	Accessible within the **same package**
protected	`protected`	Accessible within the same package **and** in **subclasses** (even in other packages)
public	`public`	Accessible from **everywhere**

3. Visibility rules

Accessed from →	Same class	Same package (other class)	Subclass in another package	Other class in another package
`private`	✅	❌	❌	❌
default	✅	✅	❌	❌
`protected`	✅	✅	✅	❌
`public`	✅	✅	✅	✅



> **TIP**
>
> Order from **most restrictive to least restrictive**: `private` → default → `protected` → `public`. A common exam line: *"private members are accessible only within the class in which they are declared."*

> **WARNING**
>
> Access specifiers apply to **data members and methods** (and classes), **never to local variables**. `private int x;` inside a method is a compile-time error.

4. Scope of variables

The **scope** of a variable is the **part of the program in which the variable can be used (is visible)**. Its **lifetime** is how long it exists in memory.

Kind of variable	Declared	Scope	Lifetime	Default value
Class variable (static variable)	In the class, with `static`, outside methods	Whole class; accessed as `ClassName.var`	From class loading till the program ends; **one copy shared** by all objects	Yes
Instance variable (data member)	In the class, without `static`, outside methods	Whole class (through an object)	As long as the object exists; **one copy per object**	Yes
Argument variable (parameter)	In the method header	Only inside that method	While the method runs	No (gets the argument's value)

Kind of variable	Declared	Scope	Lifetime	Default value
Local variable	Inside a method or block	From the declaration to the end of that **block**	While the block runs	**No** — must be initialised before use

```
class Shop
{
    static int customers = 0;          // class variable
    String item;                       // instance variable
    double price;                      // instance variable

    double bill(int qty)               // qty → argument variable
    {
        double total = price * qty;    // total → local variable
        for (int i = 1; i <= 1; i++)   // i → local to the for loop only
        {
            customers++;
        }
        return total;
    }
}
```

Shadowing: if a local variable or parameter has the **same name** as an instance variable, it **hides** the instance variable inside that method. Use `this.name` to refer to the instance variable.

5. Class variables vs instance variables

Basis	Class (static) variable	Instance variable
Keyword	`static`	none
Copies	One copy for the whole class	One copy for each object
Accessed using	Class name (`Shop.customers`)	Object (`s1.item`)
Memory allotted	When the class is loaded	When an object is created
Use	Data common to all objects (counter, rate)	Data specific to each object

> **IMPORTANT**
>
> A change in a class variable made through one object is seen by **all** objects, because they share a single copy.

6. Identify the variables — solved

```
public class Employee
{
    private static double bonusRate = 0.1;
    private String name;
    private double salary;

    public double annualPay(int months)
    {
        double pay = salary * months;
        return pay + pay * bonusRate;
```

```
    }
}
```

Variable	Kind	Access
`bonusRate`	Class variable	private
`name`, `salary`	Instance variables	private
`months`	Argument (parameter) variable	—
`pay`	Local variable	—
`annualPay()`	Method	public

7. Predict the output

```
class Counter
{
    static int total = 0;
    int mine = 0;

    void hit()
    {
        total++;
        mine++;
    }

    public static void main(String[] args)
    {
        Counter a = new Counter();
        Counter b = new Counter();
        a.hit(); a.hit(); b.hit();
        System.out.println(a.mine + " " + b.mine + " " + Counter.total);
    }
}
```

Show answer

`mine` is per object (a: 2, b: 1); `total` is shared (3). **Output:** `2 1 3`

8. Find the errors

```
class Box
{
    private double side;
    void set(double side) { side = side; }        // (1)
    double vol() { private double v = side * side * side; return v; }   // (2)
    void show() { int k; System.out.println(k); }  // (3)
}
class Test
{
    public static void main(String[] args)
    {
        Box b = new Box();
        b.side = 5;                                // (4)
```

```
    }
}
```

Show corrections

1. Shadowing — use `this.side = side;` (otherwise the data member is never set — a logical error).
2. Local variables cannot have access specifiers → `double v = side * side * side;`
3. A local variable must be initialised before use → `int k = 0;`
4. `side` is `private` in `Box` → not accessible from `Test`; use a public setter `b.set(5);`

9. One-line facts

- Encapsulation = **data + methods in one unit** with restricted access.
- **Data hiding** is achieved using the `private` access specifier.
- `private` members are accessible **only within their own class**.
- Members with **no access specifier** have **default (package)** access.
- `protected` members are accessible in the same package and in **subclasses**.
- A **static** variable is a class variable — **one copy** shared by all objects.
- **Local variables** have no default value and no access specifier.
- The scope of a loop variable declared in `for` is **only the loop**.

🧠 Test Yourself

Part A — Multiple choice

1. Which access specifier makes a member visible in subclasses of another package but **not** in unrelated classes there?
   (a) private (b) default (c) protected (d) public
2. Which is **not** allowed?
   (a) `private int a;` as a data member (b) `public void f()` (c) `public int x = 5;` inside a method (d) `static int count;` as a data member
3. *Assertion (A):* Data hiding prevents invalid values from being stored in an object.
   Reason (R): Private data can be changed only through methods, which can validate the values.
   (a) Both true; R explains A (b) Both true; R doesn't explain A (c) A true, R false (d) A false, R true
4. Three objects of a class are created. How many copies exist of a `static int n` and of an instance `int m`?
   (a) 1 and 1 (b) 3 and 3 (c) 1 and 3 (d) 3 and 1
5. `java class A { int x = 10; void f(int x) { x = x + 5; System.out.print(x + " " + this.x); } } new A().f(1);` prints (a) 6 10 (b) 15 15 (c) 6 6 (d) 15 10
6. The scope of a parameter is (a) the whole class (b) the whole program (c) the method in which it is declared (d) the block after the method
7. *Assertion (A):* A local variable declared inside a `for` loop cannot be used after the loop. *Reason (R):* The scope of a local variable ends with the block in which it is declared. (a) Both true; R explains A (b) Both true; R doesn't explain A (c) A true, R false (d) A false, R true

Part B — Short answer questions

1. Define encapsulation. How is data hiding achieved in Java?

2. Name the four access levels in order of increasing visibility.
3. Differentiate between a class variable and an instance variable.
4. What is the scope of (i) a local variable (ii) a parameter (iii) an instance variable?
5. What are getter and setter methods? Why are they used?
6. Identify the class, instance, argument and local variables: `class Car { static int made; String model; void sell(double price) { double gst = price * 0.28; } }`
7. What is shadowing? How is it resolved?

✅ Answers — Part A

Q	Ans	Reason
1	(c)	—
2	(c)	Local variables cannot have access specifiers
3	(a)	—
4	(c)	—
5	(a)	The parameter `x` (1 + 5 = 6) shadows the data member (10)
6	(c)	—
7	(a)	—

✅ Answers — Part B

1. Wrapping up data and the methods that act on it into a single unit (class) with restricted access. Data hiding is achieved by declaring data members `private` and giving access only through public methods.
2. private → default → protected → public.
3. A class variable is declared `static`; only one copy exists and it is shared by all objects; accessed with the class name. An instance variable has a separate copy in every object and is accessed through an object.
4. (i) The block in which it is declared (ii) the method in which it is declared (iii) the whole class.
5. Public methods that read (getter/accessor) and change (setter/mutator) private data members; they give controlled access and allow validation.
6. Class variable: `made`; instance variable: `model`; argument: `price`; local: `gst`.
7. When a local variable or parameter has the same name as a data member, it hides the data member inside the method. It is resolved using `this.member`.

💻 Programs in this chapter

No.	Program
Q250	Private Data with Public Getter and Setter Methods
Q251	Access Specifiers: private, default, protected and public
Q252	Kinds of Variables: Class, Instance, Local and Argument
Q253	Counting Objects with a Static (Class) Variable
Q254	BankAccount with a Private, Validated Balance

Programs Q250 – Q254

Q250 Private Data with Public Getter and Setter Methods

Define a class Student (written here as class Q_250) to demonstrate encapsulation:

```
private String name;  private int marks;
public void setName(String n)  : to assign the name; a blank name
                                 is stored as "Unknown"
public void setMarks(int m)    : to assign the marks only if m is
                                 from 0 to 100; otherwise display
                                 "Invalid marks" and keep the old
                                 value
public String getName()        : to return the name
public int getMarks()          : to return the marks
```

In main(), create an object, set the name and marks using values entered by the user, then try to set invalid marks (150), and display the data using the getter methods.

Q_250.java

```java
import java.util.Scanner;

public class Q_250
{
    private String name;
    private int marks;

    public void setName(String n)
    {
        if (n.trim().length() == 0)
            name = "Unknown";
        else
            name = n.trim();
    }

    public void setMarks(int m)
    {
        if (m >= 0 && m <= 100)
            marks = m;
        else
            System.out.println("Invalid marks");
    }

    public String getName()
    {
        return name;
    }

    public int getMarks()
    {
        return marks;
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Q_250 s = new Q_250();
        System.out.print("Enter name: ");
        s.setName(sc.nextLine());
        System.out.print("Enter marks: ");
        s.setMarks(sc.nextInt());
        System.out.println("Trying to set marks to 150...");
        s.setMarks(150);
```

```
        System.out.println("Name  : " + s.getName());
        System.out.println("Marks : " + s.getMarks());
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
name	String	to store the student's name (private)
marks	int	to store the marks (private)
n	String	to receive the name
m	int	to receive the marks
s	Q_250	to refer to the student object

Q251 Access Specifiers: private, default, protected and public

Define a class Account (written here as class Q_251) having one member of each access level:

```
private double balance        - visible only inside this class
String branch (default)       - visible inside the same package
protected String ifsc         - visible in the same package and in
                                subclasses
public String holder          - visible everywhere
```

Write public methods getBalance() and deposit(double amt) to give controlled access to the private member. In main() (which is inside the same class, so every member is accessible here), create an object, assign values to all members, deposit an amount and display all the members. Also display a table showing where each access specifier allows access.

Q_251.java

```
public class Q_251
{
    private double balance;
    String branch;
    protected String ifsc;
    public String holder;

    public double getBalance()
    {
        return balance;
    }

    public void deposit(double amt)
    {
        if (amt > 0)
            balance = balance + amt;
    }

    public static void main(String[] args)
    {
        Q_251 a = new Q_251();
        a.holder = "Ishaan Roy";
        a.branch = "Salt Lake";
        a.ifsc = "ABCD0001234";
        a.balance = 1000;
        a.deposit(500);

        System.out.println("holder (public)     : " + a.holder);
        System.out.println("branch (default)    : " + a.branch);
        System.out.println("ifsc (protected)    : " + a.ifsc);
        System.out.println("balance (private)   : " + a.getBalance());
```

```
        System.out.println();
        System.out.println("Access from     private  default  protected  public");
        System.out.println("Same class        Yes      Yes       Yes       Yes");
        System.out.println("Same package      No       Yes       Yes       Yes");
        System.out.println("Subclass (other   No       No        Yes       Yes");
        System.out.println("  package)");
        System.out.println("Other package     No       No        No        Yes");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
balance	double	to store the balance (private)
branch	String	to store the branch (default access)
ifsc	String	to store the ifsc code (protected)
holder	String	to store the holder's name (public)
amt	double	to receive the amount deposited
a	Q_251	to refer to the account object

Q252 Kinds of Variables: Class, Instance, Local and Argument

Write a class (written here as class Q_252) that shows the four kinds of variables and their scope:

```
static int count          - class variable (one copy shared by all
                            objects)
int id                    - instance variable (one copy per object)
int square(int x)         - x is an argument (parameter) variable
int sq inside square()    - a local variable
a loop variable i         - local to the for loop only
```

Each time an object is created through the method create(), count increases by 1 and the object's id is set to count. In main(), create three objects, display each id and the shared count, and call square() to display the squares of 1 to 3.

Q_252.java

```
public class Q_252
{
    static int count = 0;
    int id;

    static Q_252 create()
    {
        Q_252 ob = new Q_252();
        count++;
        ob.id = count;
        return ob;
    }

    int square(int x)
    {
        int sq = x * x;
        return sq;
    }

    public static void main(String[] args)
    {
        Q_252 a = create();
        Q_252 b = create();
        Q_252 c = create();
        System.out.println("Instance variable id : " + a.id + ", " + b.id + ", " + c.id);
```

```
        System.out.println("Class variable count : " + Q_252.count + " (one copy shared by all
objects)");

        for (int i = 1; i <= 3; i++)
            System.out.println("square(" + i + ") = " + a.square(i));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
count	int	to count objects (class variable)
id	int	to store the object's id (instance variable)
ob	Q_252	to refer to the new object (local)
x	int	to receive the number (argument variable)
sq	int	to store the square (local variable)
a	Q_252	to refer to the first object
b	Q_252	to refer to the second object
c	Q_252	to refer to the third object
i	int	to act as the loop variable (local to loop)

Q253 Counting Objects with a Static (Class) Variable

Define a class Visitor (written here as class Q_253) with a private static int total that counts how many Visitor objects have been created, a private String name, a parameterised constructor Visitor(String n) that stores the name and increases total by 1, an instance method void greet() and a static method int getTotal(). In main(), accept the names of n visitors (n entered by the user) and create an object for each, greeting each visitor; finally display the total number of visitors using getTotal().

Q_253.java

```
import java.util.Scanner;

public class Q_253
{
    private static int total = 0;
    private String name;

    Q_253(String n)
    {
        name = n;
        total++;
    }

    void greet()
    {
        System.out.println("Welcome, " + name + "! You are visitor number " + total);
    }

    static int getTotal()
    {
        return total;
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("How many visitors? ");
```

```
        int n = sc.nextInt();
        sc.nextLine();
        for (int i = 1; i <= n; i++)
        {
            System.out.print("Enter name: ");
            Q_253 v = new Q_253(sc.nextLine());
            v.greet();
        }
        System.out.println("Total visitors = " + getTotal());
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
total	int	to count the visitors (class variable)
name	String	to store the visitor's name
n	int	to receive the name / store the count of visitors
i	int	to act as the loop counter
v	Q_253	to refer to each visitor object

Q254 BankAccount with a Private, Validated Balance

Define a class BankAccount (written here as class Q_254) in which the balance cannot be changed directly from outside the class:

```
private long accNo; private String holder; private double balance
BankAccount(long a, String h)  : constructor, balance starts at 0
public boolean deposit(double amt)  : adds amt if it is positive
                                      and returns true, else false
public boolean withdraw(double amt) : subtracts amt only if it is
                                      positive and not more than
                                      the balance; returns true on
                                      success, else false
public double getBalance()          : returns the balance
public void statement()             : displays the details
```

In main(), create an account and perform the transactions: deposit 5000, withdraw 1200, withdraw 10000, deposit -50, withdraw 0. Display "Success" or "Rejected" for each and the final statement.

Q_254.java

```
public class Q_254
{
    private long accNo;
    private String holder;
    private double balance;

    Q_254(long a, String h)
    {
        accNo = a;
        holder = h;
        balance = 0;
    }

    public boolean deposit(double amt)
    {
        if (amt <= 0)
            return false;
        balance = balance + amt;
        return true;
    }
```

```
    public boolean withdraw(double amt)
    {
        if (amt <= 0 || amt > balance)
            return false;
        balance = balance - amt;
        return true;
    }

    public double getBalance()
    {
        return balance;
    }

    public void statement()
    {
        System.out.println("Account " + accNo + " (" + holder + ") balance: Rs " + balance);
    }

    public static void main(String[] args)
    {
        Q_254 acc = new Q_254(20261001L, "Megha Kar");
        System.out.println("Deposit 5000     : " + (acc.deposit(5000) ? "Success" : "Rejected"));
        System.out.println("Withdraw 1200    : " + (acc.withdraw(1200) ? "Success" : "Rejected"));
        System.out.println("Withdraw 10000   : " + (acc.withdraw(10000) ? "Success" : "Rejected"));
        System.out.println("Deposit -50      : " + (acc.deposit(-50) ? "Success" : "Rejected"));
        System.out.println("Withdraw 0       : " + (acc.withdraw(0) ? "Success" : "Rejected"));
        acc.statement();
        System.out.println("getBalance()     : " + acc.getBalance());
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
accNo	long	to store the account number
holder	String	to store the holder's name
balance	double	to store the balance
a	long	to receive the account number
h	String	to receive the holder's name
amt	double	to receive the amount
acc	Q_254	to refer to the account object

CHAPTER

14



Arrays

Single and double dimensional arrays · Declaration, initialisation and input · `length` · Linear and binary search · Bubble and selection sort · Row, column and diagonal operations

47 programs | Q255 – Q301 | 5 extra

Theory

📑 Contents

1. What is an array?
2. Single dimensional arrays
3. Memory and the `length` property
4. Array as a composite (reference) type
5. Linear search
6. Binary search
7. Bubble sort
8. Selection sort
9. Comparisons: searching and sorting
10. Double dimensional arrays
11. Predict the output
12. Find the errors
13. One-line facts
14. Test yourself
15. Programs in this chapter

1. What is an array?

An **array** is a **collection of a fixed number of elements of the same data type**, stored in **contiguous memory locations** and referred to by a **common name**. Each element is accessed using an **index (subscript)**, which starts from **0**.

Need for arrays: to store and process 50 marks, 50 separate variables would be needed. With an array, one name and a loop are enough:

```
int marks[] = new int[50];
for (int i = 0; i < 50; i++)
    marks[i] = sc.nextInt();
```

Types of arrays

Type	Shape	Example
Single dimensional (1D)	A list (one subscript)	`int a[] = new int[5];`
Double dimensional (2D)	A table of rows and columns (two subscripts)	`int m[][] = new int[3][4];`

2. Single dimensional arrays

2.1 Declaration and creation

```
int a[];            // declaration (a reference; no memory for elements yet)
a = new int[5];     // creation (memory for 5 ints, all set to 0)
int b[] = new int[5];         // both together
double[] c = new double[10];  // brackets may also be written after the type
```

2.2 Initialisation

```
int a[] = {12, 5, 8, 19, 3};              // size is taken as 5 automatically
char vowels[] = {'a', 'e', 'i', 'o', 'u'};
String days[] = {"Mon", "Tue", "Wed"};
```

> **WARNING**
>
> An initialiser list can be used **only at the time of declaration**: `int a[]; a = {1, 2, 3};` is a **syntax error**. Also `int a[5] = {…};` is invalid in Java — the size is never written in the declaration.

2.3 Accepting data and accessing elements

```
for (int i = 0; i < a.length; i++)
    a[i] = sc.nextInt();           // input

for (int i = 0; i < a.length; i++)
    System.out.print(a[i] + " ");  // output
```

Element	a[0]	a[1]	a[2]	a[3]	a[4]
Value	12	5	8	19	3

- First index = **0**; last index = **length − 1**.
- Accessing `a[5]` (or `a[-1]`) causes `ArrayIndexOutOfBoundsException` at run time.

Default values of array elements: `0` (int/long/short/byte), `0.0` (float/double), `'\u0000'` (char), `false` (boolean), `null` (String/objects).

---

3. Memory and the `length` property

`arrayName.length` gives the **number of elements** of an array. It is a **property (data member), not a method** — no brackets.

```
int a[] = new int[8];
System.out.println(a.length);   // 8
```

> **IMPORTANT**
>
> `a.length` (array, no brackets) vs `s.length()` (String, method with brackets) is a favourite exam trap.

Memory occupied = number of elements × size of the data type

Declaration	Calculation	Bytes
`int a[] = new int[10];`	10 × 4	40
`double d[] = new double[5];`	5 × 8	40
`char c[] = new char[20];`	20 × 2	40
`long m[][] = new long[3][4];`	12 × 8	96
`boolean f[] = new boolean[8];`	(JVM dependent)	—

---

4. Array as a composite (reference) type

An array is a **composite / non-primitive** data type: it is made up of many elements, and the array variable holds a **reference** to the memory block (created using `new`).

```
int a[] = {1, 2, 3};
int b[] = a;        // b refers to the SAME array — no copy is made
b[0] = 99;
System.out.println(a[0]);   // 99
```

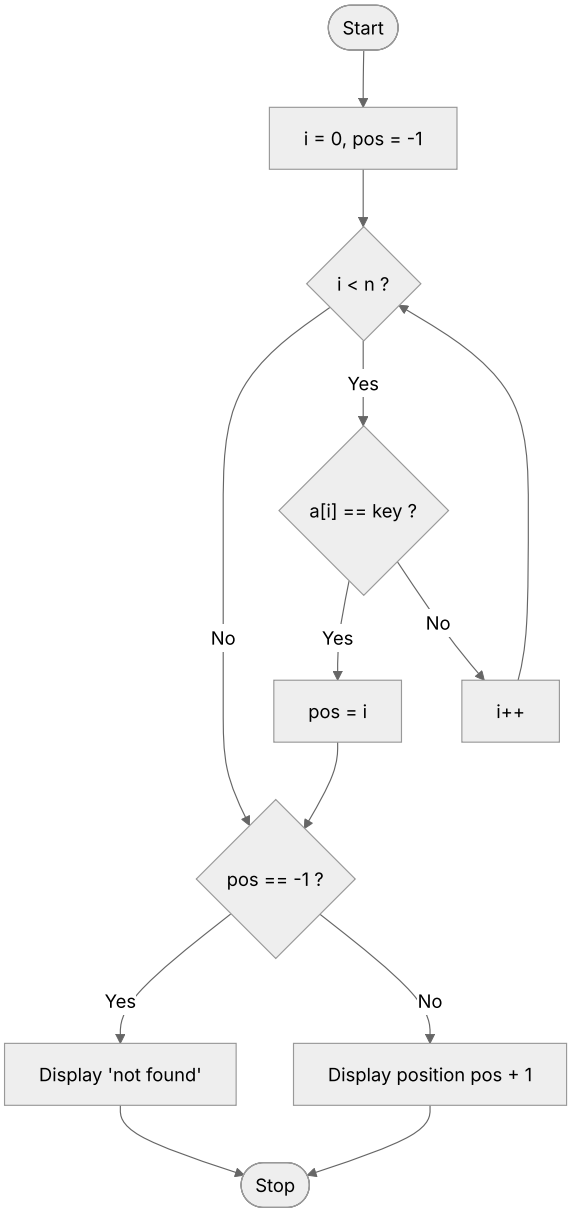
When an array is **passed to a method**, the reference is passed, so changes made to its elements inside the method **are reflected** in the caller (call by reference behaviour).

---

5. Linear search

Linear (sequential) search compares the search key with **each element one by one, from the first element**, until a match is found or the array ends. The array **need not be sorted**.

```
int pos = -1;
for (int i = 0; i < n; i++)
{
    if (a[i] == key)
    {
        pos = i;
        break;
    }
}
```



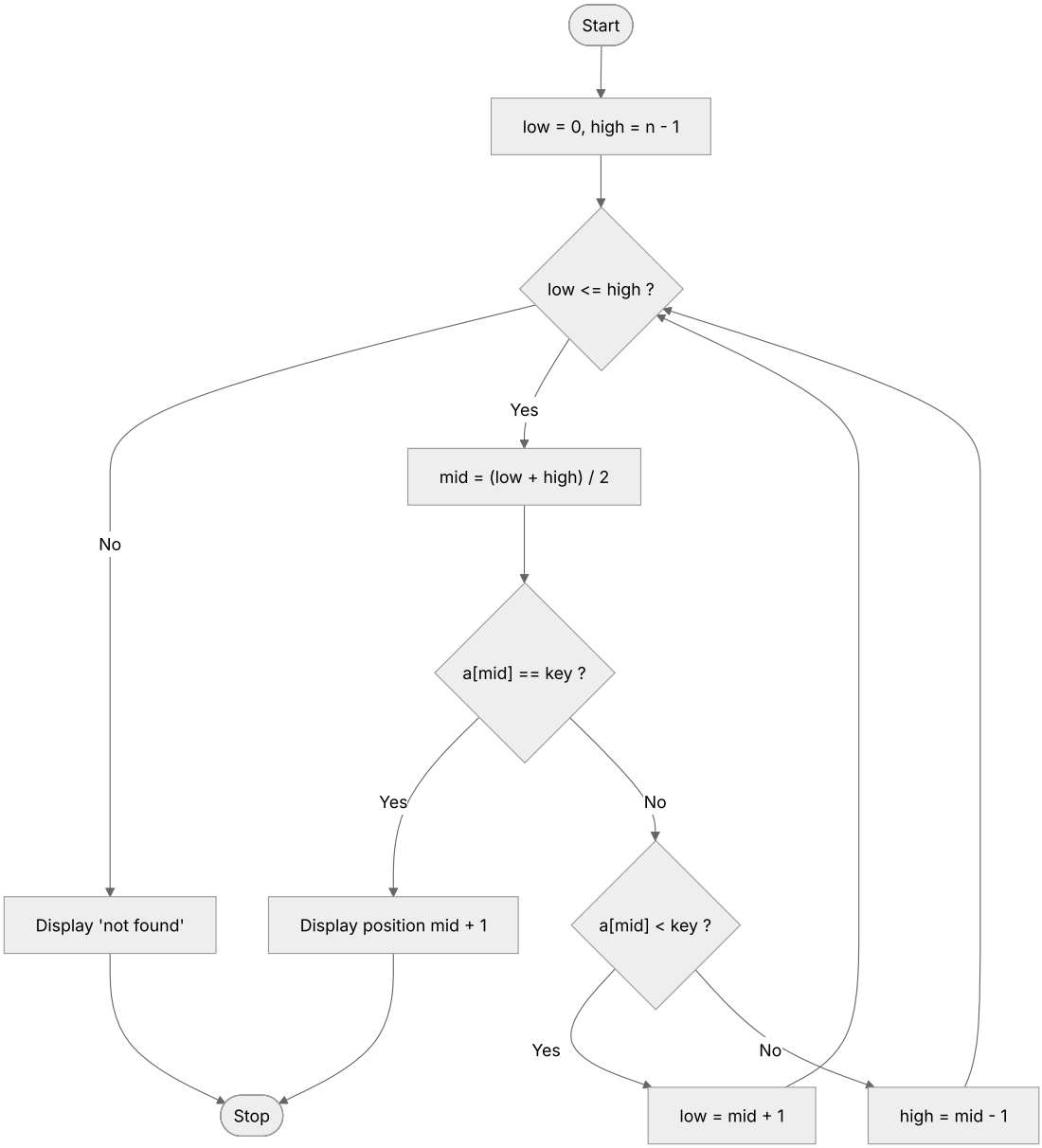
Dry run: `a = {14, -3, 27, 8}`, key = 27 → comparisons with 14 ×, −3 ×, 27 ✓ → found at index 2 (position 3) after **3 comparisons**.

---

6. Binary search

Binary search works only on a **sorted** array. It compares the key with the **middle element**; if they are not equal, it searches **only the half** in which the key can lie, repeatedly halving the search range.

```
int low = 0, high = n - 1, pos = -1;
while (low <= high)
{
    int mid = (low + high) / 2;
    if (a[mid] == key)      { pos = mid; break; }
    else if (a[mid] < key)  low = mid + 1;     // search the right half
    else                    high = mid - 1;    // search the left half
}
```



Dry run: `a = {3, 9, 14, 21, 30, 42, 57, 66}`, key = 57

Step	low	high	mid	a[mid]	Action
1	0	7	3	21	21 < 57 → low = 4
2	4	7	5	42	42 < 57 → low = 6
3	6	7	6	57	**found** at index 6

Dry run (unsuccessful): same array, key = 10

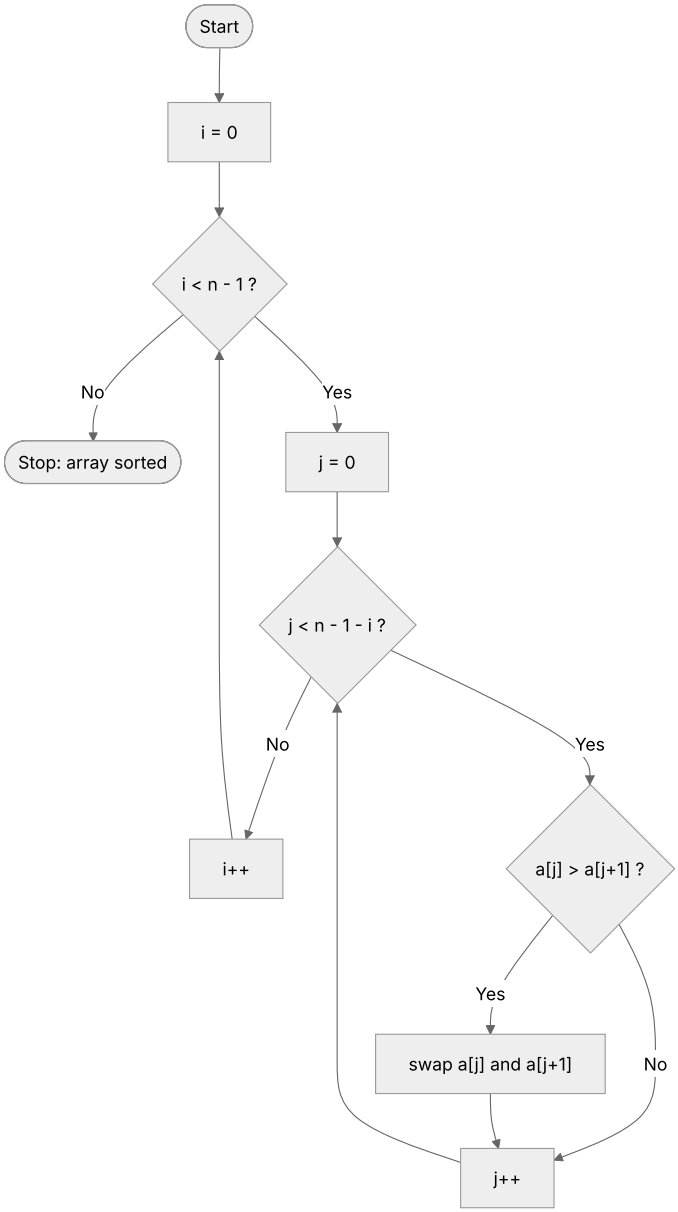
Step	low	high	mid	a[mid]	Action
1	0	7	3	21	21 > 10 → high = 2
2	0	2	1	9	9 < 10 → low = 2
3	2	2	2	14	14 > 10 → high = 1
4	2	1	—	—	low > high → **not found**

> **TIP**
>
> For an array **sorted in descending order**, reverse the two comparisons: if `a[mid] > key` then `low = mid + 1`.

7. Bubble sort

Bubble sort repeatedly compares **adjacent elements** and **swaps them if they are in the wrong order**. After each pass, the **largest** remaining element "bubbles up" to its correct place at the **end** of the unsorted part. For n elements, **n − 1 passes** are needed.

```
for (int i = 0; i < n - 1; i++)              // passes
{
    for (int j = 0; j < n - 1 - i; j++)      // comparisons in pass i
    {
        if (a[j] > a[j + 1])                 // use < for descending order
        {
            int t = a[j];
            a[j] = a[j + 1];
            a[j + 1] = t;
        }
    }
}
```



Pass-by-pass dry run: `{5, 1, 4, 2, 8}` (ascending)

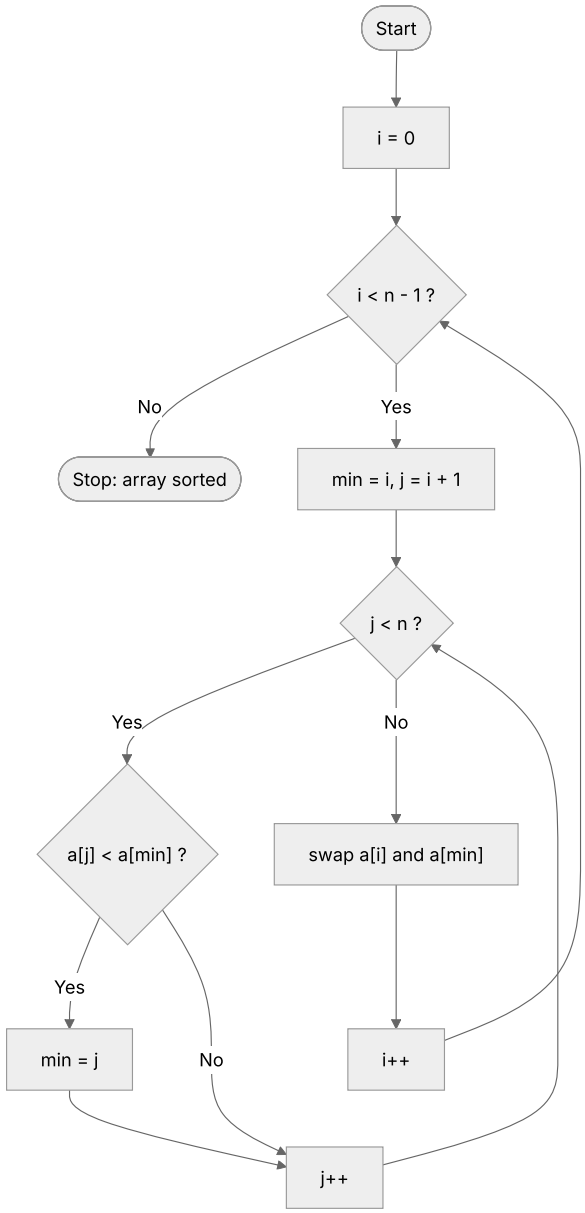
Pass	Array after the pass	Element fixed
Start	5 1 4 2 8	—
1	1 4 2 5 **8**	8
2	1 2 4 **5 8**	5
3	1 2 **4 5 8**	4
4	1 **2 4 5 8**	2 (and 1)

Pass 1 in detail: (5,1) swap → 1 5 4 2 8; (5,4) swap → 1 4 5 2 8; (5,2) swap → 1 4 2 5 8; (5,8) no swap.

8. Selection sort

Selection sort divides the array into a sorted and an unsorted part. In each pass, it **selects the smallest element** of the unsorted part and **swaps it with the first element** of the unsorted part. n − 1 passes are needed, with **at most one swap per pass**.

```java
for (int i = 0; i < n - 1; i++)
{
    int min = i;
    for (int j = i + 1; j < n; j++)
        if (a[j] < a[min])                    // use > (max) for descending order
            min = j;
    int t = a[i];
    a[i] = a[min];
    a[min] = t;
}
```



Pass-by-pass dry run: `{64, 25, 12, 22, 11}`

Pass	Smallest in unsorted part	Swap	Array after the pass
1	11	a[0] ↔ a[4]	**11** 25 12 22 64
2	12	a[1] ↔ a[2]	**11 12** 25 22 64
3	22	a[2] ↔ a[3]	**11 12 22** 25 64
4	25	none needed (a[3] ↔ a[3])	**11 12 22 25 64**

Sorting strings alphabetically: replace `a[j] > a[j + 1]` with `s[j].compareTo(s[j + 1]) > 0` (Chapter 15).

9. Comparisons: searching and sorting

Basis	Linear search	Binary search
Array must be sorted?	No	**Yes**
Method	Compares each element in turn	Compares with the middle element, halves the range
Speed for large arrays	Slow	Fast
Maximum comparisons (n = 1000)	1000	about 10
Works on	Any array	Sorted arrays only

Basis	Bubble sort	Selection sort
Compares	Adjacent elements	Each element with the current minimum
Swaps per pass	Possibly many	At most one
After pass k	Largest k elements fixed at the **end**	Smallest k elements fixed at the **beginning**
Total passes	n − 1	n − 1

10. Double dimensional arrays

A **double dimensional (2D) array** stores data in **rows and columns** (a matrix). An element is accessed as `a[row][column]`.

```
int a[][] = new int[3][4];                    // 3 rows, 4 columns, 12 elements
int b[][] = { {1, 2, 3}, {4, 5, 6} };         // 2 rows, 3 columns

for (int i = 0; i < 3; i++)                   // input row-wise
    for (int j = 0; j < 4; j++)
        a[i][j] = sc.nextInt();

for (int i = 0; i < 3; i++)                   // display in matrix format
{
    for (int j = 0; j < 4; j++)
        System.out.print(a[i][j] + "\t");
    System.out.println();
}
```

- `a.length` → number of **rows**; `a[0].length` → number of **columns**.

Standard 2D operations

Task	Logic
Sum of row i	`for (j…) rs += a[i][j];`
Sum of column j	`for (i…) cs += a[i][j];`
Left (principal) diagonal (square matrix)	elements where `i == j` → `a[i][i]`
Right diagonal	elements where `i + j == n - 1` → `a[i][n - 1 - i]`
Boundary elements	`i == 0 || i == m - 1 || j == 0 || j == n - 1`
Upper triangle (including diagonal)	`j >= i`

Task	Logic
Lower triangle (including diagonal)	`j <= i`
Transpose	`t[j][i] = a[i][j]`

Example: for

```
1 2 3
4 5 6
7 8 9
```

left diagonal = 1 + 5 + 9 = **15**; right diagonal = 3 + 5 + 7 = **15**; row 2 sum = **15**; column 3 sum = 3 + 6 + 9 = **18**.

> **NOTE**
>
> Sorting and searching techniques in the syllabus are applied only to **single dimensional** arrays.

---

11. Predict the output

Example 1

```java
int a[] = {4, 8, 2, 7, 5};
int s = 0;
for (int i = 1; i < a.length; i += 2)
    s += a[i] * a[i - 1];
System.out.println(s + " " + a[a.length - 1] + " " + a[a[2]]);
```

Show answer

i = 1: 8 × 4 = 32; i = 3: 7 × 2 = 14 → s = 46. `a[4]` = 5. `a[a[2]]` = `a[2]` = 2. **Output:** `46 5 2`

Example 2

```java
int x[] = {1, 2, 3, 4, 5};
for (int i = 0; i < x.length - 1; i++)
    x[i + 1] = x[i + 1] + x[i];
System.out.println(x[4]);
```

Show dry run

x becomes {1, 3, 6, 10, 15} (running total). **Output:** `15`

Example 3

```java
char c[] = {'J', 'A', 'V', 'A'};
for (int i = c.length - 1; i >= 0; i -= 2)
    System.out.print(c[i]);
System.out.println(c.length);
```

Show answer

i = 3 → `A`, i = 1 → `A`, then length 4. **Output:** `AA4`

Example 4 — after the **second pass** of bubble sort (ascending), what is the array `{9, 4, 7, 1, 6}`?

Show answer

Pass 1: 4 7 1 6 **9**; Pass 2: 4 1 6 **7 9**. **Answer:** `4 1 6 7 9`

Example 5 — how many comparisons does binary search make to find 30 in `{5, 10, 15, 20, 25, 30, 35}`?

Show answer

mid = 3 (20) → low = 4; mid = 5 (30) → found. **2 comparisons.**

12. Find the errors

```
int a[5] = {1, 2, 3, 4, 5};                        // (1)
int b[] = new int[];                               // (2)
for (int i = 1; i <= b.length(); i++)              // (3)
    b[i] = i;
double d[] = {1.5, 2.5}; int k = d[0];             // (4)
String s[] = new String[3]; System.out.println(s[0].length());   // (5)
```

Show corrections

1. Size cannot be written in the declaration → `int a[] = {1, 2, 3, 4, 5};`
2. Size is required with `new` → `new int[5];`
3. `length` is a property; indices go from 0 to length − 1 → `for (int i = 0; i < b.length; i++)`
4. `double` cannot be stored in `int` without a cast → `int k = (int) d[0];`
5. `s[0]` is `null` (default) → `NullPointerException`; assign strings first.

13. One-line facts

- Array index starts at **0**; the last index is **length − 1**.
- `length` for arrays is a **property** (no brackets).
- Accessing an invalid index throws `ArrayIndexOutOfBoundsException`.
- **Binary search** needs a **sorted** array; linear search does not.
- **Bubble sort** compares **adjacent** elements; **selection sort** selects the **minimum** and swaps once per pass.
- Both sorts need **n − 1 passes** for n elements.
- In a 2D array, `a.length` = rows and `a[0].length` = columns.
- The left diagonal has `i == j`; the right diagonal has `i + j == n − 1`.
- An array is a **composite (reference)** data type.

🧠 Test Yourself

Part A — Multiple choice

1. `int a[] = new int[10];` How many bytes, and what is the index of the last element?
   (a) 40, 10 (b) 40, 9 (c) 10, 9 (d) 20, 9
2. Which needs the data to be sorted? (a) Linear search (b) Binary search (c) Bubble sort (d) Selection sort

3. `int x[] = {3, 6, 9}; int y[] = x; y[1] = 0; System.out.println(x[1] + x.length);` prints
   (a) `63` (b) `3` (c) `9` (d) `03`
4. *Assertion (A):* Selection sort performs at most n − 1 swaps for n elements.
   Reason (R): In each pass, selection sort swaps only once, after finding the minimum.
   (a) Both true; R explains A (b) Both true; R doesn't explain A (c) A true, R false (d) A false, R true
5. In binary search on `{2, 4, 6, 8, 10, 12, 14}` for key 14, the values of `mid` are
   (a) 3, 5, 6 (b) 3, 4, 5, 6 (c) 4, 6 (d) 3, 6
6. After the **first pass** of selection sort (ascending) on `{7, 3, 9, 1, 5}`, the array is
   (a) 1 3 9 7 5 (b) 3 7 1 5 9 (c) 1 7 3 9 5 (d) 3 7 9 1 5
7. For `int m[][] = new int[4][6];`, `m.length` and `m[2].length` are
   (a) 6, 4 (b) 4, 6 (c) 24, 6 (d) 4, 4
8. *Assertion (A):* `int a[] = {1, 2, 3}; a[3] = 4;` causes a compile-time error.
   Reason (R): The highest valid index of `a` is 2.
   (a) Both true; R explains A (b) Both true; R doesn't explain A (c) A false, R true (d) Both false
9. Which element lies on the **right** diagonal of a 5 × 5 matrix? (a) `a[1][1]` (b) `a[1][3]` (c) `a[3][3]` (d) `a[2][1]`
10. The maximum number of comparisons in a linear search of 100 elements is (a) 7 (b) 50 (c) 100 (d) 99

Part B — Short answer questions

1. Define an array. State one advantage of using arrays.
2. Differentiate between linear search and binary search (two points).
3. Differentiate between bubble sort and selection sort (two points).
4. Write statements to (i) declare and initialise an array of 5 vowels (ii) find the number of elements of an array `arr`.
5. Show the array `{20, 15, 30, 5}` after each pass of bubble sort in descending order.
6. What is `ArrayIndexOutOfBoundsException`? Give an example.
7. Write the condition for the elements of (i) the left diagonal (ii) the boundary of an m × n matrix.
8. Calculate the memory required by `double prices[][] = new double[5][4];`

✅ Answers — Part A

Q	Ans	Reason
1	(b)	10 × 4 = 40 bytes; indices 0–9
2	(b)	—
3	(b)	`y` and `x` share the array, so `x[1]` = 0; 0 + 3 = **3**
4	(a)	—
5	(a)	mid = 3 (8 < 14, low = 4), mid = 5 (12 < 14, low = 6), mid = 6 → found
6	(a)	The minimum 1 (at index 3) is swapped with the first element 7 → `1 3 9 7 5`
7	(b)	—
8	(c)	It compiles; the error occurs at **run time** (exception). R is true
9	(b)	1 + 3 = 4 = n − 1
10	(c)	—

✅ Answers — Part B

1. A collection of a fixed number of elements of the same type stored in contiguous locations under one name and accessed by index. Advantage: many values can be processed using one name and a loop.
2. Linear search works on unsorted arrays and checks every element in turn; binary search needs a sorted array and halves the search range each time, so it is much faster for large arrays.
3. Bubble sort compares adjacent elements and may swap many times in a pass; selection sort finds the minimum of the unsorted part and swaps only once per pass.
4. (i) `char v[] = {'a', 'e', 'i', 'o', 'u'};` (ii) `int n = arr.length;`
5. Pass 1: 20 30 15 5 → Pass 2: 30 20 15 5 → Pass 3: 30 20 15 5 (already sorted).
6. A runtime error thrown when an index outside 0 to length − 1 is used, e.g. `int a[] = new int[3]; a[3] = 7;`
7. (i) `i == j` (ii) `i == 0 || i == m - 1 || j == 0 || j == n - 1`
8. 5 × 4 = 20 elements × 8 bytes = **160 bytes**.

💻 Programs in this chapter

No.	Program
Q255	Linear Search
Q256	Binary Search
Q257	Bubble Sort in Ascending Order
Q258	Selection Sort in Ascending Order
Q259	Bubble Sort in Descending Order
Q260	Selection Sort in Descending Order
Q261	Largest, Smallest, Sum and Average of Array Elements
Q262	Sum and Count of Even and Odd Elements
Q263	Count of Positive, Negative and Zero Elements
Q264	Elements Greater than the Average
Q265	Reverse an Array in Place
Q266	Second Largest Element
Q267	Frequency of Each Element
Q268	Remove Duplicate Elements
Q269	Merge Two Arrays
Q270	Split an Array into Even and Odd Arrays
Q271	Swap the First Half with the Second Half
Q272	Rotate an Array Left and Right by One Position
Q273	Insert an Element at a Given Position
Q274	Delete an Element from a Given Position
Q275	Prime Elements of an Array
Q276	Topper from Parallel Arrays of Names and Marks
Q277	Telephone Directory: Search a Name in Parallel Arrays

No.	Program	
Q278	Totals and Grades from Three Subject Arrays	
Q279	Sort Students by Marks Keeping Names Aligned	
Q280	Bubble Sort followed by Binary Search	
Q281	First n Primes and First n Fibonacci Numbers in Arrays	
Q282	Running Total (Cumulative Sum) Array	
Q283	Count of Perfect Squares in an Array	
Q284	Interchange the Largest and Smallest Elements	
Q285	Star Bar Chart of Ratings	🌟 *Extra*
Q286	Dice Roll Frequency Simulation	🌟 *Extra*
Q287	Input and Display a Matrix	
Q288	Row-wise and Column-wise Sums	
Q289	Sum of the Left and Right Diagonals	
Q290	Largest and Smallest Element of a Matrix with Positions	
Q291	Transpose of a Matrix	
Q292	Addition and Subtraction of Two Matrices	
Q293	Symmetric Matrix Check	
Q294	Boundary Elements and their Sum	
Q295	Upper and Lower Triangular Parts of a Matrix	
Q296	Even and Odd Elements of a Matrix	
Q297	Row with the Largest Sum	
Q298	Marks Matrix: Totals and Averages of Students	
Q299	Matrix Multiplication	🌟 *Extra*
Q300	Magic Square Check	🌟 *Extra*
Q301	Tic-Tac-Toe Winner Checker	🌟 *Extra*

Programs Q255 – Q301

Q255 Linear Search

> Write a program to accept n integers in a single dimensional array and a number to be searched. Search for the number using the linear search technique. If it is found, display "Search successful" and its position (counting from 1); otherwise display "Search unsuccessful". Display "Invalid size" if n is less than 1.

Q_255.java

```
import java.util.Scanner;

public class Q_255
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        if (n < 1)
        {
            System.out.println("Invalid size");
            return;
        }
        int a[] = new int[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();
        System.out.print("Enter the number to search: ");
        int key = sc.nextInt();

        int pos = -1;
        for (int i = 0; i < n; i++)
        {
            if (a[i] == key)
            {
                pos = i;
                break;
            }
        }

        if (pos != -1)
            System.out.println("Search successful, found at position " + (pos + 1));
        else
            System.out.println("Search unsuccessful");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array
a	int[]	to store the elements
i	int	to act as the index
key	int	to store the number to search
pos	int	to store the index where found

Q256 Binary Search

Write a program to accept n integers in ascending order in an array and a number to be searched. Search for the number using the binary search technique and display its position (counting from 1) if found; otherwise display "Search unsuccessful". If the numbers entered are not in ascending order, display "Array is not sorted" and stop.

Q_256.java

```java
import java.util.Scanner;

public class Q_256
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        if (n < 1)
        {
            System.out.println("Invalid size");
            return;
        }
        int a[] = new int[n];
        System.out.println("Enter " + n + " integers in ascending order:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        for (int i = 0; i < n - 1; i++)
        {
            if (a[i] > a[i + 1])
            {
                System.out.println("Array is not sorted");
                return;
            }
        }

        System.out.print("Enter the number to search: ");
        int key = sc.nextInt();
        int low = 0, high = n - 1, pos = -1;
        while (low <= high)
        {
            int mid = (low + high) / 2;
            if (a[mid] == key)
            {
                pos = mid;
                break;
            }
            else if (a[mid] < key)
                low = mid + 1;
            else
                high = mid - 1;
        }

        if (pos != -1)
            System.out.println("Search successful, found at position " + (pos + 1));
        else
            System.out.println("Search unsuccessful");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array

Name of the Variable	Data Type	Purpose / Description
a	int[]	to store the sorted elements
i	int	to act as the index
key	int	to store the number to search
low	int	to store the lower index
high	int	to store the upper index
pos	int	to store the index where found
mid	int	to store the middle index

Q257 Bubble Sort in Ascending Order

> Write a program to accept n integers in an array and arrange them in ascending order using the bubble sort technique. Display the array before and after sorting.

Q_257.java

```java
import java.util.Scanner;

public class Q_257
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        int a[] = new int[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        System.out.print("Before sorting: ");
        for (int i = 0; i < n; i++)
            System.out.print(a[i] + " ");
        System.out.println();

        for (int i = 0; i < n - 1; i++)
        {
            for (int j = 0; j < n - 1 - i; j++)
            {
                if (a[j] > a[j + 1])
                {
                    int t = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = t;
                }
            }
        }

        System.out.print("After sorting : ");
        for (int i = 0; i < n; i++)
            System.out.print(a[i] + " ");
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array
a	int[]	to store the elements
i	int	to count the passes / act as index
j	int	to act as the comparison index
t	int	to hold a value while swapping

Q258 Selection Sort in Ascending Order

> Write a program to accept n integers in an array and arrange them in ascending order using the selection sort technique. Display the sorted array.

Q_258.java

```
import java.util.Scanner;

public class Q_258
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        int a[] = new int[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        for (int i = 0; i < n - 1; i++)
        {
            int min = i;
            for (int j = i + 1; j < n; j++)
            {
                if (a[j] < a[min])
                    min = j;
            }
            int t = a[i];
            a[i] = a[min];
            a[min] = t;
        }

        System.out.print("Sorted array: ");
        for (int i = 0; i < n; i++)
            System.out.print(a[i] + " ");
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array
a	int[]	to store the elements
i	int	to mark the position being filled
min	int	to store the index of the smallest
j	int	to act as the search index

Name of the Variable	Data Type	Purpose / Description
t	int	to hold a value while swapping

Q259 Bubble Sort in Descending Order

> Write a program to accept the marks of n students in an array and arrange them in descending order using the bubble sort technique. Display the sorted marks.

Q_259.java

```
import java.util.Scanner;

public class Q_259
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of students: ");
        int n = sc.nextInt();
        double m[] = new double[n];
        System.out.println("Enter the marks:");
        for (int i = 0; i < n; i++)
            m[i] = sc.nextDouble();

        for (int i = 0; i < n - 1; i++)
        {
            for (int j = 0; j < n - 1 - i; j++)
            {
                if (m[j] < m[j + 1])
                {
                    double t = m[j];
                    m[j] = m[j + 1];
                    m[j + 1] = t;
                }
            }
        }

        System.out.print("Marks in descending order: ");
        for (int i = 0; i < n; i++)
            System.out.print(m[i] + " ");
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of students
m	double[]	to store the marks
i	int	to count the passes / act as index
j	int	to act as the comparison index
t	double	to hold a value while swapping

Q260 Selection Sort in Descending Order

> Write a program to accept n integers in an array and arrange them in descending order using the selection sort technique. Display the sorted array.

Q_260.java

```java
import java.util.Scanner;

public class Q_260
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        int a[] = new int[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        for (int i = 0; i < n - 1; i++)
        {
            int max = i;
            for (int j = i + 1; j < n; j++)
            {
                if (a[j] > a[max])
                    max = j;
            }
            int t = a[i];
            a[i] = a[max];
            a[max] = t;
        }

        System.out.print("Sorted array: ");
        for (int i = 0; i < n; i++)
            System.out.print(a[i] + " ");
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array
a	int[]	to store the elements
i	int	to mark the position being filled
max	int	to store the index of the largest
j	int	to act as the search index
t	int	to hold a value while swapping

Q261 Largest, Smallest, Sum and Average of Array Elements

> Write a program to accept n numbers in an array and find the largest element, the smallest element, the sum and the average of all the elements.

Q_261.java

```java
import java.util.Scanner;

public class Q_261
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
```

```
        if (n < 1)
        {
            System.out.println("Invalid size");
            return;
        }
        double a[] = new double[n];
        System.out.println("Enter " + n + " numbers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextDouble();

        double max = a[0], min = a[0], sum = 0;
        for (int i = 0; i < n; i++)
        {
            if (a[i] > max)
                max = a[i];
            if (a[i] < min)
                min = a[i];
            sum = sum + a[i];
        }
        System.out.println("Largest  = " + max);
        System.out.println("Smallest = " + min);
        System.out.println("Sum      = " + sum);
        System.out.println("Average  = " + (sum / n));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array
a	double[]	to store the numbers
i	int	to act as the index
max	double	to store the largest element
min	double	to store the smallest element
sum	double	to store the sum

Q262 Sum and Count of Even and Odd Elements

> Write a program to accept n integers in an array and find the sum and the count of the even elements and of the odd elements separately.

Q_262.java

```
import java.util.Scanner;

public class Q_262
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        int a[] = new int[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        long sumE = 0, sumO = 0;
        int cE = 0, cO = 0;
        for (int i = 0; i < n; i++)
        {
```

```
            if (a[i] % 2 == 0)
            {
                sumE = sumE + a[i];
                cE++;
            }
            else
            {
                sumO = sumO + a[i];
                cO++;
            }
        }
        System.out.println("Even elements: count = " + cE + ", sum = " + sumE);
        System.out.println("Odd elements : count = " + cO + ", sum = " + sumO);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array
a	int[]	to store the elements
i	int	to act as the index
sumE	long	to store the sum of even elements
sumO	long	to store the sum of odd elements
cE	int	to count the even elements
cO	int	to count the odd elements

Q263 Count of Positive, Negative and Zero Elements

> Write a program to accept n numbers in an array and count how many of them are positive, negative and zero.

Q_263.java

```
import java.util.Scanner;

public class Q_263
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        double a[] = new double[n];
        System.out.println("Enter " + n + " numbers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextDouble();

        int pos = 0, neg = 0, zero = 0;
        for (int i = 0; i < n; i++)
        {
            if (a[i] > 0)
                pos++;
            else if (a[i] < 0)
                neg++;
            else
                zero++;
        }
        System.out.println("Positive = " + pos);
        System.out.println("Negative = " + neg);
        System.out.println("Zero     = " + zero);
```

```
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array
a	double[]	to store the numbers
i	int	to act as the index
pos	int	to count positive elements
neg	int	to count negative elements
zero	int	to count zero elements

Q264 Elements Greater than the Average

> Write a program to accept the temperatures of n days in an array, find the average temperature and display the temperatures that are above the average along with their day numbers.

Q_264.java

```java
import java.util.Scanner;

public class Q_264
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of days: ");
        int n = sc.nextInt();
        if (n < 1)
        {
            System.out.println("Invalid size");
            return;
        }
        double t[] = new double[n];
        double sum = 0;
        System.out.println("Enter the temperatures:");
        for (int i = 0; i < n; i++)
        {
            t[i] = sc.nextDouble();
            sum = sum + t[i];
        }

        double avg = sum / n;
        System.out.println("Average temperature = " + avg);
        System.out.println("Days above average:");
        for (int i = 0; i < n; i++)
        {
            if (t[i] > avg)
                System.out.println("Day " + (i + 1) + " : " + t[i]);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of days
t	double[]	to store the temperatures

Name of the Variable	Data Type	Purpose / Description
sum	double	to store the sum
i	int	to act as the index
avg	double	to store the average

Q265 Reverse an Array in Place

Write a program to accept n integers in an array and reverse the array in place (without using another array) by swapping the first and last elements, the second and second-last elements, and so on. Display the reversed array.

Q_265.java

```java
import java.util.Scanner;

public class Q_265
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        int a[] = new int[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        for (int i = 0, j = n - 1; i < j; i++, j--)
        {
            int t = a[i];
            a[i] = a[j];
            a[j] = t;
        }

        System.out.print("Reversed array: ");
        for (int i = 0; i < n; i++)
            System.out.print(a[i] + " ");
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array
a	int[]	to store the elements
i	int	to act as the left index
j	int	to act as the right index
t	int	to hold a value while swapping

Q266 Second Largest Element

Write a program to accept n integers in an array and find the second largest distinct element without sorting the array. If all the elements are equal (or n < 2), display "No second largest element".

Example: 12, 45, 7, 45, 30 → Second largest = 30

Q_266.java

```java
import java.util.Scanner;

public class Q_266
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        int a[] = new int[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        if (n < 2)
        {
            System.out.println("No second largest element");
            return;
        }
        int first = a[0];
        for (int i = 1; i < n; i++)
        {
            if (a[i] > first)
                first = a[i];
        }
        boolean found = false;
        int second = 0;
        for (int i = 0; i < n; i++)
        {
            if (a[i] != first && (!found || a[i] > second))
            {
                second = a[i];
                found = true;
            }
        }

        if (found)
            System.out.println("Second largest = " + second);
        else
            System.out.println("No second largest element");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array
a	int[]	to store the elements
i	int	to act as the index
first	int	to store the largest element
found	boolean	to record whether a second exists
second	int	to store the second largest

Q267 Frequency of Each Element

Write a program to accept n integers in an array and display the frequency (number of occurrences) of each distinct element, in the order of their first appearance.

Example: 4 7 4 9 7 4 → 4 occurs 3 times, 7 occurs 2 times, 9 occurs 1 time

Q_267.java

```java
import java.util.Scanner;

public class Q_267
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        int a[] = new int[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        boolean counted[] = new boolean[n];
        System.out.println("Element\tFrequency");
        for (int i = 0; i < n; i++)
        {
            if (counted[i])
                continue;
            int f = 1;
            for (int j = i + 1; j < n; j++)
            {
                if (a[j] == a[i])
                {
                    f++;
                    counted[j] = true;
                }
            }
            System.out.println(a[i] + "\t" + f);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array
a	int[]	to store the elements
i	int	to act as the outer index
counted	boolean[]	to mark elements already counted
f	int	to store the frequency
j	int	to act as the inner index

Q268 Remove Duplicate Elements

> Write a program to accept n integers in an array and create a new array containing only the first occurrence of each element (removing the duplicates), keeping the original order. Display the new array and the number of duplicates removed.

Q_268.java

```java
import java.util.Scanner;

public class Q_268
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
```

```
        int n = sc.nextInt();
        int a[] = new int[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        int b[] = new int[n];
        int k = 0;
        for (int i = 0; i < n; i++)
        {
            boolean present = false;
            for (int j = 0; j < k; j++)
            {
                if (b[j] == a[i])
                {
                    present = true;
                    break;
                }
            }
            if (!present)
            {
                b[k] = a[i];
                k++;
            }
        }

        System.out.print("Without duplicates: ");
        for (int i = 0; i < k; i++)
            System.out.print(b[i] + " ");
        System.out.println();
        System.out.println("Duplicates removed: " + (n - k));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array
a	int[]	to store the elements
b	int[]	to store the unique elements
k	int	to count the unique elements
i	int	to act as the index of a
present	boolean	to record whether already stored
j	int	to act as the index of b

Q269 Merge Two Arrays

> Write a program to accept two arrays A (of size m) and B (of size n) and merge them into a third array C of size m + n, with the elements of A followed by the elements of B. Display array C.

Q_269.java

```
import java.util.Scanner;

public class Q_269
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the size of array A: ");
```

```
        int m = sc.nextInt();
        int a[] = new int[m];
        System.out.println("Enter the elements of A:");
        for (int i = 0; i < m; i++)
            a[i] = sc.nextInt();
        System.out.print("Enter the size of array B: ");
        int n = sc.nextInt();
        int b[] = new int[n];
        System.out.println("Enter the elements of B:");
        for (int i = 0; i < n; i++)
            b[i] = sc.nextInt();

        int c[] = new int[m + n];
        for (int i = 0; i < m; i++)
            c[i] = a[i];
        for (int i = 0; i < n; i++)
            c[m + i] = b[i];

        System.out.print("Merged array C: ");
        for (int i = 0; i < m + n; i++)
            System.out.print(c[i] + " ");
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
m	int	to store the size of a
a	int[]	to store array a
i	int	to act as the index
n	int	to store the size of b
b	int[]	to store array b
c	int[]	to store the merged array

Q270 Split an Array into Even and Odd Arrays

> Write a program to accept n integers in an array and store the even elements in one array and the odd elements in another array. Display both arrays.

Q_270.java

```
import java.util.Scanner;

public class Q_270
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        int a[] = new int[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        int even[] = new int[n];
        int odd[] = new int[n];
        int e = 0, o = 0;
        for (int i = 0; i < n; i++)
        {
```

```
            if (a[i] % 2 == 0)
                even[e++] = a[i];
            else
                odd[o++] = a[i];
        }

        System.out.print("Even array: ");
        for (int i = 0; i < e; i++)
            System.out.print(even[i] + " ");
        System.out.println();
        System.out.print("Odd array : ");
        for (int i = 0; i < o; i++)
            System.out.print(odd[i] + " ");
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array
a	int[]	to store the elements
i	int	to act as the index
even	int[]	to store the even elements
odd	int[]	to store the odd elements
e	int	to count the even elements
o	int	to count the odd elements

Q271 Swap the First Half with the Second Half

Write a program to accept n integers in an array and swap the first half of the array with the second half. If n is odd, the middle element stays in its place.

Example: 1 2 3 4 5 6 → 4 5 6 1 2 3 1 2 3 4 5 → 4 5 3 1 2

Q_271.java

```
import java.util.Scanner;

public class Q_271
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        int a[] = new int[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        int half = n / 2;
        int start2 = n - half;
        for (int i = 0; i < half; i++)
        {
            int t = a[i];
            a[i] = a[start2 + i];
            a[start2 + i] = t;
        }
```

```
        System.out.print("After swapping halves: ");
        for (int i = 0; i < n; i++)
            System.out.print(a[i] + " ");
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array
a	int[]	to store the elements
i	int	to act as the index
half	int	to store the size of each half
start2	int	to store the start of the second half
t	int	to hold a value while swapping

Q272 Rotate an Array Left and Right by One Position

Write a program to accept n integers in an array and display the array after rotating it (a) one position to the left (the first element moves to the end) and (b) one position to the right (the last element moves to the beginning), starting from the original array each time.

Example: 1 2 3 4 5 → left: 2 3 4 5 1, right: 5 1 2 3 4

Q_272.java

```java
import java.util.Scanner;

public class Q_272
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        if (n < 1)
        {
            System.out.println("Invalid size");
            return;
        }
        int a[] = new int[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        int left[] = new int[n];
        int right[] = new int[n];
        for (int i = 0; i < n; i++)
        {
            left[i] = a[(i + 1) % n];
            right[i] = a[(i - 1 + n) % n];
        }

        System.out.print("Rotated left : ");
        for (int i = 0; i < n; i++)
            System.out.print(left[i] + " ");
        System.out.println();
        System.out.print("Rotated right: ");
        for (int i = 0; i < n; i++)
            System.out.print(right[i] + " ");
```

```
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array
a	int[]	to store the elements
i	int	to act as the index
left	int[]	to store the left-rotated array
right	int[]	to store the right-rotated array

Q273 Insert an Element at a Given Position

> Write a program to accept n integers in an array, a new element and a position (from 1 to n + 1). Insert the element at that position by shifting the later elements one place to the right, and display the new array. Display "Invalid position" for a wrong position.

Q_273.java

```java
import java.util.Scanner;

public class Q_273
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        int a[] = new int[n + 1];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();
        System.out.print("Enter the element to insert: ");
        int x = sc.nextInt();
        System.out.print("Enter the position: ");
        int p = sc.nextInt();

        if (p < 1 || p > n + 1)
        {
            System.out.println("Invalid position");
        }
        else
        {
            for (int i = n; i >= p; i--)
                a[i] = a[i - 1];
            a[p - 1] = x;
            System.out.print("New array: ");
            for (int i = 0; i <= n; i++)
                System.out.print(a[i] + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of elements

Name of the Variable	Data Type	Purpose / Description
a	int[]	to store the elements with one extra place
i	int	to act as the index
x	int	to store the element to insert
p	int	to store the position

Q274 Delete an Element from a Given Position

> Write a program to accept n integers in an array and a position (from 1 to n). Delete the element at that position by shifting the later elements one place to the left, and display the remaining n - 1 elements. Display "Invalid position" for a wrong position.

Q_274.java

```java
import java.util.Scanner;

public class Q_274
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        int a[] = new int[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();
        System.out.print("Enter the position to delete: ");
        int p = sc.nextInt();

        if (p < 1 || p > n)
        {
            System.out.println("Invalid position");
        }
        else
        {
            int del = a[p - 1];
            for (int i = p - 1; i < n - 1; i++)
                a[i] = a[i + 1];
            System.out.println("Deleted element: " + del);
            System.out.print("Remaining array: ");
            for (int i = 0; i < n - 1; i++)
                System.out.print(a[i] + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of elements
a	int[]	to store the elements
i	int	to act as the index
p	int	to store the position
del	int	to store the deleted element

Q275 Prime Elements of an Array

> Write a program to accept n integers in an array and display all the prime numbers present in it along with their count.

Q_275.java

```java
import java.util.Scanner;

public class Q_275
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        int a[] = new int[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        int count = 0;
        System.out.print("Prime elements: ");
        for (int i = 0; i < n; i++)
        {
            boolean prime = a[i] > 1;
            for (int j = 2; j * j <= a[i]; j++)
            {
                if (a[i] % j == 0)
                {
                    prime = false;
                    break;
                }
            }
            if (prime)
            {
                System.out.print(a[i] + " ");
                count++;
            }
        }
        System.out.println();
        System.out.println("Count = " + count);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array
a	int[]	to store the elements
i	int	to act as the index
count	int	to count the primes
prime	boolean	to record whether a[i] is prime
j	int	to act as the trial divisor

Q276 Topper from Parallel Arrays of Names and Marks

> Write a program to accept the names and the total marks of n students in two parallel arrays. Display the name and marks of the student with the highest marks (all students in case of a tie) and the class average.

Q_276.java

```java
import java.util.Scanner;

public class Q_276
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of students: ");
        int n = sc.nextInt();
        if (n < 1)
        {
            System.out.println("Invalid size");
            return;
        }
        String name[] = new String[n];
        double marks[] = new double[n];
        for (int i = 0; i < n; i++)
        {
            sc.nextLine();
            System.out.print("Name of student " + (i + 1) + ": ");
            name[i] = sc.nextLine();
            System.out.print("Marks: ");
            marks[i] = sc.nextDouble();
        }

        double max = marks[0], sum = 0;
        for (int i = 0; i < n; i++)
        {
            if (marks[i] > max)
                max = marks[i];
            sum = sum + marks[i];
        }
        System.out.println("Highest marks: " + max);
        for (int i = 0; i < n; i++)
        {
            if (marks[i] == max)
                System.out.println("Topper: " + name[i]);
        }
        System.out.println("Class average: " + (sum / n));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of students
name	String[]	to store the names
marks	double[]	to store the marks
i	int	to act as the index
max	double	to store the highest marks
sum	double	to store the total of all marks

Q277 Telephone Directory: Search a Name in Parallel Arrays

> Write a program to store the names and telephone numbers of n people in two parallel arrays. Then accept a name and search for it using the linear search technique (ignoring case and extra spaces at the ends). If found, display the name with the telephone number; otherwise display "Name not found".

Q_277.java

```java
import java.util.Scanner;

public class Q_277
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of entries: ");
        int n = sc.nextInt();
        sc.nextLine();
        String name[] = new String[n];
        long tel[] = new long[n];
        for (int i = 0; i < n; i++)
        {
            System.out.print("Name: ");
            name[i] = sc.nextLine().trim();
            System.out.print("Telephone: ");
            tel[i] = sc.nextLong();
            sc.nextLine();
        }

        System.out.print("Enter the name to search: ");
        String key = sc.nextLine().trim();
        int pos = -1;
        for (int i = 0; i < n; i++)
        {
            if (name[i].equalsIgnoreCase(key))
            {
                pos = i;
                break;
            }
        }

        if (pos >= 0)
            System.out.println(name[pos] + " : " + tel[pos]);
        else
            System.out.println("Name not found");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of entries
name	String[]	to store the names
tel	long[]	to store the telephone numbers
i	int	to act as the index
key	String	to store the name to search
pos	int	to store the index where found

Q278 Totals and Grades from Three Subject Arrays

> Write a program to accept the marks of n students in three subjects (English, Maths and Science) in three separate arrays. Calculate each student's average and assign a grade (average >= 80: A, >= 60: B, >= 40: C, below 40: D). Display a table with roll number (1 to n), the three marks, the average and the grade, and finally the class average for each subject.

Q_278.java

```java
import java.util.Scanner;

public class Q_278
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of students: ");
        int n = sc.nextInt();
        if (n < 1)
        {
            System.out.println("Invalid size");
            return;
        }
        int eng[] = new int[n];
        int mat[] = new int[n];
        int sci[] = new int[n];
        for (int i = 0; i < n; i++)
        {
            System.out.print("Marks of student " + (i + 1) + " (Eng Maths Sci): ");
            eng[i] = sc.nextInt();
            mat[i] = sc.nextInt();
            sci[i] = sc.nextInt();
        }

        System.out.println("Roll\tEng\tMaths\tSci\tAvg\tGrade");
        int sE = 0, sM = 0, sS = 0;
        for (int i = 0; i < n; i++)
        {
            double avg = (eng[i] + mat[i] + sci[i]) / 3.0;
            char g;
            if (avg >= 80)
                g = 'A';
            else if (avg >= 60)
                g = 'B';
            else if (avg >= 40)
                g = 'C';
            else
                g = 'D';
            System.out.println((i + 1) + "\t" + eng[i] + "\t" + mat[i] + "\t" + sci[i] + "\t"
                               + Math.round(avg * 10) / 10.0 + "\t" + g);
            sE = sE + eng[i];
            sM = sM + mat[i];
            sS = sS + sci[i];
        }
        System.out.println("Subject averages: Eng " + (double) sE / n + ", Maths " + (double) sM / n +
", Sci " + (double) sS / n);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of students
eng	int[]	to store the english marks
mat	int[]	to store the maths marks
sci	int[]	to store the science marks
i	int	to act as the index
sE	int	to store the total of english marks
sM	int	to store the total of maths marks

Name of the Variable	Data Type	Purpose / Description
sS	int	to store the total of science marks
avg	double	to store a student's average
g	char	to store the grade

Q279 Sort Students by Marks Keeping Names Aligned

> Write a program to accept the names and marks of n students in two parallel arrays and arrange them in descending order of marks using the selection sort technique. Whenever two marks are interchanged, interchange the corresponding names too, so that every name stays with its marks. Display the merit list with ranks.

Q_279.java

```java
import java.util.Scanner;

public class Q_279
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of students: ");
        int n = sc.nextInt();
        String name[] = new String[n];
        int marks[] = new int[n];
        for (int i = 0; i < n; i++)
        {
            sc.nextLine();
            System.out.print("Name: ");
            name[i] = sc.nextLine();
            System.out.print("Marks: ");
            marks[i] = sc.nextInt();
        }

        for (int i = 0; i < n - 1; i++)
        {
            int max = i;
            for (int j = i + 1; j < n; j++)
            {
                if (marks[j] > marks[max])
                    max = j;
            }
            int t = marks[i];
            marks[i] = marks[max];
            marks[max] = t;
            String s = name[i];
            name[i] = name[max];
            name[max] = s;
        }

        System.out.println("Rank\tMarks\tName");
        for (int i = 0; i < n; i++)
            System.out.println((i + 1) + "\t" + marks[i] + "\t" + name[i]);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of students

Name of the Variable	Data Type	Purpose / Description
name	String[]	to store the names
marks	int[]	to store the marks
i	int	to mark the position being filled
max	int	to store the index of the highest marks
j	int	to act as the search index
t	int	to hold marks while swapping
s	String	to hold a name while swapping

Q280 Bubble Sort followed by Binary Search

> Write a program to accept n integers in an array in any order, sort them in ascending order using bubble sort, display the sorted array, and then search for a number entered by the user using binary search. Display the position of the number in the sorted array, or "Search unsuccessful".

Q_280.java

```
import java.util.Scanner;

public class Q_280
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        int a[] = new int[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        for (int i = 0; i < n - 1; i++)
        {
            for (int j = 0; j < n - 1 - i; j++)
            {
                if (a[j] > a[j + 1])
                {
                    int t = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = t;
                }
            }
        }
        System.out.print("Sorted array: ");
        for (int i = 0; i < n; i++)
            System.out.print(a[i] + " ");
        System.out.println();

        System.out.print("Enter the number to search: ");
        int key = sc.nextInt();
        int low = 0, high = n - 1, pos = -1;
        while (low <= high)
        {
            int mid = (low + high) / 2;
            if (a[mid] == key)
            {
                pos = mid;
                break;
            }
```

```
            else if (a[mid] < key)
                low = mid + 1;
            else
                high = mid - 1;
        }
        if (pos >= 0)
            System.out.println("Found at position " + (pos + 1) + " of the sorted array");
        else
            System.out.println("Search unsuccessful");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array
a	int[]	to store the elements
i	int	to count the passes / act as index
j	int	to act as the comparison index
t	int	to hold a value while swapping
key	int	to store the number to search
low	int	to store the lower index
high	int	to store the upper index
pos	int	to store the index where found
mid	int	to store the middle index

Q281 First n Primes and First n Fibonacci Numbers in Arrays

> Write a program to accept n (1 to 40) and store the first n prime numbers in one array and the first n Fibonacci numbers (0, 1, 1, 2, 3, ...) in another array. Display both arrays and the sum of the elements of each.

Q_281.java

```
import java.util.Scanner;

public class Q_281
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();
        if (n < 1 || n > 40)
        {
            System.out.println("Invalid input");
            return;
        }

        int p[] = new int[n];
        int count = 0, num = 2;
        while (count < n)
        {
            boolean prime = true;
            for (int j = 2; j * j <= num; j++)
            {
                if (num % j == 0)
                {
                    prime = false;
```

```
                    break;
                }
            }
            if (prime)
                p[count++] = num;
            num++;
        }

        long f[] = new long[n];
        f[0] = 0;
        if (n > 1)
            f[1] = 1;
        for (int i = 2; i < n; i++)
            f[i] = f[i - 1] + f[i - 2];

        long sp = 0, sf = 0;
        System.out.print("Primes   : ");
        for (int i = 0; i < n; i++)
        {
            System.out.print(p[i] + " ");
            sp = sp + p[i];
        }
        System.out.println("  (sum = " + sp + ")");
        System.out.print("Fibonacci: ");
        for (int i = 0; i < n; i++)
        {
            System.out.print(f[i] + " ");
            sf = sf + f[i];
        }
        System.out.println("  (sum = " + sf + ")");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of terms
p	int[]	to store the prime numbers
count	int	to count primes found
num	int	to store the number being tested
prime	boolean	to record whether num is prime
j	int	to act as the trial divisor
f	long[]	to store the fibonacci numbers
i	int	to act as the index
sp	long	to store the sum of primes
sf	long	to store the sum of fibonacci terms

Q282 Running Total (Cumulative Sum) Array

Write a program to accept the daily sales of n days in an array and create another array in which each element is the running total of sales up to that day (cumulative sum). Display both arrays side by side.

Example: 100 250 75 → 100 350 425

Q_282.java

```
import java.util.Scanner;
```

```java
public class Q_282
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of days: ");
        int n = sc.nextInt();
        double s[] = new double[n];
        double c[] = new double[n];
        System.out.println("Enter the daily sales:");
        for (int i = 0; i < n; i++)
            s[i] = sc.nextDouble();

        for (int i = 0; i < n; i++)
        {
            if (i == 0)
                c[i] = s[i];
            else
                c[i] = c[i - 1] + s[i];
        }

        System.out.println("Day\tSales\tRunning total");
        for (int i = 0; i < n; i++)
            System.out.println((i + 1) + "\t" + s[i] + "\t" + c[i]);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of days
s	double[]	to store the daily sales
c	double[]	to store the running totals
i	int	to act as the index

Q283 Count of Perfect Squares in an Array

> Write a program to accept n integers in an array and display the elements that are perfect squares (0, 1, 4, 9, 16, ...) and their count.

Q_283.java

```java
import java.util.Scanner;

public class Q_283
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        int a[] = new int[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        int count = 0;
        System.out.print("Perfect squares: ");
        for (int i = 0; i < n; i++)
        {
            if (a[i] >= 0)
            {
```

```
                int r = (int) Math.round(Math.sqrt(a[i]));
                if (r * r == a[i])
                {
                    System.out.print(a[i] + " ");
                    count++;
                }
            }
        }
        System.out.println();
        System.out.println("Count = " + count);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array
a	int[]	to store the elements
i	int	to act as the index
count	int	to count the perfect squares
r	int	to store the rounded square root

Q284 Interchange the Largest and Smallest Elements

Write a program to accept n integers in an array, find the positions of the largest and the smallest elements (first occurrence of each), and interchange these two elements. Display the array before and after the interchange.

Q_284.java

```
import java.util.Scanner;

public class Q_284
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        if (n < 1)
        {
            System.out.println("Invalid size");
            return;
        }
        int a[] = new int[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        System.out.print("Before: ");
        for (int i = 0; i < n; i++)
            System.out.print(a[i] + " ");
        System.out.println();

        int maxPos = 0, minPos = 0;
        for (int i = 1; i < n; i++)
        {
            if (a[i] > a[maxPos])
                maxPos = i;
            if (a[i] < a[minPos])
                minPos = i;
        }
        int t = a[maxPos];
```

```
        a[maxPos] = a[minPos];
        a[minPos] = t;

        System.out.print("After : ");
        for (int i = 0; i < n; i++)
            System.out.print(a[i] + " ");
        System.out.println();
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the size of the array
a	int[]	to store the elements
i	int	to act as the index
maxPos	int	to store the index of the largest
minPos	int	to store the index of the smallest
t	int	to hold a value while swapping

Q285 Star Bar Chart of Ratings

Extra A shop collects customer ratings from 1 to 5. Write a program to accept the ratings given by n customers (ignore any rating outside 1 to 5 and count it as invalid), count how many customers gave each rating using an array of counters, and display a horizontal bar chart of stars along with the average rating.

Example output line: 4 | **** (4)

Q_285.java

```
import java.util.Scanner;

public class Q_285
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of customers: ");
        int n = sc.nextInt();
        int freq[] = new int[6];
        int valid = 0, invalid = 0, sum = 0;

        System.out.println("Enter the ratings:");
        for (int i = 0; i < n; i++)
        {
            int r = sc.nextInt();
            if (r >= 1 && r <= 5)
            {
                freq[r]++;
                valid++;
                sum = sum + r;
            }
            else
            {
                invalid++;
            }
        }

        for (int r = 5; r >= 1; r--)
        {
            System.out.print(r + " | ");
```

```
            for (int k = 1; k <= freq[r]; k++)
                System.out.print("*");
            System.out.println(" (" + freq[r] + ")");
        }
        System.out.println("Invalid ratings ignored: " + invalid);
        if (valid > 0)
            System.out.println("Average rating: " + Math.round(sum * 100.0 / valid) / 100.0);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of customers
freq	int[]	to count each rating
valid	int	to count valid ratings
invalid	int	to count invalid ratings
sum	int	to store the total of valid ratings
i	int	to act as the loop counter
r	int	to store a rating / act as row index
k	int	to print the stars

Q286 Dice Roll Frequency Simulation

Extra Write a program to simulate rolling a die n times (n entered by the user, at least 1) using Math.random(). Store how many times each face 1 to 6 appears in an array, and display for each face its count, its percentage and a bar of # symbols (one # for every 2%). With a large n, every face should appear close to 16.67% of the time.

Q_286.java

```
import java.util.Scanner;

public class Q_286
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("How many rolls? ");
        int n = sc.nextInt();
        if (n < 1)
        {
            System.out.println("Invalid input");
            return;
        }
        int count[] = new int[7];
        for (int i = 1; i <= n; i++)
        {
            int face = (int) (Math.random() * 6) + 1;
            count[face]++;
        }

        for (int f = 1; f <= 6; f++)
        {
            double per = count[f] * 100.0 / n;
            System.out.print(f + " : " + count[f] + " (" + Math.round(per * 10) / 10.0 + "%) ");
            for (int k = 1; k <= (int) (per / 2); k++)
                System.out.print("#");
            System.out.println();
```

```
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of rolls
count	int[]	to count each face
i	int	to act as the loop counter
face	int	to store the face rolled
f	int	to act as the face index
per	double	to store the percentage
k	int	to print the bar

Q287 Input and Display a Matrix

> Write a program to accept the number of rows m and columns n of a double dimensional array, accept its elements row-wise and display the array in matrix format. Display "Invalid size" if m or n is less than 1.

Q_287.java

```
import java.util.Scanner;

public class Q_287
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter rows and columns: ");
        int m = sc.nextInt();
        int n = sc.nextInt();
        if (m < 1 || n < 1)
        {
            System.out.println("Invalid size");
            return;
        }
        int a[][] = new int[m][n];
        System.out.println("Enter the elements row-wise:");
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                a[i][j] = sc.nextInt();

        System.out.println("The matrix is:");
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
                System.out.print(a[i][j] + "\t");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
m	int	to store the number of rows
n	int	to store the number of columns

Name of the Variable	Data Type	Purpose / Description
a	int[][]	to store the matrix
i	int	to act as the row index
j	int	to act as the column index

Q288 Row-wise and Column-wise Sums

> Write a program to accept an m x n matrix and display the sum of the elements of each row and of each column, along with the matrix.

Q_288.java

```
import java.util.Scanner;

public class Q_288
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter rows and columns: ");
        int m = sc.nextInt();
        int n = sc.nextInt();
        int a[][] = new int[m][n];
        System.out.println("Enter the elements row-wise:");
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                a[i][j] = sc.nextInt();

        for (int i = 0; i < m; i++)
        {
            int rs = 0;
            for (int j = 0; j < n; j++)
            {
                System.out.print(a[i][j] + "\t");
                rs = rs + a[i][j];
            }
            System.out.println("| Row " + (i + 1) + " sum = " + rs);
        }
        for (int j = 0; j < n; j++)
        {
            int cs = 0;
            for (int i = 0; i < m; i++)
                cs = cs + a[i][j];
            System.out.println("Column " + (j + 1) + " sum = " + cs);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
m	int	to store the number of rows
n	int	to store the number of columns
a	int[][]	to store the matrix
i	int	to act as the row index
j	int	to act as the column index
rs	int	to store a row sum

Name of the Variable	Data Type	Purpose / Description
cs	int	to store a column sum

Q289 Sum of the Left and Right Diagonals

Write a program to accept a square matrix of order n and find the sum of the elements of the left (principal) diagonal and of the right diagonal.

```
Left diagonal  : a[i][i]
Right diagonal : a[i][n - 1 - i]
```

Q_289.java

```java
import java.util.Scanner;

public class Q_289
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the order of the square matrix: ");
        int n = sc.nextInt();
        int a[][] = new int[n][n];
        System.out.println("Enter the elements row-wise:");
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                a[i][j] = sc.nextInt();

        int left = 0, right = 0;
        for (int i = 0; i < n; i++)
        {
            left = left + a[i][i];
            right = right + a[i][n - 1 - i];
        }
        System.out.println("Sum of left diagonal  = " + left);
        System.out.println("Sum of right diagonal = " + right);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the order of the matrix
a	int[][]	to store the matrix
i	int	to act as the row index
j	int	to act as the column index
left	int	to store the left diagonal sum
right	int	to store the right diagonal sum

Q290 Largest and Smallest Element of a Matrix with Positions

Write a program to accept an m x n matrix and find its largest and smallest elements along with their positions (row and column numbers, counting from 1; first occurrence).

Q_290.java

```java
import java.util.Scanner;

public class Q_290
```

```
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter rows and columns: ");
        int m = sc.nextInt();
        int n = sc.nextInt();
        int a[][] = new int[m][n];
        System.out.println("Enter the elements row-wise:");
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                a[i][j] = sc.nextInt();

        int max = a[0][0], min = a[0][0];
        int mr = 0, mc = 0, nr = 0, nc = 0;
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (a[i][j] > max)
                {
                    max = a[i][j];
                    mr = i;
                    mc = j;
                }
                if (a[i][j] < min)
                {
                    min = a[i][j];
                    nr = i;
                    nc = j;
                }
            }
        }
        System.out.println("Largest  = " + max + " at row " + (mr + 1) + ", column " + (mc + 1));
        System.out.println("Smallest = " + min + " at row " + (nr + 1) + ", column " + (nc + 1));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
m	int	to store the number of rows
n	int	to store the number of columns
a	int[][]	to store the matrix
i	int	to act as the row index
j	int	to act as the column index
max	int	to store the largest element
min	int	to store the smallest element
mr	int	to store the row of the largest
mc	int	to store the column of the largest
nr	int	to store the row of the smallest
nc	int	to store the column of the smallest

Q291 Transpose of a Matrix

> Write a program to accept an m x n matrix and find its transpose (an n x m matrix in which the rows become columns). Display both matrices.

Q_291.java

```java
import java.util.Scanner;

public class Q_291
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter rows and columns: ");
        int m = sc.nextInt();
        int n = sc.nextInt();
        int a[][] = new int[m][n];
        System.out.println("Enter the elements row-wise:");
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                a[i][j] = sc.nextInt();

        int t[][] = new int[n][m];
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                t[j][i] = a[i][j];

        System.out.println("Original matrix:");
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
                System.out.print(a[i][j] + "\t");
            System.out.println();
        }
        System.out.println("Transpose:");
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < m; j++)
                System.out.print(t[i][j] + "\t");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
m	int	to store the number of rows
n	int	to store the number of columns
a	int[][]	to store the matrix
i	int	to act as the row index
j	int	to act as the column index
t	int[][]	to store the transpose

Q292 Addition and Subtraction of Two Matrices

> Write a program to accept two matrices A and B of the same order m x n and display their sum (A + B) and difference (A - B).

Q_292.java

```java
import java.util.Scanner;

public class Q_292
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter rows and columns: ");
        int m = sc.nextInt();
        int n = sc.nextInt();
        int a[][] = new int[m][n];
        int b[][] = new int[m][n];
        System.out.println("Enter matrix A:");
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                a[i][j] = sc.nextInt();
        System.out.println("Enter matrix B:");
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                b[i][j] = sc.nextInt();

        System.out.println("A + B:");
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
                System.out.print((a[i][j] + b[i][j]) + "\t");
            System.out.println();
        }
        System.out.println("A - B:");
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
                System.out.print((a[i][j] - b[i][j]) + "\t");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
m	int	to store the number of rows
n	int	to store the number of columns
a	int[][]	to store matrix a
b	int[][]	to store matrix b
i	int	to act as the row index
j	int	to act as the column index

Q293 Symmetric Matrix Check

A square matrix is said to be Symmetric if a[i][j] = a[j][i] for all i and j (it is equal to its own transpose). Write a program to accept a square matrix of order n and check whether it is symmetric. Display "Symmetric Matrix" or "Not a Symmetric Matrix".

Q_293.java

```java
import java.util.Scanner;

public class Q_293
{
```

```
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the order of the square matrix: ");
        int n = sc.nextInt();
        int a[][] = new int[n][n];
        System.out.println("Enter the elements row-wise:");
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                a[i][j] = sc.nextInt();

        boolean sym = true;
        for (int i = 0; i < n && sym; i++)
        {
            for (int j = 0; j < i; j++)
            {
                if (a[i][j] != a[j][i])
                {
                    sym = false;
                    break;
                }
            }
        }
        if (sym)
            System.out.println("Symmetric Matrix");
        else
            System.out.println("Not a Symmetric Matrix");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the order of the matrix
a	int[][]	to store the matrix
i	int	to act as the row index
j	int	to act as the column index
sym	boolean	to record whether it is symmetric

Q294 Boundary Elements and their Sum

> Write a program to accept an m x n matrix and display only its boundary elements (first row, last row, first column and last column) in matrix form, with blanks in place of the inner elements. Also display the sum of the boundary elements.

Q_294.java

```
import java.util.Scanner;

public class Q_294
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter rows and columns: ");
        int m = sc.nextInt();
        int n = sc.nextInt();
        int a[][] = new int[m][n];
        System.out.println("Enter the elements row-wise:");
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
```

```
                a[i][j] = sc.nextInt();

        int sum = 0;
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (i == 0 || i == m - 1 || j == 0 || j == n - 1)
                {
                    System.out.print(a[i][j] + "\t");
                    sum = sum + a[i][j];
                }
                else
                {
                    System.out.print("\t");
                }
            }
            System.out.println();
        }
        System.out.println("Sum of boundary elements = " + sum);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
m	int	to store the number of rows
n	int	to store the number of columns
a	int[][]	to store the matrix
i	int	to act as the row index
j	int	to act as the column index
sum	int	to store the sum of boundary elements

Q295 Upper and Lower Triangular Parts of a Matrix

Write a program to accept a square matrix of order n and display:

(a) the elements on and above the principal diagonal (upper triangular part), with 0 in place of the other elements
(b) the elements on and below the principal diagonal (lower triangular part), with 0 in place of the other elements

Q_295.java

```
import java.util.Scanner;

public class Q_295
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the order of the square matrix: ");
        int n = sc.nextInt();
        int a[][] = new int[n][n];
        System.out.println("Enter the elements row-wise:");
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                a[i][j] = sc.nextInt();

        System.out.println("Upper triangular part:");
        for (int i = 0; i < n; i++)
        {
```

```
            for (int j = 0; j < n; j++)
            {
                if (j >= i)
                    System.out.print(a[i][j] + "\t");
                else
                    System.out.print("0\t");
            }
            System.out.println();
        }
        System.out.println("Lower triangular part:");
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (j <= i)
                    System.out.print(a[i][j] + "\t");
                else
                    System.out.print("0\t");
            }
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the order of the matrix
a	int[][]	to store the matrix
i	int	to act as the row index
j	int	to act as the column index

Q296 Even and Odd Elements of a Matrix

> Write a program to accept an m x n matrix of integers and find the count and the sum of its even elements and of its odd elements.

Q_296.java

```
import java.util.Scanner;

public class Q_296
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter rows and columns: ");
        int m = sc.nextInt();
        int n = sc.nextInt();
        int a[][] = new int[m][n];
        System.out.println("Enter the elements row-wise:");
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                a[i][j] = sc.nextInt();

        int ce = 0, co = 0;
        long se = 0, so = 0;
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (a[i][j] % 2 == 0)
```

```
                {
                    ce++;
                    se = se + a[i][j];
                }
                else
                {
                    co++;
                    so = so + a[i][j];
                }
            }
        }
        System.out.println("Even elements: count = " + ce + ", sum = " + se);
        System.out.println("Odd elements : count = " + co + ", sum = " + so);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
m	int	to store the number of rows
n	int	to store the number of columns
a	int[][]	to store the matrix
i	int	to act as the row index
j	int	to act as the column index
ce	int	to count the even elements
co	int	to count the odd elements
se	long	to store the sum of even elements
so	long	to store the sum of odd elements

Q297 Row with the Largest Sum

> Write a program to accept an m x n matrix and find the row whose elements have the largest sum. Display each row sum and the row number with the largest sum (the first one in case of a tie).

Q_297.java

```
import java.util.Scanner;

public class Q_297
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter rows and columns: ");
        int m = sc.nextInt();
        int n = sc.nextInt();
        int a[][] = new int[m][n];
        System.out.println("Enter the elements row-wise:");
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                a[i][j] = sc.nextInt();

        int best = 0;
        long bestSum = 0;
        for (int i = 0; i < m; i++)
        {
            long rs = 0;
            for (int j = 0; j < n; j++)
                rs = rs + a[i][j];
```

```
            System.out.println("Row " + (i + 1) + " sum = " + rs);
            if (i == 0 || rs > bestSum)
            {
                bestSum = rs;
                best = i;
            }
        }
        System.out.println("Row with the largest sum: Row " + (best + 1) + " (" + bestSum + ")");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
m	int	to store the number of rows
n	int	to store the number of columns
a	int[][]	to store the matrix
i	int	to act as the row index
j	int	to act as the column index
best	int	to store the row with the largest sum
bestSum	long	to store the largest row sum
rs	long	to store a row sum

Q298 Marks Matrix: Totals and Averages of Students

> Write a program to store the marks of n students in 4 subjects in a double dimensional array (one row per student). Display, for each student, the four marks, the total and the average, and display the highest total in the class.

Q_298.java

```
import java.util.Scanner;

public class Q_298
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of students: ");
        int n = sc.nextInt();
        if (n < 1)
        {
            System.out.println("Invalid size");
            return;
        }
        int marks[][] = new int[n][4];
        for (int i = 0; i < n; i++)
        {
            System.out.print("Marks of student " + (i + 1) + " in 4 subjects: ");
            for (int j = 0; j < 4; j++)
                marks[i][j] = sc.nextInt();
        }

        int high = 0;
        System.out.println("Student\tS1\tS2\tS3\tS4\tTotal\tAverage");
        for (int i = 0; i < n; i++)
        {
            int total = 0;
            System.out.print((i + 1) + "\t");
```

```
            for (int j = 0; j < 4; j++)
            {
                System.out.print(marks[i][j] + "\t");
                total = total + marks[i][j];
            }
            System.out.println(total + "\t" + (total / 4.0));
            if (total > high)
                high = total;
        }
        System.out.println("Highest total = " + high);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of students
marks	int[][]	to store the marks
i	int	to act as the student index
j	int	to act as the subject index
high	int	to store the highest total
total	int	to store a student's total

Q299 Matrix Multiplication

> **Extra** Write a program to accept two matrices A (m x n) and B (p x q). If n is equal to p, find their product C = A x B, an m x q matrix with C[i][j] = sum of A[i][k] x B[k][j] for k = 0 to n - 1, and display it. Otherwise display "Multiplication not possible".

Q_299.java

```
import java.util.Scanner;

public class Q_299
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter rows and columns of A: ");
        int m = sc.nextInt();
        int n = sc.nextInt();
        int a[][] = new int[m][n];
        System.out.println("Enter matrix A:");
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                a[i][j] = sc.nextInt();
        System.out.print("Enter rows and columns of B: ");
        int p = sc.nextInt();
        int q = sc.nextInt();
        int b[][] = new int[p][q];
        System.out.println("Enter matrix B:");
        for (int i = 0; i < p; i++)
            for (int j = 0; j < q; j++)
                b[i][j] = sc.nextInt();

        if (n != p)
        {
            System.out.println("Multiplication not possible");
            return;
        }
        long c[][] = new long[m][q];
```

```
        for (int i = 0; i < m; i++)
            for (int j = 0; j < q; j++)
                for (int k = 0; k < n; k++)
                    c[i][j] = c[i][j] + (long) a[i][k] * b[k][j];

        System.out.println("A x B:");
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < q; j++)
                System.out.print(c[i][j] + "\t");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
m	int	to store the rows of a
n	int	to store the columns of a
a	int[][]	to store matrix a
i	int	to act as the row index
j	int	to act as the column index
p	int	to store the rows of b
q	int	to store the columns of b
b	int[][]	to store matrix b
c	long[][]	to store the product
k	int	to act as the summation index

Q300 Magic Square Check

Extra A Magic Square is a square matrix in which the sum of every row, every column and both diagonals is the same. Write a program to accept a square matrix of order n and check whether it is a magic square. Display "Magic Square" (with the magic sum) or "Not a Magic Square".

Example: 2 7 6 9 5 1 4 3 8 (every sum is 15)

Q_300.java

```
import java.util.Scanner;

public class Q_300
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the order of the square matrix: ");
        int n = sc.nextInt();
        int a[][] = new int[n][n];
        System.out.println("Enter the elements row-wise:");
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                a[i][j] = sc.nextInt();

        int target = 0;
        for (int j = 0; j < n; j++)
            target = target + a[0][j];
```

```
        boolean magic = true;
        int d1 = 0, d2 = 0;
        for (int i = 0; i < n; i++)
        {
            int rs = 0, cs = 0;
            for (int j = 0; j < n; j++)
            {
                rs = rs + a[i][j];
                cs = cs + a[j][i];
            }
            if (rs != target || cs != target)
                magic = false;
            d1 = d1 + a[i][i];
            d2 = d2 + a[i][n - 1 - i];
        }
        if (d1 != target || d2 != target)
            magic = false;

        if (magic)
            System.out.println("Magic Square (magic sum = " + target + ")");
        else
            System.out.println("Not a Magic Square");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the order of the matrix
a	int[][]	to store the matrix
i	int	to act as the row index
j	int	to act as the column index
target	int	to store the sum of the first row
magic	boolean	to record whether it is magic
d1	int	to store the left diagonal sum
d2	int	to store the right diagonal sum
rs	int	to store a row sum
cs	int	to store a column sum

Q301 Tic-Tac-Toe Winner Checker

Extra Write a program to accept a finished tic-tac-toe board as a 3 × 3 character array (each cell X, O or - for empty), entered one row at a time as three characters separated by spaces. Display the board and check whether X or O has won (three of the same symbol in a row, a column or a diagonal). Display "X wins", "O wins" or "No winner".

Q_301.java

```
import java.util.Scanner;

public class Q_301
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        char b[][] = new char[3][3];
        System.out.println("Enter the board, row by row (X, O or -):");
```

```
        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 3; j++)
                b[i][j] = Character.toUpperCase(sc.next().charAt(0));

        for (int i = 0; i < 3; i++)
        {
            for (int j = 0; j < 3; j++)
                System.out.print(" " + b[i][j] + " ");
            System.out.println();
        }

        char winner = '-';
        for (int i = 0; i < 3; i++)
        {
            if (b[i][0] != '-' && b[i][0] == b[i][1] && b[i][1] == b[i][2])
                winner = b[i][0];
            if (b[0][i] != '-' && b[0][i] == b[1][i] && b[1][i] == b[2][i])
                winner = b[0][i];
        }
        if (b[1][1] != '-' && ((b[0][0] == b[1][1] && b[1][1] == b[2][2]) || (b[0][2] == b[1][1] && b[1]
[1] == b[2][0])))
            winner = b[1][1];

        if (winner == 'X' || winner == 'O')
            System.out.println(winner + " wins");
        else
            System.out.println("No winner");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
b	char[][]	to store the board
i	int	to act as the row index
j	int	to act as the column index
winner	char	to store the winning symbol

CHAPTER

15



String Handling

The String class · Immutability · String methods with return types · `compareTo()` working · String arrays · Sorting and searching strings · Word extraction

49 programs | Q302 – Q350 | 6 extra

Theory

📑 Contents

1. The String class
2. Creating strings and immutability
3. String methods — complete table
4. `indexOf()` and `substring()` in detail
5. How `compareTo()` works
6. `==` vs `equals()`
7. Joining strings: `+` and `concat()`
8. char vs String
9. Extracting words from a sentence
10. String arrays — sorting and searching
11. Predict the output
12. Find the errors
13. Differentiate between
14. One-line facts
15. Test yourself
16. Programs in this chapter

1. The String class

A **string** is a **sequence of characters** enclosed in double quotes, e.g. `"ICSE 2027"`. In Java, strings are objects of the `String` class of the `java.lang` package. `String` is therefore a **composite (reference)** data type.

Each character of a string has an **index**, starting from **0**:

Index	0	1	2	3	4	5	6	7
`"COMPUTER"`	C	O	M	P	U	T	E	R

`length()` = 8; valid indices are 0 to 7.

2. Creating strings and immutability

```
String s1 = "Java";               // string literal
String s2 = new String("Java");   // using new
String s3 = "";                   // empty string, length 0
```

Immutability: a `String` object **cannot be changed** after it is created. Methods such as `toUpperCase()`, `replace()`, `trim()` and `concat()` **do not modify** the original — they **return a new String**.

```
String s = "java";
s.toUpperCase();          // result discarded — s is still "java"
s = s.toUpperCase();      // s now refers to a new object "JAVA"
```

> **WARNING**
>
> Writing `s.trim();` alone as a statement has **no effect** on `s`. Always assign the result: `s = s.trim();`. This is a very common logical error.

3. String methods — complete table

(Examples use `s = "Computer"`.)

Method	Return type	Purpose	Example	Result
`length()`	`int`	Number of characters	`s.length()`	`8`
`charAt(int i)`	`char`	Character at index i	`s.charAt(3)`	`'p'`
`indexOf(char ch)`	`int`	Index of the first occurrence of ch; −1 if absent	`s.indexOf('u')`	`4`
`indexOf(char ch, int start)`	`int`	First occurrence of ch from index start	`"banana".indexOf('a', 2)`	`3`
`indexOf(String str)`	`int`	Index where str first begins	`s.indexOf("put")`	`3`
`lastIndexOf(char ch)`	`int`	Index of the last occurrence	`"banana".lastIndexOf('a')`	`5`
`substring(int start)`	`String`	From start to the end	`s.substring(3)`	`"puter"`
`substring(int start, int end)`	`String`	From start to **end − 1**	`s.substring(0, 4)`	`"Comp"`
`toUpperCase()`	`String`	All letters in uppercase	`s.toUpperCase()`	`"COMPUTER"`
`toLowerCase()`	`String`	All letters in lowercase	`s.toLowerCase()`	`"computer"`
`trim()`	`String`	Removes leading and trailing spaces (not inner ones)	`" a b ".trim()`	`"a b"`
`concat(String str)`	`String`	Joins str at the end	`s.concat("s")`	`"Computers"`
`replace(char old, char new)`	`String`	Replaces every old char with new	`"ICSE".replace('C', 'K')`	`"IKSE"`
`replace(String old, String new)`	`String`	Replaces every old substring	`"good food".replace("oo", "e")`	`"ged fed"`
`equals(String str)`	`boolean`	Same characters, case sensitive	`s.equals("computer")`	`false`
`equalsIgnoreCase(String str)`	`boolean`	Same characters, ignoring case	`s.equalsIgnoreCase("COMPUTER")`	`true`
`compareTo(String str)`	`int`	Dictionary comparison: 0 / negative / positive	`"BAT".compareTo("BALL")`	`8`
`compareToIgnoreCase(String str)`	`int`	Same, ignoring case	`"a".compareToIgnoreCase("B")`	`-1`
`startsWith(String str)`	`boolean`	Begins with str?	`s.startsWith("Com")`	`true`
`endsWith(String str)`	`boolean`	Ends with str?	`s.endsWith("ER")`	`false`
`String.valueOf(any)`	`String`	Converts a number/char/boolean to a String	`String.valueOf(25) + 5`	`"255"`

> **IMPORTANT**
>
> **Return types are frequently asked:** `charAt()` → `char`; `indexOf()`, `lastIndexOf()`, `length()`, `compareTo()` → `int`; `equals()`, `startsWith()`, `endsWith()` → `boolean`; all the others → `String`.

Runtime errors: `charAt(i)` or `substring(…)` with an index outside the string causes **`StringIndexOutOfBoundsException`**. Calling any method on a `null` String causes **`NullPointerException`**.

---

4. `indexOf()` and `substring()` in detail

```
index:  0 1 2 3 4 5 6 7 8 9 10
s    =  B A N A N A   S P L I T
```

Expression	Result	Reason
`s.indexOf('A')`	`1`	first A
`s.lastIndexOf('A')`	`5`	last A
`s.indexOf('A', 2)`	`3`	first A from index 2
`s.indexOf("NA")`	`2`	—
`s.indexOf('Z')`	`-1`	not present
`s.substring(7)`	`"SPLIT"`	index 7 to the end
`s.substring(2, 5)`	`"NAN"`	indices 2, 3, 4
`s.substring(3, 3)`	`""`	start = end → empty string
`s.substring(0, s.indexOf(' '))`	`"BANANA"`	first word
`s.substring(s.lastIndexOf(' ') + 1)`	`"SPLIT"`	last word

> **TIP**
>
> `substring(start, end)` returns **end – start** characters. Remember: "start included, end excluded".

---

5. How `compareTo()` works

`s1.compareTo(s2)` compares the two strings **character by character** (using Unicode values):

1. At the **first position where the characters differ**, it returns `s1.charAt(i) – s2.charAt(i)`.
2. If one string is a **prefix** of the other, it returns **`s1.length() – s2.length()`**.
3. If they are identical, it returns **0**.

Expression	Working	Result	Meaning
`"APPLE".compareTo("APPLY")`	'E' – 'Y' = 69 – 89	`-20`	APPLE comes first
`"BAT".compareTo("BALL")`	'T' – 'L' = 84 – 76	`8`	BAT comes after BALL
`"CAT".compareTo("CATALOG")`	prefix: 3 – 7	`-4`	CAT comes first
`"apple".compareTo("APPLE")`	'a' – 'A' = 97 – 65	`32`	lowercase comes after uppercase
`"Zebra".compareTo("apple")`	'Z' – 'a' = 90 – 97	`-7`	uppercase Z comes before lowercase a

Expression	Working	Result	Meaning
`"ICSE".compareTo("ICSE")`	identical	`0`	equal

> **NOTE**
>
> Because all uppercase letters (65–90) come before lowercase letters (97–122), `compareTo()` puts `"Zebra"` before `"apple"`. Use **`compareToIgnoreCase()`** for true dictionary order.

In sorting: `if (a[j].compareTo(a[j + 1]) > 0)` → swap (ascending order).

6. `==` vs `equals()`

Basis	`==`	`equals()`
Type	Relational operator	Method of the String class
Compares	**References** (whether both refer to the same object)	**Contents** (characters)
Works on	Primitives and references	Objects
`new String("hi") == new String("hi")`	`false`	—
`new String("hi").equals(new String("hi"))`	—	`true`

> **WARNING**
>
> Never compare strings typed by the user with `==`. Use `equals()` or `equalsIgnoreCase()`.

7. Joining strings: `+` and `concat()`

```
String a = "Hello", b = "World";
String c = a + " " + b;          // "Hello World"
String d = a.concat(" ").concat(b);  // "Hello World"
```

Basis	`+` operator	`concat()` method
Nature	Operator	Method
Operands	Any type (converted to String)	Only a String argument
Example	`"A" + 5` → `"A5"`	`"A".concat(5)` → **error**; use `"A".concat("5")`

Mixing numbers and strings with `+` (left to right): `1 + 2 + "3"` → `"33"`; `"1" + 2 + 3` → `"123"`.

8. char vs String

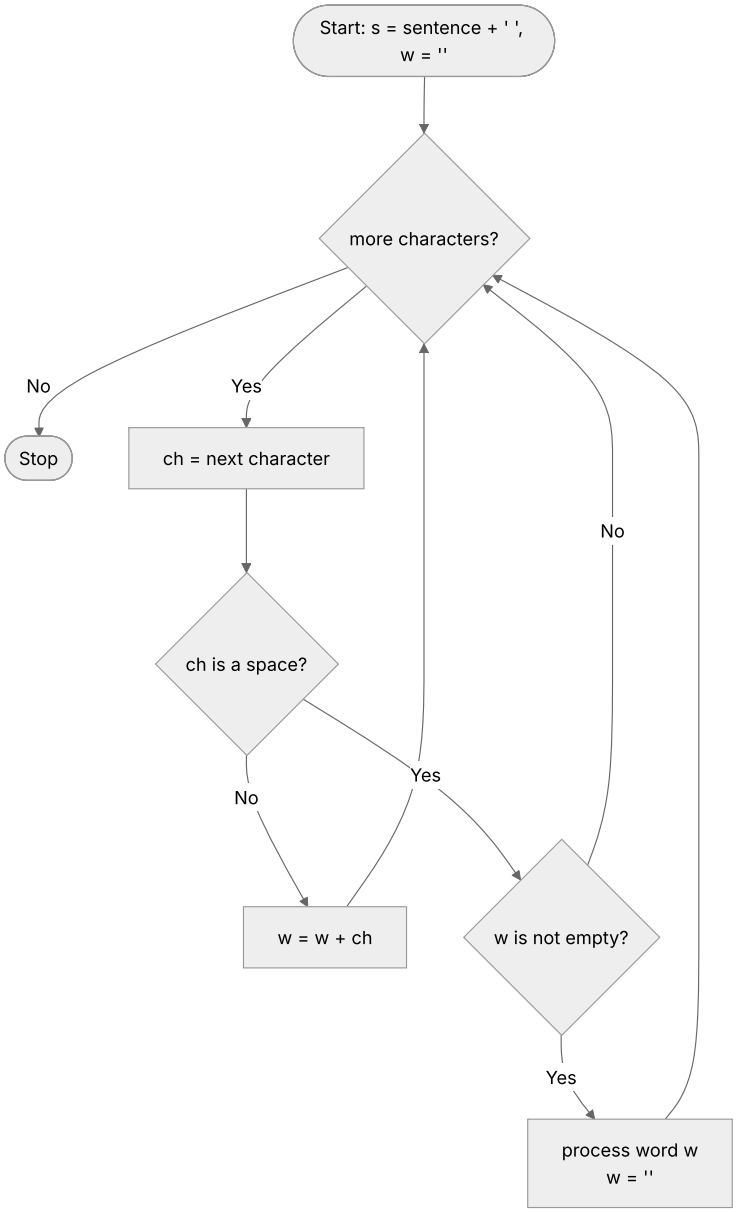
Basis	`char`	`String`
Type	Primitive	Class (reference)

Basis	`char`	`String`
Holds	Exactly one character	Zero or more characters
Quotes	Single `'A'`	Double `"A"`
Size	2 bytes	Depends on the length
Compare	`==`	`equals()`
Convert	`String s = "" + ch;` or `String.valueOf(ch)`	`char ch = s.charAt(0);`

9. Extracting words from a sentence

The standard ICSE technique (safe with extra spaces):

```
String s = sc.nextLine().trim() + " ";     // add a space so the last word is also processed
String w = "";
for (int i = 0; i < s.length(); i++)
{
    char ch = s.charAt(i);
    if (ch != ' ')
        w = w + ch;                        // build the word
    else if (w.length() > 0)               // a word has ended (skip repeated spaces)
    {
        // ... process the word w ...
        w = "";
    }
}
```



Counting words with extra spaces: a word begins wherever a non-space character follows a space (or is at index 0).

10. String arrays — sorting and searching

```
String names[] = {"Rohit", "anjali", "Kabir"};
```

Bubble sort (alphabetical):

```
for (int i = 0; i < n - 1; i++)
    for (int j = 0; j < n - 1 - i; j++)
        if (names[j].compareToIgnoreCase(names[j + 1]) > 0)
        {
            String t = names[j];
            names[j] = names[j + 1];
            names[j + 1] = t;
        }
```

Linear search: `if (names[i].equalsIgnoreCase(key))`. **Binary search on a sorted String array:** compare with `key.compareTo(names[mid])` — negative → search left, positive → search right.

11. Predict the output

```
String s = "Computer Applications";
System.out.println(s.indexOf('t') + s.lastIndexOf('t'));                // (1)
System.out.println(s.substring(9, 12).toUpperCase());                    // (2)
System.out.println(s.charAt(s.length() - 1) + 1);                        // (3)
System.out.println(s.replace('p', 'P').substring(0, 4));                 // (4)
System.out.println("ICSE".compareTo("ISC") + " " + "ab".compareTo("abcd"));  // (5)
System.out.println(s.startsWith("Comp") && s.endsWith("S"));             // (6)
String t = "  hi  ";  t.trim();  System.out.println("[" + t + "]");      // (7)
System.out.println(String.valueOf(10) + 20 + 30);                        // (8)
```

Show answers with working

#	Working	Output
1	first 't' at 5, last 't' at 16 (Applications); 5 + 16	`21`
2	characters 9, 10, 11 = "App"	`APP`
3	last char 's' (115) + 1	`116`
4	"ComPuter APPlications" → first 4	`ComP`
5	'C' − 'S' = 67 − 83 = −16; prefix 2 − 4 = −2	`-16 -2`
6	true && false (case sensitive)	`false`
7	result of `trim()` not assigned	`[ hi ]`
8	"10" + 20 + 30	`102030`

12. Find the errors

```
String s = 'Java';                          // (1)
char c = s.charAt(4);                       // (2)
int n = s.length;                           // (3)
if (s == "JAVA") System.out.println(1);     // (4)
String t = s.toUppercase();                 // (5)
boolean b = s.indexOf('a');                 // (6)
```

Show corrections

1. Strings need double quotes → `"Java"`
2. Valid indices are 0–3 → `s.charAt(3)` (otherwise `StringIndexOutOfBoundsException`)
3. For a String, `length()` is a method → `s.length()`
4. Use `equals()` / `equalsIgnoreCase()` to compare contents → `s.equalsIgnoreCase("JAVA")`
5. Method name is `toUpperCase()` (capital C)
6. `indexOf()` returns an `int` → `int b = s.indexOf('a');`

13. Differentiate between

Pair	Difference
`equals()` vs `==`	Compares contents vs compares references
`equals()` vs `compareTo()`	Returns `boolean` (equal or not) vs `int` (order: negative/0/positive)
`indexOf()` vs `charAt()`	Character → its index vs index → its character
`length` vs `length()`	Array property vs String method
`toUpperCase()` (String) vs `Character.toUpperCase()`	Whole string, returns String vs one character, returns char
`replace(char, char)` vs `substring()`	Replaces characters vs extracts a part
`trim()` vs removing all spaces	Removes only leading and trailing spaces vs all spaces
`next()` vs `nextLine()`	One word vs whole line

14. One-line facts

- `String` belongs to `java.lang` and is **immutable**.
- String indices begin at **0**; the last index is `length() - 1`.
- `substring(a, b)` returns characters from index a to **b – 1**.
- `indexOf()` returns **–1** when the character is not found.
- `compareTo()` returns **0** for equal strings.
- `equals()` is **case sensitive**; `equalsIgnoreCase()` is not.
- `trim()` removes spaces only at the **two ends**.
- `String.valueOf()` converts a number into a **String**.
- `"" + ch` converts a `char` into a String.

🧠 Test Yourself

Part A — Multiple choice

1. `"MISSISSIPPI".indexOf('S') + "MISSISSIPPI".lastIndexOf('S')` is (a) 8 (b) 9 (c) 7 (d) 10
2. What is printed? `String s = "ICSE"; s.toLowerCase(); System.out.println(s.concat("!"));`
   (a) icse! (b) ICSE! (c) ICSE (d) Error
3. *Assertion (A):* `"Apple".compareTo("apple")` returns a negative value.
   Reason (R): The Unicode value of 'A' is smaller than that of 'a'.
   (a) Both true; R explains A (b) Both true; R doesn't explain A (c) A true, R false (d) A false, R true
4. `"Programming".substring(3, 7)` returns (a) "gram" (b) "gramm" (c) "ogra" (d) "ramm"
5. Which returns a `char`? (a) `s.indexOf('a')` (b) `s.charAt(0)` (c) `s.substring(0, 1)` (d) `String.valueOf('a')`
6. `"good".replace('o', "0")` causes (a) "g00d" (b) a compile-time error (c) "g0od" (d) a runtime error
7. `String a = "Hi"; String b = "Hi"; String c = new String("Hi");` Which is **true**?
   (a) `a == c` (b) `a.equals(c)` and `a != c` (c) `b != a` (d) `c.equals("hi")`
8. What is printed? `System.out.println("abc".compareTo("abd") + "xyz".length());`
   (a) 2 (b) −1 (c) −13 (d) 4

9. `" A B ".trim().length()` is (a) 2 (b) 4 (c) 6 (d) 8
10. What is printed? `String s = "JAVA"; for (int i = 0; i < s.length(); i += 2) System.out.print(s.charAt(i) + 1);`
    (a) KW (b) 7587 (c) J1V1 (d) 7586

Part B — Short answer questions

1. What is meant by "strings are immutable"?
2. Write the return types of: `length()`, `charAt()`, `indexOf()`, `equals()`, `substring()`, `compareTo()`.
3. Give the output: `"COMPUTER".substring(2, 5) + "COMPUTER".charAt(7)`
4. Evaluate: (i) `"MANGO".compareTo("MAN")` (ii) `"cat".compareTo("dog")`
5. Write a statement to extract the last word of a sentence `s`.
6. Differentiate between `equals()` and `==` with an example.
7. Write statements to (i) count the occurrences of 'a' in `s` (ii) check whether `s` is a palindrome.
8. What does `trim()` do? Does it remove spaces between words?

✅ Answers — Part A

Q	Ans	Working
1	(a)	M0 I1 S2 S3 I4 S5 S6 I7 P8 P9 I10: first S at 2, last S at 6 → 2 + 6 = 8
2	(b)	`toLowerCase()` result discarded
3	(a)	'A' − 'a' = −32
4	(a)	indices 3–6: g r a m
5	(b)	—
6	(b)	Arguments must both be char or both be String
7	(b)	—
8	(a)	−1 + 3 = 2
9	(b)	`"A B"`
10	(b)	'J' + 1 = 75, 'V' + 1 = 87 → `7587`

✅ Answers — Part B

1. A String object cannot be changed after creation; methods that seem to modify it return a new String instead.
2. `int`, `char`, `int`, `boolean`, `String`, `int`.
3. `"MPU" + 'R'` → `MPUR`
4. (i) prefix: 5 − 3 = `2` (ii) 'c' − 'd' = `-1`
5. `String last = s.trim().substring(s.trim().lastIndexOf(' ') + 1);`
6. `==` compares references and `equals()` compares contents: `new String("A") == new String("A")` is false, but `new String("A").equals(new String("A"))` is true.
7. (i) `int c = 0; for (int i = 0; i < s.length(); i++) if (s.charAt(i) == 'a') c++;` (ii) `String r = ""; for (int i = s.length() - 1; i >= 0; i--) r = r + s.charAt(i); boolean pal = r.equals(s);`
8. It removes the leading and trailing spaces of a string. No — spaces between words remain.

💻 Programs in this chapter

No.	Program	
Q302	Palindrome Word	
Q303	Pig Latin	
Q304	Uppercase Conversion and Double-Letter Sequences	
Q305	Capitalise the First Letter of Each Word	
Q306	Initials of a Name	
Q307	Names in Alphabetical Order using Bubble Sort	
Q308	Names in Alphabetical Order using Selection Sort	
Q309	Linear Search for a Name	
Q310	Count the Words in a Sentence	
Q311	Longest Word in a Sentence	
Q312	Reverse Each Word of a Sentence	
Q313	Reverse the Order of Words	
Q314	Toggle the Case of Each Letter	
Q315	Frequency of Each Letter	
Q316	Letters of a Word in Alphabetical Order	
Q317	Special Word or Palindrome Word	
Q318	Count the Palindrome Words in a Sentence	
Q319	Frequency of a Given Word in a Sentence	
Q320	Replace a Word with Another Word	
Q321	Remove the Vowels from a Sentence	
Q322	Word Pattern: B, BL, BLU, BLUE, BLUEJ	
Q323	Word Pattern: JAVA, AVA, VA, A	
Q324	Overloading joystring()	📝 ICSE 2015
Q325	Overloading count(): Vowels and a Given Character	
Q326	Count the Words Beginning with a Vowel	
Q327	Names Starting or Ending with Given Letters	
Q328	Output of the String Class Methods	
Q329	Output of compareTo(), compareToIgnoreCase() and equals()	
Q330	Count Pairs of Consecutive Letters	
Q331	Word with the Maximum Number of Vowels	
Q332	Swap the First and Last Letters of Each Word	
Q333	Display Each Word with its Length	
Q334	Username and Domain from an Email Address	

No.	Program	
Q335	Number in Words, Digit by Digit	
Q336	Characters at Alternate Positions	
Q337	Encode a Sentence by Shifting Letters by 2	
Q338	Remove Extra Spaces from a Sentence	
Q339	Words that Begin and End with a Vowel	
Q340	Words of a Sentence in Alphabetical Order	
Q341	First Non-Repeating Character	
Q342	Anagram Check	
Q343	Pangram Check	🌟 *Extra*
Q344	Login Check using concat() and equalsIgnoreCase()	
Q345	Words that Come First and Last Alphabetically	
Q346	Number to Roman Numerals	🌟 *Extra*
Q347	Caesar Cipher: Encrypt and Decrypt	🌟 *Extra*
Q348	Word Scramble Game	🌟 *Extra*
Q349	Run-Length Encoding	🌟 *Extra*
Q350	24-Hour Time to 12-Hour Time	🌟 *Extra*

Programs Q302 – Q350

Q302 Palindrome Word

Write a program to accept a word and check whether it is a Palindrome word or not, ignoring the case of the letters. A Palindrome word reads the same from left to right and from right to left.

Example: MADAM, Level, NOON are Palindrome words.

Q_302.java

```
import java.util.Scanner;

public class Q_302
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a word: ");
        String w = sc.next();
        String rev = "";

        for (int i = w.length() - 1; i >= 0; i--)
            rev = rev + w.charAt(i);

        if (rev.equalsIgnoreCase(w))
            System.out.println("Palindrome Word");
        else
            System.out.println("Not a Palindrome Word");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
w	String	to store the word
rev	String	to store the reversed word
i	int	to act as the index

Q303 Pig Latin

Write a program to accept a word and convert it into Pig Latin. To translate a word into Pig Latin, convert it to uppercase, move all the letters before the first vowel to the end of the word, and add "AY" at the end. If the word has no vowel, simply add "AY".

Example: TROUBLE → OUBLETRAY, KING → INGKAY, apple → APPLEAY, RHYTHM → RHYTHMAY

Q_303.java

```
import java.util.Scanner;

public class Q_303
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a word: ");
        String w = sc.next().toUpperCase();
        int pos = -1;
```

```
        for (int i = 0; i < w.length(); i++)
        {
            char ch = w.charAt(i);
            if (ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U')
            {
                pos = i;
                break;
            }
        }

        String pig;
        if (pos == -1)
            pig = w + "AY";
        else
            pig = w.substring(pos) + w.substring(0, pos) + "AY";
        System.out.println("Pig Latin: " + pig);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
w	String	to store the word in uppercase
pos	int	to store the index of the first vowel
i	int	to act as the index
ch	char	to store each character
pig	String	to store the pig latin word

Q304 Uppercase Conversion and Double-Letter Sequences

Write a program to accept a sentence, convert it into uppercase and count the number of double-letter sequences in it, i.e. the number of times the same letter appears twice in succession.

Example: "She was feeding the little rabbit with an apple" Uppercase: SHE WAS FEEDING THE LITTLE RABBIT WITH AN APPLE Double-letter sequences: 4 (EE, TT, BB, PP)

Q_304.java

```
import java.util.Scanner;

public class Q_304
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine().toUpperCase();
        int count = 0;

        for (int i = 0; i < s.length() - 1; i++)
        {
            char ch = s.charAt(i);
            if (Character.isLetter(ch) && ch == s.charAt(i + 1))
                count++;
        }
        System.out.println(s);
        System.out.println("Double-letter sequences = " + count);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence in uppercase
count	int	to count the double letters
i	int	to act as the index
ch	char	to store each character

Q305 Capitalise the First Letter of Each Word

Write a program to accept a sentence and convert the first letter of each word into uppercase, leaving the other letters unchanged. The sentence may contain extra spaces; the output must have exactly one space between words and no spaces at the ends.

Example: " we are in cyber world " → "We Are In Cyber World"

Q_305.java

```
import java.util.Scanner;

public class Q_305
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine().trim() + " ";
        String res = "", w = "";

        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (ch != ' ')
            {
                w = w + ch;
            }
            else if (w.length() > 0)
            {
                w = Character.toUpperCase(w.charAt(0)) + w.substring(1);
                res = res + w + " ";
                w = "";
            }
        }
        System.out.println(res.trim());
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
res	String	to store the result
w	String	to build each word
i	int	to act as the index
ch	char	to store each character

Q306 Initials of a Name

Write a program to accept a person's full name and display it with the initials of all the names except the last name (surname), which is written in full.

Example: Mohandas Karamchand Gandhi → M.K. Gandhi Subhas Chandra Bose → S.C. Bose Tagore → Tagore

Extra spaces in the input must be ignored, and the initials must be in uppercase.

Q_306.java

```java
import java.util.Scanner;

public class Q_306
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the full name: ");
        String s = sc.nextLine().trim();
        int last = s.lastIndexOf(' ');
        String res = "";

        if (last == -1)
        {
            res = s;
        }
        else
        {
            String first = " " + s.substring(0, last);
            for (int i = 0; i < first.length() - 1; i++)
            {
                if (first.charAt(i) == ' ' && first.charAt(i + 1) != ' ')
                    res = res + Character.toUpperCase(first.charAt(i + 1)) + ".";
            }
            res = res + " " + s.substring(last + 1);
        }
        System.out.println(res);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the full name
last	int	to store the index of the last space
res	String	to store the abbreviated name
first	String	to store the names before the surname
i	int	to act as the index

Q307 Names in Alphabetical Order using Bubble Sort

Write a program to accept the names of n students in a String array and arrange them in alphabetical order using the bubble sort technique (ignore the case of letters while comparing). Display the sorted list.

Q_307.java

```java
import java.util.Scanner;

public class Q_307
{
    public static void main(String[] args)
```

```
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of names: ");
        int n = sc.nextInt();
        sc.nextLine();
        String names[] = new String[n];
        for (int i = 0; i < n; i++)
        {
            System.out.print("Name " + (i + 1) + ": ");
            names[i] = sc.nextLine().trim();
        }

        for (int i = 0; i < n - 1; i++)
        {
            for (int j = 0; j < n - 1 - i; j++)
            {
                if (names[j].compareToIgnoreCase(names[j + 1]) > 0)
                {
                    String t = names[j];
                    names[j] = names[j + 1];
                    names[j + 1] = t;
                }
            }
        }

        System.out.println("Names in alphabetical order:");
        for (int i = 0; i < n; i++)
            System.out.println(names[i]);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of names
names	String[]	to store the names
i	int	to count the passes / act as index
j	int	to act as the comparison index
t	String	to hold a name while swapping

Q308 Names in Alphabetical Order using Selection Sort

> Write a program to accept the names of n cities in a String array and arrange them in alphabetical order using the selection sort technique (ignoring case). Display the sorted list.

Q_308.java

```
import java.util.Scanner;

public class Q_308
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of cities: ");
        int n = sc.nextInt();
        sc.nextLine();
        String c[] = new String[n];
        for (int i = 0; i < n; i++)
        {
            System.out.print("City " + (i + 1) + ": ");
            c[i] = sc.nextLine().trim();
```

```
        }

        for (int i = 0; i < n - 1; i++)
        {
            int min = i;
            for (int j = i + 1; j < n; j++)
            {
                if (c[j].compareToIgnoreCase(c[min]) < 0)
                    min = j;
            }
            String t = c[i];
            c[i] = c[min];
            c[min] = t;
        }

        System.out.println("Cities in alphabetical order:");
        for (int i = 0; i < n; i++)
            System.out.println(c[i]);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of cities
c	String[]	to store the city names
i	int	to mark the position being filled
min	int	to store the index of the smallest name
j	int	to act as the search index
t	String	to hold a name while swapping

Q309 Linear Search for a Name

> Write a program to accept the names of n students in a String array and a name to be searched. Search for the name using linear search (ignoring case and extra spaces at the ends) and display "Name found at position <p>" or "Name not found".

Q_309.java

```
import java.util.Scanner;

public class Q_309
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of names: ");
        int n = sc.nextInt();
        sc.nextLine();
        String names[] = new String[n];
        for (int i = 0; i < n; i++)
        {
            System.out.print("Name " + (i + 1) + ": ");
            names[i] = sc.nextLine().trim();
        }
        System.out.print("Enter the name to search: ");
        String key = sc.nextLine().trim();

        int pos = -1;
        for (int i = 0; i < n; i++)
        {
```

```
            if (names[i].equalsIgnoreCase(key))
            {
                pos = i;
                break;
            }
        }
        if (pos != -1)
            System.out.println("Name found at position " + (pos + 1));
        else
            System.out.println("Name not found");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of names
names	String[]	to store the names
i	int	to act as the index
key	String	to store the name to search
pos	int	to store the index where found

Q310 Count the Words in a Sentence

Write a program to accept a sentence and count the number of words in it. The sentence may have extra spaces at the beginning, at the end and between words; these must not be counted as words. Display 0 for a blank sentence.

Example: " Java is fun " → Number of words = 3

Q_310.java

```
import java.util.Scanner;

public class Q_310
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine();
        int count = 0;

        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            boolean prevSpace = (i == 0) || s.charAt(i - 1) == ' ';
            if (ch != ' ' && prevSpace)
                count++;
        }
        System.out.println("Number of words = " + count);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
count	int	to count the words

Name of the Variable	Data Type	Purpose / Description
i	int	to act as the index
ch	char	to store each character
prevSpace	boolean	to check if the previous character is a space

Q311 Longest Word in a Sentence

Write a program to accept a sentence and display the longest word in it along with its length. If two or more words have the same greatest length, display the first one. Extra spaces must be handled correctly.

Example: "Tata Steel is in Jamshedpur" → Longest word: Jamshedpur (10 letters)

Q_311.java

```
import java.util.Scanner;

public class Q_311
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine().trim() + " ";
        String w = "", longest = "";

        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (ch != ' ')
            {
                w = w + ch;
            }
            else
            {
                if (w.length() > longest.length())
                    longest = w;
                w = "";
            }
        }
        if (longest.length() == 0)
            System.out.println("No words entered");
        else
            System.out.println("Longest word: " + longest + " (" + longest.length() + " letters)");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
w	String	to build each word
longest	String	to store the longest word
i	int	to act as the index
ch	char	to store each character

Q312 Reverse Each Word of a Sentence

Write a program to accept a sentence and reverse each word of it, keeping the words in their original order. Extra spaces in the input must be removed in the output.

Example: "Java is fun" → "avaJ si nuf"

Q_312.java

```java
import java.util.Scanner;

public class Q_312
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine().trim() + " ";
        String res = "", rev = "";

        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (ch != ' ')
            {
                rev = ch + rev;
            }
            else if (rev.length() > 0)
            {
                res = res + rev + " ";
                rev = "";
            }
        }
        System.out.println(res.trim());
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
res	String	to store the result
rev	String	to build each reversed word
i	int	to act as the index
ch	char	to store each character

Q313 Reverse the Order of Words

Write a program to accept a sentence and display the words in reverse order. Extra spaces in the input must be removed in the output.

Example: " I love Java programming " → "programming Java love I"

Q_313.java

```java
import java.util.Scanner;

public class Q_313
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter a sentence: ");
        String s = " " + sc.nextLine().trim();
        String res = "", w = "";

        for (int i = s.length() - 1; i >= 0; i--)
        {
            char ch = s.charAt(i);
            if (ch != ' ')
            {
                w = ch + w;
            }
            else if (w.length() > 0)
            {
                res = res + w + " ";
                w = "";
            }
        }
        System.out.println(res.trim());
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
res	String	to store the result
w	String	to build each word
i	int	to act as the index
ch	char	to store each character

Q314 Toggle the Case of Each Letter

Write a program to accept a sentence and toggle the case of each letter (uppercase to lowercase and lowercase to uppercase). Other characters remain unchanged.

Example: "Hello World 2027!" → "hELLO wORLD 2027!"

Q_314.java

```
import java.util.Scanner;

public class Q_314
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine();
        String res = "";

        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (Character.isUpperCase(ch))
                res = res + Character.toLowerCase(ch);
            else if (Character.isLowerCase(ch))
                res = res + Character.toUpperCase(ch);
            else
                res = res + ch;
        }
        System.out.println(res);
```

```
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
res	String	to store the toggled sentence
i	int	to act as the index
ch	char	to store each character

Q315 Frequency of Each Letter

> Write a program to accept a sentence and display the frequency of each letter of the alphabet present in it, ignoring case (letters that do not occur are not displayed).
>
> Example: "Hello" → E 1, H 1, L 2, O 1

Q_315.java

```java
import java.util.Scanner;

public class Q_315
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine().toUpperCase();

        System.out.println("Letter\tFrequency");
        for (char ch = 'A'; ch <= 'Z'; ch++)
        {
            int f = 0;
            for (int i = 0; i < s.length(); i++)
            {
                if (s.charAt(i) == ch)
                    f++;
            }
            if (f > 0)
                System.out.println(ch + "\t" + f);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence in uppercase
ch	char	to act as the letter being counted
f	int	to store the frequency
i	int	to act as the index

Q316 Letters of a Word in Alphabetical Order

Write a program to accept a word and arrange its letters in alphabetical order (ignoring case while comparing). Store the letters in a character array and use bubble sort.

Example: COMPUTER → CEMOPRTU

Q_316.java

```java
import java.util.Scanner;

public class Q_316
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a word: ");
        String w = sc.next();
        int n = w.length();
        char a[] = new char[n];
        for (int i = 0; i < n; i++)
            a[i] = w.charAt(i);

        for (int i = 0; i < n - 1; i++)
        {
            for (int j = 0; j < n - 1 - i; j++)
            {
                if (Character.toLowerCase(a[j]) > Character.toLowerCase(a[j + 1]))
                {
                    char t = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = t;
                }
            }
        }

        String res = "";
        for (int i = 0; i < n; i++)
            res = res + a[i];
        System.out.println("Sorted: " + res);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
w	String	to store the word
n	int	to store the length of the word
a	char[]	to store the letters
i	int	to count the passes / act as index
j	int	to act as the comparison index
t	char	to hold a letter while swapping
res	String	to store the sorted word

Q317 Special Word or Palindrome Word

A Special word is a word that begins and ends with the same letter. A Palindrome word reads the same backwards (every palindrome is also special). Write a program to accept a word and display "Palindrome Word" if it is a palindrome, "Special Word" if it is special but not a palindrome, and "Neither special nor palindrome" otherwise. Ignore the case of letters.

Example: EXISTENCE → Special Word, COMIC → Special Word, MALAYALAM → Palindrome Word

Q_317.java

```java
import java.util.Scanner;

public class Q_317
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a word: ");
        String w = sc.next().toUpperCase();
        String rev = "";
        for (int i = w.length() - 1; i >= 0; i--)
            rev = rev + w.charAt(i);

        if (rev.equals(w))
            System.out.println("Palindrome Word");
        else if (w.charAt(0) == w.charAt(w.length() - 1))
            System.out.println("Special Word");
        else
            System.out.println("Neither special nor palindrome");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
w	String	to store the word in uppercase
rev	String	to store the reversed word
i	int	to act as the index

Q318 Count the Palindrome Words in a Sentence

Write a program to accept a sentence and display the palindrome words in it and their count. Ignore case, and consider only words with at least two letters.

Example: "MOM AND DAD ARE COMING AT NOON" → MOM DAD NOON, count = 3

Q_318.java

```java
import java.util.Scanner;

public class Q_318
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine().trim() + " ";
        String w = "";
        int count = 0;

        for (int i = 0; i < s.length(); i++)
        {
```

```
            char ch = s.charAt(i);
            if (ch != ' ')
            {
                w = w + ch;
            }
            else if (w.length() > 0)
            {
                String rev = "";
                for (int j = w.length() - 1; j >= 0; j--)
                    rev = rev + w.charAt(j);
                if (w.length() > 1 && rev.equalsIgnoreCase(w))
                {
                    System.out.print(w + " ");
                    count++;
                }
                w = "";
            }
        }
        System.out.println();
        System.out.println("Number of palindrome words = " + count);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
w	String	to build each word
count	int	to count the palindrome words
i	int	to act as the index
ch	char	to store each character
rev	String	to store the reversed word
j	int	to act as the reverse index

Q319 Frequency of a Given Word in a Sentence

> Write a program to accept a sentence and a word, and count how many times the word occurs in the sentence as a complete word (ignoring case).
>
> Example: Sentence: "the quick fox and THE lazy dog near the theatre", Word: the → Frequency = 3 (theatre is not counted)

Q_319.java

```
import java.util.Scanner;

public class Q_319
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine().trim() + " ";
        System.out.print("Enter the word: ");
        String key = sc.next();
        String w = "";
        int freq = 0;

        for (int i = 0; i < s.length(); i++)
        {
```

```
            char ch = s.charAt(i);
            if (ch != ' ')
            {
                w = w + ch;
            }
            else if (w.length() > 0)
            {
                if (w.equalsIgnoreCase(key))
                    freq++;
                w = "";
            }
        }
        System.out.println("Frequency = " + freq);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
key	String	to store the word to count
w	String	to build each word
freq	int	to count the occurrences
i	int	to act as the index
ch	char	to store each character

Q320 Replace a Word with Another Word

> Write a program to accept a sentence, a word to be replaced and a new word. Replace every occurrence of the word (as a complete word, ignoring case) with the new word and display the new sentence with single spaces between words.
>
> Example: "I like tea and Tea likes me", tea → coffee "I like coffee and coffee likes me"

Q_320.java

```
import java.util.Scanner;

public class Q_320
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine().trim() + " ";
        System.out.print("Enter the word to replace: ");
        String old = sc.next();
        System.out.print("Enter the new word: ");
        String nw = sc.next();
        String res = "", w = "";
        int count = 0;

        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (ch != ' ')
            {
                w = w + ch;
            }
            else if (w.length() > 0)
            {
```

```
                if (w.equalsIgnoreCase(old))
                {
                    res = res + nw + " ";
                    count++;
                }
                else
                {
                    res = res + w + " ";
                }
                w = "";
            }
        }
        System.out.println(res.trim());
        System.out.println("Words replaced: " + count);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
old	String	to store the word to replace
nw	String	to store the new word
res	String	to store the new sentence
w	String	to build each word
count	int	to count the replacements
i	int	to act as the index
ch	char	to store each character

Q321 Remove the Vowels from a Sentence

Write a program to accept a sentence and remove all the vowels (uppercase and lowercase) from it. Display the new sentence and the number of vowels removed.

Example: "Computer Applications" → "Cmptr pplctns" (8 vowels removed)

Q_321.java

```
import java.util.Scanner;

public class Q_321
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine();
        String res = "";
        int removed = 0;

        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            char lc = Character.toLowerCase(ch);
            if (lc == 'a' || lc == 'e' || lc == 'i' || lc == 'o' || lc == 'u')
                removed++;
            else
                res = res + ch;
        }
        System.out.println(res);
```

```
        System.out.println("Vowels removed: " + removed);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
res	String	to store the sentence without vowels
removed	int	to count the vowels removed
i	int	to act as the index
ch	char	to store each character
lc	char	to store the lowercase form of ch

Q322 Word Pattern: B, BL, BLU, BLUE, BLUEJ

```
B
BL
BLU
BLUE
BLUEJ
```

Q_322.java

```
public class Q_322
{
    public static void main(String[] args)
    {
        String w = "BLUEJ";
        for (int i = 1; i <= w.length(); i++)
            System.out.println(w.substring(0, i));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
w	String	to store the word
i	int	to store the length of each part

Q323 Word Pattern: JAVA, AVA, VA, A

```
J A V A
  A V A
    V A
      A
```

Q_323.java

```
public class Q_323
{
    public static void main(String[] args)
    {
        String w = "JAVA";
        int n = w.length();
        for (int i = 0; i < n; i++)
        {
            for (int s = 1; s <= i; s++)
                System.out.print("  ");
            for (int j = i; j < n; j++)
```

```
                System.out.print(w.charAt(j) + " ");
            System.out.println();
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
w	String	to store the word
n	int	to store the length of the word
i	int	to control the rows
s	int	to print the leading spaces
j	int	to print the letters

Q324 Overloading joystring()

ICSE 2015 Design a class (written here as class Q_324) to overload a method joystring() as follows:

```
(i)   void joystring(String s, char ch1, char ch2)
      to replace every occurrence of ch1 with ch2 in s and print
      the new string
      Example: s = "TECHNALAGY", ch1 = 'A', ch2 = 'O'
               Output: TECHNOLOGY
(ii)  void joystring(String s)
      to print the position of the first space and the last
      space in s (or "No space" if there is none)
      Example: s = "Cloud computing means Internet based computing"
               Output: First index: 5, Last index: 36
(iii) void joystring(String s1, String s2)
      to join the two strings with a space between them using
      concat() and print the result
      Example: s1 = "COMMON WEALTH", s2 = "GAMES"
               Output: COMMON WEALTH GAMES
```

Write a main method to call the methods with the example values.

Q_324.java

```
public class Q_324
{
    void joystring(String s, char ch1, char ch2)
    {
        System.out.println(s.replace(ch1, ch2));
    }

    void joystring(String s)
    {
        int f = s.indexOf(' ');
        int l = s.lastIndexOf(' ');
        if (f == -1)
            System.out.println("No space");
        else
            System.out.println("First index: " + f + ", Last index: " + l);
    }

    void joystring(String s1, String s2)
    {
        System.out.println(s1.concat(" ").concat(s2));
    }

    public static void main(String[] args)
    {
        Q_324 ob = new Q_324();
```

```
        ob.joystring("TECHNALAGY", 'A', 'O');
        ob.joystring("Cloud computing means Internet based computing");
        ob.joystring("COMMON WEALTH", "GAMES");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to receive the string
ch1	char	to receive the character to replace
ch2	char	to receive the new character
f	int	to store the index of the first space
l	int	to store the index of the last space
s1	String	to receive the first string
s2	String	to receive the second string
ob	Q_324	to refer to the object

Q325 Overloading count(): Vowels and a Given Character

Design a class (written here as class Q_325) to overload a method count() as follows:

```
(i)  int count(String s)          : returns the number of vowels
                                    in s (ignoring case)
(ii) int count(String s, char ch) : returns the number of times
                                    ch occurs in s (ignoring case)
```

Write a main method to accept a sentence and a character and display both counts.

Q_325.java

```
import java.util.Scanner;

public class Q_325
{
    int count(String s)
    {
        int c = 0;
        String t = s.toLowerCase();
        for (int i = 0; i < t.length(); i++)
        {
            char ch = t.charAt(i);
            if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
                c++;
        }
        return c;
    }

    int count(String s, char ch)
    {
        int c = 0;
        for (int i = 0; i < s.length(); i++)
        {
            if (Character.toLowerCase(s.charAt(i)) == Character.toLowerCase(ch))
                c++;
        }
        return c;
    }

    public static void main(String[] args)
    {
```

```
        Scanner sc = new Scanner(System.in);
        Q_325 ob = new Q_325();
        System.out.print("Enter a sentence: ");
        String str = sc.nextLine();
        System.out.print("Enter a character: ");
        char x = sc.next().charAt(0);
        System.out.println("Number of vowels = " + ob.count(str));
        System.out.println("Occurrences of " + x + " = " + ob.count(str, x));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to receive the string
c	int	to store the count
t	String	to store s in lowercase
i	int	to act as the index
ch	char	to store/receive a character
str	String	to store the sentence entered
x	char	to store the character entered
ob	Q_325	to refer to the object

Q326 Count the Words Beginning with a Vowel

Write a program to accept a sentence and display the words that begin with a vowel and their count (ignoring case).

Example: "An apple a day keeps illness away" → An apple a illness away, count = 5

Q_326.java

```
import java.util.Scanner;

public class Q_326
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine().trim() + " ";
        String w = "";
        int count = 0;

        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (ch != ' ')
            {
                w = w + ch;
            }
            else if (w.length() > 0)
            {
                char f = Character.toUpperCase(w.charAt(0));
                if (f == 'A' || f == 'E' || f == 'I' || f == 'O' || f == 'U')
                {
                    System.out.print(w + " ");
                    count++;
                }
                w = "";
```

```
            }
        }
        System.out.println();
        System.out.println("Count = " + count);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
w	String	to build each word
count	int	to count the words
i	int	to act as the index
ch	char	to store each character
f	char	to store the first letter of a word

Q327 Names Starting or Ending with Given Letters

Write a program to accept n names in a String array. Then accept a starting string and an ending string, and display the names that begin with the starting string (using startsWith()) and the names that end with the ending string (using endsWith()). Comparisons are case sensitive.

Example: Names: Anita, Arnav, Rina, Asha, Ronit; start "A", end "a" → starting with A: Anita Arnav Asha; ending with a: Anita Rina Asha

Q_327.java

```
import java.util.Scanner;

public class Q_327
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of names: ");
        int n = sc.nextInt();
        String names[] = new String[n];
        System.out.println("Enter the names:");
        for (int i = 0; i < n; i++)
            names[i] = sc.next();
        System.out.print("Enter the starting string: ");
        String st = sc.next();
        System.out.print("Enter the ending string: ");
        String en = sc.next();

        System.out.print("Starting with " + st + ": ");
        for (int i = 0; i < n; i++)
        {
            if (names[i].startsWith(st))
                System.out.print(names[i] + " ");
        }
        System.out.println();
        System.out.print("Ending with " + en + "  : ");
        for (int i = 0; i < n; i++)
        {
            if (names[i].endsWith(en))
                System.out.print(names[i] + " ");
        }
        System.out.println();
```

```
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	int	to store the number of names
names	String[]	to store the names
i	int	to act as the index
st	String	to store the starting string
en	String	to store the ending string

Q328 Output of the String Class Methods

Write a program to store the strings s1 = " Computer Applications " and s2 = "ICSE" in variables and display the results of the following String methods:

```
s1.length()             s1.trim()               s1.trim().length()
s2.charAt(2)            s2.toLowerCase()        s2.concat(" 2027")
s1.indexOf('p')         s1.lastIndexOf('p')     s1.indexOf("tion")
s1.trim().substring(9)  s1.trim().substring(0, 8)
s2.replace('C', 'K')    s1.trim().startsWith("Comp")
s2.endsWith("se")       s2.equals("icse")       s2.equalsIgnoreCase("icse")
String.valueOf(2027)    String.valueOf(3.5) + 1
```

Q_328.java

```
public class Q_328
{
    public static void main(String[] args)
    {
        String s1 = "  Computer Applications  ";
        String s2 = "ICSE";

        System.out.println("s1.length()                 = " + s1.length());
        System.out.println("s1.trim()                   = [" + s1.trim() + "]");
        System.out.println("s1.trim().length()          = " + s1.trim().length());
        System.out.println("s2.charAt(2)                = " + s2.charAt(2));
        System.out.println("s2.toLowerCase()            = " + s2.toLowerCase());
        System.out.println("s2.concat(\" 2027\")           = " + s2.concat(" 2027"));
        System.out.println("s1.indexOf('p')             = " + s1.indexOf('p'));
        System.out.println("s1.lastIndexOf('p')         = " + s1.lastIndexOf('p'));
        System.out.println("s1.indexOf(\"tion\")           = " + s1.indexOf("tion"));
        System.out.println("s1.trim().substring(9)      = " + s1.trim().substring(9));
        System.out.println("s1.trim().substring(0, 8)   = " + s1.trim().substring(0, 8));
        System.out.println("s2.replace('C', 'K')        = " + s2.replace('C', 'K'));
        System.out.println("s1.trim().startsWith(\"Comp\") = " + s1.trim().startsWith("Comp"));
        System.out.println("s2.endsWith(\"se\")            = " + s2.endsWith("se"));
        System.out.println("s2.equals(\"icse\")            = " + s2.equals("icse"));
        System.out.println("s2.equalsIgnoreCase(\"icse\")  = " + s2.equalsIgnoreCase("icse"));
        System.out.println("String.valueOf(2027)        = " + String.valueOf(2027));
        System.out.println("String.valueOf(3.5) + 1     = " + String.valueOf(3.5) + 1);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s1	String	to store the first string
s2	String	to store the second string

Q329 Output of compareTo(), compareToIgnoreCase() and equals()

Write a program to display the results of the following comparisons and explain each result in a comment beside it:

```
"APPLE".compareTo("APPLY")          "apple".compareTo("APPLE")
"BAT".compareTo("BALL")             "CAT".compareTo("CATALOG")
"Zebra".compareTo("apple")          "Mango".compareToIgnoreCase("mango")
"ICSE".compareTo("ICSE")            "Java".equals("java")
"Java".equalsIgnoreCase("JAVA")
```

Q_329.java

```java
public class Q_329
{
    public static void main(String[] args)
    {
        String a = "APPLE", b = "APPLY", c = "apple";
        System.out.println(a.compareTo(b));                    // 'E' - 'Y' = 69 - 89 = -20
        System.out.println(c.compareTo(a));                    // 'a' - 'A' = 97 - 65 = 32
        System.out.println("BAT".compareTo("BALL"));            // 'T' - 'L' = 84 - 76 = 8
        System.out.println("CAT".compareTo("CATALOG"));        // prefix: 3 - 7 = -4 (difference in
lengths)
        System.out.println("Zebra".compareTo("apple"));        // 'Z' - 'a' = 90 - 97 = -7
        System.out.println("Mango".compareToIgnoreCase("mango")); // equal ignoring case: 0
        System.out.println("ICSE".compareTo("ICSE"));          // identical strings: 0
        System.out.println("Java".equals("java"));             // case sensitive: false
        System.out.println("Java".equalsIgnoreCase("JAVA"));   // ignores case: true
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	String	to store the string APPLE
b	String	to store the string APPLY
c	String	to store the string apple

Q330 Count Pairs of Consecutive Letters

Write a program to accept a sentence, convert it to uppercase and count the number of pairs of consecutive letters of the alphabet appearing together in it (such as AB, ST, OP, HI). Display each pair found and the count.

Example: "IT WAS NOT TOUGH FOR HIM TO RESIDE ABOVE THE HILL" → pairs: NO, GH, HI, DE, AB, HI → count = 6

Q_330.java

```java
import java.util.Scanner;

public class Q_330
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine().toUpperCase();
        int count = 0;

        for (int i = 0; i < s.length() - 1; i++)
        {
            char a = s.charAt(i);
            char b = s.charAt(i + 1);
            if (Character.isLetter(a) && Character.isLetter(b) && b == a + 1)
```

```
            {
                System.out.print("" + a + b + " ");
                count++;
            }
        }
        System.out.println();
        System.out.println("Number of consecutive pairs = " + count);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence in uppercase
count	int	to count the pairs
i	int	to act as the index
a	char	to store the current character
b	char	to store the next character

Q331 Word with the Maximum Number of Vowels

> Write a program to accept a sentence and display the word having the maximum number of vowels, along with that number. If two or more words have the same maximum, display the first one.
>
> Example: "Education is the key to opportunity" → Education (5 vowels)

Q_331.java

```
import java.util.Scanner;

public class Q_331
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine().trim() + " ";
        String w = "", best = "";
        int max = -1;

        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (ch != ' ')
            {
                w = w + ch;
            }
            else if (w.length() > 0)
            {
                int v = 0;
                for (int j = 0; j < w.length(); j++)
                {
                    char c = Character.toLowerCase(w.charAt(j));
                    if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u')
                        v++;
                }
                if (v > max)
                {
                    max = v;
                    best = w;
                }
                w = "";
```

```
            }
        }
        if (max < 0)
            System.out.println("No words entered");
        else
            System.out.println(best + " (" + max + " vowels)");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
w	String	to build each word
best	String	to store the word with most vowels
max	int	to store the maximum vowel count
i	int	to act as the index
ch	char	to store each character
v	int	to count the vowels in a word
j	int	to act as the index within a word
c	char	to store a lowercase letter of the word

Q332 Swap the First and Last Letters of Each Word

Write a program to accept a sentence and interchange the first and the last letters of each word (single-letter words remain unchanged). Display the new sentence with single spaces between words.

Example: "Mumbai is a great city" → "iumbaM si a treag yitc"

Q_332.java

```
import java.util.Scanner;

public class Q_332
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine().trim() + " ";
        String res = "", w = "";

        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (ch != ' ')
            {
                w = w + ch;
            }
            else if (w.length() > 0)
            {
                int n = w.length();
                if (n > 1)
                    w = w.charAt(n - 1) + w.substring(1, n - 1) + w.charAt(0);
                res = res + w + " ";
                w = "";
            }
        }
        System.out.println(res.trim());
```

```
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
res	String	to store the new sentence
w	String	to build each word
i	int	to act as the index
ch	char	to store each character
n	int	to store the length of a word

Q333 Display Each Word with its Length

Write a program to accept a sentence and display each word on a separate line along with the number of characters in it, followed by the total number of words.

Example: "Java is fun" → Java 4 / is 2 / fun 3 / Total words: 3

Q_333.java

```java
import java.util.Scanner;

public class Q_333
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine().trim() + " ";
        String w = "";
        int count = 0;

        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (ch != ' ')
            {
                w = w + ch;
            }
            else if (w.length() > 0)
            {
                System.out.println(w + "\t" + w.length());
                count++;
                w = "";
            }
        }
        System.out.println("Total words: " + count);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
w	String	to build each word
count	int	to count the words

Name of the Variable	Data Type	Purpose / Description
i	int	to act as the index
ch	char	to store each character

Q334 Username and Domain from an Email Address

Write a program to accept an email address and extract the username (the part before @) and the domain (the part after @) using indexOf(), lastIndexOf() and substring(). The address is valid only if it contains exactly one @, the @ is neither the first nor the last character, and the domain contains a dot that is not its first or last character. Display "Invalid email address" otherwise.

Example: aarav.sen@school.edu.in → Username: aarav.sen, Domain: school.edu.in

Q_334.java

```
import java.util.Scanner;

public class Q_334
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter an email address: ");
        String e = sc.nextLine().trim();
        int at = e.indexOf('@');
        boolean valid = at > 0 && at == e.lastIndexOf('@') && at < e.length() - 1;

        String user = "", domain = "";
        if (valid)
        {
            user = e.substring(0, at);
            domain = e.substring(at + 1);
            int dot = domain.indexOf('.');
            if (dot <= 0 || domain.charAt(domain.length() - 1) == '.')
                valid = false;
        }

        if (valid)
        {
            System.out.println("Username: " + user);
            System.out.println("Domain  : " + domain);
        }
        else
        {
            System.out.println("Invalid email address");
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
e	String	to store the email address
at	int	to store the position of @
valid	boolean	to record whether the address is valid
user	String	to store the username
domain	String	to store the domain
dot	int	to store the position of the first dot

Q335 Number in Words, Digit by Digit

Write a program to accept a non-negative integer, convert it into a String using String.valueOf(), and display each digit in words using a switch statement.

Example: 4052 → FOUR ZERO FIVE TWO

Q_335.java

```java
import java.util.Scanner;

public class Q_335
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        long n = sc.nextLong();
        if (n < 0)
        {
            System.out.println("Invalid input");
            return;
        }
        String s = String.valueOf(n);
        String res = "";

        for (int i = 0; i < s.length(); i++)
        {
            switch (s.charAt(i))
            {
                case '0': res = res + "ZERO "; break;
                case '1': res = res + "ONE "; break;
                case '2': res = res + "TWO "; break;
                case '3': res = res + "THREE "; break;
                case '4': res = res + "FOUR "; break;
                case '5': res = res + "FIVE "; break;
                case '6': res = res + "SIX "; break;
                case '7': res = res + "SEVEN "; break;
                case '8': res = res + "EIGHT "; break;
                default:  res = res + "NINE ";
            }
        }
        System.out.println(res.trim());
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
n	long	to store the number
s	String	to store the number as a string
res	String	to store the words
i	int	to act as the index

Q336 Characters at Alternate Positions

Write a program to accept a word and form two new words: one from the characters at odd positions (1st, 3rd, 5th, ...) and another from the characters at even positions (2nd, 4th, ...).

Example: COMPUTER → odd positions: CMUE, even positions: OPTR

Q_336.java

```java
import java.util.Scanner;

public class Q_336
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a word: ");
        String w = sc.next();
        String odd = "", even = "";

        for (int i = 0; i < w.length(); i++)
        {
            if (i % 2 == 0)
                odd = odd + w.charAt(i);
            else
                even = even + w.charAt(i);
        }
        System.out.println("Odd positions : " + odd);
        System.out.println("Even positions: " + even);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
w	String	to store the word
odd	String	to store characters at odd positions
even	String	to store characters at even positions
i	int	to act as the index

Q337 Encode a Sentence by Shifting Letters by 2

> Write a program to accept a sentence and encode it by replacing each letter with the letter two places after it in the alphabet, wrapping around at the end (Y becomes A, Z becomes B, y becomes a, z becomes b). The case of the letters is kept, and other characters remain unchanged.
>
> Example: "Zebra crossing at 5 pm!" → "Bgdtc etquukpi cv 5 ro!"

Q_337.java

```java
import java.util.Scanner;

public class Q_337
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine();
        String res = "";

        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (Character.isUpperCase(ch))
                ch = (char) ('A' + (ch - 'A' + 2) % 26);
            else if (Character.isLowerCase(ch))
                ch = (char) ('a' + (ch - 'a' + 2) % 26);
            res = res + ch;
        }
        System.out.println(res);
```

```
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
res	String	to store the encoded sentence
i	int	to act as the index
ch	char	to store each character

Q338 Remove Extra Spaces from a Sentence

> Write a program to accept a sentence and remove all extra spaces from it, so that there is no space at the beginning or the end and exactly one space between words. Display the sentence before and after (within square brackets) and the number of spaces removed.

Q_338.java

```java
import java.util.Scanner;

public class Q_338
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine();
        String t = s.trim();
        String res = "";

        for (int i = 0; i < t.length(); i++)
        {
            char ch = t.charAt(i);
            if (ch != ' ' || t.charAt(i - 1) != ' ')
                res = res + ch;
        }
        System.out.println("Before: [" + s + "]");
        System.out.println("After : [" + res + "]");
        System.out.println("Spaces removed: " + (s.length() - res.length()));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
t	String	to store the trimmed sentence
res	String	to store the cleaned sentence
i	int	to act as the index
ch	char	to store each character

Q339 Words that Begin and End with a Vowel

Write a program to accept a sentence and display the words that begin and end with a vowel (ignoring case), and their count.

Example: "Anna ate an orange in Asia" → Anna ate orange Asia, count = 4

Q_339.java

```java
import java.util.Scanner;

public class Q_339
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine().trim() + " ";
        String vowels = "AEIOU";
        String w = "";
        int count = 0;

        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (ch != ' ')
            {
                w = w + ch;
            }
            else if (w.length() > 0)
            {
                char f = Character.toUpperCase(w.charAt(0));
                char l = Character.toUpperCase(w.charAt(w.length() - 1));
                if (vowels.indexOf(f) != -1 && vowels.indexOf(l) != -1)
                {
                    System.out.print(w + " ");
                    count++;
                }
                w = "";
            }
        }
        System.out.println();
        System.out.println("Count = " + count);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
vowels	String	to store the vowels
w	String	to build each word
count	int	to count the words
i	int	to act as the index
ch	char	to store each character
f	char	to store the first letter
l	char	to store the last letter

Q340 Words of a Sentence in Alphabetical Order

Write a program to accept a sentence, store its words in a String array (first count the words, then extract them), arrange the words in alphabetical order (ignoring case) using bubble sort, and display them as a sentence.

Example: "the quick brown fox" → "brown fox quick the"

Q_340.java

```
import java.util.Scanner;

public class Q_340
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine().trim() + " ";

        int n = 0;
        for (int i = 0; i < s.length() - 1; i++)
        {
            if (s.charAt(i) != ' ' && s.charAt(i + 1) == ' ')
                n++;
        }

        String words[] = new String[n];
        String w = "";
        int k = 0;
        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (ch != ' ')
            {
                w = w + ch;
            }
            else if (w.length() > 0)
            {
                words[k++] = w;
                w = "";
            }
        }

        for (int i = 0; i < n - 1; i++)
        {
            for (int j = 0; j < n - 1 - i; j++)
            {
                if (words[j].compareToIgnoreCase(words[j + 1]) > 0)
                {
                    String t = words[j];
                    words[j] = words[j + 1];
                    words[j + 1] = t;
                }
            }
        }

        String res = "";
        for (int i = 0; i < n; i++)
            res = res + words[i] + " ";
        System.out.println(res.trim());
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
n	int	to store the number of words
i	int	to act as the index
words	String[]	to store the words
w	String	to build each word
k	int	to count the words stored
ch	char	to store each character
j	int	to act as the comparison index
t	String	to hold a word while swapping
res	String	to store the sorted sentence

Q341 First Non-Repeating Character

Write a program to accept a word and display the first character that does not repeat anywhere in the word (ignoring case). If every character repeats, display "No non-repeating character".

Example: "swiss" → w, "Programming" → P, "aabb" → No non-repeating character

Q_341.java

```
import java.util.Scanner;

public class Q_341
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a word: ");
        String w = sc.next();
        String low = w.toLowerCase();
        int pos = -1;

        for (int i = 0; i < low.length(); i++)
        {
            char ch = low.charAt(i);
            if (low.indexOf(ch) == low.lastIndexOf(ch))
            {
                pos = i;
                break;
            }
        }
        if (pos == -1)
            System.out.println("No non-repeating character");
        else
            System.out.println("First non-repeating character: " + w.charAt(pos));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
w	String	to store the word
low	String	to store the word in lowercase

Name of the Variable	Data Type	Purpose / Description
pos	int	to store the position found
i	int	to act as the index
ch	char	to store each character

Q342 Anagram Check

Two words or phrases are anagrams if they contain the same letters the same number of times (ignoring case and spaces), e.g. LISTEN and SILENT, "Dormitory" and "Dirty room". Write a program to accept two strings and check whether they are anagrams, using an array of 26 counters. Display "Anagrams" or "Not Anagrams".

Q_342.java

```java
import java.util.Scanner;

public class Q_342
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the first string: ");
        String a = sc.nextLine().toUpperCase();
        System.out.print("Enter the second string: ");
        String b = sc.nextLine().toUpperCase();
        int count[] = new int[26];

        for (int i = 0; i < a.length(); i++)
        {
            char ch = a.charAt(i);
            if (ch >= 'A' && ch <= 'Z')
                count[ch - 'A']++;
        }
        for (int i = 0; i < b.length(); i++)
        {
            char ch = b.charAt(i);
            if (ch >= 'A' && ch <= 'Z')
                count[ch - 'A']--;
        }

        boolean ana = true;
        for (int i = 0; i < 26; i++)
        {
            if (count[i] != 0)
            {
                ana = false;
                break;
            }
        }
        if (ana)
            System.out.println("Anagrams");
        else
            System.out.println("Not Anagrams");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
a	String	to store the first string
b	String	to store the second string

Name of the Variable	Data Type	Purpose / Description
count	int[]	to store the letter count differences
i	int	to act as the index
ch	char	to store each character
ana	boolean	to record whether they are anagrams

Q343 Pangram Check

Extra A Pangram is a sentence that contains every letter of the English alphabet at least once, e.g. "The quick brown fox jumps over the lazy dog". Write a program to accept a sentence and check whether it is a pangram. If it is not, display the missing letters.

Q_343.java

```
import java.util.Scanner;

public class Q_343
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine().toUpperCase();
        String missing = "";

        for (char ch = 'A'; ch <= 'Z'; ch++)
        {
            if (s.indexOf(ch) == -1)
                missing = missing + ch + " ";
        }
        if (missing.length() == 0)
            System.out.println("Pangram");
        else
            System.out.println("Not a Pangram. Missing letters: " + missing.trim());
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence in uppercase
missing	String	to store the missing letters
ch	char	to act as the letter being checked

Q344 Login Check using concat() and equalsIgnoreCase()

A school portal creates a username by joining the first name, a dot and the last name in lowercase using concat(), e.g. Ria Sen → ria.sen; the password is "Icse@2027". Write a program to accept a student's first name and last name, generate and display the username, then accept a username and a password for login. The username is checked with equalsIgnoreCase() and the password with equals() (case sensitive). Display "Login successful", "Wrong username" or "Wrong password".

Q_344.java

```
import java.util.Scanner;

public class Q_344
```

```
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        final String PASS = "Icse@2027";
        System.out.print("Enter first name and last name: ");
        String f = sc.next();
        String l = sc.next();
        String user = f.toLowerCase().concat(".").concat(l.toLowerCase());
        System.out.println("Your username is " + user);

        System.out.print("Login - username: ");
        String u = sc.next();
        System.out.print("Login - password: ");
        String p = sc.next();

        if (!u.equalsIgnoreCase(user))
            System.out.println("Wrong username");
        else if (!p.equals(PASS))
            System.out.println("Wrong password");
        else
            System.out.println("Login successful");
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
PASS	String	to store the correct password
f	String	to store the first name
l	String	to store the last name
user	String	to store the generated username
u	String	to store the username entered
p	String	to store the password entered

Q345 Words that Come First and Last Alphabetically

> Write a program to accept a sentence and find the word that comes first and the word that comes last in alphabetical (dictionary) order, using compareToIgnoreCase().
>
> Example: "India is my country" → First: country, Last: my

Q_345.java

```
import java.util.Scanner;

public class Q_345
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = sc.nextLine().trim() + " ";
        String w = "", first = "", last = "";

        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (ch != ' ')
            {
                w = w + ch;
```

```
            }
            else if (w.length() > 0)
            {
                if (first.length() == 0 || w.compareToIgnoreCase(first) < 0)
                    first = w;
                if (last.length() == 0 || w.compareToIgnoreCase(last) > 0)
                    last = w;
                w = "";
            }
        }
        if (first.length() == 0)
        {
            System.out.println("No words entered");
        }
        else
        {
            System.out.println("First alphabetically: " + first);
            System.out.println("Last alphabetically : " + last);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the sentence
w	String	to build each word
first	String	to store the first word alphabetically
last	String	to store the last word alphabetically
i	int	to act as the index
ch	char	to store each character

Q346 Number to Roman Numerals

Extra Write a program to accept a number from 1 to 3999 and convert it into Roman numerals, using the symbols and values below stored in two parallel arrays (from the largest value to the smallest, repeatedly subtracting the largest value that fits).

```
M=1000 CM=900 D=500 CD=400 C=100 XC=90 L=50 XL=40 X=10 IX=9
V=5 IV=4 I=1
```

Example: 1994 → MCMXCIV, 2027 → MMXXVII. Display "Out of range" for other numbers.

Q_346.java

```
import java.util.Scanner;

public class Q_346
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int val[] = {1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1};
        String sym[] = {"M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V", "IV", "I"};
        System.out.print("Enter a number (1-3999): ");
        int n = sc.nextInt();

        if (n < 1 || n > 3999)
        {
            System.out.println("Out of range");
            return;
        }
```

```
        String roman = "";
        int num = n;
        for (int i = 0; i < val.length; i++)
        {
            while (num >= val[i])
            {
                roman = roman + sym[i];
                num = num - val[i];
            }
        }
        System.out.println(n + " = " + roman);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
val	int[]	to store the values of the symbols
sym	String[]	to store the roman symbols
n	int	to store the number
roman	String	to store the roman numeral
num	int	to store the value left to convert
i	int	to act as the index

Q347 Caesar Cipher: Encrypt and Decrypt

Extra In a Caesar cipher, each letter is shifted forward by a key k (from 1 to 25) in the alphabet, wrapping around after Z; decryption shifts it back by k. Write a program with the methods:

```
String encrypt(String s, int k)
String decrypt(String s, int k)
```

that keep the case of letters and leave other characters unchanged. In main(), accept a message and a key, display the encrypted text, and decrypt it back to show the original message. Display "Invalid key" for a key outside 1 to 25.

Q_347.java

```
import java.util.Scanner;

public class Q_347
{
    static String encrypt(String s, int k)
    {
        String res = "";
        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (Character.isUpperCase(ch))
                ch = (char) ('A' + (ch - 'A' + k) % 26);
            else if (Character.isLowerCase(ch))
                ch = (char) ('a' + (ch - 'a' + k) % 26);
            res = res + ch;
        }
        return res;
    }

    static String decrypt(String s, int k)
    {
        return encrypt(s, 26 - k);
    }
```

```
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the message: ");
        String msg = sc.nextLine();
        System.out.print("Enter the key (1-25): ");
        int key = sc.nextInt();

        if (key < 1 || key > 25)
        {
            System.out.println("Invalid key");
            return;
        }
        String enc = encrypt(msg, key);
        System.out.println("Encrypted: " + enc);
        System.out.println("Decrypted: " + decrypt(enc, key));
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to receive the text
k	int	to receive the key
res	String	to build the result
i	int	to act as the index
ch	char	to store each character
msg	String	to store the message
key	int	to store the key
enc	String	to store the encrypted text

Q348 Word Scramble Game

Extra Write a word scramble game. Store ten Java keywords in a String array. The computer picks one at random, scrambles it by swapping each letter with a randomly chosen letter of the word (if the result is the same as the word, it swaps the first two letters), and shows the scrambled word. The player gets three attempts to guess the word (ignoring case). Display "Correct!" or "Wrong, try again", and after three wrong attempts display the answer.

Q_348.java

```
import java.util.Scanner;

public class Q_348
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String words[] = {"public", "static", "string", "switch", "double",
                          "return", "import", "boolean", "private", "while"};
        String word = words[(int) (Math.random() * words.length)];
        int n = word.length();
        char c[] = new char[n];
        for (int i = 0; i < n; i++)
            c[i] = word.charAt(i);

        for (int i = 0; i < n; i++)
        {
```

```
            int r = (int) (Math.random() * n);
            char t = c[i];
            c[i] = c[r];
            c[r] = t;
        }
        String mixed = "";
        for (int i = 0; i < n; i++)
            mixed = mixed + c[i];
        if (mixed.equals(word))
            mixed = word.substring(1, 2) + word.substring(0, 1) + word.substring(2);

        System.out.println("Unscramble this Java keyword: " + mixed.toUpperCase());
        boolean won = false;
        for (int tries = 1; tries <= 3 && !won; tries++)
        {
            System.out.print("Attempt " + tries + ": ");
            String g = sc.next();
            if (g.equalsIgnoreCase(word))
            {
                won = true;
                System.out.println("Correct!");
            }
            else
            {
                System.out.println("Wrong, try again");
            }
        }
        if (!won)
            System.out.println("The word was: " + word);
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
words	String[]	to store the keywords
word	String	to store the chosen word
n	int	to store the length of the word
c	char[]	to store the letters to scramble
i	int	to act as the index
r	int	to store a random index
t	char	to hold a letter while swapping
mixed	String	to store the scrambled word
won	boolean	to record whether the player won
tries	int	to count the attempts
g	String	to store the player's guess

Q349 Run-Length Encoding

Extra Run-length encoding compresses a string by replacing each run of the same character with the character followed by the length of the run. Write a program to accept a word and display its run-length encoding and the compression (length of the encoded word compared with the original).

Example: aaabbcdddd → a3b2c1d4

Q_349.java

```java
import java.util.Scanner;

public class Q_349
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a word: ");
        String s = sc.next();
        String res = "";
        int i = 0;

        while (i < s.length())
        {
            char ch = s.charAt(i);
            int run = 1;
            while (i + run < s.length() && s.charAt(i + run) == ch)
                run++;
            res = res + ch + run;
            i = i + run;
        }
        System.out.println("Encoded: " + res);
        System.out.println("Original length " + s.length() + ", encoded length " + res.length());
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
s	String	to store the word
res	String	to store the encoded word
i	int	to store the start of a run
ch	char	to store the character of the run
run	int	to store the length of the run

Q350 24-Hour Time to 12-Hour Time

Extra Write a program to accept a time in 24-hour format as a string "HH:MM" (for example 13:05) and convert it into 12-hour format with AM or PM (1:05 PM). Midnight 00:MM is 12:MM AM and noon 12:MM is 12:MM PM. Validate the input: it must have the form of two digits, a colon and two digits, with hours 00-23 and minutes 00-59; otherwise display "Invalid time".

Q_350.java

```java
import java.util.Scanner;

public class Q_350
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter time (HH:MM): ");
        String t = sc.next();
        boolean ok = t.length() == 5 && t.charAt(2) == ':'
                  && Character.isDigit(t.charAt(0)) && Character.isDigit(t.charAt(1))
                  && Character.isDigit(t.charAt(3)) && Character.isDigit(t.charAt(4));

        int h = 0, m = 0;
        if (ok)
        {
```

```
            h = Integer.parseInt(t.substring(0, 2));
            m = Integer.parseInt(t.substring(3));
            if (h > 23 || m > 59)
                ok = false;
        }

        if (!ok)
        {
            System.out.println("Invalid time");
        }
        else
        {
            String ampm = (h < 12) ? "AM" : "PM";
            int h12 = h % 12;
            if (h12 == 0)
                h12 = 12;
            String mm = t.substring(3);
            System.out.println(h12 + ":" + mm + " " + ampm);
        }
    }
}
```

VARIABLE DESCRIPTION

Name of the Variable	Data Type	Purpose / Description
t	String	to store the time entered
ok	boolean	to record whether the time is valid
h	int	to store the hours
m	int	to store the minutes
ampm	String	to store am or pm
h12	int	to store the hour in 12-hour format
mm	String	to store the minutes as text